PHP

CHEAT SHEETS

A Complete Reference for Modern PHP 8

Syntax & Types · Strings · match · Functions & Closures · Arrays · Classes & Enums · Property Hooks · Composer · PHPUnit & Pest · Forms & Sessions · PDO · Security

133 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–04	Introduction & Setup
05–07	Output & Comments
08–15	Variables & Data Types
16–21	Strings
22–27	Operators
28–35	Control Flow

CORE PHP

36–40	Error Handling
41–50	Functions
51–61	Arrays & Data Structures
62–76	Object-Oriented PHP
77–84	Files & I/O

ECOSYSTEM & WEB

85–93	Modules, Composer & Standard Library
94–101	Testing, Debugging & Tooling
102–108	Advanced Topics
109–118	Web Fundamentals
119–124	Databases with PDO
125–130	Security
131–133	Quick Reference

PHP

Table of Contents

133 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

01 PHP History & Versions
02 Installation & Configur…
03 Running PHP
04 Program Structure & P…

OUTPUT & COMMENTS 05 – 07

05 echo, print & Output
06 Dumping & Inspecting …
07 Comments & PHPDoc

VARIABLES & DATA TYPES 08 – 15

08 Variables & Assignment
09 Constants & Magic Co…
10 Integers
11 Floats
12 Booleans & Truthiness
13 Null
14 Type Juggling & Casting
15 Type Checking

STRINGS 16 – 21

16 String Basics & Interpo…
17 String Formatting
18 String Functions — Sea…
19 String Functions — Tra…
20 Substrings, Indexing & …
21 Multibyte Strings & UT…

OPERATORS 22 – 27

22 Arithmetic Operators
23 Comparison Operators
24 Logical Operators
25 Bitwise Operators
26 Assignment & Operato…
27 Null-Safe & Special Op…

CONTROL FLOW 28 – 35

28 if, elseif & else
29 Ternary & Conditional …
30 switch
31 match Expressions
32 for Loops
33 foreach Loops
34 while & do-while
35 Loop Controls

ERROR HANDLING 36 – 40

36 Errors vs Exceptions
37 try, catch & finally
38 Throwing & SPL Except…
39 Custom Exceptions
40 Global Error Handlers

FUNCTIONS 41 – 50

41 Defining & Calling Func…
42 Parameters & Argume…
43 Variadic Parameters & …
44 Parameter & Return Ty…
45 Variable Scope
46 Anonymous Functions …
47 Arrow Functions
48 Callables & First-Class…
49 Recursion
50 Generators & yield

ARRAYS & DATA STRUCTURES 51 – 61

51 Array Basics
52 Adding & Removing Ele…
53 Searching Arrays
54 Transforming Arrays
55 Sorting Arrays
56 Slicing, Merging & Buil…
57 Set Operations & Keys
58 Array Destructuring
59 Multidimensional Arrays
60 SPL Data Structures
61 Iterators & Iterables

OBJECT-ORIENTED PHP 62 – 76

62 Classes & Objects
63 Properties & Readonly
64 Constructors & Destru…
65 Visibility & Access Co…
66 Property Hooks
67 Static Members & Clas…
68 Inheritance
69 Abstract Classes
70 Interfaces
71 Traits
72 Enums
73 Magic Methods — Prop…
74 Magic Methods — Clon…
75 Cloning & Comparing …
76 Reflection & Object Uti…

FILES & I/O 77 – 84

77 Reading Files
78 Writing & Appending F…
79 File Paths & File Info
80 Directories & File Man…
81 JSON
82 CSV
83 XML & HTML Parsing
84 Streams & Wrappers

MODULES, COMPOSER & STANDARD LIBRARY 85 – 93

85 include & require
86 Namespaces
87 Autoloading & PSR-4
88 Composer Basics
89 Composer — Scripts &…
90 Math & Random
91 Dates — date() & Time…
92 DateTime Objects
93 System & Environment

TESTING, DEBUGGING & TOOLING 94 – 101

94 PHPUnit Basics
95 PHPUnit — Data Provid…
96 Pest
97 Debugging with Xdebug
98 Static Analysis — PHPS…
99 Code Style & Refactori…
100 Logging — Monolog &…
101 HTTP Clients — cURL …

ADVANCED TOPICS 102 – 108

102 Regular Expressions (…
103 Attributes
104 Serialization
105 Fibers & Async PHP
106 CLI Scripts
107 Performance, OPcach…
108 Memory, Weak Refere…

WEB FUNDAMENTALS 109 – 118

109 The Request Lifecycle
110 Superglobals
111 Handling Forms
112 Input Validation & Filt…
113 Headers, Redirects & …
114 Cookies
115 Sessions
116 File Uploads
117 Templating & Views
118 Routing & JSON APIs

DATABASES WITH PDO 119 – 124

119 PDO — Connecting
120 PDO — Prepared Stat…
121 PDO — Fetching Resul…
122 PDO — Transactions …
123 MySQLi
124 ORMs & Query Builde…

SECURITY 125 – 130

125 XSS & Output Escapi…
126 SQL Injection
127 CSRF & Session Secu…
128 Password Hashing
129 Hashing, HMAC & Enc…
130 Secure Configuration …

QUICK REFERENCE 131 – 133

131 PHP 8.x Feature Timel…
132 Quick Reference — S…
133 Quick Reference — C…

PHP History & Versions

WHAT PHP IS

PHP is a general-purpose scripting language built for the web. Rasmus Lerdorf released *Personal Home Page Tools* in 1995; today the name is the recursive *PHP: Hypertext Preprocessor*. The reference implementation runs on the **Zend Engine**, which compiles source to opcodes and executes them. PHP 7 roughly doubled performance over PHP 5; PHP 8 added a JIT, union types, `match`, attributes, enums and much more. New features arrive through public **RFCs** that core developers vote on.

NOTES

- One minor release every year, around late November
- Each release gets 2 years of active support + 2 years of security fixes
- There was no PHP 6 — the Unicode rewrite was abandoned
- **PHP-FIG** publishes PSRs: shared standards for autoloading, logging, HTTP
- Docs and changelogs live at `php.net`

CHECKING YOUR VERSION

```php
<?php
echo PHP_VERSION;          // "8.5.0"
echo PHP_MAJOR_VERSION;    // 8
echo PHP_MINOR_VERSION;    // 5
echo PHP_OS_FAMILY;        // "Linux", "Darwin", "Windows"

if (version_compare(PHP_VERSION, '8.4.0', '<')) {
    exit('This script needs PHP 8.4 or newer');
}

// Feature check without a version number
if (function_exists('array_find')) {
    $first = array_find([1, 4, 9], fn($n) => $n > 3);
}

phpinfo();                 // full config page (dev only)
```

CODE EXPLAINED

- → `PHP_VERSION` — full version string of the running engine
- → `version_compare()` — compares versions correctly (`8.10 > 8.9`)
- → `function_exists()` — test for a feature instead of a version
- → `phpinfo()` — prints build, extensions and ini values
- → Never leave `phpinfo()` reachable on a public server

MAJOR MILESTONES

VERSION	YEAR · HIGHLIGHTS
`PHP 3`	1998 · Zeev Suraski & Andi Gutmans rewrite
`PHP 4`	2000 · Zend Engine 1
`PHP 5`	2004 · New object model, exceptions
`PHP 5.3`	2009 · Namespaces, closures
`PHP 7.0`	2015 · Engine rewrite, scalar types, `??`
`PHP 7.4`	2019 · Typed properties, `fn` arrows
`PHP 8.0`	2020 · JIT, `match`, named args

PHP 8 RELEASES

VERSION	HEADLINE FEATURES
`8.1`	Enums, readonly, fibers, `never`
`8.2`	Readonly classes, DNF types
`8.3`	Typed class constants, `json_validate()`
`8.4`	Property hooks, asymmetric visibility
`8.5`	Pipe operator `|>`, `clone with`

KEY TERMS

TERM	MEANING
`Zend Engine`	The interpreter core
`SAPI`	Server API: CLI, FPM, Apache module
`RFC`	Proposal + vote for a language change
`PECL`	C extension repository
`PSR`	PHP-FIG standard recommendation
`Composer`	Dependency manager

⚠ COMMON MISTAKES

- Following tutorials written for PHP 5 — `mysql_*` functions and old idioms are gone
- Running an end-of-life version in production — no security fixes
- Checking `PHP_VERSION` with string comparison instead of `version_compare()`

✓ TIPS & BEST PRACTICES

- Target a supported version (8.3+) for new projects
- Read the *Migration* appendix in the manual before upgrading
- Pin the version in `composer.json` (`"php": "^8.3"`)

QUICK SUMMARY

- Zend Engine compiles to opcodes
- Yearly releases, ~4 years of support
- PHP 8 = modern typed language
- Standards come from RFCs and PSRs

Installation & Configuration

GETTING PHP

Install PHP from your OS package manager, a bundle such as XAMPP or Laragon, or run it in Docker. The **CLI binary** (`php`) is all you need to run scripts and the built-in server. Behaviour is controlled by **php.ini**; each SAPI (CLI, FPM, Apache) can load a different ini file. Features such as `pdo_mysql`, `mbstring` and `intl` come from **extensions** that must be installed and enabled.

NOTES

- `php -v` shows version; `php -m` lists loaded extensions
- `php --ini` shows which ini files were loaded
- Extra ini snippets live in a `conf.d/` folder
- Ship `php.ini-development` locally, `php.ini-production` on servers
- Restart FPM / Apache after editing ini files

INSTALL & INSPECT

```
# macOS (Homebrew)
brew install php

# Debian / Ubuntu
sudo apt install php-cli php-mbstring php-xml php-pdo

# Windows: winget install PHP.PHP.8.4  (or XAMPP)

php -v                  # version + Zend Engine
php -m                  # loaded extensions
php --ini               # loaded php.ini path(s)
php -i | grep memory    # one ini setting (CLI)
php -r 'echo ini_get("memory_limit");'

# Docker: run a script with no local install
docker run --rm -v "$PWD":/app -w /app \
  php:8.4-cli php script.php
```

CODE EXPLAINED

- → Package names vary by distro (`php8.4-cli` on some)
- → `-m` — confirm an extension such as `pdo_mysql` is enabled
- → `--ini` — the first thing to check when a setting won't apply
- → `-r` — run inline code without `<?php` tags
- → `php:8.4-cli` — official image; use `-fpm` / `-apache` for servers

COMMON INI SETTINGS

DIRECTIVE	PURPOSE
`error_reporting`	Which errors are raised (E_ALL)
`display_errors`	Print errors to output (dev only)
`memory_limit`	Max memory per script
`max_execution_time`	Seconds before timeout (web)
`upload_max_filesize`	Largest single upload
`date.timezone`	Default timezone

ESSENTIAL EXTENSIONS

EXTENSION	USED FOR
`mbstring`	Multibyte (UTF-8) strings
`intl`	Locale formatting, ICU
`pdo_mysql`	MySQL via PDO
`curl`	HTTP requests
`zip`	Composer, archives
`opcache`	Bytecode cache (speed)

INSTALL OPTIONS

OPTION	BEST FOR
`OS packages`	Servers, Linux desktops
`Homebrew`	macOS development
`XAMPP / Laragon`	Windows all-in-one
`Docker`	Matching production exactly
`Herd / Valet`	Laravel-style local sites

⚠ COMMON MISTAKES

- Editing the wrong php.ini — CLI and FPM often use different files
- Forgetting to restart FPM after an ini change
- Leaving `display_errors=On` in production

✓ TIPS & BEST PRACTICES

- Run `php --ini` before editing any configuration
- Put custom settings in a small `conf.d/99-local.ini` file
- Use the same major.minor version locally and in production

QUICK SUMMARY

- `php -v` / `-m` / `--ini` to inspect
- php.ini controls behaviour per SAPI
- Extensions add features
- Docker gives exact versions

Running PHP

WAYS TO RUN CODE

PHP runs through a **SAPI** (Server API). The **CLI** runs scripts from a terminal and is used for tools, cron jobs and tests. For the web, **PHP-FPM** runs a pool of worker processes behind nginx or Apache; `mod_php` embeds PHP inside Apache. During development the **built-in server** (`php -S`) serves a folder with no web server at all. Every request starts with a fresh state.

NOTES

- CLI has no execution time limit by default
- `php -S` is single-process — never use it in production
- A router script handles every request (`php -S host:port router.php`)
- `PHP_SAPI` tells you which SAPI is running
- `php -l file.php` checks syntax without running

CLI, REPL & BUILT-IN SERVER

```
php hello.php               # run a script
php hello.php Ana 42        # args land in $argv
php -r 'echo 2 ** 10;'      # inline code -> 1024
php -a                      # interactive shell
php -l src/User.php         # lint (syntax check)

php -S localhost:8000           # serve current folder
php -S localhost:8000 -t public  # docroot = public/
php -S 0.0.0.0:8000 router.php   # every request -> router

# Executable script (chmod +x tool.php)
#!/usr/bin/env php
<?php
echo "Running under " . PHP_SAPI . PHP_EOL;  // cli
```

CODE EXPLAINED

→ `$argv[0]` is the script name; arguments follow

→ `-a` — REPL; end statements with `;`

→ `-t public` — expose only the public folder

→ router.php returns `false` to serve static files as-is

→ Shebang line lets you run `./tool.php` directly

CLI FLAGS

FLAG	EFFECT
`-r 'code'`	Run code (no <?php)
`-a`	Interactive shell
`-l`	Syntax check
`-S host:port`	Built-in web server
`-t dir`	Document root for -S
`-d key=val`	Override an ini value
`-c file`	Use a specific php.ini

SAPIS

SAPI	WHERE IT RUNS
`cli`	Terminal, cron, workers
`fpm-fcgi`	PHP-FPM behind nginx/Apache
`apache2handler`	mod_php inside Apache
`cli-server`	`php -S` dev server
`embed / frankenphp`	Embedded runtimes

WEB STACKS

STACK	NOTES
`nginx + FPM`	Most common in production
`Apache + FPM`	`.htaccess` support
`Apache + mod_php`	Older, simple
`Caddy / FrankenPHP`	Worker mode, HTTPS built in
`php -S`	Development only

⚠ COMMON MISTAKES

- Deploying with `php -S` — it handles one request at a time
- Writing `<?php` in `php -r` code — it's a syntax error there
- Expecting variables to persist between web requests

✓ TIPS & BEST PRACTICES

- Use `php -l` in a pre-commit hook
- Pass `-t public` so config files are never served
- Check `PHP_SAPI === 'cli'` before running CLI-only code

QUICK SUMMARY

- CLI for scripts, FPM for the web
- `php -S` for quick local serving
- `-r` inline, `-a` REPL, `-l` lint
- Each request starts fresh

Sheet 04

Program Structure & PHP Tags

<?php · <?= · omitting ?> · semicolons · declare(strict_types=1) · mixing HTML

ANATOMY OF A PHP FILE

The parser only runs code between **PHP tags**. Everything outside `<?php ... ?>` is sent to the output unchanged, which is how PHP mixes with HTML. Statements end with a semicolon; blocks use braces. Files that contain only PHP should **omit the closing tag** (?>) so stray whitespace can't leak into the response. `declare(strict_types=1);` must be the very first statement to take effect.

NOTES

- `<?=` is short for `<?php echo` and is always enabled
- Short open tag `<?` depends on ini — don't use it
- The last statement before `?>` doesn't need a semicolon
- Whitespace and newlines are not significant (unlike Python)
- Case-insensitive: keywords, function and class names; case-sensitive: variables

A TYPICAL SCRIPT FILE

```php
<?php
declare(strict_types=1);          // must be first

namespace App;                    // optional

use App\Models\User;              // imports

require __DIR__ . '/vendor/autoload.php';

function greet(string $name): string
{
    return "Hello, {$name}!";
}

echo greet('Ana'), PHP_EOL;
// no closing ?> in pure-PHP files
```

CODE EXPLAINED

- → `declare(strict_types=1)` — scalar types are not coerced in calls made from this file
- → `namespace` must come before any other code except `declare`
- → `use` imports a class name into this file
- → `__DIR__` — folder of the current file; keeps paths stable
- → Omitting `?>` avoids "headers already sent" bugs

TAG FORMS

TAG	MEANING
`<?php ... ?>`	Standard PHP block
`<?= $x ?>`	Echo shortcut
`<? ... ?>`	Short tag — avoid
`?>`	Ends PHP mode; implies `;`
`(no ?>)`	Recommended for PHP-only files

TEMPLATE MIXING

CODE	OUTPUT
`<h1><?= $title ?></h1>`	Heading with value
`<?php if ($ok): ?>`	Start a block
`<p>Yes</p>`	Sent only if $ok
`<?php endif; ?>`	End the block
`<?php foreach ($xs as $x): ?>`	Loop over HTML

CASE RULES

ITEM	CASE-SENSITIVE?
`$variables`	Yes
`constants`	Yes
`functions`	No (`STRLEN()` works)
`classes / methods`	No
`keywords`	No (`IF`, `ECHO`)

⚠ COMMON MISTAKES

- Whitespace before `<?php` or after `?>` — breaks `header()` and sessions
- Putting `declare(strict_types=1)` after other code — fatal error
- Relative `require 'x.php'` paths — they resolve from the working directory

✓ TIPS & BEST PRACTICES

- One class per file, no closing tag
- Start every file with `declare(strict_types=1);`
- Use `<?=` and alternative syntax in templates

QUICK SUMMARY

- Code lives between `<?php` and `?>`
- Outside tags = raw output
- Omit `?>` in PHP-only files
- `strict_types` goes first

echo, print & Output

SENDING OUTPUT

`echo` and `print` are **language constructs**, not functions, so parentheses are optional. `echo` takes one or more comma-separated arguments and returns nothing; `print` takes exactly one and returns 1. In a web request the output becomes the response body; on the CLI it goes to STDOUT. **Output buffering** (`ob_start()`) captures output into a string instead of sending it.

NOTES

- `echo` is slightly faster and far more common
- Comma-separated `echo a, b` avoids building a joined string
- `PHP_EOL` = newline for the current OS
- Use `"\n"` for web output; HTML ignores newlines anyway
- `printf()` / `sprintf()` format output (see Sheet 17)

PRINTING VALUES

```php
<?php
$name = 'Ana';
$n = 3;

echo 'Hello';                    // Hello
echo 'Hi ', $name, PHP_EOL;      // Hi Ana + newline
echo "You have $n messages\n";   // interpolation
print "Done\n";                  // print returns 1

echo 'Sum: ' . (2 + 3) . "\n";   // parens around math

// Capture output into a string
ob_start();
echo '<p>Buffered</p>';
$html = ob_get_clean();          // '<p>Buffered</p>'

fwrite(STDERR, "warning\n");     // CLI error stream
```

CODE EXPLAINED

→ Commas pass several arguments to `echo`

→ Double quotes interpolate `$n`; single quotes don't

→ `(2 + 3)` — parens make the order obvious; PHP 8 ranks `+` above `.`

→ `ob_get_clean()` returns the buffer and stops buffering

→ `STDERR` — constant stream, CLI only

ECHO VS PRINT

FEATURE	ECHO / PRINT
Arguments	many / one
Return value	none / 1
Use in expression	no / yes
Parentheses	optional / optional
Speed	marginally faster / slower

OUTPUT BUFFERING

FUNCTION	DOES
`ob_start()`	Begin capturing
`ob_get_clean()`	Return + discard buffer
`ob_get_contents()`	Read buffer, keep it
`ob_end_flush()`	Send buffer, stop
`flush()`	Push output to client

NEWLINES

VALUE	USE
`PHP_EOL`	Platform newline (CLI)
`"\n"`	LF (double quotes only)
`'\n'`	Literal backslash-n
` `	Line break in HTML
`nl2br($s)`	Convert \n to

⚠ COMMON MISTAKES

- `echo '\n'` — single quotes print backslash-n literally
- Echoing before `header()` or `session_start()` — headers already sent
- Echoing user input without `htmlspecialchars()` — XSS

✓ TIPS & BEST PRACTICES

- Prefer `echo` with commas for several values
- Use output buffering to render templates into strings
- Escape everything you echo into HTML

QUICK SUMMARY

- `echo a, b` — many args
- `print x` — one arg, returns 1
- `ob_start()` / `ob_get_clean()` capture
- Escape HTML output

Dumping & Inspecting Values

SEEING INSIDE VARIABLES

`echo` can't show arrays or objects. For debugging, `print_r()` prints a readable structure, `var_dump()` adds **types and lengths**, and `var_export()` prints **valid PHP code** you can paste back. Pass `true` as the second argument to `print_r()` or `var_export()` to get a string instead of output. For production, write to the log with `error_log()` rather than the page.

NOTES

- `var_dump()` is the only one that shows `bool` and `null` clearly
- Wrap dumps in `<pre>` in a browser to keep indentation
- With Xdebug, `var_dump()` output is formatted and limited
- `json_encode($x, JSON_PRETTY_PRINT)` is handy for APIs
- Symfony's `dump()` / Laravel's `dd()` are library helpers

COMPARING THE DUMPERS

```php
<?php
$user = ['id' => 7, 'admin' => false, 'tags' => null];

print_r($user);
// Array ( [id] => 7 [admin] => [tags] => )

var_dump($user);
// array(3) { ["id"]=> int(7)
//   ["admin"]=> bool(false) ["tags"]=> NULL }

var_export($user);
// array ( 'id' => 7, 'admin' => false,
//   'tags' => NULL, )

$s = print_r($user, true);       // as a string
error_log('user: ' . $s);        // to the error log

echo '<pre>'; var_dump($user); echo '</pre>';
```

CODE EXPLAINED

- → `print_r` hides `false` and `null` — both print as empty
- → `var_dump` shows the type of every element
- → `var_export` output is executable PHP
- → `true` second arg returns the text
- → `error_log()` writes to the server log, not the page

WHICH DUMPER?

FUNCTION	BEST FOR
`print_r()`	Quick look at arrays
`var_dump()`	Exact types and values
`var_export()`	Generating PHP code
`json_encode()`	API-style output
`debug_zval_dump()`	Value plus refcount

DEBUG HELPERS

FUNCTION	PURPOSE
`debug_print_backtrace()`	Print call stack
`debug_backtrace()`	Call stack as array
`get_debug_type($x)`	Readable type name
`error_log($msg)`	Write to log
`spl_object_id($o)`	Unique object id

RETURN INSTEAD OF PRINT

CALL	RETURNS
`print_r($x, true)`	string
`var_export($x, true)`	string
`var_dump($x)`	null (always prints)
`json_encode($x)`	string|false
`ob_start(); var_dump();`	capture via buffer

⚠ COMMON MISTAKES

- Reading `print_r` output as "empty" when the value was `false`
- Leaving `var_dump()` calls in committed code
- Dumping secrets (passwords, tokens) into logs

✓ TIPS & BEST PRACTICES

- Use `var_dump()` when types matter
- Use Xdebug step debugging instead of scattering dumps
- Search for `var_dump(` before every commit

QUICK SUMMARY

- `print_r` — readable
- `var_dump` — types
- `var_export` — PHP code
- `error_log` — log, not page

Comments & PHPDoc

COMMENTS AND DOC BLOCKS

PHP has single-line comments (`//` and `#`) and block comments (`/* ... */`). A block that starts with `/**` is a **DocBlock**: editors, static analysers and documentation tools read its `@tags`. Since PHP 8, `#[` starts an **attribute**, not a comment, so prefer `//` for single lines. Native type declarations replace many DocBlock tags; keep tags for detail the type system can't express.

NOTES

- Block comments do not nest
- A single-line comment ends at `?>` as well as at a newline
- PHPStan and Psalm read generic types such as `array<int, User>`
- DocBlocks go directly above the element they describe
- Don't duplicate a type that's already in the signature

COMMENT STYLES & DOCBLOCKS

```php
<?php
// Single-line comment
# Also single-line (avoid: looks like #[attributes])

/*
 * Block comment spanning lines
 */

/**
 * Find active users by role.
 *
 * @param list<string> $roles  role names to match
 * @return array<int, User>    keyed by user id
 * @throws InvalidArgumentException if $roles is empty
 */
function findUsers(array $roles, int $limit = 50): array
{
    /** @var User[] $rows */
    $rows = [];
    return $rows;
}
```

CODE EXPLAINED

→ `list<string>` — more precise than the native `array` type

→ `@return array<int, User>` — key and value types

→ `@throws` — documents exceptions callers should handle

→ `$limit` has a native type, so no `@param` is needed

→ Inline `@var` tells tools the type of a local variable

COMMENT SYNTAX

SYNTAX	KIND
`// text`	Single line
`# text`	Single line (shell style)
`/* text */`	Block
`/** text */`	DocBlock
`#[Attr]`	Attribute — not a comment

COMMON TAGS

TAG	DOCUMENTS
`@param T $x`	Parameter type + note
`@return T`	Return type
`@throws E`	Possible exception
`@var T`	Variable/property type
`@deprecated`	Don't use
`@see / @link`	Cross-reference

TYPE SYNTAX IN DOCS

TYPE	MEANING
`int[]`	Array of ints
`list<int>`	0-indexed sequential list
`array<string, int>`	Keyed map
`?User`	User or null
`class-string<T>`	Class name string
`non-empty-string`	Static-analysis type

⚠ COMMON MISTAKES

- Using `#` comments at line start next to attributes — confusing to read
- DocBlock types that contradict the native types
- Commenting *what* the code does instead of *why*

✓ TIPS & BEST PRACTICES

- Let native types carry the basics; DocBlocks add precision
- Run PHPStan so DocBlock types are checked
- Delete commented-out code — version control remembers it

QUICK SUMMARY

- `//` and `/* */` for notes
- `/** */` DocBlocks for tools
- `@param` `@return` `@throws` `@var`
- `#[...]` is an attribute

Variables & Assignment

$name · by value · by reference =& · variable variables $$ · isset() · unset()

VARIABLES

Every variable starts with $ and is created the first time you assign to it; there is no declaration keyword. Variables are **dynamically typed**: the value has a type, the variable doesn't. Assignment **copies the value** — arrays included (copy-on-write keeps this cheap). Objects are handles, so copying a variable that holds an object still points at the same object. =& makes two names share one value.

NOTES

- Names: letter or underscore first, then letters, digits, underscores
- Names are case-sensitive: `$user` ≠ `$User`
- Reading an undefined variable gives a **Warning** and `null`
- `$this` is reserved inside classes
- Convention: `$camelCase` (PSR) or `$snake_case` — be consistent

ASSIGN, COPY, REFERENCE

```php
<?php
$name = 'Ana';
$count = 3;
$a = $b = 0;                // chained assignment

$list = [1, 2];
$copy = $list;              // independent copy
$copy[] = 3;                // $list is still [1, 2]

$ref = &$list;              // alias, not a copy
$ref[] = 99;                // $list is now [1, 2, 99]

isset($name);               // true (exists, not null)
unset($name);               // remove the variable
isset($name);               // false

$field = 'count';
echo $$field;               // 3 (variable variable)
```

CODE EXPLAINED

→ Chained `$a = $b = 0` assigns right to left

→ Arrays copy on assignment; changes don't leak back

→ `&$list` — both names now refer to the same value

→ `isset()` is false for undefined **and** for `null`

→ `$$field` reads the variable named by `$field` — avoid in real code

ASSIGNMENT FORMS

CODE	EFFECT
`$a = 5`	Assign by value
`$a = &$b`	Assign by reference
`$a = $b = 0`	Chained
`[$a, $b] = [1, 2]`	Destructure (Sheet 58)
`$a ??= 'x'`	Assign if null/unset
`unset($a)`	Destroy variable

EXISTENCE CHECKS

CHECK	TRUE WHEN
`isset($x)`	Defined and not null
`empty($x)`	Undefined or falsy
`is_null($x)`	Value is null (warns if undefined)
`$x ?? 'd'`	Use 'd' if unset/null
`array_key_exists`	Key exists, even if null

COPY SEMANTICS

TYPE	ON ASSIGNMENT
`int, float, bool`	Copied
`string`	Copied (copy-on-write)
`array`	Copied (copy-on-write)
`object`	Handle copied — same object
`resource`	Handle copied

⚠ COMMON MISTAKES

- Expecting `$b = $arr` to share changes — arrays are copied
- Expecting `$b = $obj` to copy an object — use `clone`
- Using `isset()` to test a key whose value may be `null`

✓ TIPS & BEST PRACTICES

- Initialise variables before use; enable `E_ALL` to catch typos
- Avoid references unless you need in-place edits
- Prefer clear names over variable variables

QUICK SUMMARY

- `$name = value` creates it
- Arrays copy, objects share
- `=&` aliases
- `isset` / `unset` / `??`

Constants & Magic Constants

const · define() · defined() · __LINE__ · __FILE__ · __DIR__ · PHP_INT_MAX · PHP_VERSION

VALUES THAT DON'T CHANGE

A constant holds a value that can't be reassigned. Use `const` at the top level or inside a class; use `define()` when the name or value is computed at runtime or the code is inside a function or `if`. Constants have **no $** and are global (or class-scoped). PHP also has **magic constants** such as `__DIR__` and `__LINE__` whose value depends on where they appear.

NOTES

- Constant values: scalars, arrays, enums, and objects (via `new` in const expressions, 8.1+)
- Convention: `UPPER_SNAKE_CASE`
- `const` names are resolved at compile time and respect namespaces
- Undefined constant → `Error` in PHP 8 (no more string fallback)
- Class constants: `self::MAX`, `Order::STATUS_NEW` (Sheet 67)

DEFINING & USING CONSTANTS

```php
<?php
const APP_NAME = 'Shop';
const RATES = ['std' => 0.2, 'low' => 0.05];

if (!defined('DEBUG')) {
    define('DEBUG', getenv('APP_DEBUG') === '1');
}

echo APP_NAME;                  // Shop
echo RATES['low'];              // 0.05
echo constant('APP_NAME');      // dynamic lookup

echo __FILE__;   // /app/src/config.php
echo __DIR__;    // /app/src
echo __LINE__;   // 15
echo PHP_INT_MAX;               // 9223372036854775807
```

CODE EXPLAINED

- → `const` at file level — simple, compile-time
- → `define()` inside `if` — runtime, conditional
- → `defined('NAME')` — avoids redefinition errors
- → `constant()` reads a constant by string name
- → `__DIR__` is the directory of **this** file

CONST VS DEFINE()

ASPECT	CONST / DEFINE()
Where	top level, class / anywhere
Inside if / function	no / yes
Computed name	no / yes
Namespaced	yes / only if name includes it
Class constants	yes / no

MAGIC CONSTANTS

CONSTANT	VALUE
__LINE__	Current line number
__FILE__	Full path of this file
__DIR__	Directory of this file
__FUNCTION__	Function name
__CLASS__	Class name with namespace
__METHOD__	Class::method
__NAMESPACE__	Current namespace

BUILT-IN CONSTANTS

CONSTANT	EXAMPLE
PHP_VERSION	"8.5.0"
PHP_EOL	"\n" on Linux
PHP_INT_MAX	92233720368547758
PHP_INT_SIZE	8 (bytes)
PHP_FLOAT_EPSILON	2.2204460492503E-
DIRECTORY_SEPARATOR	/ or \
E_ALL	All error levels

⚠ COMMON MISTAKES

- Writing `$APP_NAME` — constants have no `$`
- Calling `define()` twice for one name — Warning, value unchanged
- Using `const` inside an `if` block — syntax error

✓ TIPS & BEST PRACTICES

- Prefer class constants or enums over global constants
- Use `__DIR__` for all include paths
- Keep secrets in environment variables, not constants in code

QUICK SUMMARY

- `const X = 1;` at top level
- `define()` at runtime
- No `$`, can't reassign
- Magic constants: `__DIR__`, `__LINE__`

Integers

WHOLE NUMBERS

`int` is a signed whole number, 64 bits on all modern platforms (`PHP_INT_MIN` to `PHP_INT_MAX`). Write literals in decimal, hex (`0x`), octal (`0o` or a leading `0`) or binary (`0b`), with `_` separators for readability. When an arithmetic result goes past `PHP_INT_MAX` PHP **silently switches to `float`**, losing precision. `/` always returns a float if the result isn't whole; use `intdiv()` for integer division.

NOTES

- `_` separators can't start, end or double up: `1__000` is invalid
- `0o17` (8.1+) is clearer than legacy `017`
- `%` works on ints; `fmod()` for floats
- Use the `bcmath` or `gmp` extension for arbitrary precision
- `is_int()` checks type; `ctype_digit()` checks a digit string

LITERALS, LIMITS & CONVERSION

```php
<?php
$dec = 1_000_000;           // 1000000
$hex = 0xFF;                // 255
$oct = 0o17;                // 15
$bin = 0b1010;              // 10

var_dump(PHP_INT_MAX + 1);  // float(9.2233720368548E+18)

var_dump(7 / 2);            // float(3.5)
var_dump(8 / 2);            // int(4)
var_dump(intdiv(7, 2));     // int(3)
var_dump(7 % 3, -7 % 3);    // int(1), int(-1)

(int) '42abc';              // 42 (no error)
intval('0x1A', 16);         // 26
intval('101', 2);           // 5
(int) 3.99;                 // 3 (truncates)
```

CODE EXPLAINED

→ Literal prefixes pick the base; the value is the same `int`

→ Overflow turns the result into a `float`

→ `/` gives `int` only when the division is exact

→ `%` takes the sign of the left operand

→ `(int)` truncates toward zero and ignores trailing junk

LITERAL SYNTAX

LITERAL	VALUE
`42`	Decimal
`-17`	Negative
`0x2A`	Hex = 42
`0o52 / 052`	Octal = 42
`0b101010`	Binary = 42
`1_000`	1000

INTEGER FUNCTIONS

FUNCTION	R
`intdiv(7, 2)`	3
`abs(-5)`	5
`intval('12', 10)`	1 b
`is_int($x)`	T c
`filter_var($s, FILTER_VALIDATE_INT)`	i f
`PHP_INT_MAX`	L i

BASE CONVERSION

FUNCTION	EXAMPLE
`decbin(10)`	'1010'
`bindec('1010')`	10
`dechex(255)`	'ff'
`hexdec('ff')`	255
`decoct(8)`	'10'
`base_convert('ff', 16, 2)`	'11111111'

⚠ COMMON MISTAKES

- Validating input with `(int)` — `'12abc'` becomes `12` silently
- Assuming integer math can't overflow — it becomes a float
- Legacy octal `0755` read as decimal by humans

✓ TIPS & BEST PRACTICES

- Validate with `filter_var(..., FILTER_VALIDATE_INT)`
- Use `intdiv()` when you need whole-number division
- Use `0o` for octal literals in new code

QUICK SUMMARY

- 64-bit signed
- `0x` `0o` `0b` and `_`
- Overflow → float
- `intdiv()` / `%` / `intval()`

Floats

FLOATING-POINT NUMBERS

`float` is an IEEE-754 double: about 15–17 significant digits. Many decimal fractions (like `0.1`) have no exact binary form, so **never compare floats with ==** — compare the difference against a tolerance. Division by zero throws `DivisionByZeroError`; other invalid results produce `INF` or `NAN`. For money, store integer cents or use **BCMath** (string-based decimals).

NOTES

- `precision` ini controls echo output; `serialize_precision=-1` gives shortest round-trip
- `NAN` is not equal to anything, including itself
- `round()` rounds half away from zero by default
- PHP 8.4 added `RoundingMode` enum for `round()`
- `number_format()` formats for display (Sheet 17)

PRECISION & SPECIAL VALUES

```php
<?php
$a = 1.5;  $b = 1.5e3;  $c = 7E-10;   // 1500.0, 0.0000000007

var_dump(0.1 + 0.2 == 0.3);           // bool(false)
var_dump(abs(0.1 + 0.2 - 0.3) < PHP_FLOAT_EPSILON); // true

echo round(2.5);          // 3
echo round(-2.5);         // -3
echo round(3.14159, 2);   // 3.14
echo floor(3.7), ' ', ceil(3.2);  // 3 4

var_dump(log(0));         // float(-INF)
var_dump(sqrt(-1));       // float(NAN)
is_nan(sqrt(-1));         // true
is_finite(1e308 * 10);    // false

echo bcadd('0.1', '0.2', 2);  // "0.30"
```

CODE EXPLAINED

- → `e` notation works for large and small values
- → `0.1 + 0.2` is `0.30000000000000004`
- → `PHP_FLOAT_EPSILON` is a tolerance for values near 1
- → `floor()` / `ceil()` return floats
- → `bcadd()` adds decimal strings exactly (scale 2)

ROUNDING

FUNCTION	3.456 / -3.5
`round($x)`	3 / -4
`round($x, 2)`	3.46 / -3.5
`floor($x)`	3 / -4
`ceil($x)`	4 / -3
`(int) $x`	3 / -3
`number_format($x, 1)`	'3.5' / '-3.5'

SPECIAL VALUES

VALUE	TEST
`INF / -INF`	is_infinite()
`NAN`	is_nan()
`finite number`	is_finite()
`PHP_FLOAT_MAX`	Largest float
`PHP_FLOAT_EPSILON`	Smallest 1+x≠1
`PHP_FLOAT_DIG`	Reliable digits (15)

BCMATH

FUNCTION	PURPOSE
`bcadd($a, $b, 2)`	Add
`bcsub($a, $b, 2)`	Subtract
`bcmul($a, $b, 2)`	Multiply
`bcdiv($a, $b, 2)`	Divide
`bccomp($a, $b, 2)`	Compare (-1/0/1)
`new BcMath\Number('1.5')`	Object API (8.4)

⚠ COMMON MISTAKES

- Comparing floats with `==` or `===`
- Storing money as floats — rounding drift adds up
- Testing `$x == NAN` — always false; use `is_nan()`

✓ TIPS & BEST PRACTICES

- Store currency as integer cents or BCMath strings
- Compare with a tolerance suited to your data
- Round only for display, not mid-calculation

QUICK SUMMARY

- IEEE-754 double
- Never `==` floats
- `INF` / `NAN` checks
- BCMath for exact decimals

Booleans & Truthiness

TRUE, FALSE AND TRUTHINESS

`bool` has two values, `true` and `false` (case-insensitive). Any value can be used in a condition; PHP converts it to `bool`. A short list of values is **falsy** — everything else is truthy. The surprise is that the string `"0"` is falsy while `"0.0"`, `" "` and `"false"` are truthy. `empty($x)` is `!isset($x) || !$x` and never warns about undefined variables.

NOTES

- Empty arrays are falsy; all objects are truthy
- `(bool)` / `boolval()` convert explicitly
- `true` echoes as `1`; `false` echoes as an empty string
- `filter_var($s, FILTER_VALIDATE_BOOL)` parses "yes", "on", "1"
- Comparison operators always return a `bool`

TRUTHINESS IN PRACTICE

```php
<?php
$vals = [0, 0.0, '', '0', [], null, '0.0', ' ', 'false', [0]];
foreach ($vals as $v) {
    echo json_encode($v), ' => ', $v ? 'truthy' : 'falsy', "\n";
}
// 0 falsy, 0.0 falsy, "" falsy, "0" falsy, [] falsy,
// null falsy, "0.0" truthy, " " truthy, "false" truthy,
// [0] truthy

echo true;     // 1
echo false;    // (nothing)

var_dump(empty($missing));                    // true, no warning
var_dump(filter_var('off', FILTER_VALIDATE_BOOL)); // false
var_dump((bool) 'false');                     // true
```

CODE EXPLAINED

→ Eight falsy values: `false 0 0.0 -0.0 "" "0" [] null`

→ A non-empty array is truthy even if it holds only `0`

→ `echo false` prints nothing — use `var_dump` to see it

→ `empty()` is safe on undefined variables

→ Casting the string `'false'` gives `true`

FALSY VALUES

VALUE	TYPE
`false`	bool
`0` / `-0`	int
`0.0` / `-0.0`	float
`""`	empty string
`"0"`	string zero
`[]`	empty array
`null`	null

CONVERSIONS

EXPRESSION	RESULT
`(bool) 'abc'`	true
`(bool) '0'`	false
`(int) true`	1
`(string) false`	""
`boolval([])`	false
`true + true`	2

FILTER_VALIDATE_BOOL

INPUT	RESULT
`'1', 'true', 'on', 'yes'`	true
`'0', 'false', 'off', 'no', ''`	false
`'maybe'`	false
`with FILTER_NULL_ON_FAILURE`	null for 'maybe'

⚠ COMMON MISTAKES

- `if ($count)` where `0` is a valid result — test `!== null` instead
- `if (strpos($s, 'a'))` — position `0` is falsy (use `str_contains`)
- Parsing "false" from config with `(bool)`

✓ TIPS & BEST PRACTICES

- Use strict comparisons when `0` or `""` is meaningful
- Use `FILTER_VALIDATE_BOOL` for user/config input
- Use `var_dump` when debugging booleans

QUICK SUMMARY

- `true` / `false`
- `"0"` and `[]` are falsy
- `empty()` = not set or falsy
- Validate booleans with `filter_var`

Null

THE ABSENCE OF A VALUE

`null` is the only value of type `null`. A variable is `null` when assigned `null`, after `unset()`, or when read without being defined (with a Warning). Functions without a `return` value return `null`. Type declarations reject `null` unless you allow it with `?Type` or `Type|null`. The `??` and `?->` operators exist to make null handling short and safe (Sheet 27).

NOTES

- `null`, `NULL` and `Null` are the same
- `isset($x)` is false when `$x` is `null`
- A typed property without a default is **uninitialized**, not `null`
- `null == false`, `null == 0`, `null == ''` are all true; use `===`
- In arrays, a key can exist with a `null` value

WORKING WITH NULL

```php
<?php
function findUser(int $id): ?User      // may return null
{
    return $id === 1 ? new User('Ana') : null;
}

$user = findUser(2);
if ($user === null) {
    echo "not found\n";
}

$name = $user?->name ?? 'guest';       // 'guest'

class Order {
    public ?string $note = null;       // nullable, default null
    public int $total;                 // uninitialized!
}
$o = new Order();
var_dump($o->note);                    // NULL
// echo $o->total;   // Error: must not be accessed
                     //        before initialization
```

CODE EXPLAINED

→ `?User` return type — `User` or `null`

→ Compare with `=== null` (or `is_null()`)

→ `?->` stops at null; `??` supplies a default

→ `?string $note = null` — explicitly nullable

→ Typed property with no default throws on read until set

NULL CHECKS

CODE	NOTES
`$x === null`	Clearest, fastest
`is_null($x)`	Same, as a function
`isset($x)`	false if null or undefined
`$x ?? $d`	Default when null/unset
`$a?->b`	null if $a is null
`array_key_exists($k, $a)`	Key with null value

LOOSE EQUALITY

EXPRESSION	RESULT
`null == false`	true
`null == 0`	true
`null == ''`	true
`null == []`	true
`null === false`	false
`null <=> 0`	0

NULLABLE TYPES

SYNTAX	MEANING
`?int`	int or null
`int|null`	same (union form)
`?int $x = null`	Optional, nullable param
`mixed`	Includes null already
`null (8.2)`	Standalone null type
`int $x = null`	Deprecated implicit nullable (8.4)

⚠ COMMON MISTAKES

- `int $x = null` without `?` — implicit nullable is deprecated in 8.4
- Using `== null` and matching `0` or `''`
- Reading an uninitialized typed property

✓ TIPS & BEST PRACTICES

- Return `null` for "not found", throw for real errors
- Give nullable properties an explicit `= null`
- Reach for `??` and `?->` before nested `if` checks

QUICK SUMMARY

- `null` = no value
- `?Type` allows null
- Compare with `=== null`
- Uninitialized ≠ null

Type Juggling & Casting

(int) · (string) · (array) · settype() · numeric strings · PHP 8 string-number comparison

AUTOMATIC AND EXPLICIT CONVERSION

PHP converts types automatically when an operator or function expects a different type — this is **type juggling**. `'5' + 3` is 8; `'5' . 3` is `'53'`. Casting with `(int)`, `(float)`, `(string)`, `(bool)`, `(array)` or `(object)` converts explicitly. PHP 8 made comparisons between numbers and **non-numeric** strings compare as strings, so `0 == 'abc'` is finally `false`.

NOTES

- Arithmetic on a non-numeric string throws `TypeError` in PHP 8
- Leading-numeric `'12abc' + 1` works with a Warning
- `declare(strict_types=1)` stops juggling for scalar parameters
- `(array)` on an object exposes its properties
- `settype($x, 'int')` converts the variable in place

JUGGLING AND CASTS

```php
<?php
var_dump('5' + 3);        // int(8)
var_dump('5' . 3);        // string(2) "53"
var_dump('1.5' * 2);      // float(3)
var_dump('12abc' + 1);    // int(13) + Warning
// 'abc' + 1;             // TypeError (PHP 8)

var_dump(0 == 'abc');     // false in PHP 8 (true in 7)
var_dump('1' == '01');    // true  (numeric strings)
var_dump('10' == '1e1');  // true
var_dump(100 == '1e2');   // true

var_dump((int) '0x1A');   // int(0)
var_dump((string) 1.0);   // string(1) "1"
var_dump((array) 'hi');   // array(1) { [0]=> "hi" }
var_dump((bool) '0');     // bool(false)
```

CODE EXPLAINED

→ `+` converts strings to numbers; `.` converts numbers to strings

→ PHP 8 throws instead of treating `'abc'` as `0`

→ Two numeric strings compare as numbers

→ Casts don't parse hex strings

→ Casting a scalar to array wraps it

CAST OPERATORS

CAST	SAME AS
`(int)`	intval()
`(float)`	floatval()
`(string)`	strval()
`(bool)`	boolval()
`(array)`	wrap scalar / object props
`(object)`	stdClass from array

TO INT

VALUE	(INT)
`'42'`	42
`' 42'`	42
`'4.9'`	4
`'abc'`	0
`true`	1
`null`	0
`[1, 2]`	1

PHP 8 COMPARISON

EXPRESSION	7.X / 8.X
`0 == 'abc'`	true / false
`0 == ''`	true / false
`'1' == '01'`	true / true
`'abc' == 0`	true / false
`null == 0`	true / true
`42 == '42abc'`	true / false

⚠ COMMON MISTAKES

- Relying on PHP 7 comparison rules after upgrading
- Casting user input with `(int)` and calling it validation
- Concatenating with `+` (JavaScript habit)

✓ TIPS & BEST PRACTICES

- Use `===` and `!==` by default
- Turn on `strict_types` in every file
- Validate input with `filter_var()` before casting

QUICK SUMMARY

- Juggling = automatic conversion
- Casts: `(int)` `(string)` …
- PHP 8 fixed number/string `==`
- Prefer strict comparison

Type Checking

FINDING OUT WHAT YOU HAVE

Use the `is_*()` family to test a value's type at runtime, `instanceof` for classes and interfaces, and `get_debug_type()` (PHP 8) for a readable name in error messages. **Type declarations** on parameters, returns and properties make the engine check for you and throw `TypeError` on a mismatch — prefer them over manual checks wherever you can.

NOTES

- `gettype()` returns legacy names like `"double"` and `"NULL"`
- `get_debug_type()` returns `"float"`, `"null"` and class names
- `is_numeric()` accepts numeric strings like `"1e3"`
- `is_callable()` and `is_iterable()` test capabilities
- `instanceof` accepts a class name string on the right

RUNTIME TYPE TESTS

```php
<?php
$values = [42, 4.2, '42', true, null, [1], new DateTime()];

foreach ($values as $v) {
    echo str_pad(get_debug_type($v), 10), gettype($v), "\n";
}
// int       integer
// float     double
// string    string
// bool      boolean
// null      NULL
// array     array
// DateTime  object

is_numeric('1e3');               // true
is_numeric('0x1A');              // false
$d = new DateTime();
$d instanceof DateTimeInterface; // true

function area(float $r): float { return M_PI * $r ** 2; }
// area('big');  // TypeError: must be of type float
```

CODE EXPLAINED

- → `get_debug_type()` gives the name you'd write in a type
- → `gettype()` uses old names — avoid for new code
- → `is_numeric()` is true for int/float and numeric strings
- → `instanceof` works with parent classes and interfaces
- → Typed parameters throw `TypeError` automatically

IS_* FUNCTIONS

FUNCTION	TRUE FOR
`is_int()`	int
`is_float()`	float
`is_string()`	string
`is_bool()`	bool
`is_array()`	array
`is_object()`	any object
`is_null()`	null

CAPABILITY TESTS

FUNCTION	TRUE WHEN
`is_numeric()`	Number or numeric string
`is_scalar()`	int, float, string, bool
`is_iterable()`	array or Traversable
`is_countable()`	array or Countable
`is_callable()`	Can be called
`is_resource()`	Open resource

TYPE NAME FUNCTIONS

FUNCTION	DATETIME / 1.0
`get_debug_type()`	DateTime / float
`gettype()`	object / double
`get_class()`	DateTime / TypeError
`$o::class`	DateTime / TypeError
`var_dump()`	object(DateTime) / float(1)

⚠ COMMON MISTAKES

- Checking `gettype($x) == 'float'` — it returns `"double"`
- Using `is_numeric()` to validate an integer ID — it accepts `"1.5"`
- Hand-written type checks where a type declaration would do

✓ TIPS & BEST PRACTICES

- Declare types; check manually only at system boundaries
- Use `get_debug_type()` in exception messages
- Check interfaces with `instanceof`, not class names

QUICK SUMMARY

- `is_*()` for scalars
- `instanceof` for objects
- `get_debug_type()` for messages
- Type declarations throw `TypeError`

String Basics & Interpolation

'single' vs "double" · "$var" · "{$obj->prop}" · escape sequences · heredoc · nowdoc · . concatenation

STRINGS

A PHP string is a sequence of **bytes**, not characters — `strlen()` counts bytes (Sheet 21 covers UTF-8). **Single quotes** are literal: only `\'` and `\\` are escapes. **Double quotes** expand variables and escape sequences like `\n`. **Heredoc** (`<<<EOT`) behaves like double quotes over many lines; **nowdoc** (`<<<'EOT'`) behaves like single quotes. Join strings with the `.` operator.

NOTES

- Strings are mutable values but copied on assignment
- Wrap complex expressions in braces: `"{$user->name}"`
- Array keys in strings: `"$row[id]"` or `"{$row['id']}"`
- Closing heredoc marker may be indented (7.3+); that indentation is stripped
- `"${var}"` interpolation is deprecated in 8.2 — use `{$var}`

QUOTES, INTERPOLATION & HEREDOC

```php
<?php
$name = 'Ana';
$user = ['id' => 7];
$cart = new stdClass();
$cart->total = 9.5;

echo 'Hi $name\n';          // Hi $name\n (literal)
echo "Hi $name\n";          // Hi Ana + newline
echo "ID: {$user['id']}";   // ID: 7
echo "Total: {$cart->total}";
echo 'Hi ' . $name . '!';   // concatenation

$html = <<<HTML
    <p>Hello, {$name}</p>
    <p>Items: {$user['id']}</p>
    HTML;                    // indent removed

$raw = <<<'SQL'
    SELECT * FROM t WHERE x = '$notVar'
    SQL;
```

CODE EXPLAINED

→ Single quotes print `$name` and `\n` as typed

→ Braces make array/property access unambiguous

→ `.` joins; `.=` appends (`$s .= 'x'`)

→ Heredoc interpolates; closing marker sets the indent

→ Nowdoc never interpolates — good for SQL and templates

ESCAPE SEQUENCES

ESCAPE	MEANING (DOUBLE QUOTES)
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\$`	Literal dollar
`\"`	Double quote
`\u{1F600}`	Unicode code point
`\x41`	Hex byte (A)

QUOTE STYLES

STYLE	INTERPOLATES?
`'single'`	No
`"double"`	Yes
`<<<EOT heredoc`	Yes
`<<<'EOT' nowdoc`	No
`` `backticks` ``	Runs shell command!

INTERPOLATION FORMS

SYNTAX	USE
`"$var"`	Simple variable
`"$arr[key]"`	Unquoted key, simple
`"{$arr['key']}"`	Quoted key
`"{$obj->a->b}"`	Chained props
`"{$obj->m()}"`	Method call
`"${var}"`	Deprecated (8.2)

⚠ COMMON MISTAKES

- `'\n'` in single quotes — prints backslash-n
- Using `+` to join strings — that's numeric addition
- Backticks as quotes — they execute a shell command

✓ TIPS & BEST PRACTICES

- Use single quotes when nothing needs interpolating
- Always use `{$...}` braces for anything beyond a plain variable
- Use nowdoc for SQL or code snippets

QUICK SUMMARY

- `'…'` literal, `"…"` interpolates
- `.` concatenates
- Heredoc / nowdoc for blocks
- Strings are bytes

String Formatting

FORMATTED OUTPUT

`sprintf()` builds a string from a **format** with placeholders; `printf()` prints it directly. A placeholder is `%` + optional flags, width and precision + a type letter: `%05.2f` means float, 2 decimals, padded with zeros to width 5. `number_format()` adds thousands separators for display, `str_pad()` pads to a fixed width, and `vsprintf()` takes the values as an array.

NOTES

- `%%` prints a literal percent sign
- Argument numbering: `%1$s` reuses the first argument
- `%'*10s` pads with `*` instead of spaces
- `-` flag left-aligns; `+` forces a sign
- For locales and currencies use `NumberFormatter` (intl)

SPRINTF, PRINTF & FRIENDS

```php
<?php
$item = 'Coffee'; $qty = 3; $price = 4.5;

printf("%s x%d = %.2f\n", $item, $qty, $qty * $price);
// Coffee x3 = 13.50

echo sprintf('%05.1f', 3.14159);     // 003.1
echo sprintf('%-8s|', 'ab');         // 'ab      |'
echo sprintf('%08.3f', -1.5);        // -001.500
echo sprintf('%x %o %b', 255, 8, 5); // ff 10 101
echo sprintf('%1$s-%1$s', 'go');     // go-go
echo sprintf('%u%%', 50);            // 50%

echo number_format(1234567.891);          // 1,234,568
echo number_format(1234567.891, 2);       // 1,234,567.89
echo number_format(1234.5, 2, ',', '.'); // 1.234,50

echo str_pad('7', 3, '0', STR_PAD_LEFT); // 007
echo vsprintf('%s is %d', ['Bo', 30]);    // Bo is 30
```

CODE EXPLAINED

- → `%.2f` — two decimal places, rounded
- → Width counts the whole output including sign and dot
- → `%-8s` — left-aligned in 8 columns
- → `%1$s` — positional argument, reusable
- → `number_format()` returns a string for display only

TYPE SPECIFIERS

SPEC	OUTPUT
`%s`	String
`%d`	Signed integer
`%u`	Unsigned integer
`%f / %F`	Float (locale / non-locale)
`%e`	Scientific 1.2e+3
`%x / %X`	Hex lower / upper
`%b / %o`	Binary / octal

FLAGS & WIDTH

FORMAT	RESULT FOR 42
`%5d`	' 42'
`%-5d|`	'42 |'
`%05d`	'00042'
`%+d`	'+42'
`%'#6d`	'####42'
`%.1f`	'42.0'

RELATED FUNCTIONS

FUNCTION	DOES
`printf()`	Print, return length
`sprintf()`	Return string
`vsprintf()`	Args as array
`fprintf($fh, ...)`	Write to stream
`number_format()`	1,234.50
`str_pad()`	Pad to width

⚠ COMMON MISTAKES

- Formatting money with `%f` and floats — round to cents first
- Passing fewer arguments than placeholders — `ArgumentCountError`
- Doing math on `number_format()` output — it's a string with commas

✓ TIPS & BEST PRACTICES

- Keep numbers numeric until the moment you display them
- Use positional `%1$s` in translatable strings
- Use `NumberFormatter` for locale-aware output

QUICK SUMMARY

- `sprintf('%.2f', $x)`
- `%s %d %f %x`
- Flags: `- + 0 'c`
- `number_format()` for display

String Functions — Search

FINDING TEXT

PHP 8 added three readable helpers — `str_contains()`, `str_starts_with()` and `str_ends_with()` — that return a plain `bool`. Use `strpos()` when you need the **position**: it returns the 0-based index or `false`, so always compare with `!== false`. The `i` variants (`stripos`, `stristr`) ignore ASCII case. `strstr()` returns the rest of the string from the match.

NOTES

- All of these are byte-based and case-sensitive unless noted
- `str_contains($s, '')` is always `true`
- Negative offsets search from the end (`strpos($s, 'x', -5)`)
- Use `mb_strpos()` when positions must be in characters
- Use `preg_match()` for patterns (Sheet 102)

SEARCH FUNCTIONS

```php
<?php
$s = 'The quick brown fox';

str_contains($s, 'quick');      // true
str_starts_with($s, 'The');     // true
str_ends_with($s, 'Fox');       // false (case)

strpos($s, 'quick');            // 4
strpos($s, 'The');              // 0  <- falsy!
strpos($s, 'cat');              // false
stripos($s, 'FOX');             // 16
strrpos('a-b-c', '-');          // 3 (last)

if (strpos($s, 'The') !== false) { /* found */ }

strstr('ana@example.com', '@');        // '@example.com'
strstr('ana@example.com', '@', true);  // 'ana'
strrchr('a/b/c.txt', '/');             // '/c.txt'
substr_count('banana', 'an');          // 2
```

CODE EXPLAINED

- → `str_*` helpers return `bool` — no `!== false` dance
- → Position `0` is a real match but falsy
- → `stripos()` — case-insensitive `strpos()`
- → `strstr(..., true)` returns the part **before** the match
- → `substr_count()` doesn't count overlaps

BOOLEAN CHECKS

FUNCTION	TRUE WHEN
`str_contains($h, $n)`	$n anywhere
`str_starts_with($h, $n)`	$h begins with $n
`str_ends_with($h, $n)`	$h ends with $n
`in_array($s, $list)`	Exact match in list
`preg_match($re, $s)`	Pattern matches (1)

POSITION FUNCTIONS

FUNCTION	RETURNS
`strpos()`	First index or false
`stripos()`	Same, ignore case
`strrpos()`	Last index or false
`strripos()`	Last, ignore case
`strspn()`	Length of leading mask match
`strcspn()`	Length before any mask char

EXTRACTING BY MATCH

FUNCTION	RETURNS
`strstr($s, $n)`	From match to end
`strstr($s, $n, true)`	Before match
`stristr()`	strstr, ignore case
`strrchr($s, $c)`	From last char to end
`substr_count()`	Occurrences
`similar_text()`	Similarity score

⚠ COMMON MISTAKES

- `if (strpos($s, 'x'))` — fails when the match is at index 0
- Using `strpos` + `!== false` when `str_contains` says it clearly
- Expecting `stripos` to fold non-ASCII case — use `mb_stripos`

✓ TIPS & BEST PRACTICES

- Reach for `str_contains` / `str_starts_with` first
- Always compare `strpos()` with `!== false`
- Normalise case once with `strtolower()` for repeated checks

QUICK SUMMARY

- `str_contains` / `_starts_with` / `_ends_with`
- `strpos()` → index or false
- `i` = case-insensitive
- `strstr()` returns the tail

String Functions — Transform

CHANGING STRINGS

String functions never modify their argument — they **return a new string**, so assign the result. Case functions (`strtoupper`, `ucfirst`, …) are byte-based and ASCII-only since PHP 8.2; use `mb_*` for other alphabets. `trim()` removes whitespace (or chosen characters) from the ends. `str_replace()` replaces every occurrence and accepts arrays for many replacements at once.

NOTES

- `str_replace()` has a 4th by-reference `$count` argument
- `str_ireplace()` ignores case
- `strtr($s, ['a' => 'b'])` replaces longest keys first, in one pass
- `trim($s, '/')` strips custom characters
- `nl2br()` and `strip_tags()` are for HTML text

TRANSFORM FUNCTIONS

```php
<?php
$s = '  hello world  ';

trim($s);                    // 'hello world'
rtrim('path/', '/');         // 'path'
strtoupper('abc');           // 'ABC'
ucfirst('hello');            // 'Hello'
ucwords('hello big world');  // 'Hello Big World'
lcfirst('Hello');            // 'hello'

str_replace('world', 'PHP', 'hello world');   // 'hello PHP'
str_replace(['a', 'b'], ['1', '2'], 'abc');   // '12c'
str_replace('o', '0', 'foo boo', $n);         // $n = 4

strtr('Hi all', ['Hi' => 'Hello', 'all' => 'you']);
str_repeat('=-', 3);         // '=-=-=-'
strrev('abc');               // 'cba'
wordwrap('The quick brown fox', 10, "\n", true);
```

CODE EXPLAINED

- → `trim()` default set: space, `\t \n \r \0 \x0B`
- → `ucwords()` uppercases the first letter of each word
- → Array search + array replace pair up by position
- → `$n` receives the replacement count
- → `wordwrap(..., true)` also breaks long words

CASE & WHITESPACE

FUNCTION	RESULT
`strtoupper('aB')`	'AB'
`strtolower('aB')`	'ab'
`ucfirst('ab')`	'Ab'
`ucwords('a b')`	'A B'
`trim(' a ')`	'a'
`ltrim / rtrim`	One side only

REPLACE & BUILD

FUNCTION	DOES
`str_replace($a, $b, $s)`	Replace all
`str_ireplace()`	Case-insensitive
`substr_replace($s, $r, 0, 2)`	Replace a range
`strtr($s, $map)`	Multi-replace, one pass
`str_repeat($s, $n)`	Repeat
`strrev($s)`	Reverse (bytes)

HTML-RELATED

FUNCTION	DOES
`nl2br($s)`	\n →
`strip_tags($s)`	Remove tags
`htmlspecialchars($s)`	Escape < > & " '
`html_entity_decode()`	& → &
`wordwrap($s, 40)`	Hard wrap lines

⚠ COMMON MISTAKES

- Calling `trim($s);` without using the result
- Chained `str_replace` calls that re-replace earlier output — use `strtr`
- `strtoupper('é')` — unchanged; use `mb_strtoupper`

✓ TIPS & BEST PRACTICES

- Assign the result: `$s = trim($s);`
- Use `strtr()` with an array for template-style swaps
- Use `mb_*` functions for user-facing text

QUICK SUMMARY

- Functions return new strings
- `trim` / case functions
- `str_replace` / `strtr`
- `mb_*` for non-ASCII

Substrings, Indexing & Splitting

substr() · $str[0] · $str[-1] · explode() · implode() · str_split() · strtok()

PIECES OF STRINGS

`substr($s, $start, $length)` extracts part of a string; negative values count from the end. You can read a single byte with `$s[0]` or `$s[-1]`. `explode()` splits on a delimiter into an array and `implode()` joins an array back into a string. `str_split()` cuts into fixed-size chunks and `strtok()` walks tokens one at a time.

NOTES

- `substr()` returns `''` (not `false`) when out of range in PHP 8
- Reading `$s[99]` past the end gives a Warning
- `explode()` with a limit keeps the rest in the last element
- `implode()` argument order: glue first, then array
- Use `mb_substr()` / `mb_str_split()` for UTF-8 text

SLICING & SPLITTING

```php
<?php
$s = 'Hello, World';

substr($s, 0, 5);        // 'Hello'
substr($s, 7);           // 'World'
substr($s, -5, 3);       // 'Wor'
$s[0];                   // 'H'
$s[-1];                  // 'd'

$parts = explode(',', 'a,b,,c');      // ['a','b','','c']
explode(',', 'a,b,c', 2);             // ['a', 'b,c']
explode(',', 'a,b,c', -1);            // ['a', 'b']
implode(' | ', ['x', 'y', 'z']);      // 'x | y | z'

str_split('abcdef', 2);               // ['ab','cd','ef']

$tok = strtok('one two  three', ' ');
while ($tok !== false) {
    echo $tok, "\n";                  // one, two, three
    $tok = strtok(' ');
}
```

CODE EXPLAINED

→ `substr($s, start, length)` — length is optional

→ Negative start counts back from the end

→ `explode` keeps empty pieces (`''` between `,,`)

→ Negative limit drops that many pieces from the end

→ `strtok()` skips repeated delimiters

SUBSTR() PATTERNS

CALL	'ABCDEF'
`substr($s, 2)`	'cdef'
`substr($s, 0, 3)`	'abc'
`substr($s, -2)`	'ef'
`substr($s, 1, -1)`	'bcde'
`substr($s, 10)`	''
`$s[2]`	'c'

SPLIT FUNCTIONS

FUNCTION	SPLITS BY
`explode($d, $s)`	Delimiter string
`str_split($s, $n)`	Fixed byte length
`mb_str_split($s)`	Characters
`preg_split($re, $s)`	Regex
`str_getcsv($s)`	CSV fields
`strtok($s, $chars)`	Token stream

JOIN & COUNT

FUNCTION	DOES
`implode($glue, $arr)`	Join pieces
`join()`	Alias of implode
`strlen($s)`	Byte length
`mb_strlen($s)`	Character length
`str_word_count($s)`	Word count
`count_chars($s, 3)`	Unique bytes

⚠ COMMON MISTAKES

- Using `$s[0]` on UTF-8 text — returns one byte of a multibyte char
- Forgetting that `explode` on `''` returns `['']`, not `[]`
- Passing `implode($arr, ',')` (old order) — `TypeError` in PHP 8

✓ TIPS & BEST PRACTICES

- Use `array_filter(explode(...))` to drop empty parts
- Use `explode` with a limit for `key=value` pairs
- Use `mb_substr()` for anything a user typed

QUICK SUMMARY

- `substr($s, start, len)`
- `$s[0]` / `$s[-1]`
- `explode` ↔ `implode`
- `str_split` / `strtok`

Multibyte Strings & UTF-8

UTF-8 TEXT

Core string functions count **bytes**. In UTF-8 a character takes 1–4 bytes, so `strlen('café')` is 5. The **mbstring** extension provides character-aware versions: `mb_strlen`, `mb_substr`, `mb_strtoupper` and friends. Set the internal encoding to UTF-8 (the default since PHP 5.6) and use `mb_*` for anything a human reads. The **intl** extension adds normalisation, transliteration and grapheme functions.

NOTES

- `mb_str_pad()` arrived in PHP 8.3
- `mb_trim()`, `mb_ucfirst()`, `mb_lcfirst()` arrived in PHP 8.4
- `mb_check_encoding($s, 'UTF-8')` validates input
- Emoji with modifiers are several code points — use `grapheme_strlen()`
- Save source files as UTF-8 without BOM

BYTES VS CHARACTERS

```php
<?php
$s = 'Café über';

strlen($s);                  // 11 (bytes)
mb_strlen($s);               // 9  (characters)
substr($s, 0, 4);            // 'Caf' + half of é (broken)
mb_substr($s, 0, 4);         // 'Café'
strtoupper($s);              // 'CAFé üBER' (ASCII only)
mb_strtoupper($s);           // 'CAFÉ ÜBER'
mb_str_pad('né', 5, '*');    // 'né***' (8.3+)
mb_str_split('añb');         // ['a', 'ñ', 'b']

mb_check_encoding($input, 'UTF-8');   // validate
mb_convert_encoding($latin1, 'UTF-8', 'ISO-8859-1');

$e = "👍🏽";
mb_strlen($e);               // 2 (code points)
grapheme_strlen($e);         // 1 (what users see)
```

CODE EXPLAINED

- → `é` and `ü` are 2 bytes each in UTF-8
- → `substr()` can cut a character in half
- → `strtoupper()` ignores non-ASCII letters (PHP 8.2+)
- → `mb_convert_encoding()` converts legacy text
- → `grapheme_*` (intl) counts user-visible characters

BYTE → MB EQUIVALENTS

BYTE FUNCTION	MB VERSION
`strlen`	mb_strlen
`substr`	mb_substr
`strpos`	mb_strpos
`strtoupper`	mb_strtoupper
`str_split`	mb_str_split
`str_pad`	mb_str_pad (8.3)
`trim`	mb_trim (8.4)

ENCODING TOOLS

FUNCTION	PURPOSE
`mb_check_encoding()`	Is it valid UTF-8?
`mb_convert_encoding()`	Convert charset
`mb_detect_encoding()`	Guess charset
`iconv()`	Convert (iconv ext)
`mb_convert_case($s, MB_CASE_TITLE)`	Title Case
`Normalizer::normalize()`	NFC / NFD (intl)

UNITS OF TEXT

UNIT
`Bytes (strlen)`
`Code points (mb_strlen)`
`Graphemes (grapheme_strlen)`
`Normalized NFC (Normalizer)`

⚠ COMMON MISTAKES

- Truncating names with `substr()` — produces invalid UTF-8
- Assuming `strlen()` is a character count for validation rules
- Mixing encodings in one database

✓ TIPS & BEST PRACTICES

- Use `mb_*` for all user-visible text
- Validate incoming text with `mb_check_encoding()`
- Use UTF-8 (utf8mb4) end to end: files, DB, HTTP headers

QUICK SUMMARY

- Core functions count bytes
- `mb_*` counts characters
- intl `grapheme_*` for emoji
- UTF-8 everywhere

Arithmetic Operators

DOING MATH

PHP has the usual arithmetic operators plus ** for powers. / returns an `int` only when both operands are ints and the result is exact; otherwise a `float`. % is the integer remainder and takes the sign of the left operand. Dividing or taking % by zero throws **`DivisionByZeroError`**. Numeric strings are converted automatically; non-numeric strings throw `TypeError`.

NOTES

- `**` is right-associative: `2 ** 3 ** 2` is `2 ** 9`
- `-2 ** 2` is `-4` (power binds tighter than unary minus)
- `%` truncates floats to int (`7.9 % 2` is `1` , deprecated 8.1+)
- `PHP_INT_MAX + 1` overflows to `float`
- `fdiv()` returns `INF` / `NAN` instead of throwing

OPERATORS IN ACTION

```php
<?php
echo 7 + 2, ' ', 7 - 2, ' ', 7 * 2;   // 9 5 14
var_dump(7 / 2);                      // float(3.5)
var_dump(6 / 2);                      // int(3)
var_dump(7 % 3);                      // int(1)
var_dump(-7 % 3);                     // int(-1)
var_dump(2 ** 10);                    // int(1024)
var_dump(2 ** -1);                    // float(0.5)
var_dump(fmod(7.5, 2));               // float(1.5)

$i = 5;
echo $i++;   // 5 (then $i = 6)
echo ++$i;   // 7

try {
    echo 1 % 0;
} catch (DivisionByZeroError $e) {
    echo $e->getMessage();            // Modulo by zero
}
var_dump(fdiv(1, 0));                 // float(INF)
```

CODE EXPLAINED

→ `/` gives int only for exact int division

→ `%` result has the sign of the dividend

→ Negative exponent produces a float

→ Postfix `$i++` returns the old value; prefix returns the new

→ `1 / 0` and `1 % 0` both throw in PHP 8

OPERATORS

OP	EXAMPLE → RESULT
`+`	5 + 2 → 7
`-`	5 - 2 → 3
`*`	5 * 2 → 10
`/`	5 / 2 → 2.5
`%`	5 % 2 → 1
`**`	5 ** 2 → 25
`-$x`	Negation

INCREMENT / DECREMENT

CODE	VALUE, THEN $I
`$i++`	old, +1
`++$i`	new (+1)
`$i--`	old, -1
`--$i`	new (-1)
`'a'++`	'b' (string increment)
`'z'++`	'aa'

MATH HELPERS

FUNCTION	USE
`intdiv($a, $b)`	Integer division
`fmod($a, $b)`	Float remainder
`fdiv($a, $b)`	IEEE division (INF)
`abs() / max() / min()`	Basics
`pow($b, $e)`	Same as **
`sqrt() / pi()`	Roots, π

⚠ COMMON MISTAKES

- Using `%` with floats — operands are truncated to int
- Expecting `-2 ** 2` to be `4`
- Using `++` on strings — prefer `str_increment()` (8.3+)

✓ TIPS & BEST PRACTICES

- Use `intdiv()` when you want an int result
- Parenthesise mixed operators for readability
- Catch `DivisionByZeroError` where a divisor comes from input

QUICK SUMMARY

- `+ - * / % **`
- `/` may return float
- Zero divisor throws
- `++$i` vs `$i++`

Comparison Operators

COMPARING VALUES

== compares **after type juggling**; === requires the **same type and value**. Prefer the strict forms === and !== unless you specifically want conversion. The **spaceship** operator <=> returns −1, 0 or 1 and is ideal for sort callbacks. Arrays compare element by element; objects compare property by property with == and by identity with ===.

NOTES

- `<>` is an alias of `!=`
- Strings compare byte by byte (`'Z' < 'a'`)
- Two numeric strings compare numerically: `'10' == '1e1'`
- `===` on arrays also checks key order and types
- `match` uses strict `===` comparison (Sheet 31)

LOOSE, STRICT & SPACESHIP

```php
<?php
var_dump(1 == '1');           // true
var_dump(1 === '1');          // false
var_dump(null == false);      // true
var_dump('abc' == 0);         // false (PHP 8)
var_dump('1e3' == '1000');    // true (numeric strings)

echo 1 <=> 2;                 // -1
echo 'b' <=> 'a';             // 1
echo [1, 2] <=> [1, 2];       // 0

var_dump([1, 2] == [2 => 1]);           // false
var_dump(['a' => 1, 'b' => 2]
      == ['b' => 2, 'a' => 1]);         // true
var_dump(['a' => 1, 'b' => 2]
     === ['b' => 2, 'a' => 1]);         // false (order)

usort($people, fn($a, $b) => $a->age <=> $b->age);
```

CODE EXPLAINED

→ `==` converts; `===` doesn't

→ PHP 8 compares a number and non-numeric string as strings

→ `<=>` compares arrays element by element

→ Loose array equality ignores key order

→ `<=>` in `usort` sorts ascending; swap sides for descending

OPERATORS

OP	TRUE WHEN
`$a == $b`	Equal after juggling
`$a === $b`	Same type and value
`$a != $b`	Not loosely equal
`$a !== $b`	Not identical
`$a < $b / >`	Less / greater
`$a <=> $b`	-1, 0, 1

LOOSE SURPRISES

EXPRESSION	RESULT
`'1' == '01'`	true
`'10' == '1e1'`	true
`100 == '1e2'`	true
`null == []`	true
`'abc' == 'ABC'`	false
`[] == false`	true

OBJECT COMPARISON

EXPRESSION	TRUE WHEN
`$a == $b`	Same class, equal props
`$a === $b`	Same instance
`$a instanceof C`	$a is a C
`spl_object_id($a)`	Identity number
`$a <=> $b`	Compares props

⚠ COMMON MISTAKES

- Using `==` to compare tokens or hashes — use `hash_equals()`
- Writing `=` instead of `==` in a condition
- Sorting with a callback that returns `bool` instead of `<=>`

✓ TIPS & BEST PRACTICES

- Default to `===` / `!==`
- Use `<=>` in every sort comparator
- Use `hash_equals()` for secrets (timing-safe)

QUICK SUMMARY

- `===` strict, `==` loose
- `<=>` → -1 / 0 / 1
- Arrays: `==` ignores order
- Objects: `===` = same instance

Logical Operators

COMBINING CONDITIONS

&&, || and ! combine boolean conditions and always return `bool`. Both && and || **short-circuit**: the right side runs only if needed. PHP also has word forms `and`, `or` and `xor`, but `and`/`or` have **lower precedence than =**, which causes real bugs: `$ok = true and false;` sets `$ok` to `true`. Use the symbol forms in expressions.

NOTES

- `xor` is true when exactly one side is true (no symbol form)
- `!` binds tightly: `!$a == $b` means `(!$a) == $b`
- Operands are converted to bool using truthiness rules
- `$x || throw new E()` works because `throw` is an expression (8.0)
- `or die()` idiom is old style — prefer exceptions

SHORT-CIRCUIT & PRECEDENCE

```php
<?php
$age = 20; $member = false;

if ($age >= 18 && !$member) { echo "adult guest\n"; }
if ($age < 13 || $age > 65) { echo "discount\n"; }

// Short-circuit: right side skipped
$user = null;
if ($user !== null && $user->isAdmin()) { /* safe */ }

// Precedence trap
$a = true and false;   // ($a = true) and false
var_dump($a);          // bool(true)
$b = (true and false); // bool(false)
$c = true && false;    // bool(false)

var_dump(true xor true);   // false
var_dump(true xor false);  // true

$config = load() ?: throw new RuntimeException('none');
```

CODE EXPLAINED

→ `&&` stops at the first falsy operand

→ Null check before the method call prevents an error

→ `and` loses to `=`, so the assignment happens first

→ Parentheses or `&&` give the expected result

→ `throw` in an expression aborts when the left side is falsy

OPERATORS

OP	RESULT
`$a && $b`	Both truthy
`$a || $b`	Either truthy
`!$a`	Not $a
`$a and $b`	Like && (low precedence)
`$a or $b`	Like || (low precedence)
`$a xor $b`	Exactly one truthy

TRUTH TABLE

A, B	&& / || / XOR
T, T	T / T / F
T, F	F / T / T
F, T	F / T / T
F, F	F / F / F

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
1	!
2	< > <= >= == === <=>
3	&&
4	||
5	?? then ?:
6	= += .= …
7	and, xor, or

⚠ COMMON MISTAKES

- `$ok = check() and other();` — `$ok` ignores `other()`
- `!$a == $b` when you meant `!($a == $b)`
- Putting expensive calls first in `||` chains

✓ TIPS & BEST PRACTICES

- Use `&&` / `||` everywhere; avoid `and` / `or`
- Order conditions cheapest and most likely first
- Add parentheses whenever you mix `&&` and `||`

QUICK SUMMARY

- `&& || !`
- Short-circuit evaluation
- `and` / `or` bind below `=`
- `xor` = exactly one

Bitwise Operators

WORKING WITH BITS

Bitwise operators act on the binary digits of integers. `&` keeps bits set in both, `|` in either, `^` in exactly one, `~` flips every bit, and `<<` / `>>` shift left or right. PHP uses them most for **flags**: many built-in functions take options combined with `|`, such as `JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES`, and error levels like `E_ALL & ~E_DEPRECATED`.

NOTES

- Operands are converted to int; on two strings they work byte-wise
- `>>` is an arithmetic shift (keeps the sign)
- Shifting by a negative amount throws `ArithmeticError`
- `&` binds tighter than `|` but looser than `==` — use parentheses
- See the Bitwise sheet set for deeper tricks

FLAGS & MASKS

```php
<?php
const READ  = 0b001;   // 1
const WRITE = 0b010;   // 2
const EXEC  = 0b100;   // 4

$perm = READ | WRITE;            // 3  (0b011)
$canWrite = ($perm & WRITE) !== 0;  // true
$perm |= EXEC;                   // add a flag  -> 7
$perm &= ~WRITE;                 // remove flag -> 5
$perm ^= READ;                   // toggle      -> 4

echo 1 << 3;      // 8
echo 64 >> 2;     // 16
echo ~0;          // -1
printf('%08b', 0b1010 ^ 0b0110);  // 00001100

echo json_encode(['a/b' => 'é'],
    JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
error_reporting(E_ALL & ~E_DEPRECATED);
```

CODE EXPLAINED

→ Each flag is a different power of two

→ `&` with a flag tests it; wrap in parentheses before comparing

→ `|=` sets, `&= ~` clears, `^=` toggles

→ `<< n` multiplies by 2^n; `>> n` divides

→ Built-in option constants combine the same way

OPERATORS

OP	6 (110) OP 3 (011)
`&`	2 (010)
`|`	7 (111)
`^`	5 (101)
`~6`	-7
`6 << 1`	12
`6 >> 1`	3

FLAG IDIOMS

CODE	EFFECT
`$f | FLAG`	Set
`$f & ~FLAG`	Clear
`$f ^ FLAG`	Toggle
`($f & FLAG) !== 0`	Test
`($f & MASK) === MASK`	All in mask set

BUILT-IN FLAG FAMILIES

PREFIX	USED BY
`E_*`	error_reporting()
`JSON_*`	json_encode / decode
`FILE_*`	file_put_contents, file
`ENT_*`	htmlspecialchars
`PREG_*`	preg_* functions
`SORT_*`	sort functions

⚠ COMMON MISTAKES

- `$f & FLAG === FLAG` — compares first; add parentheses
- Using `&&` when you meant `&` (or vice versa)
- Defining flags that aren't powers of two

✓ TIPS & BEST PRACTICES

- Use `0b` literals so flag bits are visible
- Prefer enums or arrays of options for new APIs
- Use `printf('%b')` to debug masks

QUICK SUMMARY

- `& | ^ ~ << >>`
- Combine options with `|`
- Test with `($f & X) !== 0`
- Clear with `& ~X`

Assignment & Operator Precedence

= · += · .= · **= · =& · precedence table · associativity

ASSIGNMENT AND EVALUATION ORDER

= stores the right-hand value and the whole expression evaluates to that value, so `$a = $b = 5` works. **Compound** operators combine an operation with assignment: `$total += $price`, `$html .= '<li>'`. When an expression mixes operators, **precedence** decides what binds first and **associativity** decides the direction for equal precedence. When in doubt, add parentheses.

NOTES

- Assignment is right-associative: `$a = ($b = 5)`
- `.=` is the idiomatic way to build strings
- `??=` assigns only if the left side is null or unset
- In PHP 8, `+` and `-` bind tighter than `.`
- Non-associative operators (`==` , `<`) can't be chained: `1 < $x < 5` is an error

COMPOUND ASSIGNMENT

```php
<?php
$total = 0;
$total += 10;      // 10
$total -= 3;       // 7
$total *= 2;       // 14
$total /= 4;       // 3.5
$total **= 2;      // 12.25
$n = 17; $n %= 5;  // 2

$html = '<ul>';
foreach (['a', 'b'] as $item) {
    $html .= "<li>$item</li>";
}
$html .= '</ul>';

$opts['debug'] ??= false;   // set if missing

echo 'Sum: ' . 1 + 2;       // 'Sum: 3' in PHP 8
echo 2 + 3 * 4;             // 14
echo (2 + 3) * 4;           // 20
echo 2 ** 3 ** 2;           // 512 (right-assoc)
```

CODE EXPLAINED

- → Each compound operator reads, computes, writes back
- → `.=` appends to the string in place
- → `??=` won't overwrite an existing non-null value
- → PHP 8: `+` evaluates before `.`
- → `**` groups from the right

COMPOUND OPERATORS

OP	SAME AS
`$a += $b`	$a = $a + $b
`$a -= $b`	$a = $a - $b
`$a *= $b`	$a = $a * $b
`$a /= $b`	$a = $a / $b
`$a .= $b`	$a = $a . $b
`$a ??= $b`	$a = $a ?? $b
`$a |= $b`	$a = $a | $b

PRECEDENCE (HIGH → LOW)

GROUP	OPERATORS
`clone new`	Object creation
`**`	Power (right)
`++ -- ~ (int) @`	Unary
`instanceof, !`	Type, not
`* / %`	Multiplicative
`+ -`	Additive
`<< >> then .`	Shift, then concat

PRECEDENCE (CONT.)

GROUP	OPERATORS
`< <= > >=`	Relational
`== != === <=>`	Equality
`& ^ |`	Bitwise
`&& ||`	Logical
`??`	Null coalesce (right)
`? :`	Ternary (non-assoc)
`= += …`	Assignment (right)

⚠ COMMON MISTAKES

- `1 < $x < 5` — write `1 < $x && $x < 5`
- Relying on precedence across `.` and `+` without parentheses
- Using `=` where `==` was meant inside `if`

✓ TIPS & BEST PRACTICES

- Parenthesise anything a reader might misread
- Use `.=` in loops instead of building arrays when output is small
- Use `??=` to fill defaults in option arrays

QUICK SUMMARY

- `=` returns the value
- Compound: `+= .= ??=`
- Precedence then associativity
- Parentheses beat memorising

Null-Safe & Special Operators

SHORTCUTS FOR COMMON PATTERNS

?? (null coalescing) returns the left side unless it is `null` or undefined — **without warnings**. `?:` (short ternary) returns the left side if it is **truthy**. `?->` (nullsafe) stops a property or method chain and returns `null` when it hits `null`. `...` spreads arrays into calls and array literals. PHP 8.5 adds the **pipe operator** `|>`, which passes a value into a callable.

NOTES

- `??` checks for null only; `?:` treats `0`, `''`, `'0'` as missing too
- `??` and `?->` don't raise undefined-index/variable warnings
- `?->` can't be used for writing (`$a?->b = 1` is an error)
- `@` hides warnings but slows code and hides bugs
- `|>` right side must be a callable taking one argument

NULL HANDLING, SPREAD & PIPE

```php
<?php
$page = $_GET['page'] ?? 1;             // no warning if missing
$name = $input['name'] ?: 'Anonymous'; // '' -> Anonymous
$opts['limit'] ??= 20;                  // default if unset

$city = $user?->getAddress()?->city;    // null if any is null
$len  = $user?->name ? strlen($user->name) : 0;

function sum(int ...$n): int { return array_sum($n); }
$nums = [1, 2, 3];
echo sum(...$nums);                     // 6
$all = [...$nums, 4, ...[5, 6]];        // [1..6]

$ok = @file_get_contents('missing.txt');  // false, silent

// PHP 8.5 pipe operator
$slug = '  Hello World  '
    |> trim(...)
    |> strtolower(...)
    |> (fn($s) => str_replace(' ', '-', $s));
// 'hello-world'
```

CODE EXPLAINED

- → `??` — default for missing keys and nulls
- → `?:` — default for any falsy value
- → `?->` — short-circuits the whole rest of the chain
- → `...$nums` unpacks into arguments or array elements
- → Each `|>` step receives the previous result

?? VS ?:

LEFT VALUE	?? 'D' / ?: 'D'
`null`	'd' / 'd'
`undefined`	'd' / 'd' + Warning
`0`	0 / 'd'
`''`	'' / 'd'
`'0'`	'0' / 'd'
`'x'`	'x' / 'x'

SPECIAL OPERATORS

OP	PURPOSE
`??`	Null coalescing
`??=`	Assign if null
`?:`	Short ternary (Elvis)
`?->`	Nullsafe call/fetch
`...`	Spread / variadic
`@`	Suppress errors
`|>`	Pipe (8.5)

SPREAD RULES

USE	NOTES
`f(...$arr)`	Args from array
`f(...$assoc)`	String keys → named args
`[...$a, ...$b]`	Merge lists
`[...$assoc]`	String keys allowed (8.1)
`[...$gen]`	Any Traversable

⚠ COMMON MISTAKES

- Using `?:` when `0` is a valid value — use `??`
- Silencing errors with `@` instead of checking return values
- Expecting `?->` to also guard array access — use `??` for keys

✓ TIPS & BEST PRACTICES

- Use `??` for all request/array defaults
- Replace nested null checks with `?->`
- Use `|>` for readable transformation chains (8.5+)

QUICK SUMMARY

- `??` = null default
- `?:` = falsy default
- `?->` = null-safe chain
- `...` spread, `|>` pipe

if, elseif & else

CONDITIONAL BRANCHES

`if` runs a block when its condition is truthy; `elseif` tests further conditions in order and `else` catches everything left. Only the first matching branch runs. Braces are optional for one statement but always recommended. In templates, the **alternative syntax** (`if (...): … endif;`) reads better between chunks of HTML. **Guard clauses** return early and keep the main path unindented.

NOTES

- `elseif` (one word) and `else if` behave the same with braces
- With the colon syntax only `elseif` works
- Conditions use truthiness rules (Sheet 12)
- Assignments in conditions need extra parentheses for clarity
- Long `elseif` chains on one value → consider `match`

BRANCHES, GUARDS & TEMPLATES

```php
<?php
function shipping(float $total, string $country): float
{
    if ($total <= 0) {
        throw new InvalidArgumentException('Empty cart');
    }                                  // guard clause

    if ($total >= 100) {
        return 0.0;
    } elseif ($country === 'US') {
        return 5.0;
    } else {
        return 15.0;
    }
}

if (($user = findUser($id)) !== null) {
    echo $user->name;
}
?>
<?php if ($loggedIn): ?>
    <a href="/logout">Log out</a>
<?php else: ?>
    <a href="/login">Log in</a>
<?php endif; ?>
```

CODE EXPLAINED

→ Guard clause handles the bad case and leaves

→ Branches are tested top to bottom; first match wins

→ `else` has no condition

→ Parenthesise an assignment used as a condition

→ Colon syntax pairs with `endif;` in HTML templates

SYNTAX FORMS

FORM	NOTES
`if (c) { … }`	Standard
`if (c) stmt;`	One statement, no braces
`elseif (c) { … }`	Next test
`else { … }`	Fallback
`if (c): … endif;`	Alternative syntax
`else: / elseif (c):`	Inside colon form

COMMON CONDITIONS

CONDITION	MEANING
`if ($x)`	Truthy
`if ($x === null)`	Exactly null
`if (isset($a['k']))`	Key set, not null
`if (!empty($list))`	Non-empty
`if ($o instanceof C)`	Type check
`if (in_array($v, $xs, true))`	Strict membership

WHEN TO USE WHAT

SITUATION	USE
`One condition`	if
`Two outcomes, one value`	ternary ?:
`Default for null`	??
`Many values of one var`	match
`Early exit`	guard clause + return

⚠ COMMON MISTAKES

- `if ($a = 5)` — assigns instead of comparing
- Mixing `{` braces and `:` syntax in one `if`
- Deep nesting instead of returning early

✓ TIPS & BEST PRACTICES

- Always use braces in PHP-only code
- Return early on invalid input
- Use alternative syntax only in templates

QUICK SUMMARY

- `if` / `elseif` / `else`
- First match wins
- Colon syntax for templates
- Guard clauses flatten code

Ternary & Conditional Expressions

?: · short ternary ?: · nested ternary needs parens · ?? as default · ternary in templates

CHOOSING A VALUE INLINE

The ternary operator `cond ? a : b` is an **expression**, so it produces a value you can assign, return or echo. The **short ternary** `a ?: b` returns `a` if it's truthy, otherwise `b`. Nesting ternaries **without parentheses is a fatal error** since PHP 8 because the old left-to-right grouping was confusing. For null-only defaults, `??` is safer than `?:`.

NOTES

- Only one branch is evaluated
- Keep ternaries to one line and one decision
- `?:` evaluates the left side once
- Use `match` when there are three or more outcomes
- Ternaries work well inside `<?= ?>` in templates

TERNARY PATTERNS

```php
<?php
$age = 17;
$label = $age >= 18 ? 'adult' : 'minor';   // 'minor'

echo $count === 1 ? 'item' : 'items';

$title = $post['title'] ?: 'Untitled';     // '' -> Untitled
$limit = $_GET['limit'] ?? 10;             // null-only

// Nested: parentheses are required
$size = $n < 10 ? 'small' : ($n < 100 ? 'medium' : 'large');
// $n < 10 ? 'a' : $n < 100 ? 'b' : 'c';  // Fatal error

// Better for 3+ outcomes
$size = match (true) {
    $n < 10  => 'small',
    $n < 100 => 'medium',
    default  => 'large',
};

return $user?->isAdmin() ? redirect('/admin') : view('home');
?>
<li class="<?= $active ? 'active' : '' ?>">Home</li>
```

CODE EXPLAINED

→ Condition, then value if true, then value if false

→ `?:` falls back on any falsy value

→ `??` falls back only on null or missing

→ Unparenthesised nesting is rejected at compile time

→ `match(true)` reads better than nested ternaries

FORMS

EXPRESSION	RESULT
`$c ? $a : $b`	$a if truthy else $b
`$a ?: $b`	$a if truthy else $b
`$a ?? $b`	$a unless null/unset
`$c ? $a : ($d ? $x : $y)`	Nested (parens)
`match (true) { … }`	Many branches

?: VS ??

$A	$A ?: 'D' / $A ?? 'D'
`null`	'd' / 'd'
`0`	'd' / 0
`''`	'd' / ''
`false`	'd' / false
`'x'`	'x' / 'x'

GOOD USES

USE	EXAMPLE
`Pluralise`	$n === 1 ? '' : 's'
`CSS class`	$on ? 'on' : 'off'
`Default title`	$t ?: 'Untitled'
`Return choice`	return $ok ? $a : $b
`Config default`	$cfg['x'] ?? 5

⚠ COMMON MISTAKES

- Nesting ternaries without parentheses
- Using `?:` for a value where `0` is valid
- Cramming side effects into ternary branches

✓ TIPS & BEST PRACTICES

- One ternary per expression
- Use `??` for missing keys, `?:` for empty strings
- Switch to `match` for three or more outcomes

QUICK SUMMARY

- `c ? a : b` is an expression
- `a ?: b` short form
- Nested needs parens
- `match` for many outcomes

switch

MULTI-WAY BRANCH ON ONE VALUE

`switch` compares one expression against several `case` values using **loose `==` comparison** and runs code from the first match **until a `break`**. Leaving out `break` makes execution **fall through** into the next case — useful for grouping cases, dangerous by accident. `default` runs when nothing matches. For most new code, `match` (Sheet 31) is shorter and stricter.

NOTES

- `continue` inside `switch` acts like `break` (and warns)
- Cases can be expressions, not just literals
- `switch` is a statement — it doesn't produce a value
- Alternative syntax: `switch (...): … endswitch;`
- `default` can appear anywhere but put it last

SWITCH WITH GROUPING & FALL-THROUGH

```php
<?php
$day = 'sat';

switch ($day) {
    case 'sat':
    case 'sun':                     // grouped cases
        $type = 'weekend';
        break;
    case 'fri':
        echo "Almost there\n";      // falls through ...
    case 'mon':
    case 'tue':
        $type = 'weekday';          // ... into here
        break;
    default:
        $type = 'unknown';
}

switch ('1') {
    case 1:                         // '1' == 1 -> matches
        echo "loose match\n";
        break;
}

switch (true) {                     // range checks
    case $score >= 90: $grade = 'A'; break;
    case $score >= 80: $grade = 'B'; break;
    default:           $grade = 'C';
}
```

CODE EXPLAINED

→ Empty cases stack to share one body

→ No `break` after `fri` → runs the next body too

→ `default` handles unmatched values

→ Comparison is `==`, so `'1'` matches `1`

→ `switch (true)` turns cases into conditions

SWITCH VS MATCH

FEATURE	SWITCH / MATCH
Comparison	== / ===
Returns value	no / yes
Fall-through	yes / no
Needs break	yes / no
No match	nothing / UnhandledMatchError
Multi-statement bodies	yes / no

KEYWORDS

KEYWORD	ROLE
`switch (expr)`	Value to test
`case v:`	Candidate value
`break;`	Leave the switch
`default:`	No case matched
`endswitch;`	Alt-syntax end
`return`	Also exits (in functions)

LOOSE MATCH GOTCHAS

SWITCH / CASE	MATCHES
`'1'` / `1`	yes
`0` / `'a'`	no (PHP 8)
`null` / `false`	yes
`''` / `null`	yes
`true` / `'yes'`	yes

⚠ COMMON MISTAKES

- Forgetting `break` — silent fall-through
- Relying on loose comparison with mixed types
- Using `switch` just to assign one value — use `match`

✓ TIPS & BEST PRACTICES

- Comment intentional fall-through: `// fall through`
- Use `return` instead of `break` inside small functions
- Prefer `match` for value mapping

QUICK SUMMARY

- Loose `==` comparison
- `break` ends a case
- Fall-through without it
- `default` catches the rest

match Expressions

MATCH (PHP 8.0)

`match` maps a value to a result. It is an **expression** — it returns a value — and it compares with **strict ===**. Each arm is a single expression; there's no fall-through and no `break`. Several values can share an arm separated by commas. If nothing matches and there's no `default`, PHP throws `UnhandledMatchError`, so missing cases show up immediately instead of silently doing nothing.

NOTES

- Arm results can be any expression, including `throw`
- `match (true)` lets each arm be a condition
- Only the matching arm's expression is evaluated
- Trailing comma after the last arm is allowed
- Combine with enums for exhaustive mapping

MAPPING VALUES WITH MATCH

```php
<?php
$status = 404;

$message = match ($status) {
    200, 201 => 'OK',
    301, 302 => 'Redirect',
    404      => 'Not Found',
    500      => 'Server Error',
    default  => 'Unknown',
};                                  // 'Not Found'

$price = match ($plan) {
    'free'  => 0,
    'pro'   => 12,
    default => throw new DomainException("Bad plan: $plan"),
};

$grade = match (true) {
    $score >= 90 => 'A',
    $score >= 80 => 'B',
    $score >= 70 => 'C',
    default      => 'F',
};

echo match ('1') { 1 => 'int', '1' => 'string' }; // string
```

CODE EXPLAINED

- → Comma list: any of these values selects the arm
- → Result is assigned directly — no temp variable
- → `throw` works as an arm expression
- → `match (true)` picks the first true condition
- → Strict comparison: `'1'` doesn't match `1`

SYNTAX

PART	MEANING
`match (subject)`	Value to compare
`v => expr,`	One arm
`a, b => expr,`	Several values
`default => expr,`	Fallback
`};`	Ends the expression

BEHAVIOUR

RULE	EFFECT
`Comparison`	Strict ===
`No match, no default`	UnhandledMatc
`Fall-through`	Never
`Arm body`	One expressior
`Evaluation`	Arms tried top bottom

MULTI-LINE LOGIC

NEED	APPROACH
`Several statements`	Call a function in the arm
`Side effect only`	Arm calls a method
`Shared code`	Group values with commas
`Range test`	match (true)
`Exhaustive enum`	One arm per case, no default

⚠ COMMON MISTAKES

- Expecting loose comparison like `switch`
- Trying to put several statements in an arm
- Adding `default` to enum matches — it hides new cases

✓ TIPS & BEST PRACTICES

- Use `match` for value-to-value mapping
- Let `UnhandledMatchError` flag impossible states
- Move complex arm logic into small functions

QUICK SUMMARY

- Expression that returns a value
- Strict `===`
- No fall-through or break
- Unmatched → `UnhandledMatchError`

for Loops

COUNTING LOOPS

`for (init; condition; step)` runs the init once, checks the condition before each pass, and runs the step after each pass. Use it when you need an **index or counter**; for walking arrays, `foreach` (Sheet 33) is simpler and works with any keys. Each section can hold several comma-separated expressions, and any section may be empty.

NOTES

- `for (;;)` is an infinite loop — exit with `break`
- The condition is re-evaluated every iteration
- Store `count($arr)` in a variable before looping
- Loop variables stay defined after the loop ends
- Alternative syntax: `for (...): … endfor;`

FOR LOOP PATTERNS

```php
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i, ' ';                    // 1 2 3 4 5
}

for ($i = 10; $i > 0; $i -= 3) {
    echo $i, ' ';                    // 10 7 4 1
}

$items = ['a', 'b', 'c'];
for ($i = 0, $n = count($items); $i < $n; $i++) {
    echo "$i: {$items[$i]}\n";
}

for ($i = 0, $j = 9; $i < $j; $i++, $j--) {
    // two counters moving toward each other
}

for ($row = 1; $row <= 3; $row++) {
    for ($col = 1; $col <= 3; $col++) {
        echo $row * $col, "\t";
    }
    echo "\n";                       // multiplication grid
}
echo $i;                             // still defined
```

CODE EXPLAINED

→ Init `$i = 1`, test `$i <= 5`, step `$i++`

→ Steps can be any expression, like `$i -= 3`

→ `$n = count(...)` runs once, not every pass

→ Comma-separated expressions in init and step

→ Nested loops: inner runs fully per outer pass

PARTS OF A FOR LOOP

PART	RUNS
`init`	Once, before the loop
`condition`	Before every pass
`body`	While condition is truthy
`step`	After every pass
`break`	Leaves immediately

FOR VS FOREACH

NEED	USE
Walk every array element	foreach
Need the key	foreach ($a as $k => $v)
Count a fixed number	for
Step by 2, go backward	for
Two indexes at once	for
Iterate a generator	foreach

COMMON RANGES

LOOP HEADER	VALUES
`$i = 0; $i < $n; $i++`	0 … n−1
`$i = 1; $i <= $n; $i++`	1 … n
`$i = $n - 1; $i >= 0; $i--`	n−1 … 0
`$i = 0; $i < $n; $i += 2`	Even indexes
`foreach (range(1, 5) as $i)`	1 … 5

⚠ COMMON MISTAKES

- `$i <= count($a)` — off by one, reads past the end
- Calling `count()` in the condition of a huge loop
- Using `for` with index access on arrays that aren't 0-indexed lists

✓ TIPS & BEST PRACTICES

- Prefer `foreach` for arrays
- Hoist expensive calls out of the condition
- Give nested loop counters meaningful names

QUICK SUMMARY

- `for (init; cond; step)`
- Use for counters and steps
- Hoist `count()`
- `foreach` for arrays

foreach Loops

foreach as $v · $k => $v · by reference &$v · unset after reference · destructuring · endforeach

WALKING ARRAYS AND ITERABLES

`foreach` visits every element of an array or **Traversable** object (iterators, generators) in order. `as $v` gives values; `as $k => $v` gives keys too. It works on a **copy** of the array, so changing `$v` doesn't change the array — iterate **by reference** (`&$v`) for in-place edits, then `unset($v)`. You can destructure each element directly in the loop header.

NOTES

- Order is insertion order, not key order
- Iterating a non-iterable gives a Warning
- Adding elements during a by-value loop doesn't affect it
- `foreach` over an object without `Iterator` walks public properties
- Alternative syntax: `foreach (...): … endforeach;`

FOREACH PATTERNS

```php
<?php
$prices = ['tea' => 3, 'cake' => 5];

foreach ($prices as $price) {
    echo $price, ' ';                    // 3 5
}

foreach ($prices as $item => $price) {
    echo "$item costs $price\n";
}

foreach ($prices as &$price) {           // by reference
    $price *= 2;
}
unset($price);                           // break the reference!
// $prices = ['tea' => 6, 'cake' => 10]

$users = [['id' => 1, 'name' => 'Ana'], ['id' => 2, 'name' => 'Bo']];
foreach ($users as ['id' => $id, 'name' => $name]) {
    echo "$id: $name\n";                 // destructuring
}

foreach ([[1, 2], [3, 4]] as [$x, $y]) {
    echo $x + $y, ' ';                   // 3 7
}
```

CODE EXPLAINED

→ Value-only form when keys don't matter

→ `$k => $v` exposes the key

→ `&$price` writes back into the array

→ `unset($price)` prevents the classic dangling-reference bug

→ Keyed and list destructuring work in the header

FORMS

HEADER	GIVES
`foreach ($a as $v)`	Values
`foreach ($a as $k => $v)`	Keys + values
`foreach ($a as &$v)`	Writable values
`foreach ($a as [$x, $y])`	List destructure
`foreach ($a as ['k' => $x])`	Keyed destructure
`foreach ($gen as $v)`	Generator output

LOOP HELPERS

NEED	USE
`Index while looping`	$k => $v on a list
`Keys only`	foreach (array_keys($a) as $k)
`First/last flag`	$k === array_key_last($a)
`Stop early`	break
`Skip one`	continue
`Transform all`	array_map() instead

REFERENCE BUG

STEP	RESULT
`foreach ($a as &$v) {}`	$v → last element
`(no unset)`	reference remains
`foreach ($a as $v) {}`	rewrites last element
`Fix`	unset($v) after loop

⚠ COMMON MISTAKES

- Skipping `unset($v)` after a reference loop
- Modifying `$v` and expecting the array to change
- Looping over `null` from a failed lookup

✓ TIPS & BEST PRACTICES

- Prefer `array_map` / `array_filter` for pure transforms
- Use destructuring for rows from a database
- Guard with `?? []` when the array may be missing

QUICK SUMMARY

- `as $v` / `as $k => $v`
- By-value copy by default
- `&$v` + `unset($v)`
- Destructure in the header

while & do-while

CONDITION-DRIVEN LOOPS

`while` checks its condition **before** each pass, so the body may run zero times. `do … while` checks **after**, so the body always runs at least once. Use them when you don't know the number of iterations in advance: reading lines until end of file, fetching rows until none are left, retrying until success. Make sure something in the body moves toward the exit.

NOTES

- `do … while` needs a semicolon after `while (...)`
- `while (true)` + `break` is fine for event loops
- Assignment in the condition is idiomatic for readers and cursors
- Compare with `!== false` when `0` or `''` are valid values
- Alternative syntax: `while (...): … endwhile;`

WHILE AND DO-WHILE

```php
<?php
$n = 3;
while ($n > 0) {
    echo $n--, ' ';                 // 3 2 1
}

$fh = fopen('log.txt', 'r');
while (($line = fgets($fh)) !== false) {
    echo trim($line), "\n";
}
fclose($fh);

while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    process($row);                  // until fetch returns false
}

$attempt = 0;
do {
    $ok = tryConnect();
    $attempt++;
} while (!$ok && $attempt < 3);     // runs at least once

while (true) {
    $job = $queue->pop();
    if ($job === null) break;
    $job->run();
}
```

CODE EXPLAINED

→ The counter changes inside the body, so the loop ends

→ Wrap the assignment in parentheses before `!==`

→ `fetch()` returns `false` when rows run out

→ `do … while` retries at least once

→ Infinite loop with an explicit `break`

LOOP CHOICE

SITUATION	LOOP
`Known count`	for
`Every array element`	foreach
`Unknown count, check first`	while
`Run at least once`	do … while
`Endless worker`	while (true) + break

READER IDIOMS

CODE	STOPS WHEN
`while (($l = fgets($f)) !== false)`	EOF
`while ($r = $stmt->fetch())`	No more rows
`while (!feof($f))`	EOF (check after read!)
`while (($k = key($a)) !== null)`	Array end
`while ($q->valid())`	Iterator done

EXIT TOOLS

KEYWORD	EFFECT
`break`	Leave loop
`break 2`	Leave two loops
`continue`	Next iteration
`return`	Leave function
`exit`	Stop the script

⚠ COMMON MISTAKES

- Forgetting to change the loop variable — infinite loop
- `while (!feof($f))` that processes a final `false` line
- Missing the `;` after `do { } while (...)`

✓ TIPS & BEST PRACTICES

- Use a generator + `foreach` for unbounded streams
- Add a max-iterations guard to retry loops
- Use `!== false` with readers that can return `'0'`

QUICK SUMMARY

- `while` checks first
- `do … while` runs once minimum
- Assign-in-condition for readers
- Always progress toward exit

Loop Controls

CHANGING LOOP FLOW

`break` leaves the loop immediately; `continue` skips to the next iteration. Both take an optional **level number** to affect outer loops: `break 2` leaves two nested loops. `switch` counts as a loop level for this purpose. `exit` (alias `die`) ends the whole script, optionally printing a message or returning an exit code. `goto` exists but is almost never the right tool.

NOTES

- The level must be a literal integer (no variables)
- `continue` inside `switch` behaves like `break` and warns
- `exit(1)` sets the CLI exit status; `exit('msg')` prints
- `exit` still runs shutdown functions and destructors
- `goto` can't jump into a loop or switch

BREAK, CONTINUE & LEVELS

```php
<?php
foreach ([1, 2, 3, 4, 5, 6] as $n) {
    if ($n % 2 === 0) continue;      // skip evens
    if ($n > 4) break;               // stop at 5
    echo $n, ' ';                    // 1 3
}

$grid = [[1, 2], [3, 99], [5, 6]];
foreach ($grid as $r => $row) {
    foreach ($row as $c => $v) {
        if ($v === 99) {
            echo "found at $r,$c\n";
            break 2;                 // leave both loops
        }
    }
}

foreach ($files as $f) {
    switch (pathinfo($f, PATHINFO_EXTENSION)) {
        case 'tmp':
            continue 2;              // next file, not next case
    }
    process($f);
}

if (!$config) exit("No config\n");  // stop script
```

CODE EXPLAINED

- → `continue` jumps to the next element
- → `break` ends the loop at the first `$n > 4`
- → `break 2` exits the inner and outer loop
- → `continue 2` skips past the `switch` to the next file
- → `exit` with a string prints it and stops

KEYWORDS

KEYWORD	EFFECT
`break`	Leave innermost loop/switch
`break n`	Leave n levels
`continue`	Next iteration
`continue n`	Next iteration of level n
`return`	Leave current function
`exit / die`	End script

EXIT FORMS

CALL	RESULT
`exit;`	Status 0
`exit(0)`	Success
`exit(1)`	Failure (CLI)
`exit('text')`	Print, status 0
`die('text')`	Same as exit

ALTERNATIVES

INSTEAD OF	CONSIDER
`break 3`	Extract a function, return
`goto`	Loops or functions
`exit in library`	Throw an exception
`Flag variables`	break n
`continue chains`	Filter first

⚠ COMMON MISTAKES

- Using `continue` inside `switch` meaning "next loop pass" — use `continue 2`
- Calling `exit` deep inside reusable code
- `break $n` with a variable — not allowed

✓ TIPS & BEST PRACTICES

- Extract nested loops into a function and `return`
- Use exit codes in CLI scripts so callers can react
- Throw exceptions instead of `die()` in apps

QUICK SUMMARY

- `break` leaves, `continue` skips
- Level numbers for nesting
- `switch` is a level
- `exit` stops everything

Errors vs Exceptions

TWO ERROR SYSTEMS

PHP reports problems in two ways. **Diagnostics** (warnings, notices, deprecations) are messages with a level such as `E_WARNING`; by default execution continues. **Throwables** interrupt execution until caught. `Throwable` has two branches: `Exception` for conditions your code expects to handle, and `Error` for engine failures such as `TypeError`, `ValueError` and calling an undefined function.

NOTES

- Uncaught throwables become a "Fatal error: Uncaught …"
- Many PHP 7 fatal errors became catchable `Error` s in PHP 8
- `display_errors` controls output; `log_errors` writes to the log
- Development: `E_ALL` + display on; production: log only
- Catch `Throwable` only at the top level of an app

LEVELS AND HIERARCHY

```php
<?php
error_reporting(E_ALL);               // report everything
ini_set('display_errors', '1');       // dev only

echo $undefined;        // Warning: Undefined variable
$arr = [];
echo $arr['x'];         // Warning: Undefined array key
strlen();               // ArgumentCountError (an Error)

try {
    intdiv(1, 0);
} catch (DivisionByZeroError $e) {
    echo get_class($e), ': ', $e->getMessage(), "\n";
}

try {
    new DateTime('not a date');
} catch (Exception $e) {
    echo 'Exception: ', $e->getMessage(), "\n";
}

try {
    undefined_function();
} catch (Error $e) {
    echo 'Engine error: ', $e->getMessage();
}
```

CODE EXPLAINED

→ `E_ALL` includes notices and deprecations

→ Undefined variables/keys are Warnings in PHP 8

→ Wrong argument count throws, it doesn't warn

→ `DivisionByZeroError` extends `ArithmeticError` → `Error`

→ Calling an unknown function throws `Error`

ERROR LEVELS

CONSTANT	MEANING
`E_ERROR`	Fatal; script stops
`E_WARNING`	Problem; continues
`E_NOTICE`	Minor issue
`E_DEPRECATED`	Feature will be removed
`E_USER_*`	Raised by trigger_error()
`E_ALL`	Everything

THROWABLE TREE

CLASS	PARENT
`Exception`	Throwable
`Error`	Throwable
`TypeError`	Error
`ValueError`	Error
`ArithmeticError`	Error
`DivisionByZeroError`	ArithmeticError
`ErrorException`	Exception

RECOMMENDED INI

SETTING	DEV / PROD
`error_reporting`	E_ALL / E_ALL
`display_errors`	On / Off
`log_errors`	On / On
`error_log`	path / path
`display_startup_errors`	On / Off

⚠ COMMON MISTAKES

- Hiding warnings instead of fixing them
- Showing errors to visitors in production
- Catching `Exception` and missing `TypeError` (it's an `Error`)

✓ TIPS & BEST PRACTICES

- Develop with `E_ALL` and display on
- Convert warnings to exceptions with an error handler (Sheet 40)
- Log everything in production

QUICK SUMMARY

- Diagnostics vs throwables
- `Exception` vs `Error`
- `E_ALL` always
- Display in dev, log in prod

try, catch & finally

try · catch · multi-catch A|B · catch without variable · finally · rethrow · getMessage()

HANDLING EXCEPTIONS

Wrap code that may throw in `try`. Each `catch` names one or more types; the **first matching catch** runs, so order them from most specific to most general. `catch (A | B $e)` handles several types in one block, and since PHP 8 the variable is optional. `finally` runs whether the `try` succeeded, threw, or returned — use it for cleanup like closing files or releasing locks.

NOTES

- A `catch` matches subclasses of the named type
- Rethrow with `throw $e;` or wrap: `throw new X('…', 0, $e)`
- A `return` in `finally` overrides earlier returns — avoid it
- Uncaught exceptions bubble up through calling functions
- `getMessage()`, `getCode()`, `getFile()`, `getLine()`, `getTrace()`

TRY / CATCH / FINALLY

```php
<?php
function loadConfig(string $path): array
{
    $fh = fopen($path, 'r')
        ?: throw new RuntimeException("Can't open $path");
    try {
        $json = stream_get_contents($fh);
        return json_decode($json, true,
            flags: JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        throw new RuntimeException("Bad JSON in $path", 0, $e);
    } finally {
        fclose($fh);                    // always runs
    }
}

try {
    $cfg = loadConfig('app.json');
} catch (InvalidArgumentException | DomainException $e) {
    echo 'Input problem: ', $e->getMessage();
} catch (RuntimeException $e) {
    error_log($e);                      // __toString has trace
    $cfg = [];
} catch (Throwable) {                   // no variable (8.0)
    exit(1);
}
```

CODE EXPLAINED

- → `?: throw` — throw as an expression when `fopen` fails
- → `JSON_THROW_ON_ERROR` turns JSON errors into exceptions
- → Wrapping keeps the original as `getPrevious()`
- → `finally` closes the file even after `return`
- → Multi-catch with `|`; broadest type last

EXCEPTION METHODS

METHOD	RETURNS
`getMessage()`	Message text
`getCode()`	Integer code
`getPrevious()`	Wrapped throwable
`getFile()` / `getLine()`	Where thrown
`getTrace()`	Stack as array
`getTraceAsString()`	Stack as text

CATCH FORMS

SYNTAX	USE
`catch (E $e)`	One type
`catch (A | B $e)`	Several types
`catch (E)`	Don't need the object
`catch (Throwable $t)`	Anything (top level)
`finally { … }`	Cleanup

CONTROL FLOW

SITUATION	RUNS
`No exception`	try, finally
`Caught`	try (part), catch, finally
`Not caught`	try (part), finally, then bubbles
`return in try`	finally, then return
`exit() in try`	finally does NOT run

⚠ COMMON MISTAKES

- Empty `catch` blocks that swallow errors
- Catching `Exception` before a more specific type
- Returning from `finally`

✓ TIPS & BEST PRACTICES

- Catch only what you can handle; let the rest bubble
- Wrap low-level exceptions with context
- Log the exception object — `__toString()` includes the trace

QUICK SUMMARY

- First matching `catch` wins
- `A | B` multi-catch
- `finally` always cleans up
- Wrap with `$previous`

Throwing & SPL Exceptions

SIGNALLING FAILURE

`throw` raises any object that implements `Throwable` (in practice, an `Exception` subclass). Since PHP 8 `throw` is an **expression**, so it can appear after `??`, `?:`, in arrow functions and in `match` arms. The **SPL exceptions** give you ready-made types with clear meanings: `LogicException` branches for bugs in the calling code, `RuntimeException` branches for problems only detectable at runtime.

NOTES

- Throw specific types so callers can catch selectively
- Messages should say what was wrong and with which value
- Built-in functions throw `ValueError` / `TypeError` for bad arguments
- Don't throw `Error` subclasses yourself — they're for the engine
- Document thrown types with `@throws`

THROWING THE RIGHT TYPE

```php
<?php
function setAge(int $age): void
{
    if ($age < 0 || $age > 150) {
        throw new InvalidArgumentException(
            "Age must be 0-150, got $age"
        );
    }
}

function withdraw(float $amt, float $balance): float
{
    if ($amt > $balance) {
        throw new UnderflowException('Insufficient funds');
    }
    return $balance - $amt;
}

$id = $_GET['id'] ?? throw new OutOfBoundsException('No id');
$fn = fn($x) => $x ?: throw new LengthException('Empty');

try {
    str_repeat('x', -1);
} catch (ValueError $e) {
    echo $e->getMessage();
    // str_repeat(): Argument #2 ($times) must be
    //   greater than or equal to 0
}
```

CODE EXPLAINED

→ `InvalidArgumentException` — caller passed a bad value

→ `UnderflowException` — a runtime condition, not a bug

→ `?? throw` — fail fast on a missing input

→ `throw` inside an arrow function body

→ Engine argument checks throw `ValueError`

LOGICEXCEPTION FAMILY

CLASS	MEANING
`LogicException`	Bug in program logic
`InvalidArgumentException`	Bad argument value
`DomainException`	Value outside valid domain
`LengthException`	Invalid length
`OutOfRangeException`	Bad index (compile-time)
`BadFunctionCallException`	Bad callback

RUNTIMEEXCEPTION FAMILY

CLASS	MEANING
`RuntimeException`	Runtime-only problem
`OutOfBoundsException`	Key/index not present
`OverflowException`	Container full
`UnderflowException`	Container empty
`RangeException`	Result out of range
`UnexpectedValueException`	Unexpected return value

ENGINE ERRORS

CLASS	THROWN WHEN
`TypeError`	Wrong type passed/returned
`ValueError`	Right type, invalid value
`ArgumentCountError`	Too few arguments
`ArithmeticError`	Bad math (shift by -1)
`UnhandledMatchError`	match had no arm
`JsonException`	JSON_THROW_ON_ERROR

⚠ COMMON MISTAKES

- Throwing plain `Exception` for everything
- Messages with no context ("Error occurred")
- Using exceptions for normal control flow (e.g. "not found" in a loop)

✓ TIPS & BEST PRACTICES

- Pick the most specific SPL class, or subclass it (Sheet 39)
- Include the bad value in the message
- Use `?? throw` for required inputs

QUICK SUMMARY

- `throw new X('msg')`
- `throw` is an expression (8.0)
- Logic = caller bug, Runtime = environment
- Engine throws `TypeError` / `ValueError`

Custom Exceptions

YOUR OWN EXCEPTION TYPES

Create a custom exception by extending `Exception` or, better, the closest SPL class. A custom type lets callers catch **your** failures specifically and can carry extra data such as an order ID or HTTP status. Give a package a common base class or **marker interface** so users can catch everything from it in one place. Pass the original exception as `$previous` to keep the full chain.

NOTES

- Constructor signature: `($message = '', $code = 0, ?Throwable $previous = null)`
- Named constructors (`static fromId()`) keep messages consistent
- Mark leaf exception classes `final` unless you expect subclasses
- Don't put secrets in messages; they end up in logs
- Keep exception classes in an `Exception/` namespace

A SMALL EXCEPTION HIERARCHY

```php
<?php
namespace Shop\Exception;

interface ShopException extends \Throwable {}   // marker

class OrderNotFound extends \RuntimeException
    implements ShopException
{
    public function __construct(
        public readonly int $orderId,
        ?\Throwable $previous = null,
    ) {
        parent::__construct("Order #$orderId not found",
            404, $previous);
    }

    public static function forId(int $id): self
    {
        return new self($id);
    }
}

try {
    throw OrderNotFound::forId(42);
} catch (OrderNotFound $e) {
    echo $e->getMessage(), ' ', $e->getCode();  // ... 404
    echo $e->orderId;                           // 42
} catch (ShopException $e) {
    // any other error from this package
}
```

CODE EXPLAINED

→ Marker interface groups all package exceptions

→ Extend the closest SPL type, not bare `Exception`

→ Promoted readonly property carries context

→ `parent::__construct()` sets message, code, previous

→ Catch the specific type or the package interface

DESIGNING EXCEPTIONS

CHOICE	RECOMMENDATION
`Base class`	Closest SPL exception
`Grouping`	Package marker interface
`Extra data`	Readonly properties
`Creation`	Static named constructors
`Naming`	Describe the problem (OrderNotFound)
`Code`	Optional; HTTP status or enum

CHAINING

METHOD	USE
`new X($m, $c, $prev)`	Wrap a lower error
`$e->getPrevious()`	Get wrapped error
`while ($e = $e->getPrevious())`	Walk the chain
`(string) $e`	Includes chain + traces
`$e->getTraceAsString()`	Readable stack

WHERE TO CATCH

LAYER	ACTION
`Data access`	Wrap driver errors
`Service`	Throw domain exceptions
`Controller`	Map to HTTP response
`Front controller`	Catch Throwable, log, 500
`CLI`	Catch, print, exit code

⚠ COMMON MISTAKES

- Overriding `getMessage()` — it's `final` ; pass the message to the parent
- Losing the original error by not passing `$previous`
- A custom class for every possible problem

✓ TIPS & BEST PRACTICES

- Create exceptions for failures callers will handle differently
- Name them after the problem, not the module
- Expose context via typed properties

QUICK SUMMARY

- Extend an SPL exception
- Marker interface per package
- Readonly context properties
- Always chain `$previous`

Global Error Handlers

set_error_handler() · set_exception_handler() · ErrorException · register_shutdown_function() · error_get_last() · trigger_error()

CATCHING WHAT SLIPS THROUGH

`set_error_handler()` intercepts warnings, notices and deprecations; the common pattern converts them to `ErrorException` so they behave like exceptions. `set_exception_handler()` receives any **uncaught** throwable — your last chance to log and show a friendly error page. True fatal errors (like running out of memory) can only be observed from a **shutdown function** using `error_get_last()`.

INSTALLING HANDLERS

```php
<?php
set_error_handler(function (
    int $no, string $msg, string $file, int $line,
): bool {
    if (!(error_reporting() & $no)) {
        return false;                 // silenced with @ or level off
    }
    throw new ErrorException($msg, 0, $no, $file, $line);
});

set_exception_handler(function (Throwable $e): void {
    error_log((string) $e);
    http_response_code(500);
    echo 'Something went wrong. Please try again.';
});

register_shutdown_function(function (): void {
    $err = error_get_last();
    $fatal = [E_ERROR, E_PARSE, E_CORE_ERROR];
    if ($err && in_array($err['type'], $fatal, true)) {
        error_log("Fatal: {$err['message']}");
    }
});

trigger_error('Use newApi() instead', E_USER_DEPRECATED);
echo $undefined;                      // now throws ErrorException
```

NOTES

- The error handler must return `true` to stop PHP's default handling
- Respect `error_reporting()` and the `@` operator inside your handler
- The exception handler ends the script after it runs
- `trigger_error()` raises `E_USER_*` diagnostics from your code
- Frameworks install these handlers for you

CODE EXPLAINED

→ Bitmask check honours `@` and `error_reporting`

→ `ErrorException` keeps the severity and location

→ Exception handler: log, send 500, show safe text

→ Shutdown function sees fatal errors after the fact

→ With the handler, warnings become exceptions

HANDLER FUNCTIONS

FUNCTION	HANDLES
`set_error_handler()`	Warnings, notices, deprecations
`set_exception_handler()`	Uncaught throwables
`register_shutdown_function()`	Script end (incl. fatal)
`error_get_last()`	Last error as array
`restore_error_handler()`	Pop previous handler
`trigger_error($m, $lvl)`	Raise E_USER_*

NOT CATCHABLE BY HANDLERS

ERROR	OBSERVE VIA
`E_ERROR (out of memory)`	shutdown function
`E_PARSE in main file`	server log
`E_CORE_ERROR`	shutdown function
`E_COMPILE_ERROR`	shutdown function
`Max execution time`	shutdown function

TRIGGER_ERROR LEVELS

LEVEL	USE
`E_USER_NOTICE`	Infor
`E_USER_WARNING`	Recc prob
`E_USER_DEPRECATED`	API g away
`E_USER_ERROR`	Dep in 8. thro inste

⚠ COMMON MISTAKES

- Ignoring `error_reporting()` — `@` stops working
- Echoing stack traces to users in production
- Using `E_USER_ERROR` (deprecated in 8.4)

✓ TIPS & BEST PRACTICES

- Install handlers first thing in your front controller
- Turn warnings into exceptions during development
- Log with full context; show users a generic message

QUICK SUMMARY

- `set_error_handler` → `ErrorException`
- `set_exception_handler` = last resort
- Shutdown function for fatals
- `trigger_error` for deprecations

Defining & Calling Functions

FUNCTIONS

`function name(params): type { … }` defines a named function. Top-level functions are **hoisted**: you can call them before the line that defines them in the same file. `return` sends a value back and ends the call; a function without `return` (or with a bare `return;`) gives `null`. Function names are global (within a namespace), case-insensitive, and can't be redeclared.

NOTES

- Functions defined inside `if` or another function exist only after that code runs
- Return several values as an array and destructure them
- `void` return type means "returns nothing"
- `never` return type means "always throws or exits"
- Built-in functions can't be redefined

DEFINE, CALL, RETURN

```php
<?php
echo greet('Ana');                  // works: hoisted

function greet(string $name): string
{
    return "Hello, $name!";
}

function logLine(string $msg): void
{
    echo date('H:i'), " $msg\n";  // no return value
}

function minMax(array $nums): array
{
    if ($nums === []) {
        return [null, null];        // early return
    }
    return [min($nums), max($nums)];
}

[$lo, $hi] = minMax([4, 1, 9]);    // 1, 9

if (!function_exists('str_slug')) {
    function str_slug(string $s): string { return strtolower($s); }
}
```

CODE EXPLAINED

→ Calling before the definition works for top-level functions

→ Return type after the colon

→ `void` functions can use bare `return;`

→ Return an array and destructure it at the call site

→ `function_exists()` guards optional helper definitions

ANATOMY

PART	ROLE
`function`	Keyword
`name`	camelCase or snake_case
`(params)`	Inputs (Sheet 42)
`: type`	Return type
`{ … }`	Body
`return`	Output and exit

RETURN TYPES

TYPE	MEANS
`string, int, …`	That type
`?string`	Type or null
`void`	No value
`never`	Doesn't return
`static / self`	Same class (methods)
`mixed`	Anything

INTROSPECTION

FUNCTION	RETURNS
`function_exists('f')`	bool
`is_callable('f')`	bool
`get_defined_functions()`	['internal', 'user']
`new ReflectionFunction('f')`	Details
`__FUNCTION__`	Current name

⚠ COMMON MISTAKES

- Defining the same function twice (e.g. a file included twice) — fatal error
- Echoing inside a function instead of returning the value
- Relying on hoisting for functions declared inside `if`

✓ TIPS & BEST PRACTICES

- Give functions one job and a verb name
- Declare parameter and return types
- Use `require_once` or autoloaded classes to avoid redeclaration

QUICK SUMMARY

- `function f(): T { }`
- Top-level functions are hoisted
- No return → `null`
- Return arrays for several values

Parameters & Arguments

default values · pass by reference & · named arguments · optional-before-required deprecation · nullable defaults

PASSING DATA IN

Parameters are **passed by value**: the function gets a copy (objects pass their handle, so methods can still change the object). Give a parameter a **default** to make it optional. Prefix with & to pass **by reference** so the function can modify the caller's variable. PHP 8 **named arguments** let you pass values by parameter name, skip optional ones, and make calls self-documenting.

NOTES

- Defaults must be constant expressions (or `new` objects, 8.1+)
- Optional parameters should come after required ones
- Too few arguments → `ArgumentCountError`; extras are ignored by user functions
- Named and positional args can mix; positional must come first
- Renaming a parameter is a breaking change once callers use names

DEFAULTS, REFERENCES & NAMED ARGS

```php
<?php
function makeTag(string $tag, string $text = '',
                 array $attrs = [], bool $close = true): string
{
    $a = '';
    foreach ($attrs as $k => $v) $a .= " $k=\"$v\"";
    return "<$tag$a>$text" . ($close ? "</$tag>" : '');
}

makeTag('p', 'Hi');                       // <p>Hi</p>
makeTag('br', close: false);              // <br>
makeTag(tag: 'a', text: 'Go', attrs: ['href' => '/']);

function addTax(float &$price, float $rate = 0.2): void
{
    $price *= 1 + $rate;                  // changes caller's var
}
$p = 10.0;
addTax($p);
echo $p;                                  // 12

htmlspecialchars($s, double_encode: false); // skip middle args
```

CODE EXPLAINED

→ Defaults make `$text`, `$attrs`, `$close` optional

→ Named argument skips all the defaults in between

→ Names can be given in any order

→ `&$price` writes back to `$p`

→ Named args work with built-in functions too

PARAMETER KINDS

SYNTAX	MEANING
`int $x`	Required
`int $x = 5`	Optional with default
`?int $x = null`	Optional, nullable
`int &$x`	By reference
`int ...$xs`	Variadic (Sheet 43)
`public int $x`	Promoted (constructors)

CALLING STYLES

CALL	NOTES
`f(1, 2)`	Positional
`f(b: 2, a: 1)`	Named, any order
`f(1, c: 3)`	Mixed: positional first
`f(...$args)`	Spread list
`f(...['a' => 1])`	Spread as named (8.1)

ERRORS

PROBLEM	RESULT
`Missing required arg`	ArgumentCo
`Wrong type`	TypeError
`Unknown named arg`	Error
`Same param twice`	Error
`Positional after named`	Compile erro

⚠ COMMON MISTAKES

- Required parameter after an optional one — deprecated
- Passing a literal to a by-reference parameter
- Long boolean parameter lists called positionally (`f(true, false, true)`)

✓ TIPS & BEST PRACTICES

- Use named arguments for booleans and option flags
- Return new values instead of using references
- Keep parameter lists short; group options into an object

QUICK SUMMARY

- By value by default
- Defaults → optional
- `&` → by reference
- Named args: `f(name: value)`

Variadic Parameters & Spread

…$args · spread in calls · string-key spread · func_get_args() · named + variadic

ANY NUMBER OF ARGUMENTS

A **variadic** parameter `...$items` collects all remaining arguments into an array. It must be the last parameter and can be typed (`int ...$nums`) so every element is checked. The same `...` operator **spreads** an array or Traversable into separate arguments at the call site. With string keys, spread values become named arguments, and a variadic collects unknown named arguments with their names as keys.

NOTES

- Only one variadic parameter, and it must be last
- A variadic can be by reference: `&...$refs`
- `func_get_args()` is the old way — prefer `...`
- Spreading an empty array passes no arguments
- `array_merge(...$arrays)` merges a list of arrays

COLLECTING AND SPREADING

```php
<?php
function sum(int ...$nums): int
{
    return array_sum($nums);
}
sum();                          // 0
sum(1, 2, 3);                   // 6
$list = [4, 5, 6];
sum(...$list);                  // 15
sum(1, ...$list, ...[7]);       // 23

function tag(string $name, string ...$classes): string
{
    return "<$name class=\"" . implode(' ', $classes) . '">';
}
tag('div', 'card', 'shadow');  // <div class="card shadow">

function opts(...$o) { return $o; }
opts(1, debug: true);           // [0 => 1, 'debug' => true]

$args = ['tag' => 'p', 'text' => 'Hi'];
makeTag(...$args);              // spread as named args

$all = array_merge(...$chunks);   // flatten one level
```

CODE EXPLAINED

→ `int ...$nums` — zero or more ints in an array

→ `...$list` at the call spreads elements as arguments

→ Mix fixed args with several spreads

→ Named extras land in the variadic with string keys

→ Spread a list of arrays into `array_merge()`

TWO USES OF ...

WHERE	MEANING
`function f(...$a)`	Collect (variadic)
`f(...$a)`	Spread into call
`[...$a, ...$b]`	Spread into array
`fn(...$a) => …`	Variadic arrow fn
`strlen(...)`	First-class callable (Sheet 48)

VARIADIC RULES

RULE	DETAIL
Position	Last parameter only
Typing	Each element checked
Default	Not allowed
Empty	Becomes []
Named extras	String keys in array

HANDY SPREADS

CODE	RESULT
`max(...$nums)`	Largest
`array_merge(...$lists)`	Flatten one level
`sprintf($fmt, ...$vals)`	Fill placeholders
`new Point(...$xy)`	Construct from array
`array_push($a, ...$more)`	Append many

⚠ COMMON MISTAKES

- Putting a parameter after the variadic
- Spreading an array with string keys into a function without those parameter names
- Using `func_get_args()` and losing type checks

✓ TIPS & BEST PRACTICES

- Type variadic parameters
- Use spread instead of `call_user_func_array()`
- Prefer an explicit array parameter when callers already have an array

QUICK SUMMARY

- `...$x` in params collects
- `...$x` in calls spreads
- Must be last
- String keys → named args

Parameter & Return Types

int|string · ?int · void · never · mixed · static · A&B · DNF types

TYPE DECLARATIONS

Types on parameters, return values and properties are **checked at runtime**: a wrong type throws `TypeError`. In the default **coercive mode**, scalar arguments are converted when safe (`'5'` → 5). With `declare(strict_types=1)` at the top of the **calling** file, no conversion happens (except int → float). Unions (`int|string`), intersections (A&B) and DNF types (`(A&B)|null`) describe richer contracts.

NOTES

- strict_types affects calls made *from* the file that declares it
- Return types are always checked against the function's own file mode
- `mixed` = any value including null; it can't be nullable
- `iterable` = `array|Traversable` (8.2 alias)
- `false`, `true` and `null` exist as standalone types (8.2)

TYPE FORMS

```php
<?php
declare(strict_types=1);

function area(float $w, float $h): float { return $w * $h; }
area(2, 3);              // int → float allowed: 6.0
// area('2', 3);         // TypeError in strict mode

function idOf(int|string $id): string { return (string) $id; }
function find(int $id): ?User { return null; }
function note(string $m): void { echo $m; }
function fail(string $m): never { throw new Exception($m); }

function save(Countable&Traversable $items): void {}
function load((Countable&Traversable)|null $x): void {}

class Builder {
    public function where(string $c): static { return $this; }
}
```

CODE EXPLAINED

- → Strict mode still widens `int` to `float`
- → `int|string` — union accepts either
- → `?User` — nullable shorthand
- → `A&B` — value must implement both
- → `static` return — the called class (fluent APIs)

SCALAR & SPECIAL TYPES

TYPE	ACCEPTS
`int / float`	Numbers
`string / bool`	Text / flags
`array`	Any array
`callable`	Callable values (not properties)
`iterable`	array or Traversable
`object`	Any object
`mixed`	Anything

COMPOSITE TYPES

SYNTAX	SINCE
`?T`	7.1
`A|B`	8.0
`mixed / static`	8.0
`never / A&B`	8.1
`(A&B)|C (DNF)`	8.2
`true / false / null`	8.2

COERCIVE VS STRICT

CALL	COERCIVE / STRICT
`f(int) with '5'`	5 / TypeError
`f(int) with 5.0`	5 / TypeError
`f(int) with 5.5`	Deprecated / TypeError
`f(float) with 5`	5.0 / 5.0
`f(string) with 5`	'5' / TypeError
`f(bool) with 1`	true / TypeError

⚠ COMMON MISTAKES

- Putting `declare(strict_types=1)` anywhere but the first statement
- Using `?mixed` — `mixed` already includes null
- Typing `callable` on a property — not allowed

✓ TIPS & BEST PRACTICES

- Enable strict types in every file
- Type every parameter, return and property
- Use PHPDoc generics for array contents (Sheet 98)

QUICK SUMMARY

- Runtime-checked types
- `strict_types` stops coercion
- Unions `|`, intersections `&`
- `void`, `never`, `static`

Variable Scope

WHERE VARIABLES LIVE

Each function has its own **local scope**: it can't see variables from the file or from its caller, and its variables disappear when it returns. Unlike many languages, PHP has **no block scope** — a variable created inside `if` or a loop is visible for the rest of the function. `static` variables keep their value between calls. **Superglobals** such as $_GET are visible everywhere.

NOTES

- The top level of a script is the *global* scope
- `global $x;` imports a global into a function (avoid)
- `$GLOBALS` is read-only as a whole array since PHP 8.1
- Closures capture outer variables only with `use` (Sheet 46)
- Arrow functions capture automatically (Sheet 47)

LOCAL, GLOBAL & STATIC

```php
<?php
$rate = 0.2;

function price(float $net): float
{
    // echo $rate;              // Warning: undefined
    global $rate;               // import (not recommended)
    return $net * (1 + $rate);
}

function priceBetter(float $net, float $rate): float
{
    return $net * (1 + $rate); // pass it in instead
}

function counter(): int
{
    static $n = 0;              // initialised once
    return ++$n;
}
counter(); counter();
echo counter();                 // 3

if (true) { $inside = 'visible'; }
echo $inside;                   // 'visible' — no block scope
```

CODE EXPLAINED

→ Functions can't see `$rate` without help

→ `global` links the local name to the global variable

→ Passing values in keeps functions testable

→ `static $n` survives across calls

→ Variables set inside `if` remain afterwards

SCOPES

SCOPE	CONTAINS
`Global`	Top-level script variables
`Function`	Params + locals
`Method`	Same, plus $this
`Closure`	Params + use() vars
`Arrow fn`	Params + auto-captured
`Superglobal`	Everywhere

SUPERGLOBALS

VARIABLE	HOLDS
`$_GET / $_POST`	Request data
`$_SERVER`	Server + request info
`$_COOKIE`	Cookies
`$_SESSION`	Session data
`$_FILES`	Uploads
`$_ENV`	Environment
`$GLOBALS`	All globals

STATIC VARIABLES

BEHAVIOUR	DETAIL
`Initialised`	Once, first call
`Lifetime`	Whole request
`Visible`	Only inside that function
`Initialiser`	Any expression (8.3+)
`In methods`	Shared by all instances

⚠ COMMON MISTAKES

- Expecting a function to see a variable from the calling code
- Relying on `global` — hidden dependencies
- Using `static` variables as a cache that never clears in long-running workers

✓ TIPS & BEST PRACTICES

- Pass dependencies as parameters
- Return results instead of writing to globals
- Keep top-level script code small

QUICK SUMMARY

- Functions have local scope
- No block scope
- `static` persists between calls
- Avoid `global`

Anonymous Functions & Closures

function () use ($x) · use (&$x) · Closure::bind() · bindTo() · static function · $this binding

FUNCTIONS AS VALUES

An **anonymous function** (`function () { … }`) is an expression that creates a `Closure` object. You can store it, pass it to other functions, and return it. A closure doesn't see outer variables automatically: list them in **`use (…)`**. `use ($x)` copies the value at creation time; `use (&$x)` binds by reference. Closures created inside a class method automatically get `$this`.

NOTES

- The `use` value is captured when the closure is created, not when it runs
- `static function () { … }` doesn't bind `$this` (slightly cheaper)
- `Closure::bind()` / `bindTo()` change `$this` and scope
- `$closure->call($obj, …)` binds and calls in one step
- Closures can be type-hinted as `Closure` or `callable`

USE, REFERENCES & BINDING

```php
<?php
$tax = 0.2;
$withTax = function (float $p) use ($tax): float {
    return $p * (1 + $tax);
};
$tax = 0.5;                         // closure still uses 0.2
echo $withTax(10);                  // 12

$count = 0;
$inc = function () use (&$count) { $count++; };
$inc(); $inc();
echo $count;                        // 2

function multiplier(int $n): Closure
{
    return function (int $x) use ($n) { return $x * $n; };
}
$triple = multiplier(3);
echo $triple(5);                    // 15

class Counter { private int $n = 41; }
$peek = function () { return ++$this->n; };
echo Closure::bind($peek, new Counter, Counter::class)(); // 42
```

CODE EXPLAINED

- → `use ($tax)` snapshots the value
- → `use (&$count)` shares the variable
- → Returning a closure creates a function factory
- → Each call to `multiplier()` captures its own `$n`
- → `bind()` gives access to private members of another class

CLOSURE SYNTAX

FORM	NOTES
`function () { }`	Anonymous function
`function () use ($a) { }`	Capture by value
`function () use (&$a) { }`	Capture by reference
`static function () { }`	No $this
`fn () => expr`	Arrow fn (Sheet 47)

CLOSURE METHODS

METHOD	DOES
`Closure::bind($f, $obj, $scope)`	Copy with new $this
`$f->bindTo($obj)`	Same, instance form
`$f->call($obj, ...$args)`	Bind + call
`Closure::fromCallable($c)`	Wrap a callable
`$f->__invoke(...)`	Same as $f(…)

COMMON USES

USE	EXAMPLE
`Callbacks`	array_map(…)
`Sorting`	usort($a, fu …)
`Factories`	return func
`Middleware`	$next($requ
`Lazy values`	fn() => expe

⚠ COMMON MISTAKES

- Expecting changes after creation to show up with `use ($x)`
- Forgetting `use` and getting "Undefined variable"
- Capturing large objects in long-lived closures — memory stays alive

✓ TIPS & BEST PRACTICES

- Prefer arrow functions for one-line callbacks
- Use `static` closures in classes when you don't need `$this`
- Capture by value unless you truly need shared state

QUICK SUMMARY

- `function () use ($x) { }`
- `use` copies; `&` shares
- Closures are objects
- `bind()` changes `$this`

Arrow Functions

SHORT CLOSURES (PHP 7.4)

`fn(params) => expression` is a compact closure whose body is **one expression** that is returned automatically. Arrow functions **capture outer variables by value automatically** — no `use` list. They can't modify outer variables and can't contain statements, so for multi-line logic use a regular closure. Types, defaults, by-reference parameters and variadics all work.

NOTES

- Captured values are copied when the arrow function is created
- Assigning to an outer variable inside has no outside effect
- Arrow functions can be nested; each captures from its parent
- `$this` is captured automatically inside methods
- `throw` is allowed as the expression (PHP 8)

ARROW FUNCTIONS IN PRACTICE

```php
<?php
$nums = [1, 2, 3, 4, 5];
$factor = 10;

$scaled = array_map(fn($n) => $n * $factor, $nums);
// [10, 20, 30, 40, 50] — $factor captured automatically

$evens = array_filter($nums, fn(int $n): bool => $n % 2 === 0);

usort($users, fn($a, $b) => $a->age <=> $b->age);

$adder = fn($x) => fn($y) => $x + $y;   // nested
echo $adder(2)(3);                      // 5

$total = 0;
$add = fn($n) => $total += $n;          // modifies a copy
$add(5);
echo $total;                            // 0

$get = fn(array $a, string $k) => $a[$k]
    ?? throw new OutOfBoundsException($k);
```

CODE EXPLAINED

- → No `use` needed — `$factor` is captured
- → Typed params and return type work like normal functions
- → Ideal inline comparator for `usort()`
- → Inner arrow fn captures `$x` from the outer one
- → Outer `$total` is untouched — capture is by value

ARROW VS CLOSURE

FEATURE	FN / FUNCTION
Body	One expression / statements
Capture	Automatic / use()
By-reference capture	No / use (&$x)
Implicit return	Yes / No
$this	Auto / Auto in methods
Since	7.4 / 5.3

GOOD FITS

TASK	EXAMPLE
Map	array_map(fn($x) => $x * 2, $a)
Filter	array_filter($a, fn($x) => $x > 0)
Sort	usort($a, fn($x, $y) => $x <=> $y)
Key extract	fn($u) => $u->id
Default	fn() => new Cache()

NOT ALLOWED

INSIDE FN	USE INSTEAD
Multiple statements	function () { }
foreach / if blocks	function or match
Modify outer var	use (&$x)
return keyword	(implicit)
echo statement	print() is an expression

⚠ COMMON MISTAKES

- Expecting `fn` to change outer variables
- Trying to write `return` inside an arrow function
- Cramming complex logic onto one line

✓ TIPS & BEST PRACTICES

- Default to `fn` for short callbacks
- Switch to a named function once logic grows
- Add types to arrow parameters for clarity

QUICK SUMMARY

- `fn($x) => expr`
- Auto-captures by value
- One expression only
- Great for map/filter/sort

Callables & First-Class Callables

THINGS YOU CAN CALL

A **callable** is any value PHP can call: a function name string, a closure, an `[object, 'method']` or `[Class::class, 'staticMethod']` array, a `'Class::method'` string, or an object with `__invoke()`. PHP 8.1's **first-class callable syntax** `strlen(...)` or `$obj->method(...)` creates a `Closure` directly, with IDE and static-analysis support — prefer it over strings and arrays.

NOTES

- `callable` can be used as a parameter type, not a property type
- String and array callables break silently when renamed — `(...)` doesn't
- `(...)` keeps the original scope, so private methods work
- `is_callable()` checks before calling
- Call any callable directly: `$f(…)`

CALLABLE FORMS

```php
<?php
class Str {
    public static function up(string $s): string
    { return strtoupper($s); }
    public function wrap(string $s): string
    { return "[$s]"; }
}
$str = new Str();

array_map('trim', [' a ', ' b ']);            // string
array_map([$str, 'wrap'], ['a']);             // [obj, method]
array_map([Str::class, 'up'], ['a']);         // static
array_map('Str::up', ['a']);                  // 'Class::method'

$len  = strlen(...);                          // Closure (8.1)
$wrap = $str->wrap(...);
$up   = Str::up(...);
echo $len('hello'), $wrap('x'), $up('y');     // 5[x]Y

class Doubler {
    public function __invoke(int $n) { return $n * 2; }
}
$d = new Doubler();
echo $d(21);                                  // 42

is_callable([$str, 'missing']);               // false
call_user_func($wrap, 'z');                   // '[z]'
```

CODE EXPLAINED

→ Strings name global functions

→ Arrays pair an object or class with a method name

→ `f(...)` turns any call target into a `Closure`

→ An invokable object is called like a function

→ `call_user_func()` is rarely needed now — call `$f()`

CALLABLE FORMS

FORM	EXAMPLE
`Function name`	'strlen'
`Closure`	fn($x) => $x
`Instance method`	[$obj, 'save']
`Static method`	[Foo::class, 'make']
`Static string`	'Foo::make'
`Invokable`	new Doubler()
`First-class`	strlen(…)

FIRST-CLASS SYNTAX

CODE	CREATES
`strlen(...)`	Closure for strlen
`$o->m(...)`	Closure bound to $o
`Foo::m(...)`	Closure for static m
`$f(...)`	Closure for callable $f
`new Foo(...)`	Not allowed

CALLING HELPERS

FUNCTION	USE
`$f($a, $b)`	Direct call
`$f(...$args)`	Spread args
`call_user_func($f, $a)`	Legacy style
`call_user_func_array($f, $args)`	Legacy spread
`is_callable($f)`	Check first
`Closure::fromCallable($f)`	Pre-8.1 conversion

⚠ COMMON MISTAKES

- Typing a property as `callable` — use `Closure`
- String callables that break after a rename
- Passing `[$this, 'privateMethod']` to outside code — not callable there

✓ TIPS & BEST PRACTICES

- Use `name(...)` syntax for callbacks
- Type properties as `Closure` and parameters as `callable`
- Use `__invoke` for single-action classes

QUICK SUMMARY

- Strings, arrays, closures, invokables
- `f(...)` → Closure (8.1)
- Call with `$f()`
- `is_callable()` to check

Recursion

FUNCTIONS THAT CALL THEMSELVES

A recursive function solves a problem by calling itself on a smaller part of it. Every recursive function needs a **base case** that returns without recursing. Recursion fits naturally on trees: nested arrays, directories, menus, JSON. PHP has **no tail-call optimisation** and no fixed recursion limit — very deep recursion exhausts memory — so convert deep or unbounded recursion into a loop with an explicit stack.

NOTES

- Xdebug limits nesting with `xdebug.max_nesting_level`
- Recursive closures need `use (&$fn)` or `static fn` via a variable
- Memoise repeated sub-results with a `static` array
- SPL `RecursiveIteratorIterator` walks trees without recursion in your code
- Generators with `yield from` make lazy recursive walks

RECURSIVE PATTERNS

```php
<?php
function factorial(int $n): int
{
    return $n <= 1 ? 1 : $n * factorial($n - 1);  // base case
}

function fib(int $n): int
{
    static $memo = [];
    if ($n < 2) return $n;
    return $memo[$n] ??= fib($n - 1) + fib($n - 2);
}
fib(90);                          // fast thanks to $memo

function flatten(array $a): array
{
    $out = [];
    foreach ($a as $v) {
        if (is_array($v)) array_push($out, ...flatten($v));
        else $out[] = $v;
    }
    return $out;
}
flatten([1, [2, [3, [4]]]]);      // [1, 2, 3, 4]

$walk = function (array $tree, int $d = 0) use (&$walk) {
    foreach ($tree as $k => $sub) {
        echo str_repeat('  ', $d), $k, "\n";
        $walk($sub, $d + 1);
    }
};
```

CODE EXPLAINED

→ Base case `$n <= 1` stops the recursion

→ `static $memo` caches results across calls

→ `??=` computes each value once

→ Spread the recursive result into the output

→ A closure refers to itself through `use (&$walk)`

RECURSION CHECKLIST

ITEM	WHY
`Base case`	Stops the recursion
`Progress`	Each call moves toward base
`Depth`	Stack grows per call
`Memo`	Avoid repeated work
`Iterative option`	For deep inputs

BUILT-IN RECURSIVE HELPERS

FUNCTION	DOES
`array_walk_recursive()`	Visit every leaf
`array_merge_recursive()`	Deep merge
`array_replace_recursive()`	Deep override
`count($a, COUNT_RECURSIVE)`	Count all levels
`RecursiveArrayIterator`	Iterate nested arrays
`RecursiveDirectoryIterator`	Walk folders

RECURSION → LOOP

RECURSIVE	ITERATIVE
`call f(child)`	push child on $stack
`return value`	accumulate in variable
`base case`	stack empty
`depth arg`	store [node, depth] pairs
`lazy walk`	generator + yield from

⚠ COMMON MISTAKES

- Missing or unreachable base case — runs out of memory
- Exponential recursion like naive `fib()` without memoisation
- Recursing over user-supplied deep data without a depth limit

✓ TIPS & BEST PRACTICES

- Write the base case first
- Use SPL recursive iterators for trees
- Add a `$maxDepth` parameter for untrusted input

QUICK SUMMARY

- Base case + smaller step
- No tail-call optimisation
- Memoise with `static`
- Use iterators/loops for deep data

Generators & yield

LAZY SEQUENCES

A function containing `yield` is a **generator**: calling it returns a `Generator` object without running the body. Each `foreach` step resumes the body until the next `yield`, which hands out one value. Only the current value lives in memory, so generators can process huge files or infinite sequences. `yield from` delegates to another generator or array; `return` provides a final value via `getReturn()`.

NOTES

- A generator can only be iterated once (no rewind after start)
- `yield $key => $value` controls the keys
- `$gen->send($x)` resumes and makes `yield` evaluate to `$x`
- `iterator_to_array($g, false)` ignores keys (avoids overwrites)
- Return type: `Generator`, `iterable` or `Iterator`

GENERATORS IN ACTION

```php
<?php
function readLines(string $path): Generator
{
    $fh = fopen($path, 'r');
    try {
        while (($line = fgets($fh)) !== false) {
            yield rtrim($line, "\n");     // one line at a time
        }
    } finally {
        fclose($fh);
    }
}
foreach (readLines('huge.log') as $n => $line) {
    if (str_contains($line, 'ERROR')) echo "$n: $line\n";
}

function naturals(): Generator {
    for ($i = 1; ; $i++) yield $i;      // infinite
}

function all(): Generator {
    yield 0;
    yield from [1, 2];                  // delegate
    yield from naturals();
}

function job(): Generator { yield 1; return 'done'; }
$g = job();
foreach ($g as $v) {}
echo $g->getReturn();                   // done
```

CODE EXPLAINED

→ Body pauses at each `yield`

→ `finally` closes the file even if the loop breaks early

→ Keys default to 0, 1, 2, …

→ An infinite generator is fine when the consumer stops

→ `getReturn()` reads the `return` value after completion

GENERATOR METHODS

METHOD	DOES
`current()`	Current yielded value
`key()`	Current key
`next()`	Resume to next yield
`send($v)`	Resume, yield returns $v
`valid()`	Not finished?
`getReturn()`	Final return value
`throw($e)`	Throw at the yield point

YIELD FORMS

SYNTAX	PRODUCES
`yield $v`	Value, auto key
`yield $k => $v`	Explicit key
`yield`	null value
`$x = yield $v`	Receives send()
`yield from $iter`	Everything from $iter
`return $r`	Final value

ARRAY VS GENERATOR

ASPECT	ARRAY / GENERATOR
`Memory`	All items / One item
`Reuse`	Many times / Once
`count()`	Yes / No
`Random access`	Yes / No
`Infinite`	No / Yes
`Start cost`	Builds all / Lazy

⚠ COMMON MISTAKES

- Iterating a generator twice — Exception: already closed
- `iterator_to_array()` on `yield from` output — duplicate keys overwrite
- Calling `count()` on a generator

✓ TIPS & BEST PRACTICES

- Stream big files and query results with generators
- Pass `false` to `iterator_to_array()` for lists
- Use `try/finally` inside generators for cleanup

QUICK SUMMARY

- `yield` makes a lazy iterator
- One value in memory at a time
- `yield from` delegates
- Iterate once only

Array Basics

[] · array() · indexed vs associative · key casting · count() · ordered hash map

ONE TYPE, MANY ROLES

A PHP array is an **ordered map**: every element has a key and a value, and elements keep insertion order. With automatic integer keys it acts as a list; with string keys it acts as a dictionary. Keys can only be `int` or `string` — other key types are cast. Arrays are **values**: assigning or passing one gives a copy (cheaply, using copy-on-write).

NOTES

- `[]` short syntax replaces `array()` (5.4+)
- Appending with `$a[] = x` uses the next integer key (max int key + 1)
- `array_is_list($a)` (8.1) checks for keys 0…n-1 in order
- Reading a missing key gives a Warning and `null`
- Trailing commas are allowed in array literals

CREATING AND READING ARRAYS

```php
<?php
$fruits = ['apple', 'banana', 'cherry'];   // list
$user = [                                  // map
    'name'  => 'Ana',
    'age'   => 30,
    'roles' => ['admin', 'editor'],        // nested
];

echo $fruits[0];              // apple
echo $user['name'];           // Ana
echo $user['roles'][1];       // editor
echo count($fruits);          // 3

$fruits[] = 'date';           // append, key 3
$user['email'] = 'a@x.io';    // add key

$mixed = [5 => 'a', 'b', '7' => 'c', true => 'd', 1.9 => 'e'];
// [5 => 'a', 6 => 'b', 7 => 'c', 1 => 'e']

var_dump(array_is_list($fruits));   // true
var_dump(array_is_list($mixed));    // false
```

CODE EXPLAINED

→ Without keys, PHP assigns 0, 1, 2, …

→ `=>` sets a key explicitly

→ Chain brackets for nested access

→ `$a[] =` appends with the next integer key

→ `'7'` → 7, `true` → 1, `1.9` → 1 (overwrites `'d'`)

KEY CASTING

KEY GIVEN	STORED AS
`'8'`	int 8
`'08'`	string '08'
`1.9`	int 1 (deprecated 8.1)
`true / false`	1 / 0
`null`	'' (empty string)
`array / object`	TypeError

BASIC OPERATIONS

CODE	DOES
`count($a)`	Number of elements
`$a[$k]`	Read
`$a[$k] = $v`	Write / add
`$a[] = $v`	Append
`unset($a[$k])`	Remove (keys not reindexed)
`isset($a[$k])`	Exists and not null

LIST VS MAP

ASPECT	LIST / MAP
Keys	0…n-1 / strings or sparse ints
Check	array_is_list() true / false
JSON	[…] / { … }
Loop	for or foreach / foreach
Sort	sort() / asort() or ksort()

⚠ COMMON MISTAKES

- Using float or bool keys and losing data to collisions
- Expecting `unset()` to reindex a list — use `array_values()`
- Reading `$a['missing']` without `??`

✓ TIPS & BEST PRACTICES

- Use short `[]` syntax and trailing commas
- Use `??` when a key may be missing
- Use objects/DTOs for fixed structures instead of big maps

QUICK SUMMARY

- Ordered int/string → value map
- List or dictionary
- Arrays copy on assign
- `count()`, `[]`, `unset()`

Adding & Removing Elements

$a[] = · array_push() · array_pop() · array_shift() · array_unshift() · unset() · array_splice()

CHANGING ARRAY CONTENTS

Append with `$a[] = $v` (fastest) or `array_push()` for several values. `array_pop()` removes from the end and `array_shift()` from the front; `array_unshift()` adds to the front. These functions take the array **by reference** and modify it in place. `unset($a[$k])` removes one key but leaves a gap in list indexes; `array_splice()` removes or inserts in the middle and **reindexes** numeric keys.

NOTES

- `array_shift()` / `array_unshift()` reindex numeric keys — O(n)
- `array_pop()` and `array_shift()` return `null` on an empty array
- Return value of `array_push()` is the new count
- Use `SplQueue` for heavy queue workloads (Sheet 60)
- `array_values()` renumbers keys 0…n-1

PUSH, POP, SHIFT, SPLICE

```php
<?php
$stack = [1, 2];
$stack[] = 3;                    // [1, 2, 3]
array_push($stack, 4, 5);        // [1, 2, 3, 4, 5]
$top = array_pop($stack);        // 5; [1, 2, 3, 4]

$queue = ['a', 'b', 'c'];
$first = array_shift($queue);    // 'a'; ['b', 'c']
array_unshift($queue, 'z');      // ['z', 'b', 'c']

$list = ['a', 'b', 'c', 'd'];
unset($list[1]);                 // [0 => 'a', 2 => 'c', 3 => 'd']
$list = array_values($list);     // ['a', 'c', 'd']

$n = [1, 2, 3, 4, 5];
$removed = array_splice($n, 1, 2);        // [2, 3]
// $n = [1, 4, 5]
array_splice($n, 1, 0, ['x', 'y']);       // insert
// $n = [1, 'x', 'y', 4, 5]
array_splice($n, -1, 1, 'last');          // replace end
```

CODE EXPLAINED

- → `$a[] =` and `array_push()` add to the end
- → `array_pop()` returns the removed value
- → `array_shift()` takes the first element
- → `unset()` leaves a hole; `array_values()` fixes it
- → `array_splice($a, offset, length, replacement)`

ENDS OF AN ARRAY

FUNCTION	EFFECT
`$a[] = $v`	Append one
`array_push($a, ...$v)`	Append many
`array_pop($a)`	Remove last
`array_unshift($a, ...$v)`	Prepend
`array_shift($a)`	Remove first
`array_key_last($a)`	Last key (no change)

MIDDLE OF AN ARRAY

FUNCTION	EFFECT
`unset($a[$k])`	Remove key
`array_splice($a, $i, $n)`	Remove n at i
`array_splice($a, $i, 0, $x)`	Insert at i
`array_slice($a, $i, $n)`	Copy part (Sheet 56)
`array_values($a)`	Reindex
`array_diff_key($a, $rm)`	Remove keys

STACK & QUEUE IDIOMS

STRUCTURE	ADD / REMOVE
`Stack (LIFO)`	$a[] / array_pop
`Queue (FIFO)`	$a[] / array_shift
`Deque`	push/unshift / pop/shift
`Big queue`	SplQueue
`Priority`	SplPriorityQueue

⚠ COMMON MISTAKES

- `array_push($a, $b)` where `$b` is an array — nests it
- Looping with `for` over a list after `unset()` — missing indexes
- Using `array_shift()` in a loop over a huge array — slow

✓ TIPS & BEST PRACTICES

- Prefer `$a[] = $v` for single appends
- Reindex with `array_values()` after removals
- Use `array_splice()` for in-place middle edits

QUICK SUMMARY

- Append `$a[]`, `array_push`
- `array_pop` / `array_shift`
- `unset` leaves gaps
- `array_splice` edits the middle

Searching Arrays

in_array() · array_search() · array_key_exists() · isset vs array_key_exists · array_find() · array_any() · array_all()

FINDING VALUES AND KEYS

`in_array()` tells you whether a value exists; `array_search()` returns its **key** (or `false`). Both use loose comparison unless you pass `true` as the third argument — always do. For keys, `isset()` is fastest but treats `null` values as missing; `array_key_exists()` doesn't. PHP 8.4 added `array_find()`, `array_find_key()`, `array_any()` and `array_all()` for callback-based searches.

NOTES

- `array_search()` can return `0` — compare with `!== false`
- Searching values is O(n); key lookups are O(1)
- For many lookups, flip the list once: `isset(array_flip($list)[$v])`
- `array_keys($a, $v)` returns every key holding `$v`
- `array_find()` returns `null` when nothing matches

SEARCH FUNCTIONS

```php
<?php
$ids = [10, 20, 30];
in_array('20', $ids);           // true  (loose)
in_array('20', $ids, true);     // false (strict)
array_search(30, $ids, true);   // 2
array_search(99, $ids, true);   // false

$cfg = ['debug' => null, 'env' => 'prod'];
isset($cfg['debug']);            // false (value is null)
array_key_exists('debug', $cfg); // true

$users = [
    ['id' => 1, 'name' => 'Ana', 'admin' => true],
    ['id' => 2, 'name' => 'Bo',  'admin' => false],
];
// PHP 8.4
$bo  = array_find($users, fn($u) => $u['name'] === 'Bo');
$key = array_find_key($users, fn($u) => $u['id'] === 2); // 1
array_any($users, fn($u) => $u['admin']);    // true
array_all($users, fn($u) => $u['admin']);    // false

$col = array_column($users, 'name', 'id');   // [1=>'Ana', 2=>'Bo']
```

CODE EXPLAINED

- → Third argument `true` = strict comparison
- → `array_search` gives the key, not the value
- → `isset` ignores keys whose value is `null`
- → `array_find` returns the first matching element
- → `array_column` builds a lookup table for O(1) access

VALUE SEARCH

FUNCTION	RETURNS
`in_array($v, $a, true)`	bool
`array_search($v, $a, true)`	key or false
`array_keys($a, $v, true)`	all keys
`array_find($a, $fn)`	value or null (8.4)
`array_find_key($a, $fn)`	key or null (8.4)
`array_filter($a, $fn)`	all matches

KEY CHECKS

CHECK	NULL VALUE?
`isset($a[$k])`	false
`array_key_exists($k, $a)`	true
`$a[$k] ?? $d`	uses $d
`array_key_first($a)`	first key
`array_key_last($a)`	last key

PREDICATES (8.4)

FUNCTION	TRUE WHEN
`array_any($a, $fn)`	At least one matches
`array_all($a, $fn)`	All match
`!array_any(...)`	None match
`Callback args`	($value, $key)
`Empty array`	any → false, all → true

⚠ COMMON MISTAKES

- Loose `in_array()` — `in_array('abc', [0])` was true in PHP 7
- `if (array_search(...))` when the match is at key 0
- Using `isset()` on keys that may hold `null`

✓ TIPS & BEST PRACTICES

- Always pass `true` for strict search
- Index data by key for repeated lookups
- Use the 8.4 helpers instead of `foreach` + `break`

QUICK SUMMARY

- `in_array` / `array_search` (strict!)
- `isset` vs `array_key_exists`
- 8.4: `array_find`, `array_any`, `array_all`
- Lookups by key are fastest

Transforming Arrays

MAP, FILTER, REDUCE

`array_map()` builds a new array by applying a callback to each element; `array_filter()` keeps elements whose callback returns truthy (or non-falsy values with no callback); `array_reduce()` folds everything into one value. These return new arrays and leave the input alone. `array_walk()` instead modifies elements in place via a by-reference parameter. Note the **inconsistent argument order** between them.

NOTES

- `array_map($fn, $a)` — callback first; `array_filter($a, $fn)` — array first
- `array_map` keeps keys with one array; reindexes with several
- `array_filter` **preserves keys** — wrap in `array_values()` for a list
- `ARRAY_FILTER_USE_KEY` / `_USE_BOTH` pass keys to the callback
- `array_map(null, $a, $b)` zips arrays together

PIPELINES ON ARRAYS

```php
<?php
$nums = [1, 2, 3, 4, 5, 6];

$squares = array_map(fn($n) => $n ** 2, $nums);
$evens   = array_filter($nums, fn($n) => $n % 2 === 0);
// [1 => 2, 3 => 4, 5 => 6]  keys kept!
$evens   = array_values($evens);          // [2, 4, 6]
$sum     = array_reduce($nums, fn($c, $n) => $c + $n, 0);

$clean = array_filter(['a', '', null, 0, 'b']);  // ['a', 4 => 'b']

$prices = ['tea' => 3, 'cake' => 5];
array_walk($prices, function (&$p, $item) { $p *= 1.2; });

$rows = [['id' => 7, 'name' => 'Ana'], ['id' => 9, 'name' => 'Bo']];
array_column($rows, 'name');              // ['Ana', 'Bo']
array_column($rows, 'name', 'id');        // [7 => 'Ana', 9 => 'Bo']
array_combine(['a', 'b'], [1, 2]);        // ['a' => 1, 'b' => 2]
array_map(null, [1, 2], ['x', 'y']);      // [[1,'x'], [2,'y']]
```

CODE EXPLAINED

- → `array_map` returns the transformed copy
- → `array_filter` keeps original keys
- → `array_reduce($a, $fn, $initial)` — carry, then item
- → No callback: `array_filter` removes falsy values
- → `array_column` picks one field, optionally keyed

CORE FUNCTIONS

FUNCTION	RETURNS
`array_map($fn, $a)`	Transformed copy
`array_filter($a, $fn)`	Subset (keys kept)
`array_reduce($a, $fn, $init)`	Single value
`array_walk($a, $fn)`	true; edits in place
`array_column($a, $col, $key)`	Column values
`array_combine($keys, $vals)`	Map from two lists

USEFUL VARIANTS

CODE	EFFECT
`array_filter($a)`	Drop falsy
`array_filter($a, $f, ARRAY_FILTER_USE_KEY)`	Test keys
`array_filter($a, $f, ARRAY_FILTER_USE_BOTH)`	($v, $k)
`array_map($f, $a, $b)`	Parallel arrays
`array_map(null, $a, $b)`	Zip
`array_flip($a)`	Swap keys/values

AGGREGATES

FUNCTION	RESULT
`array_sum($a)`	Total
`array_product($a)`	Product
`count($a)`	Size
`max($a) / min($a)`	Extremes
`array_count_values($a)`	Frequencies
`implode(',', $a)`	Join

⚠ COMMON MISTAKES

- Swapping argument order between `array_map` and `array_filter`
- Assuming `array_filter` output is a list
- Forgetting `&` in `array_walk` callbacks — nothing changes

✓ TIPS & BEST PRACTICES

- Chain with variables or the `|>` pipe (8.5)
- Use `array_column` for lookup maps
- Use a plain `foreach` when the logic gets long

QUICK SUMMARY

- map → transform
- filter → subset (keys kept)
- reduce → one value
- `array_column` for fields

Sorting Arrays

ORDERING ARRAYS

Sort functions modify the array **in place** and return `true`. The name tells you what they do: plain `sort` sorts values and **reindexes**; `a`-prefixed functions keep key ⇄ value association; `k` functions sort by key; `r` means reverse; `u` means you supply a comparison callback, usually written with `<=>`. Since PHP 8.0 all sorts are **stable**: equal elements keep their original order.

NOTES

- `sort()` throws away string keys
- Callbacks return negative, zero or positive — `<=>` does this
- Sort by several fields: `[$a->x, $a->y] <=> [$b->x, $b->y]`
- `SORT_NATURAL` orders `img2` before `img10`
- `array_multisort()` sorts parallel arrays together

SORTING EXAMPLES

```php
<?php
$n = [3, 1, 2];
sort($n);                       // [1, 2, 3]
rsort($n);                      // [3, 2, 1]

$ages = ['Bo' => 30, 'Ana' => 25, 'Cy' => 35];
asort($ages);                   // by value, keep keys
// ['Ana' => 25, 'Bo' => 30, 'Cy' => 35]
ksort($ages);                   // by key
arsort($ages);                  // by value, descending

$users = [
    ['name' => 'Bo',  'age' => 30],
    ['name' => 'Ana', 'age' => 30],
    ['name' => 'Cy',  'age' => 25],
];
usort($users, fn($a, $b) =>
    [$a['age'], $a['name']] <=> [$b['age'], $b['name']]);
// Cy 25, Ana 30, Bo 30

$files = ['img12', 'img10', 'IMG2'];
sort($files, SORT_NATURAL | SORT_FLAG_CASE);
// ['IMG2', 'img10', 'img12']
```

CODE EXPLAINED

→ `sort` / `rsort` reindex 0…n-1

→ `asort` sorts values but keeps keys

→ `ksort` orders by key

→ Array `<=>` compares element by element — multi-field sort

→ Flags combine natural order with case-insensitivity

SORT FUNCTIONS

FUNCTION	SORTS BY / KEYS
`sort / rsort`	value / reindexed
`asort / arsort`	value / kept
`ksort / krsort`	key / kept
`usort`	callback on values / reindexed
`uasort`	callback on values / kept
`uksort`	callback on keys / kept

SORT FLAGS

FLAG	COMPARES
`SORT_REGULAR`	Default rules
`SORT_NUMERIC`	As numbers
`SORT_STRING`	As strings
`SORT_NATURAL`	Human order
`SORT_FLAG_CASE`	+ case-insensitive
`SORT_LOCALE_STRING`	Locale rules

COMPARATOR PATTERNS

CALLBACK	ORDER
`$a <=> $b`	Ascending
`$b <=> $a`	Descending
`$a->x <=> $b->x`	By property
`[$a->x, $a->y] <=> [$b->x, $b->y]`	Multi-key
`strcmp($a, $b)`	Binary string
`strnatcasecmp($a, $b)`	Natural, no case

⚠ COMMON MISTAKES

- `$sorted = sort($a);` — `$sorted` is `true`
- Using `sort()` on a map and losing keys
- Returning a bool from a `usort` callback

✓ TIPS & BEST PRACTICES

- Pick the `a` / `k` variant whenever keys matter
- Write comparators with `<=>`
- Sort a copy if the original order matters later

QUICK SUMMARY

- Sorts modify in place
- `a` keeps keys, `k` by key
- `u` + `<=>` callback
- Stable since 8.0

Slicing, Merging & Building

COMBINING AND BUILDING ARRAYS

`array_slice()` copies a portion without changing the original. To combine arrays, `array_merge()` **appends** numeric keys (renumbering them) and **overwrites** duplicate string keys with later values. The `+` operator is different: it keeps the **left** value for duplicate keys of either type. The spread syntax `[...$a, ...$b]` behaves like `array_merge()`. Builders like `range()` and `array_fill()` create arrays from scratch.

NOTES

- `array_slice($a, $i, $n, true)` preserves integer keys
- `+` is handy for defaults: `$opts + $defaults`
- `array_replace()` overwrites by key without renumbering
- `array_chunk($a, $n, true)` keeps keys in each chunk
- `range('a', 'e')` works on single characters

SLICE, MERGE, BUILD

```php
<?php
$a = ['x', 'y', 'z', 'w'];
array_slice($a, 1, 2);        // ['y', 'z']
array_slice($a, -2);          // ['z', 'w']

$m1 = ['a' => 1, 'b' => 2, 5 => 'five'];
$m2 = ['b' => 20, 'c' => 30, 5 => 'FIVE'];
array_merge($m1, $m2);
// ['a'=>1, 'b'=>20, 0=>'five', 'c'=>30, 1=>'FIVE']
$m1 + $m2;
// ['a'=>1, 'b'=>2, 5=>'five', 'c'=>30]
[...$m1, ...$m2];             // same as array_merge

$defaults = ['color' => 'blue', 'size' => 'M'];
$opts = ['size' => 'L'] + $defaults;  // size L, color blue

range(1, 10, 3);              // [1, 4, 7, 10]
range('a', 'e');              // ['a','b','c','d','e']
array_fill(0, 3, 0);          // [0, 0, 0]
array_fill_keys(['a', 'b'], 0);       // ['a'=>0, 'b'=>0]
array_chunk([1, 2, 3, 4, 5], 2);      // [[1,2],[3,4],[5]]
array_pad([1, 2], 4, 0);              // [1, 2, 0, 0]
```

CODE EXPLAINED

→ Negative offset slices from the end

→ `array_merge` : later string keys win, ints renumbered

→ `+` : first array wins for any duplicate key

→ `+` with defaults on the right fills gaps

→ `range(start, end, step)`

COMBINING

METHOD	DUPLICATE KEY → / INT KEYS
`array_merge($a, $b)`	later wins / renumbered
`[...$a, ...$b]`	later wins / renumbered
`$a + $b`	first wins / kept
`array_replace($a, $b)`	later wins / kept
`array_merge_recursive()`	values merged into arrays

BUILDING

FUNCTION	RESULT
`range(1, 5)`	[1..5]
`range(0, 1, 0.25)`	Floats
`array_fill(5, 2, 'x')`	[5=>'x', 6=>'x']
`array_fill_keys($keys, $v)`	Same value per key
`array_pad($a, $n, $v)`	Grow to n
`str_split / explode`	From strings

SPLITTING

FUNCTION	RETURNS
`array_slice($a, $i, $n)`	Part (copy)
`array_chunk($a, $n)`	List of chunks
`array_splice($a, …)`	Removed part (in place)
`array_keys($a)`	Keys only
`array_values($a)`	Values, reindexed
`array_filter twice`	Partition by a test

⚠ COMMON MISTAKES

- Using `+` to concatenate lists — `[1] + [2]` is `[1]`
- Expecting `array_merge` to keep numeric keys
- Merging inside a loop (`$all = array_merge($all, $x)`) — O(n²)

✓ TIPS & BEST PRACTICES

- Collect pieces in an array, then `array_merge(...$pieces)` once
- Use `$opts + $defaults` for option arrays
- Use `array_replace` to override by key

QUICK SUMMARY

- `array_slice` copies part
- `array_merge` renumbers, later wins
- `+` keeps first
- `range` , `array_fill` , `array_chunk`

Set Operations & Keys

ARRAYS AS SETS

PHP has no built-in set type, but arrays work well. `array_unique()` removes duplicate values; `array_diff()` returns values of the first array not found in the others; `array_intersect()` returns values found in all. `_key` variants compare keys and `_assoc` variants compare both. For fast membership tests, store items as **keys** (`$set[$item] = true`). PHP 8.5 adds `array_first()` and `array_last()`.

NOTES

- These functions compare as strings (`(string) $a === (string) $b`) by default
- Results keep the keys of the first array
- `array_unique()` keeps the first occurrence
- `u` variants (`array_udiff`) take a comparison callback
- `array_key_first()` / `array_key_last()` exist since 7.3

SET OPERATIONS

```php
<?php
$a = ['red', 'green', 'blue', 'green'];
$b = ['green', 'yellow'];

array_unique($a);           // [0=>'red', 1=>'green', 2=>'blue']
array_diff($a, $b);         // [0=>'red', 2=>'blue']
array_intersect($a, $b);    // [1=>'green', 3=>'green']
array_values(array_unique(array_merge($a, $b)));  // union

$x = ['id' => 1, 'name' => 'Ana', 'pw' => 'secret'];
array_diff_key($x, ['pw' => 0]);           // drop 'pw'
array_intersect_key($x, array_flip(['id', 'name']));  // whitelist

$set = [];
foreach (['a', 'b', 'a'] as $tag) {
    $set[$tag] = true;      // keys = unique members
}
isset($set['a']);           // O(1) membership
array_keys($set);           // ['a', 'b']

array_first(['x' => 10, 'y' => 20]);   // 10 (8.5)
array_last(['x' => 10, 'y' => 20]);    // 20 (8.5)
```

CODE EXPLAINED

- → Duplicates removed; first keys kept
- → `diff` = in first, not in others
- → `intersect` = in first and all others
- → `array_intersect_key` + `array_flip` = pick allowed fields
- → Keys-as-set gives constant-time lookups

VALUE COMPARISONS

FUNCTION	KEEPS
`array_unique($a)`	First of each value
`array_diff($a, $b)`	In $a, not $b
`array_intersect($a, $b)`	In both
`array_udiff($a, $b, $fn)`	Custom compare
`array_diff_assoc($a, $b)`	Key+value differ

KEY COMPARISONS

FUNCTION	KEEPS
`array_diff_key($a, $b)`	Keys not in $b
`array_intersect_key($a, $b)`	Keys in $b
`array_flip($a)`	Values become keys
`array_keys($a)`	All keys
`array_key_first($a)`	First key
`array_key_last($a)`	Last key

FIRST / LAST ELEMENT

CODE	NOTES
`array_first($a)`	Value or null (8.5)
`array_last($a)`	Value or null (8.5)
`$a[array_key_first($a)]`	7.3+
`reset($a)`	Moves pointer, needs variable
`end($a)`	Same, last element

⚠ COMMON MISTAKES

- Expecting `array_diff` results to be reindexed
- Running `array_unique` on arrays of arrays — string conversion fails
- Using `in_array` in a loop instead of a keyed set

✓ TIPS & BEST PRACTICES

- Model sets as `[$member => true]`
- Whitelist input fields with `array_intersect_key`
- Wrap results in `array_values()` when you need a list

QUICK SUMMARY

- `array_unique`, `diff`, `intersect`
- `_key` / `_assoc` variants
- Keys make fast sets
- `array_first` / `array_last` (8.5)

Array Destructuring

[$a, $b] = · list() · ['x' => $x] = · nested · swap · foreach destructuring

UNPACKING ARRAYS INTO VARIABLES

Destructuring assigns array elements to several variables at once. Use `[$a, $b] = $arr` for lists (or the older `list($a, $b)`), and `['key' => $var] = $arr` for maps. You can skip positions, nest patterns, assign by reference and destructure directly in `foreach`. It's the natural way to receive multiple return values from a function.

NOTES

- Missing keys produce a Warning and `null`
- You can't mix keyed and positional entries in one pattern
- Skip elements with empty slots: `[, $second] = $a`
- References: `[&$a, $b] = $arr` (7.3+)
- Works with any array, not just lists

DESTRUCTURING PATTERNS

```php
<?php
[$first, $second] = ['a', 'b'];
[, , $third] = [1, 2, 3];            // skip two

['name' => $name, 'age' => $age] = ['name' => 'Ana', 'age' => 30];

[$a, $b] = [1, 2];
[$a, $b] = [$b, $a];                 // swap: a=2, b=1

[[$x1, $y1], [$x2, $y2]] = [[0, 0], [3, 4]];    // nested

function stats(array $n): array {
    return ['min' => min($n), 'max' => max($n)];
}
['min' => $lo, 'max' => $hi] = stats([3, 9, 1]);

$points = [['x' => 1, 'y' => 2], ['x' => 3, 'y' => 4]];
foreach ($points as ['x' => $x, 'y' => $y]) {
    echo "($x, $y) ";
}

[$k, $v] = explode('=', 'color=red', 2);   // 'color', 'red'
```

CODE EXPLAINED

→ Positional pattern matches keys 0, 1, …

→ Empty slots skip values

→ Keyed pattern can list keys in any order

→ Swap without a temp variable

→ `explode` + destructuring parses key=value pairs

PATTERN FORMS

PATTERN	MATCHES
`[$a, $b]`	Keys 0, 1
`list($a, $b)`	Same (old syntax)
`[, $b]`	Skip key 0
`['k' => $v]`	Key 'k'
`[[$a], [$b]]`	Nested
`[&$a]`	By reference

WHERE IT WORKS

CONTEXT	EXAMPLE
`Assignment`	[$a, $b] = f()
`foreach`	foreach ($rows as [$id, $n])
`Keyed foreach`	foreach ($r as ['id' => $id])
`Swap`	[$a, $b] = [$b, $a]
`Parsing`	[$h, $m] = explode(':', $t)

EDGE CASES

SITUATION	RESULT
`Fewer elements`	null + Warning
`Extra elements`	Ignored
`Mixed keyed/positional`	Compile error
`Non-array right side`	Variables become null
`String on right`	null (strings unsupported)

⚠ COMMON MISTAKES

- Destructuring a map positionally — keys 0, 1 don't exist
- Not passing a limit to `explode` before destructuring
- Mixing `[$a, 'k' => $b]` in one pattern

✓ TIPS & BEST PRACTICES

- Return keyed arrays and destructure by name
- Use destructuring in `foreach` over DB rows
- Prefer small value objects for widely shared shapes

QUICK SUMMARY

- `[$a, $b] = $list`
- `['k' => $v] = $map`
- Skip, nest, swap
- Works in `foreach`

Multidimensional Arrays

nested arrays · array_column() · grouping by key · matrix loops · deep copy behavior · array_walk_recursive()

ARRAYS OF ARRAYS

Arrays can contain other arrays to any depth: tables of rows, trees of categories, matrices of numbers. Access nested values by chaining brackets, and use ?? to avoid warnings on missing levels. Because arrays are values, copying a nested array copies everything (lazily) — but arrays of **objects** copy only the handles. Common jobs are extracting a column, grouping rows by a field, and walking all leaves.

NOTES

- Writing to a missing nested key auto-creates the levels
- `$a['x']['y'] ?? null` is safe even if `'x'` is missing
- `count($a, COUNT_RECURSIVE)` counts all elements at all levels
- Use `json_encode` with `JSON_PRETTY_PRINT` to inspect deep data
- Large row sets: prefer generators over loading everything

WORKING WITH NESTED DATA

```php
<?php
$orders = [
    ['id' => 1, 'user' => 'ana', 'total' => 30],
    ['id' => 2, 'user' => 'bo',  'total' => 15],
    ['id' => 3, 'user' => 'ana', 'total' => 20],
];

echo $orders[1]['user'];                 // bo
$byId   = array_column($orders, null, 'id');  // index rows by id
$totals = array_column($orders, 'total');      // [30, 15, 20]

$byUser = [];
foreach ($orders as $o) {
    $byUser[$o['user']][] = $o;          // group (auto-creates)
}
// ['ana' => [..2 rows..], 'bo' => [..1 row..]]

$sum  = fn($rows) => array_sum(array_column($rows, 'total'));
$sums = array_map($sum, $byUser);        // ['ana' => 50, 'bo' => 15]

$grid = [[1, 2, 3], [4, 5, 6]];
foreach ($grid as $r => $row) {
    foreach ($row as $c => $val) { /* $grid[$r][$c] */ }
}
$city = $data['user']['address']['city'] ?? 'unknown';
```

CODE EXPLAINED

- → Chain brackets for nested access
- → `array_column($rows, null, 'id')` keys whole rows by id
- → `$g[$key][] =` builds groups in one pass
- → Combine `array_map` + `array_column` for per-group sums
- → `??` guards every missing level

NESTED ACCESS

CODE	NOTES
`$a['x']['y']`	Read (warns if missing)
`$a['x']['y'] ?? $d`	Safe read
`$a['x']['y'] = 1`	Creates levels
`isset($a['x']['y'])`	Safe check
`unset($a['x']['y'])`	Remove leaf

ROW HELPERS

CODE	RESULT
`array_column($r, 'c')`	One column
`array_column($r, 'c', 'id')`	id => c
`array_column($r, null, 'id')`	id => row
`usort($r, fn …)`	Sort rows
`array_filter($r, fn …)`	Filter rows

DEEP HELPERS

FUNCTION	DOES
`array_walk_recursive()`	Visit leaves
`array_merge_recursive()`	Merge deeply
`array_replace_recursive()`	Override deeply
`count($a, COUNT_RECURSIVE)`	Count all
`json_encode($a, JSON_PRETTY_PRINT)`	Inspect

⚠ COMMON MISTAKES

- Deep nesting where a class would be clearer
- Reading deep keys without `??`
- Expecting array copies to deep-copy objects inside them

✓ TIPS & BEST PRACTICES

- Index rows by id with `array_column(…, null, 'id')`
- Group in one `foreach` pass
- Convert stable shapes to DTO classes

QUICK SUMMARY

- Arrays nest to any depth
- `??` for safe deep reads
- `array_column` for rows
- Group with `$g[$k][] = $row`

SPL Data Structures

SPECIALISED CONTAINERS

The Standard PHP Library (SPL) ships classic data structures as classes. `SplStack` and `SplQueue` are doubly linked lists with stack and queue methods; `SplMinHeap`, `SplMaxHeap` and `SplPriorityQueue` keep items ordered by priority; `SplFixedArray` is a fixed-size integer-indexed array; `SplObjectStorage` maps **objects** to data. Use them when plain arrays get slow or unclear for the job.

NOTES

- SPL structures are objects — passed by handle, not copied
- `SplPriorityQueue` with equal priorities has no guaranteed order
- Iterating a heap or priority queue removes items
- `WeakMap` (8.0) is an object-keyed map that doesn't keep keys alive
- `ArrayObject` wraps an array in an object with array syntax

USING SPL CONTAINERS

```php
<?php
$stack = new SplStack();
$stack->push('a'); $stack->push('b');
echo $stack->pop();                 // b (LIFO)

$queue = new SplQueue();
$queue->enqueue('job1'); $queue->enqueue('job2');
echo $queue->dequeue();             // job1 (FIFO)

$pq = new SplPriorityQueue();
$pq->insert('low', 1);
$pq->insert('urgent', 10);
echo $pq->extract();                // urgent

$heap = new SplMinHeap();
foreach ([5, 1, 3] as $n) $heap->insert($n);
echo $heap->extract();              // 1

$fixed = new SplFixedArray(3);
$fixed[0] = 'x';                    // index 3 → RuntimeException

$seen = new SplObjectStorage();
$seen[$user] = ['visits' => 3];     // object as key
count($seen);                       // 1
```

CODE EXPLAINED

- → `push` / `pop` work from the top
- → `enqueue` / `dequeue` work first-in, first-out
- → Highest priority number comes out first
- → Min-heap always yields the smallest value
- → `SplObjectStorage` keys by object identity

STRUCTURES

CLASS	BEST FOR
`SplStack`	LIFO
`SplQueue`	FIFO
`SplDoublyLinkedList`	Both ends
`SplMinHeap` / `SplMaxHeap`	Smallest / largest first
`SplPriorityQueue`	Scheduling by priority
`SplFixedArray`	Fixed-size int-indexed array
`SplObjectStorage`	Object sets/maps

KEY METHODS

METHOD	CLASS
`push()` / `pop()` / `top()`	SplStack
`enqueue()` / `dequeue()`	SplQueue
`insert()` / `extract()`	Heaps, PQ
`isEmpty()` / `count()`	All
`attach()` / `contains()`	SplObjectStorage
`toArray()` / `setSize()`	SplFixedArray

ARRAY ALTERNATIVES

NEED	PLAIN ARRAY WAY
`Stack`	$a[] / array_pop
`Small queue`	$a[] / array_shift
`Priority`	usort each time
`Object keys`	spl_object_id($o) as key
`Weak object keys`	WeakMap

⚠ COMMON MISTAKES

- Iterating an `SplPriorityQueue` and expecting it to stay full
- Using `array_shift` on huge arrays instead of `SplQueue`
- Assuming FIFO order for equal priorities

✓ TIPS & BEST PRACTICES

- Reach for SPL when a plain array obscures intent
- Use `WeakMap` for per-object caches
- Add a sequence number to priorities for stable ordering

QUICK SUMMARY

- Stack, queue, heap, priority queue
- Fixed arrays and object maps
- Objects, not values
- Iteration may consume items

Iterators & Iterables

MAKING OBJECTS LOOPABLE

`foreach` works on arrays and on objects that implement **`Traversable`** — through either `Iterator` (you write `current`, `key`, `next`, `rewind`, `valid`) or `IteratorAggregate` (you return an iterator, often a generator, from `getIterator()`). `IteratorAggregate` + `yield` is almost always simpler. Add `Countable` for `count()` and `ArrayAccess` for `$obj[...]` syntax.

NOTES

- `Traversable` can't be implemented directly — use one of the two
- The `iterable` type accepts arrays and Traversables
- `iterator_to_array()` converts any Traversable to an array
- `iterator_count()` counts by walking the iterator
- SPL provides many ready iterators (filter, limit, caching)

ITERATORAGGREGATE WITH YIELD

```php
<?php
class Playlist implements IteratorAggregate, Countable
{
    /** @var list<string> */
    private array $songs = [];

    public function add(string $song): static
    {
        $this->songs[] = $song;
        return $this;
    }

    public function getIterator(): Generator
    {
        foreach ($this->songs as $i => $song) {
            yield $i + 1 => $song;          // 1-based keys
        }
    }

    public function count(): int { return count($this->songs); }
}

$p = (new Playlist())->add('Intro')->add('Outro');
foreach ($p as $n => $song) echo "$n. $song\n";
echo count($p);                             // 2

$long = new CallbackFilterIterator(
    new ArrayIterator(['a', 'bbb', 'cc']),
    fn($s) => strlen($s) > 1,
);
iterator_to_array($long, false);            // ['bbb', 'cc']
```

CODE EXPLAINED

→ `IteratorAggregate` needs only `getIterator()`

→ Returning a generator keeps the code short

→ Generators can set custom keys

→ `Countable` makes `count($p)` work

→ SPL iterators decorate other iterators lazily

INTERFACES

INTERFACE	YOU IMPLEMENT
`Iterator`	current, key, next, rewind, valid
`IteratorAggregate`	getIterator()
`Countable`	count()
`ArrayAccess`	offsetGet/Set/Exists/Unset
`Stringable`	__toString()

SPL ITERATORS

CLASS	DOES
`ArrayIterator`	Iterate an array
`LimitIterator`	Offset + limit
`CallbackFilterIterator`	Filter lazily
`CachingIterator`	Look ahead (hasNext)
`AppendIterator`	Chain iterators
`RecursiveIteratorIterator`	Flatten trees

HELPERS

FUNCTION	RESULT
`iterator_to_array($it)`	array
`iterator_to_array($it, false)`	list
`iterator_count($it)`	int
`iterator_apply($it, $fn)`	Call per item
`is_iterable($x)`	bool

⚠ COMMON MISTAKES

- Implementing `Iterator` by hand when a generator would do
- Returning an array from `getIterator()` — must be Traversable
- Iterating a generator-backed object twice when it's a one-shot generator

✓ TIPS & BEST PRACTICES

- Prefer `IteratorAggregate` + `yield`
- Type parameters as `iterable` to accept both arrays and iterators
- Use SPL iterators for lazy pipelines

QUICK SUMMARY

- `Iterator` or `IteratorAggregate`
- Generators make it easy
- `iterable` type
- `iterator_to_array()` to convert

Classes & Objects

CLASSES AND INSTANCES

A **class** defines properties (data) and methods (behaviour); `new` creates an **object** from it. Use `->` to reach an object's members and `$this` inside methods to refer to the current object. A variable holds a **handle** to the object, so assigning or passing it shares the same object — use `clone` for a copy. PHP 8.4 lets you call a method directly on `new Foo()` without wrapping it in parentheses.

NOTES

- One class per file, file named after the class (PSR-4)
- Class names are case-insensitive but write them consistently
- `ClassName::class` gives the fully qualified name as a string
- Objects are destroyed when the last reference goes away
- `stdClass` is the generic empty object (from `(object)` or `json_decode`)

DEFINE, CREATE, USE

```php
<?php
class Product
{
    public string $name;
    public float $price = 0.0;

    public function priceWithTax(float $rate = 0.2): float
    {
        return round($this->price * (1 + $rate), 2);
    }
}

$p = new Product();
$p->name = 'Mug';
$p->price = 8.5;
echo $p->priceWithTax();          // 10.2

$q = $p;                          // same object
$q->price = 10;
echo $p->price;                   // 10

$copy = clone $p;                 // independent copy
var_dump($p instanceof Product);  // true
echo Product::class;              // 'Product'

echo new Product()->priceWithTax(); // PHP 8.4: no extra ()
```

CODE EXPLAINED

→ Typed properties with optional defaults

→ `$this->price` reads the current object's property

→ Assignment copies the handle, not the object

→ `clone` makes a shallow copy

→ PHP 8.4 allows member access right after `new`

SYNTAX

CODE	MEANING
`class Foo { }`	Declare
`new Foo()`	Instantiate
`$o->prop`	Property
`$o->method()`	Method call
`$this`	Current object
`Foo::class`	Name string

OBJECT FUNCTIONS

FUNCTION	RETURNS
`get_class($o)`	Class name
`$o::class`	Class name (8.0)
`get_object_vars($o)`	Accessible props
`method_exists($o, 'm')`	bool
`property_exists($o, 'p')`	bool
`spl_object_id($o)`	Unique int id

VALUE VS HANDLE

ACTION	EFFECT
`$b = $a`	Same object
`$b = clone $a`	Shallow copy
`f($a)`	Function can modify object
`$a == $b`	Same class + props
`$a === $b`	Same instance

⚠ COMMON MISTAKES

- Expecting `$b = $a` to copy an object
- Using `$this` in a static method — Error
- Accessing a typed property before it's set

✓ TIPS & BEST PRACTICES

- Initialise required data in the constructor (Sheet 64)
- Keep properties private and expose methods
- Prefer immutable objects where practical

QUICK SUMMARY

- `class` defines, `new` creates
- `->` and `$this`
- Variables hold handles
- `clone` to copy

Properties & Readonly

typed properties · defaults · readonly (8.1) · readonly classes (8.2) · dynamic properties deprecated · #[AllowDynamicProperties]

OBJECT STATE

Properties should be **typed**. A typed property without a default starts **uninitialized** and throws if read before it's assigned. `readonly` properties (8.1) can be set **once**, from inside the class scope, and never changed — perfect for value objects. A `readonly class` (8.2) makes every property readonly. Creating undeclared **dynamic properties** is deprecated since 8.2.

NOTES

- Readonly properties must be typed and have no default
- A readonly property holding an object can still mutate that object
- 8.3 allows re-initialising readonly props inside `__clone()`
- `stdClass` still allows dynamic properties
- `#[AllowDynamicProperties]` opts a legacy class back in

TYPED AND READONLY PROPERTIES

```php
<?php
class User
{
    public int $id;                    // uninitialized
    public string $name = 'guest';     // default
    public ?string $email = null;      // nullable
    public array $roles = [];
    public readonly DateTimeImmutable $created;

    public function __construct()
    {
        $this->created = new DateTimeImmutable();   // set once
    }
}

$u = new User();
// echo $u->id;      // Error: must not be accessed before init
$u->id = 7;
// $u->created = new DateTimeImmutable();  // Error: readonly
// $u->nickname = 'A';  // Deprecated: dynamic property

readonly class Money
{
    public function __construct(
        public int $amount,
        public string $currency,
    ) {}
}
$m = new Money(500, 'EUR');
// $m->amount = 1;   // Error: Cannot modify readonly property
```

CODE EXPLAINED

→ No default → uninitialized until assigned

→ `?string ... = null` → explicitly nullable

→ Readonly set once in the constructor

→ Undeclared property assignment is deprecated

→ `readonly class` + promoted properties = compact value object

PROPERTY DECLARATIONS

DECLARATION	STARTS AS
`public int $x;`	Uninitialized
`public int $x = 0;`	0
`public ?int $x = null;`	null
`public $x;`	null (untyped)
`public readonly int $x;`	Uninitialized, set once
`public static int $n = 0;`	Class-level (Sheet 67)

READONLY RULES

RULE	DETAIL
`Typed`	Required
`Default value`	Not allowed
`Set from`	Inside the class scope
`Times set`	Exactly once
`unset()`	Not allowed after init
`Clone`	Re-init in __clone (8.3)

CHECKING PROPERTIES

CODE	USE
`isset($o->p)`	Set and not null
`property_exists($o, 'p')`	Declared?
`$o->p ?? $d`	Safe read
`get_object_vars($o)`	All visible props
`ReflectionProperty::isInitialized`	Typed prop set?

⚠ COMMON MISTAKES

- Reading a typed property before assigning it
- Relying on dynamic properties (deprecated 8.2)
- Thinking readonly makes nested objects immutable

✓ TIPS & BEST PRACTICES

- Type every property
- Use readonly for identity and value objects
- Use `DateTimeImmutable` inside readonly objects

QUICK SUMMARY

- Typed properties
- Uninitialized ≠ null
- `readonly` = write once
- Dynamic props deprecated

Constructors & Destructors

__construct · constructor promotion · new in initializers · static factories · __destruct

SETTING UP OBJECTS

`__construct()` runs when new creates an object; use it to require and validate the data an object needs. **Constructor promotion** (8.0) declares and assigns a property in one step by adding a visibility keyword to a parameter. new in initializers (8.1) allows default objects as parameter defaults. **Static factory methods** give alternative, named ways to build an object. `__destruct()` runs when the object is destroyed.

NOTES

- A child class must call `parent::__construct()` itself
- Promoted params can be `readonly` and have defaults
- Constructors may be `private` (factories, singletons)
- Destructors run at end of request or when refcount drops to 0
- Don't throw from destructors

PROMOTION, FACTORIES & DESTRUCTORS

```php
<?php
class Point
{
    public function __construct(
        public readonly float $x = 0.0,
        public readonly float $y = 0.0,
    ) {}

    public static function fromArray(array $a): static
    {
        return new static($a['x'], $a['y']);
    }

    public static function origin(): static
    {
        return new static();
    }
}

$p = new Point(3, 4);
$q = Point::fromArray(['x' => 1, 'y' => 2]);
$r = new Point(y: 5);              // named arg

class Logger
{
    public function __construct(
        private Clock $clock = new SystemClock(),  // 8.1
    ) {}
}

class TempFile
{
    public function __construct(public string $path) {}
    public function __destruct() { @unlink($this->path); }
}
```

CODE EXPLAINED

→ Visibility on a parameter promotes it to a property

→ Defaults work like normal parameters

→ `new static` respects subclasses

→ `new` in a default creates the object when omitted

→ Destructor cleans up when the object goes away

CONSTRUCTOR FORMS

FORM	EXAMPLE
`Classic`	$this->x = $x;
`Promoted`	public int $x
`Promoted readonly`	public readonly int $x
`Default object`	Clock $c = new SystemClock()
`Private`	private function __construct()
`Parent call`	parent::__construct($x)

FACTORY METHODS

NAME	PURPOSE
`fromArray()`	Build from array
`fromString()`	Parse input
`create()`	Generic factory
`withX()`	Copy with change
`origin() / empty()`	Common instance
`tryFrom()`	Return null on failure

LIFECYCLE

EVENT	HOOK
`new`	__construct()
`clone`	__clone()
`Last reference gone`	__destruct()
`Script ends`	__destruct() on remaining
`exit()`	Destructors still run

⚠ COMMON MISTAKES

- Forgetting `parent::__construct()` in a child
- Doing heavy work (DB, HTTP) in constructors
- Relying on destructor timing for important logic

✓ TIPS & BEST PRACTICES

- Promote properties to cut boilerplate
- Validate arguments in the constructor
- Add named static factories for alternative inputs

QUICK SUMMARY

- `__construct()` on `new`
- Promotion: `public int $x`
- Static factories
- `__destruct()` on cleanup

Visibility & Access Control

public · protected · private · private(set) asymmetric visibility (8.4) · getters & setters · encapsulation

WHO CAN TOUCH WHAT

Visibility controls access to properties, methods and constants. `public` members are accessible anywhere, `protected` ones from the class and its subclasses, `private` ones only from the declaring class. PHP 8.4 adds **asymmetric visibility**: `public private(set)` lets everyone read a property but only the class write it — no getter boilerplate needed.

NOTES

- Visibility is per class, not per object: an instance can access another instance's private members
- Private members aren't inherited for access, but still exist
- Methods without a keyword are `public`; properties need one (or `var`)
- `protected(set)` allows subclasses to write
- Asymmetric visibility requires a typed property

VISIBILITY IN PRACTICE

```php
<?php
class BankAccount
{
    public private(set) float $balance = 0.0;   // 8.4
    protected string $currency = 'EUR';
    private array $log = [];

    public function deposit(float $amt): void
    {
        if ($amt <= 0) throw new DomainException('Must be > 0');
        $this->balance += $amt;
        $this->record("deposit $amt");
    }

    private function record(string $msg): void
    {
        $this->log[] = $msg;
    }

    public function sameCurrency(BankAccount $o): bool
    {
        return $this->currency === $o->currency; // allowed
    }
}

$acct = new BankAccount();
$acct->deposit(50);
echo $acct->balance;        // 50 (public read)
// $acct->balance = 1e6;    // Error: private(set)
// $acct->record('x');      // Error: private method
```

CODE EXPLAINED

→ `public private(set)` — read anywhere, write inside only

→ `protected` — visible to subclasses

→ `private` — this class only

→ Methods enforce rules before changing state

→ Same-class instances can read each other's protected/private data

ACCESS MATRIX

KEYWORD	CLASS / CHILD / OUTSIDE
`public`	yes / yes / yes
`protected`	yes / yes / no
`private`	yes / no / no
`public private(set)`	write: yes / no / no
`public protected(set)`	write: yes / yes / no

APPLIES TO

MEMBER	DEFAULT
Methods	public
Properties	must declare
Constants	public
Constructor	public
Promoted params	as declared

ENCAPSULATION OPTIONS

NEED	APPROACH
Read-only to outsiders	public private(set)
Never changes	readonly
Validation on write	set hook (Sheet 66)
Computed value	get hook / method
Hide entirely	private

⚠ COMMON MISTAKES

- Making everything `public` by default
- Writing trivial getters/setters in 8.4+ when `private(set)` would do
- Expecting `private` methods to be overridable by children

✓ TIPS & BEST PRACTICES

- Start with `private`; widen only when needed
- Expose behaviour (methods), not raw data
- Use asymmetric visibility for read-only public state

QUICK SUMMARY

- `public` / `protected` / `private`
- Class-level, not object-level
- `private(set)` (8.4)
- Default to private

Property Hooks

LOGIC ON PROPERTY ACCESS (8.4)

Property hooks attach code to reading (`get`) or writing (`set`) a property, so you can validate or compute values while keeping plain `$obj->prop` syntax. A **backed** property stores a value and its hooks can refer to it with `$this->prop`. A **virtual** property has hooks that never touch its own storage — it's computed from other data. Interfaces can declare properties with required hooks.

NOTES

- `set` receives the new value as `$value` (or a named param)
- Short form: `get => expression;` and `set => expression;`
- A virtual property with only `get` is read-only
- Hooks are incompatible with `readonly`
- Child classes can override individual hooks

GET AND SET HOOKS

```php
<?php
class User
{
    public string $email {
        set(string $value) {
            if (!filter_var($value, FILTER_VALIDATE_EMAIL)) {
                throw new InvalidArgumentException("Bad email");
            }
            $this->email = strtolower($value);   // backing store
        }
    }

    public function __construct(
        public string $first,
        public string $last,
    ) {}

    public string $fullName {                    // virtual
        get => "$this->first $this->last";
    }
}

$u = new User('Ana', 'Lee');
$u->email = 'ANA@Example.com';
echo $u->email;        // ana@example.com
echo $u->fullName;     // Ana Lee
// $u->fullName = 'x'; // Error: no set hook

interface HasArea { public float $area { get; } }
```

CODE EXPLAINED

- → `set` validates then writes the backing value
- → Inside the hook, `$this->email` is the raw storage
- → Promoted properties work alongside hooked ones
- → Virtual: computed on each read, no storage
- → Interfaces can require a readable property

HOOK FORMS

SYNTAX	MEANING
`get { return …; }`	Full getter
`get => expr;`	Short getter
`set { … }`	Setter, uses $value
`set(T $v) { … }`	Typed/named param
`set => expr;`	Short: stores expr
`{ get; }`	Interface requirement

BACKED VS VIRTUAL

ASPECT	BACKED / VIRTUAL
`Storage`	Yes / No
`Refers to $this->prop`	Yes / No
`Default value`	Allowed / No
`Only get`	Read/write / Read-only
`Serialised`	Yes / No

COMPARED WITH

APPROACH	TRADE-OFF
`Public property`	No validation
`Getter/setter methods`	Verbose calls
`__get / __set`	Untyped, slow, magic
`private(set)`	Read-only outside, no logic
`Hooks`	Typed, per-property logic

⚠ COMMON MISTAKES

- Calling the hook recursively via another accessor
- Combining hooks with `readonly`
- Using hooks for heavy work — reads look cheap

✓ TIPS & BEST PRACTICES

- Use `set` hooks for validation and normalisation
- Use virtual `get` hooks for derived values
- Keep hook bodies short

QUICK SUMMARY

- `get` / `set` on properties (8.4)
- Backed = stored, virtual = computed
- Interfaces can require hooks
- Replaces most getters/setters

Static Members & Class Constants

static · self:: · static:: late static binding · parent:: · class constants · typed constants (8.3)

CLASS-LEVEL MEMBERS

`static` properties and methods belong to the **class**, not to an instance; access them with `::`. **Class constants** are fixed values on the class; PHP 8.3 lets you type them. `self::` refers to the class where the code is written, while `static::` refers to the class that was **actually called** — this is **late static binding**, essential for inheritance-friendly factories.

NOTES

- Static methods have no `$this`
- Static properties are shared across all instances (per request)
- `final` constants can't be overridden (8.1)
- `Foo::{$name}` dynamic constant fetch (8.3)
- Prefer dependency injection over static state

SELF::, STATIC:: AND CONSTANTS

```php
<?php
class Model
{
    public const string TABLE = 'models';      // typed (8.3)
    final public const int VERSION = 2;
    protected static int $count = 0;

    public static function create(): static
    {
        static::$count++;
        return new static();                   // called class
    }

    public static function table(): string
    {
        return static::TABLE;                  // late binding
    }

    public static function selfTable(): string
    {
        return self::TABLE;                    // always Model
    }
}

class User extends Model
{
    public const string TABLE = 'users';
}

echo User::table();       // users
echo User::selfTable();   // models
$u = User::create();      // a User, not a Model
echo Model::VERSION;      // 2
```

CODE EXPLAINED

- → Constants are read with `Class::NAME`
- → `final` constants can't be redefined in children
- → `new static()` creates the subclass that was called
- → `static::TABLE` resolves in the child
- → `self::TABLE` is fixed to the defining class

SCOPE RESOLUTION

KEYWORD	REFERS TO
`self::`	Class where code is written
`static::`	Class actually called
`parent::`	Parent class
`Foo::`	Named class
`$obj::`	Object's class (8.0)

STATIC MEMBERS

CODE	MEANING
`public static $x`	Shared property
`public static function f()`	No $this
`Foo::$x`	Read static prop
`Foo::f()`	Call static method
`static $n inside method`	Function static var

CLASS CONSTANTS

FEATURE	SINCE
`const X = 1`	5.3
`Visibility on constants`	7.1
`final const`	8.1
`Enum cases as constants`	8.1
`Typed const string X`	8.3
`Foo::{$name}`	8.3

⚠ COMMON MISTAKES

- Using `self::` in factories, so subclasses get the parent type
- Global state hidden in static properties
- Calling instance methods statically — Error

✓ TIPS & BEST PRACTICES

- Use `static::` and `new static` in base-class factories
- Type your class constants (8.3)
- Keep static methods pure (no shared state)

QUICK SUMMARY

- `::` for class members
- `self` = here, `static` = called
- Typed constants (8.3)
- Avoid static state

Inheritance

EXTENDING CLASSES

`class Child extends ParentClass` inherits all public and protected members. A child can **override** a method by redefining it, and call the original with `parent::method()`. PHP supports **single inheritance** only — use interfaces and traits to share more. `final` stops a class from being extended or a method from being overridden. The `#[\Override]` attribute (8.3) makes PHP check that a method really overrides something.

NOTES

- Overrides may widen visibility (protected → public), never narrow it
- Return types may be narrower (covariant)
- Parameter types may be wider (contravariant)
- Constructors are exempt from signature compatibility rules
- Favour composition when the relationship isn't "is-a"

EXTENDS, PARENT:: AND #[\OVERRIDE]

```php
<?php
class Animal
{
    public function __construct(protected string $name) {}

    public function speak(): string
    {
        return "{$this->name} makes a sound";
    }
}

class Dog extends Animal
{
    public function __construct(string $name, private string $breed)
    {
        parent::__construct($name);
    }

    #[\Override]
    public function speak(): string
    {
        return parent::speak() . ' — Woof!';
    }
}

final class Puppy extends Dog {}
// class X extends Puppy {}   // Fatal: cannot extend final

$d = new Dog('Rex', 'Lab');
echo $d->speak();     // Rex makes a sound — Woof!
var_dump($d instanceof Animal);   // true
```

CODE EXPLAINED

- → Child constructor must call `parent::__construct()`
- → `parent::speak()` reuses the original logic
- → `#[\Override]` errors if no parent method exists
- → `final class` can't be extended
- → A `Dog` is also an `Animal`

KEYWORDS

KEYWORD	EFFECT
`extends`	Inherit from a class
`parent::`	Call parent's version
`final class`	No subclasses
`final function`	No overriding
`abstract`	Must be extended (Sheet 69)
`#[\Override]`	Verify override (8.3)

SIGNATURE RULES

PART	CHILD MAY
`Return type`	Be more specific
`Parameter type`	Be more general
`Extra params`	Only with defaults
`Visibility`	Be wider
`static vs instance`	Not change

WHAT'S INHERITED

MEMBER	INHERITED?
`public`	Yes
`protected`	Yes
`private`	No (exists, not accessible)
`Constants`	Yes (unless private)
`Static props`	Shared unless redeclared
`Constructor`	Yes, if child has none

⚠ COMMON MISTAKES

- Forgetting `parent::__construct()`
- Deep inheritance trees that are hard to follow
- Changing parameter types in incompatible ways

✓ TIPS & BEST PRACTICES

- Mark classes `final` unless designed for extension
- Add `#[\Override]` to every overriding method
- Prefer interfaces + composition for shared behaviour

QUICK SUMMARY

- `extends` single parent
- `parent::` reaches up
- `final` blocks extension
- `#[\Override]` (8.3)

Abstract Classes

INCOMPLETE BASE CLASSES

An **abstract class** can't be instantiated; it exists to be extended. It may contain normal methods and properties plus **abstract methods** — signatures without bodies that every concrete child must implement. This makes abstract classes ideal for the **template method** pattern: the base class defines the algorithm and children fill in specific steps.

NOTES

- Any class with an abstract method must be declared `abstract`
- Children must implement abstract methods with compatible signatures
- Abstract classes can have constructors and state
- A class can extend only one abstract class
- Abstract methods can be `protected`

TEMPLATE METHOD PATTERN

```php
<?php
abstract class Report
{
    public function __construct(protected array $rows) {}

    // Template method: fixed algorithm
    public function render(): string
    {
        return $this->header() . $this->body() . $this->footer();
    }

    abstract protected function header(): string;
    abstract protected function body(): string;

    protected function footer(): string      // default step
    {
        return "Rows: " . count($this->rows) . "\n";
    }
}

final class CsvReport extends Report
{
    protected function header(): string { return "id,name\n"; }

    protected function body(): string
    {
        return implode('', array_map(
            fn($r) => "{$r['id']},{$r['name']}\n", $this->rows));
    }
}

echo (new CsvReport([['id' => 1, 'name' => 'Ana']]))->render();
// new Report([]);   // Error: Cannot instantiate abstract class
```

CODE EXPLAINED

- → `render()` defines the order of steps
- → Abstract steps have no body
- → `footer()` is a default that children may override
- → Concrete child implements every abstract method
- → Instantiating the abstract base is an Error

ABSTRACT VS INTERFACE

FEATURE	ABSTRACT / INTERFACE
`Instantiate`	No / No
`Method bodies`	Yes / No
`Properties`	Yes / Hooked only (8.4)
`Constructor`	Yes / Signature only
`Multiple per class`	One / Many
`Constants`	Yes / Yes

RULES

RULE	DETAIL
`abstract method`	No body; ends with ;
`Containing class`	Must be abstract
`Child`	Implements all or is abstract
`Visibility`	Child same or wider
`Static abstract`	Allowed

WHEN TO USE

SITUATION	CHOICE
`Shared code + contract`	Abstract class
`Contract only`	Interface
`Shared code, no is-a`	Trait or composition
`Fixed algorithm, varying steps`	Template method
`Many implementations`	Interface

⚠ COMMON MISTAKES

- Making a class abstract with no abstract members and no reason
- Putting too much logic in the base class
- Using abstract classes where an interface is enough

✓ TIPS & BEST PRACTICES

- Pair an interface with an optional abstract base
- Keep template methods `final` if steps must not change
- Name bases clearly: `AbstractReport` or `Report`

QUICK SUMMARY

- Can't instantiate
- Abstract methods = required steps
- Can hold code and state
- Template method pattern

Interfaces

CONTRACTS

An **interface** lists public methods (and, since 8.4, hooked properties) that implementing classes must provide, without saying how. A class can `implement` **many** interfaces, and interfaces can `extend` several others. Type your parameters against interfaces rather than concrete classes so you can swap implementations in tests or later — this is the heart of dependency injection.

NOTES

- All interface methods are public
- Interfaces may define constants (overridable since 8.1 unless `final`)
- Any class with `__toString()` automatically implements `Stringable`
- Built-ins: `Countable`, `ArrayAccess`, `IteratorAggregate`, `JsonSerializable`
- Name by capability (`Cache`, `Clock`) or with an `-able` suffix

DEFINING AND IMPLEMENTING INTERFACES

```php
<?php
interface Cache
{
    public function get(string $key): mixed;
    public function set(string $key, mixed $v, int $ttl = 0): void;
}

final class ArrayCache implements Cache, Countable
{
    private array $items = [];

    public function get(string $key): mixed
    {
        return $this->items[$key] ?? null;
    }

    public function set(string $key, mixed $v, int $ttl = 0): void
    {
        $this->items[$key] = $v;
    }

    public function count(): int { return count($this->items); }
}

function warm(Cache $cache): void       // depends on the contract
{
    $cache->set('config', ['debug' => false]);
}

$c = new ArrayCache();
warm($c);
var_dump($c instanceof Cache, count($c));   // true, 1
```

CODE EXPLAINED

→ Interface methods have signatures only

→ `implements` several interfaces at once

→ All declared methods must be implemented

→ Functions type-hint the interface, not the class

→ `Countable` makes `count()` work on the object

SYNTAX

CODE	MEANING
`interface I { }`	Declare
`class C implements I, J`	Implement many
`interface I extends J, K`	Extend many
`const X = 1;`	Interface constant
`public T $p { get; }`	Property (8.4)
`$o instanceof I`	Check

BUILT-IN INTERFACES

INTERFACE	ENABLES
`Countable`	count($o)
`ArrayAccess`	$o['k']
`IteratorAggregate`	foreach
`JsonSerializable`	json_encode($o)
`Stringable`	(string) $o
`Throwable`	throw (via Exception/Error)

DESIGN TIPS

GUIDELINE	WHY
`Small interfaces`	Easy to implement
`Type against them`	Swappable implementations
`One per role`	Interface segregation
`Return interfaces`	Hide concrete types
`Fakes in tests`	Implement the interface

⚠ COMMON MISTAKES

- Giant interfaces that force empty methods
- Type-hinting concrete classes everywhere
- Changing an interface signature without updating implementers

✓ TIPS & BEST PRACTICES

- Depend on interfaces, construct concretes at the edge
- Keep interfaces focused on one role
- Use PSR interfaces (logging, cache, HTTP) where they exist

QUICK SUMMARY

- Public method contracts
- Implement many
- Type against interfaces
- Built-ins add language features

Traits

REUSING CODE WITHOUT INHERITANCE

A **trait** is a bundle of methods and properties copied into a class with `use TraitName;`. Traits solve the single-inheritance limit for code reuse: any number of unrelated classes can share them. When two traits define the same method, resolve the conflict with `insteadof`, and rename or change visibility with `as`. Traits can declare abstract methods to require something from the using class.

NOTES

- Class methods override trait methods; trait methods override inherited ones
- Traits can't be instantiated or used as types
- Traits can have static members and (8.2+) constants
- `self` inside a trait refers to the using class
- Prefer composition when the shared code needs its own state or dependencies

USING AND COMBINING TRAITS

```php
<?php
trait HasTimestamps
{
    public ?DateTimeImmutable $createdAt = null;

    public function touch(): void
    {
        $this->createdAt = new DateTimeImmutable();
    }
}

trait SoftDeletes
{
    public bool $deleted = false;
    public function delete(): void { $this->deleted = true; }
    abstract public function id(): int;      // requirement
}

trait Loggable   { public function log($m) { echo $m; } }
trait Debuggable { public function log($m) { var_dump($m); } }

class Post
{
    use HasTimestamps, SoftDeletes;
    use Loggable, Debuggable {
        Loggable::log insteadof Debuggable;  // pick one
        Debuggable::log as protected dump;   // keep other, renamed
    }

    public function id(): int { return 1; }
}

$p = new Post();
$p->touch();
$p->log('saved');
```

CODE EXPLAINED

→ A trait adds properties and methods to the class

→ `abstract` method — the class must provide it

→ Several traits in one `use` statement

→ `insteadof` resolves a name clash

→ `as` aliases and can change visibility

CONFLICT RESOLUTION

SYNTAX	EFFECT
`A::m insteadof B`	Use A's m
`B::m as other`	Alias B's m
`m as protected`	Change visibility
`m as private alias`	Both
`Class defines m`	Class wins

TRAIT CONTENTS

MEMBER	ALLOWED?
Methods	Yes
Properties	Yes
Abstract methods	Yes
Static methods/props	Yes
Constants	Yes (8.2)
Implements interface	No (the class does)

PRECEDENCE

SOURCE	PRIORITY
`Class's own method`	Highest
`Trait method`	Middle
`Inherited method`	Lowest
`Same property in class + trait`	Must be compatible
`class_uses($o)`	List used traits

⚠ COMMON MISTAKES

- Using traits as a dumping ground for unrelated helpers
- Hidden dependencies — trait assumes properties the class may not have
- Trying to type-hint a trait

✓ TIPS & BEST PRACTICES

- Declare requirements with abstract methods
- Pair a trait with an interface: interface for type, trait for default code
- Keep traits small and focused

QUICK SUMMARY

- `use Trait;` copies code in
- `insteadof` / `as` resolve clashes
- Abstract requirements
- Not a type

Enums

ENUMERATIONS (8.1)

An `enum` defines a type with a fixed set of possible values called **cases**. A **pure enum** has cases only; a **backed enum** gives each case a scalar value (`int` or `string`) for storing in databases and JSON. Enums are objects: each case is a singleton, comparisons use `===`, and enums can have methods, constants, static methods and implement interfaces. They pair perfectly with `match`.

NOTES

- `Suit::Hearts` is an instance of `Suit`
- Backed enums: `->value` ; all enums: `->name`
- `from()` throws `ValueError` ; `tryFrom()` returns `null`
- Enums can't have state (no properties besides name/value)
- Enum constants in `const` expressions work (`Status::Active->value` since 8.2)

PURE AND BACKED ENUMS

```php
<?php
enum Status: string
{
    case Draft = 'draft';
    case Published = 'published';
    case Archived = 'archived';

    public function label(): string
    {
        return match ($this) {
            self::Draft     => 'Draft',
            self::Published => 'Live',
            self::Archived  => 'Archived',
        };
    }

    public static function visible(): array
    {
        return [self::Published];
    }
}

$s = Status::from('published');     // Status::Published
$t = Status::tryFrom('nope');       // null
echo $s->value, ' ', $s->name;      // published Published
echo $s->label();                   // Live
var_dump($s === Status::Published); // true
Status::cases();                    // all three cases

enum Suit { case Hearts; case Spades; }   // pure enum
function paint(Suit $s): void {}          // type-safe param
```

CODE EXPLAINED

- → `: string` makes it a backed enum
- → `match ($this)` is exhaustive — no default needed
- → `from()` converts stored values back to cases
- → `->value` for storage, `->name` for the identifier
- → Pure enums are perfect as parameter types

ENUM API

MEMBER	RETURNS
`Enum::Case`	The case instance
`Enum::cases()`	List of all cases
`Enum::from($v)`	Case or ValueError
`Enum::tryFrom($v)`	Case or null
`$case->name`	Case name string
`$case->value`	Backing value

ALLOWED IN ENUMS

FEATURE	OK?
`Methods / static methods`	Yes
`Constants`	Yes
`implements interfaces`	Yes
`Traits (no props)`	Yes
`Properties / state`	No
`new Enum()`	No

ENUMS VS CONSTANTS

ASPECT	CLASS CONSTANTS / ENUM
`Type-safe param`	No / Yes
`List all values`	Reflection / cases()
`Invalid value`	Any int/string / Impossible
`Methods per value`	No / Yes
`Exhaustive match`	No / Yes

⚠ COMMON MISTAKES

- Comparing `$case == 'draft'` — compare `$case->value` or the case itself
- Using `from()` on untrusted input without catching `ValueError`
- Adding `default` to enum `match` and hiding missing cases

✓ TIPS & BEST PRACTICES

- Store `->value` , rebuild with `from()`
- Put labels and rules as enum methods
- Prefer enums over string/int constants

QUICK SUMMARY

- `enum X: string { case A = 'a'; }`
- Pure vs backed
- `from` / `tryFrom` / `cases`
- Methods + `match`

Magic Methods — Property & Call Overloading

INTERCEPTING ACCESS

Magic methods start with __ and are called by PHP automatically. `__get`, `__set`, `__isset` and `__unset` fire when code touches **inaccessible or undefined** properties; `__call` and `__callStatic` fire for inaccessible or undefined methods. `__toString()` controls string conversion and `__invoke()` makes the object callable. They power proxies, fluent query builders and ORMs — but they hide behaviour from IDEs and static analysis.

NOTES

- Magic property methods don't fire for accessible properties
- `__toString()` must return a string
- Document magic members with `@property` / `@method` tags
- Property hooks (8.4) replace most `__get` / `__set` uses
- Magic calls are slower than real methods

OVERLOADING PROPERTIES AND CALLS

```php
<?php
/** @property string $title  @method string getTitle() */
class Record
{
    private array $data = [];

    public function __get(string $n): mixed
    { return $this->data[$n] ?? null; }

    public function __set(string $n, mixed $v): void
    { $this->data[$n] = $v; }

    public function __isset(string $n): bool
    { return isset($this->data[$n]); }

    public function __call(string $m, array $args): mixed
    {
        if (str_starts_with($m, 'get')) {
            return $this->{lcfirst(substr($m, 3))};
        }
        throw new BadMethodCallException("No method $m");
    }

    public function __toString(): string
    { return json_encode($this->data); }
}

$r = new Record();
$r->title = 'Hi';          // __set
echo $r->title;            // __get -> Hi
echo $r->getTitle();       // __call -> Hi
echo $r;                   // __toString -> {"title":"Hi"}
```

CODE EXPLAINED

→ Docblock tags tell tools about magic members

→ Property methods route to an internal array

→ `__isset` keeps `isset()` and `??` working

→ `__call` receives the method name and arguments

→ `echo $obj` triggers `__toString()`

PROPERTY OVERLOADING

METHOD	TRIGGERED BY
`__get($n)`	Reading $o->n
`__set($n, $v)`	Writing $o->n
`__isset($n)`	isset() / empty()
`__unset($n)`	unset($o->n)

CALL & CONVERSION

METHOD	TRIGGERED BY
`__call($m, $args)`	$o->m(…)
`__callStatic($m, $args)`	C::m(…)
`__toString()`	(string) $o, echo
`__invoke(...)`	$o(…)
`__debugInfo()`	var_dump($o)

MAGIC VS MODERN

MAGIC	MODERN ALTERNATIVE
`__get / __set`	Property hooks (8.4)
`__call getters`	Real methods
`__toString`	Stringable + method
`__callStatic facades`	Dependency injection
`__invoke`	First-class callables

⚠ COMMON MISTAKES

- Forgetting `__isset` — `isset()` and `??` return false
- Silently returning null for typos in `__get`
- Expecting magic methods to fire for public properties

✓ TIPS & BEST PRACTICES

- Throw on unknown names instead of returning null
- Add `@property` / `@method` docs for tooling
- Use hooks or real methods when possible

QUICK SUMMARY

- `__get` / `__set` / `__isset` / `__unset`
- `__call` / `__callStatic`
- `__toString`, `__invoke`
- Use sparingly

Magic Methods — Cloning & Serialization

__clone · __serialize · __unserialize · __sleep · __wakeup · __debugInfo · __set_state

LIFECYCLE HOOKS

These magic methods run at specific points in an object's life. `__clone()` runs on the **copy** after `clone`, letting you deep-copy nested objects. `__serialize()` / `__unserialize()` (7.4) control how `serialize()` stores the object and how it's rebuilt — they replace the older `__sleep()` / `__wakeup()`. `__debugInfo()` customises `var_dump()` output and `__set_state()` supports `var_export()`.

NOTES

- `__clone` runs with `$this` = the new copy
- If both exist, `__serialize` wins over `__sleep`
- `__unserialize` runs instead of the constructor
- Never `unserialize()` untrusted input (Sheet 104)
- Hide secrets from dumps with `__debugInfo()` or `#[\SensitiveParameter]`

__CLONE, __SERIALIZE & __DEBUGINFO

```php
<?php
class Order
{
    public function __construct(
        public array $items,
        public DateTime $placed,
        private string $apiKey,
    ) {}

    public function __clone(): void
    { $this->placed = clone $this->placed; }      // deep copy

    public function __serialize(): array
    { return ['items' => $this->items, 'at' => $this->placed]; }

    public function __unserialize(array $d): void
    {
        [$this->items, $this->placed] = [$d['items'], $d['at']];
        $this->apiKey = '';                       // never stored
    }

    public function __debugInfo(): array
    { return ['items' => $this->items, 'apiKey' => '***']; }
}

$a = new Order(['mug'], new DateTime(), 'secret');
$b = clone $a;                        // $b->placed is its own
$copy = unserialize(serialize($a));   // via __(un)serialize
var_dump($a);                         // uses __debugInfo
```

CODE EXPLAINED

→ `clone` is shallow; `__clone` copies nested objects

→ `__serialize` returns a plain array to store

→ Secrets are left out of the serialised form

→ `__unserialize` rebuilds state (constructor not called)

→ `__debugInfo` masks sensitive fields in dumps

LIFECYCLE METHODS

METHOD	RUNS ON
`__construct`	new
`__clone`	clone (on the copy)
`__serialize`	serialize()
`__unserialize`	unserialize()
`__destruct`	Object freed
`__set_state`	var_export() output eval

OLD VS NEW SERIALIZATION

OLD	NEW
`__sleep(): array of prop names`	__serialize(): data array
`__wakeup(): void`	__unserialize(array $d)
`Serializable interface`	Deprecated (8.1)
`Private props by name`	Any shape you like

DEBUG OUTPUT

FUNCTION	USES
`var_dump($o)`	__debugInfo()
`print_r($o)`	__debugInfo()
`var_export($o)`	__set_state format
`json_encode($o)`	JsonSerializable
`(string) $o`	__toString()

⚠ COMMON MISTAKES

- Shallow clones sharing mutable `DateTime` or collections
- Implementing the deprecated `Serializable` interface
- Leaking passwords via `var_dump` in logs

✓ TIPS & BEST PRACTICES

- Use immutable objects to avoid clone surprises
- Prefer JSON over `serialize()` for storage
- Mask secrets in `__debugInfo()`

QUICK SUMMARY

- `__clone` for deep copies
- `__serialize` / `__unserialize`
- `__debugInfo` for dumps
- Avoid `Serializable`

Cloning & Comparing Objects

clone · shallow vs deep copy · clone with (8.5) · == vs === on objects · spl_object_id() · WeakMap

COPIES AND IDENTITY

`clone $obj` makes a **shallow copy**: scalar and array properties are copied, but properties holding objects still point to the same objects unless `__clone()` copies them. PHP 8.5's **`clone with`** syntax copies and changes properties in one step — even `readonly` ones — ideal for immutable "withers". Compare objects with == (same class, equal properties) or === (same instance).

NOTES

- `clone` doesn't call the constructor
- Arrays inside objects are copied (copy-on-write)
- `spl_object_id()` is unique only among live objects
- `WeakMap` lets you attach data to objects without leaking memory
- Immutable objects remove most cloning worries

CLONE, WITHERS & COMPARISON

```php
<?php
final class Money
{
    public function __construct(
        public readonly int $amount,
        public readonly string $currency,
    ) {}

    public function withAmount(int $amount): static
    {
        return clone($this, ['amount' => $amount]);   // PHP 8.5
    }
}

$a = new Money(100, 'EUR');
$b = $a->withAmount(250);          // new object, EUR kept
$c = new Money(100, 'EUR');

var_dump($a == $c);                // true  (equal props)
var_dump($a === $c);               // false (different objects)
var_dump($a === $a);               // true

$cfg = new stdClass();
$cfg->db = new stdClass();
$copy = clone $cfg;
$copy->db->host = 'x';             // also changes $cfg->db!

$meta = new WeakMap();
$meta[$a] = ['cached' => true];    // freed when $a is freed
echo spl_object_id($a);            // e.g. 12
```

CODE EXPLAINED

- → `clone($obj, [...])` sets new values on the copy
- → Works on readonly properties (8.5)
- → `==` compares class and property values
- → `===` checks identity
- → Shallow clone shares nested objects

COPY BEHAVIOUR

PROPERTY TYPE	AFTER CLONE
`int / string / bool`	Copied
`array`	Copied
`object`	Same object shared
`readonly`	Copied (can't change without 8.5 clone with)
`resource`	Same resource

COMPARISON

EXPRESSION	TRUE WHEN
`$a == $b`	Same class, == props
`$a === $b`	Same instance
`$a <=> $b`	Compares props in order
`spl_object_id($a) === spl_object_id($b)`	Same instance
`$a instanceof $b`	$a is $b's class

WITHER STYLES

VERSION	CODE
Any	$c = clone $this; $c->x = $x; (mutable)
8.3	Re-init readonly in __clone
8.5	clone($this, ['x' => $x])
Any	new static($x, $this->y)

⚠ COMMON MISTAKES

- Assuming `clone` is deep
- Using `==` when identity matters (e.g. entity tracking)
- Keying a plain array by `spl_object_id` and leaking memory

✓ TIPS & BEST PRACTICES

- Make value objects readonly and add withers
- Use `clone with` on PHP 8.5+
- Use `WeakMap` for per-object caches

QUICK SUMMARY

- `clone` = shallow copy
- `clone($o, [...])` (8.5)
- `==` values, `===` identity
- `WeakMap` for metadata

Reflection & Object Utilities

::class · get_class() · method_exists() · property_exists() · get_object_vars() · ReflectionClass · ReflectionMethod

INSPECTING CODE AT RUNTIME

Simple questions — what class is this, does it have that method — are answered by functions such as `get_class()`, `method_exists()` and `property_exists()`. The **Reflection API** goes much further: `ReflectionClass`, `ReflectionMethod`, `ReflectionProperty` and friends expose names, types, attributes, doc comments and parameters, and can even create objects without constructors. Frameworks use it for dependency injection, routing and serialisation.

NOTES

- Reflection ignores visibility for reading metadata
- `setAccessible()` is no longer needed (8.1+)
- `getAttributes()` reads `#[...]` attributes (Sheet 103)
- Reflection is slower — cache results in hot paths
- `class_exists()` triggers autoloading by default

INTROSPECTION AND REFLECTION

```php
<?php
class User
{
    public function __construct(
        public int $id,
        private string $email = '',
    ) {}
    public function greet(string $g = 'Hi'): string
    { return $g; }
}
$u = new User(1);

echo $u::class;                      // User
method_exists($u, 'greet');          // true
property_exists(User::class, 'email');// true (even private)
get_object_vars($u);                 // ['id' => 1] from outside
class_exists('User');                // true

$rc = new ReflectionClass(User::class);
foreach ($rc->getConstructor()->getParameters() as $p) {
    echo $p->getName(), ': ', $p->getType(),
         $p->isOptional() ? ' (optional)' : '', "\n";
}
// id: int
// email: string (optional)

$rp = new ReflectionProperty(User::class, 'email');
echo $rp->getValue($u);              // '' — private read
$rm = new ReflectionMethod(User::class, 'greet');
echo $rm->invoke($u, 'Hello');       // Hello
$blank = $rc->newInstanceWithoutConstructor();
```

CODE EXPLAINED

- → `::class` on an object gives its class name
- → `property_exists` sees private properties too
- → `get_object_vars` respects the caller's scope
- → Reflection lists constructor parameters and types — the basis of autowiring
- → Reflection can read private values and call methods

QUICK FUNCTIONS

FUNCTION	RETURNS
`get_class($o)`	Class name
`get_parent_class($o)`	Parent or false
`class_implements($o)`	Interfaces
`class_uses($o)`	Traits
`get_class_methods($c)`	Method names
`is_a($o, C::class)`	Is-a check

REFLECTION CLASSES

CLASS	DESCRIBES
`ReflectionClass`	Class / interface
`ReflectionMethod`	Method
`ReflectionProperty`	Property
`ReflectionFunction`	Function / closure
`ReflectionParameter`	Parameter
`ReflectionNamedType`	Type info
`ReflectionEnum`	Enum

USEFUL METHODS

METHOD	USE
`getMethods() / getProperties()`	List members
`getAttributes(A::class)`	Read attributes
`getDocComment()`	DocBlock text
`newInstanceArgs($args)`	Construct
`isFinal() / isAbstract()`	Modifiers
`getShortName()`	Name without namespace

⚠ COMMON MISTAKES

- Using reflection in tight loops without caching
- Calling private methods via reflection in production code to work around design
- `class_exists($name, false)` when you need autoloading

✓ TIPS & BEST PRACTICES

- Use simple functions for simple checks
- Cache reflection results (frameworks do this)
- Use attributes + reflection for metadata-driven code

QUICK SUMMARY

- `::class`, `get_class()`
- `method_exists`, `property_exists`
- ReflectionClass & friends
- Basis of DI and attributes

Reading Files

GETTING DATA OUT OF FILES

For small files, `file_get_contents()` reads everything into a string and `file()` reads lines into an array. For large files, open a **stream** with `fopen()` and read a line (`fgets`) or a chunk (`fread`) at a time so memory stays flat. Every read function returns `false` on failure and emits a Warning — check the result. `SplFileObject` wraps the same operations in an iterable object.

NOTES

- `file()` flags: `FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES`
- Always `fclose()` handles you open (or let them go out of scope)
- `file_get_contents()` also reads URLs if `allow_url_fopen` is on
- Check `is_readable()` first for nicer error messages
- Read binary files with mode `'rb'`

WHOLE FILE, LINES, STREAMS

```php
<?php
$text = file_get_contents(__DIR__ . '/notes.txt');
if ($text === false) {
    throw new RuntimeException('Cannot read notes.txt');
}

$lines = file('list.txt', FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);

$fh = fopen('big.log', 'r');
while (($line = fgets($fh)) !== false) {
    // one line at a time, constant memory
}
fclose($fh);

$fh = fopen('photo.jpg', 'rb');
$header = fread($fh, 8);                // first 8 bytes
fclose($fh);

$file = new SplFileObject('data.txt');
foreach ($file as $n => $line) {
    echo $n + 1, ': ', rtrim($line), "\n";
}
```

CODE EXPLAINED

- → `=== false` distinguishes failure from an empty file
- → `file()` with flags gives clean lines
- → `fgets()` loop streams very large files
- → `fread($fh, n)` reads up to n bytes
- → `SplFileObject` is iterable line by line

READ FUNCTIONS

FUNCTION	RETURNS
`file_get_contents($p)`	Whole file string
`file($p)`	Array of lines
`fgets($fh)`	Next line or false
`fread($fh, $n)`	Up to n bytes
`fgetc($fh)`	One character
`stream_get_contents($fh)`	Rest of stream

FOPEN READ MODES

MODE	MEANING
`r`	Read, start at beginning
`r+`	Read + write, no truncate
`rb`	Read binary
`Must exist`	Yes for r / r+
`Pointer`	Starts at 0

STREAM HELPERS

FUNCTION	USE
`feof($fh)`	At end?
`ftell($fh)`	Current position
`fseek($fh, $n)`	Move pointer
`rewind($fh)`	Back to start
`fclose($fh)`	Close
`filesize($p)`	Size in bytes

⚠ COMMON MISTAKES

- Loading a multi-GB log with `file_get_contents()`
- `while (!feof($fh))` that processes a final `false`
- Building paths from user input without validation (path traversal)

✓ TIPS & BEST PRACTICES

- Use `__DIR__` to build paths
- Stream with `fgets()` or a generator for big files
- Throw on `false` instead of continuing

QUICK SUMMARY

- `file_get_contents` / `file` for small
- `fopen` + `fgets` for big
- Check for `false`
- `SplFileObject` iterates lines

Writing & Appending Files

PUTTING DATA INTO FILES

`file_put_contents()` writes a string (or array of strings) in one call, creating the file if needed and replacing it unless you pass `FILE_APPEND`. Add `LOCK_EX` so concurrent writers don't interleave. For incremental writes, open a handle with a write mode and call `fwrite()`. The mode decides whether the file is truncated, appended to, or must not already exist.

NOTES

- Return value: bytes written, or `false` on failure
- The directory must exist — create it with `mkdir(..., recursive: true)`
- Write to a temp file and `rename()` for atomic replacement
- `PHP_EOL` vs `"\n"` : pick one line ending per file format
- Check permissions with `is_writable()`

WRITE, APPEND, ATOMIC REPLACE

```php
<?php
file_put_contents('out.txt', "Hello\n");          // overwrite
file_put_contents('app.log', date('c') . " start\n",
                  FILE_APPEND | LOCK_EX);            // append safely
file_put_contents('list.txt', ["a\n", "b\n"]);    // array OK

$fh = fopen('report.csv', 'w');                      // truncate
fwrite($fh, "id,name\n");
fwrite($fh, "1,Ana\n");
fclose($fh);

$fh = fopen('new.lock', 'x');     // fails if file exists
if ($fh === false) { exit("Already running\n"); }

// Atomic write: readers never see a half-written file
$tmp = tempnam(dirname($path), 'tmp');
file_put_contents($tmp, json_encode($data));
rename($tmp, $path);

$dir = __DIR__ . '/cache/2026';
is_dir($dir) || mkdir($dir, 0775, recursive: true);
```

CODE EXPLAINED

→ Default mode replaces the file

→ `FILE_APPEND | LOCK_EX` — append with an exclusive lock

→ `'w'` truncates before writing

→ `'x'` creates only if missing — a simple lock

→ `rename()` within one filesystem is atomic

WRITE MODES

MODE	CREATES? / TRUNCATES? / POINTER
`w`	yes / yes / start
`a`	yes / no / end (always)
`x`	only if new / — / start
`c`	yes / no / start
`w+` / `a+` / `x+` / `c+`	Same, plus read
`b` suffix	Binary (e.g. wb)

FILE_PUT_CONTENTS FLAGS

FLAG	EFFECT
`FILE_APPEND`	Add to end
`LOCK_EX`	Exclusive lock while writing
`FILE_USE_INCLUDE_PATH`	Search include_path
`(none)`	Replace contents

FILE OPERATIONS

FUNCTION	DOES
`copy($a, $b)`	Copy file
`rename($a, $b)`	Move / rename
`unlink($p)`	Delete
`touch($p)`	Create / update mtime
`chmod($p, 0644)`	Permissions
`tempnam($dir, $prefix)`	Unique temp file

⚠ COMMON MISTAKES

- Concurrent appends without `LOCK_EX`
- Writing into a directory that doesn't exist
- Ignoring a `false` return — disk full or permissions

✓ TIPS & BEST PRACTICES

- Use temp file + `rename()` for config or cache writes
- Prefer a logging library over hand-rolled log files
- Keep writable dirs outside the web root

QUICK SUMMARY

- `file_put_contents()` one-shot
- `FILE_APPEND | LOCK_EX`
- Modes `w a x c`
- Temp + `rename()` = atomic

File Paths & File Info

WORKING WITH PATHS

Relative paths in PHP resolve against the **current working directory**, which differs between the CLI and web servers — so build paths from `__DIR__`. `pathinfo()`, `basename()` and `dirname()` split paths into parts; `realpath()` resolves `..` and symlinks and returns `false` if the path doesn't exist. Status functions like `file_exists()`, `is_file()` and `filesize()` answer questions about a path.

NOTES

- `/` works as a separator on Windows too
- `dirname($p, 2)` goes up two levels
- Stat results are cached — call `clearstatcache()` after changes
- `realpath()` is the basis of path-traversal checks
- `sys_get_temp_dir()` gives the temp folder

PATH FUNCTIONS

```php
<?php
$path = '/var/www/app/public/img/logo.final.png';

basename($path);                  // logo.final.png
basename($path, '.png');          // logo.final
dirname($path);                   // /var/www/app/public/img
dirname($path, 2);                // /var/www/app/public
pathinfo($path, PATHINFO_EXTENSION);  // png
pathinfo($path);
// ['dirname' => …, 'basename' => 'logo.final.png',
//  'extension' => 'png', 'filename' => 'logo.final']

$config = __DIR__ . '/../config/app.php';   // stable
realpath($config);                // /var/www/app/config/app.php

// Path traversal guard
$base = realpath(__DIR__ . '/uploads');
$full = realpath($base . '/' . $_GET['file']);
$inBase = $full !== false
    && str_starts_with($full, $base . DIRECTORY_SEPARATOR);
if (!$inBase) {
    http_response_code(403); exit;
}

is_file($full) && filesize($full);   // bytes
date('Y-m-d', filemtime($full));     // last modified
```

CODE EXPLAINED

- → `basename` with a suffix strips it
- → `dirname($p, n)` climbs n levels
- → `filename` excludes only the last extension
- → `__DIR__` keeps paths independent of the working directory
- → Compare resolved path against the allowed base

PATH PARTS

FUNCTION	LOGO.FINAL.PNG
`basename($p)`	logo.final.png
`pathinfo($p, PATHINFO_FILENAME)`	logo.final
`pathinfo($p, PATHINFO_EXTENSION)`	png
`dirname($p)`	parent folder
`realpath($p)`	absolute or false

STATUS CHECKS

FUNCTION	TRUE WHEN
`file_exists($p)`	File or dir exists
`is_file($p)`	Regular file
`is_dir($p)`	Directory
`is_readable($p)`	Can read
`is_writable($p)`	Can write
`is_link($p)`	Symlink

FILE INFO

FUNCTION	RETURNS
`filesize($p)`	Bytes
`filemtime($p)`	Modified timestamp
`fileperms($p)`	Permission bits
`mime_content_type($p)`	MIME guess
`stat($p)`	All metadata
`disk_free_space('/')`	Free bytes

⚠ COMMON MISTAKES

- Relative includes that break when run from another directory
- Trusting `pathinfo()` extension for security
- Concatenating user input into paths without `realpath()` checks

✓ TIPS & BEST PRACTICES

- Build every path from `__DIR__`
- Validate uploads by content, not extension
- Use a filesystem library (Flysystem) for cloud storage

QUICK SUMMARY

- `__DIR__` for stable paths
- `basename` / `dirname` / `pathinfo`
- `realpath` for safety
- `is_file`, `filesize`, `filemtime`

Sheet 80

Directories & File Management

mkdir() · scandir() · glob() · RecursiveDirectoryIterator · rename() · copy() · unlink() · rmdir()

FOLDERS AND THEIR CONTENTS

Create directories with `mkdir()` (pass `recursive: true` for nested paths) and remove empty ones with `rmdir()`. List contents with `scandir()` (includes `.` and `..`) or match patterns with `glob()`. To walk a whole tree, combine `RecursiveDirectoryIterator` with `RecursiveIteratorIterator`. Deleting a non-empty directory means removing its files first.

NOTES

- `mkdir` permissions are reduced by the process umask
- `glob()` returns `[]` when nothing matches
- `GLOB_BRACE` (`*.{jpg,png}`) isn't available on every platform
- `FilesystemIterator::SKIP_DOTS` hides `.` and `..`
- `rename()` can move across directories on the same disk

LIST, WALK, DELETE

```php
<?php
mkdir(__DIR__ . '/storage/cache', 0775, recursive: true);

$entries = array_diff(scandir('.'), ['.', '..']);
$images  = glob(__DIR__ . '/img/*.{jpg,png}', GLOB_BRACE);
$dirs    = glob('*', GLOB_ONLYDIR);

$skip = FilesystemIterator::SKIP_DOTS;
$it = new RecursiveIteratorIterator(
    new RecursiveDirectoryIterator('src', $skip)
);
foreach ($it as $file) {               // SplFileInfo
    if ($file->getExtension() === 'php') {
        echo $file->getPathname(), ' ', $file->getSize(), "\n";
    }
}

function rrmdir(string $dir): void
{
    $skip = FilesystemIterator::SKIP_DOTS;
    $it = new RecursiveIteratorIterator(
        new RecursiveDirectoryIterator($dir, $skip),
        RecursiveIteratorIterator::CHILD_FIRST
    );
    foreach ($it as $f) {
        $p = $f->getPathname();
        $f->isDir() ? rmdir($p) : unlink($p);
    }
    rmdir($dir);
}
```

CODE EXPLAINED

- → Named `recursive:` argument creates parents
- → Remove `.` and `..` from `scandir()` output
- → `glob` brace pattern matches several extensions
- → Recursive iterator yields `SplFileInfo` objects
- → `CHILD_FIRST` deletes contents before folders

DIRECTORY FUNCTIONS

FUNCTION	DOES
`mkdir($d, 0775, true)`	Create (nested)
`rmdir($d)`	Remove empty dir
`scandir($d)`	List names
`glob($pattern)`	Match paths
`is_dir($d)`	Check
`getcwd() / chdir()`	Working dir

GLOB PATTERNS

PATTERN	MATCHES
`*.php`	PHP files here
`src/*/*.php`	One level down
`*.{jpg,png}`	With GLOB_BRACE
`file?.txt`	Single char
`[abc]*.txt`	Char class
`GLOB_ONLYDIR`	Directories only

SPLFILEINFO METHODS

METHOD	RETURNS
`getFilename()`	Name
`getPathname()`	Full path
`getExtension()`	Extension
`getSize()`	Bytes
`getMTime()`	Modified time
`isDir() / isFile()`	Type

⚠ COMMON MISTAKES

- Forgetting `.` and `..` in `scandir()` results
- `rmdir()` on a non-empty directory — fails
- Recursive delete on a path built from user input

✓ TIPS & BEST PRACTICES

- Use iterators for large trees (lazy)
- Guard destructive operations with `realpath()` checks
- Use Symfony Filesystem for robust cross-platform helpers

QUICK SUMMARY

- `mkdir(…, recursive: true)`
- `scandir` / `glob`
- Recursive iterators walk trees
- Empty before `rmdir`

JSON

ENCODING AND DECODING JSON

`json_encode()` turns PHP values into JSON text; `json_decode()` turns JSON into PHP values — objects (`stdClass`) by default, or **associative arrays** when the second argument is `true`. Without flags, errors return `null`/`false` silently, so always pass **`JSON_THROW_ON_ERROR`**. Classes control their JSON form by implementing `JsonSerializable`. PHP 8.3 adds `json_validate()` to check syntax cheaply.

NOTES

- Lists encode as `[…]`, other arrays as `{…}`
- Strings must be valid UTF-8 or encoding fails
- `1.0` encodes as `1` unless you pass `JSON_PRESERVE_ZERO_FRACTION`
- Big integers in JSON may lose precision as floats
- Public properties are encoded for plain objects

ENCODE, DECODE, CUSTOMISE

```php
<?php
$data = ['name' => 'Ana', 'tags' => ['php', 'web'], 'url' => 'https://x.io/a'];

echo json_encode($data);
// {"name":"Ana","tags":["php","web"],"url":"https:\/\/x.io\/a"}
echo json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES
                      | JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);

$obj = json_decode('{"a":1,"b":[2,3]}');            // stdClass
echo $obj->b[0];                                    // 2
$arr = json_decode('{"a":1}', true, 512, JSON_THROW_ON_ERROR);
echo $arr['a'];                                     // 1

try {
    json_decode('{bad json}', flags: JSON_THROW_ON_ERROR);
} catch (JsonException $e) {
    echo $e->getMessage();                          // Syntax error
}
json_validate('{"ok":true}');                       // true (8.3)

final class User implements JsonSerializable {
    public function __construct(private int $id, private string $pw) {}
    public function jsonSerialize(): array { return ['id' => $this->id]; }
}
echo json_encode(new User(7, 'secret'));            // {"id":7}
```

CODE EXPLAINED

- → Default output escapes `/` and non-ASCII
- → Combine flags with `|` for readable output
- → `true` as 2nd arg → associative arrays
- → `JSON_THROW_ON_ERROR` turns failures into `JsonException`
- → `jsonSerialize()` hides private data like passwords

ENCODE FLAGS

FLAG	EFFECT
`JSON_PRETTY_PRINT`	Indented output
`JSON_UNESCAPED_SLASHES`	Keep /
`JSON_UNESCAPED_UNICODE`	Keep é, 😀
`JSON_THROW_ON_ERROR`	Throw JsonException
`JSON_FORCE_OBJECT`	[] as {}
`JSON_HEX_TAG`	Escape < > (HTML-safe)

DECODE OPTIONS

OPTION	EFFECT
`$assoc = true`	Arrays not objects
`JSON_OBJECT_AS_ARRAY`	Same as flag
`$depth`	Max nesting (512)
`JSON_BIGINT_AS_STRING`	Keep big ints exact
`JSON_THROW_ON_ERROR`	Throw on bad input
`json_validate($s)`	Check only (8.3)

PHP ↔ JSON

PHP	JSON
`['a', 'b']`	["a","b"]
`['k' => 1]`	{"k":1}
`[]`	[]
`null / true`	null / true
`stdClass`	{}
`1.5`	1.5

⚠ COMMON MISTAKES

- Not checking errors — `json_decode` returns `null` silently
- Encoding non-UTF-8 strings
- Expecting an empty array to encode as `{}`

✓ TIPS & BEST PRACTICES

- Always use `JSON_THROW_ON_ERROR`
- Decode to arrays (`true`) for simple data
- Implement `JsonSerializable` for API output

QUICK SUMMARY

- `json_encode` / `json_decode`
- `true` → arrays
- `JSON_THROW_ON_ERROR`
- `JsonSerializable`, `json_validate`

CSV

COMMA-SEPARATED VALUES

`fgetcsv()` reads one CSV record from a stream and returns an array of fields, handling quotes and embedded commas and newlines. `fputcsv()` writes an array as a properly quoted line. `str_getcsv()` parses a single CSV string. For large files, read row by row. Since PHP 8.4, not passing the `escape` parameter is deprecated: pass `escape: ''` for RFC 4180 behaviour.

NOTES

- Default separator `,`, enclosure `"`
- A blank line returns `[null]`
- Excel expects a UTF-8 BOM to detect encoding
- Use `;` as separator for many European Excel locales
- Combine header row + `array_combine()` for keyed rows

READING AND WRITING CSV

```php
<?php
// Read with header row → associative arrays
$fh = fopen('users.csv', 'r');
$header = fgetcsv($fh, escape: '');
$rows = [];
while (($fields = fgetcsv($fh, escape: '')) !== false) {
    if ($fields === [null]) continue;        // blank line
    $rows[] = array_combine($header, $fields);
}
fclose($fh);

// Write
$out = fopen('export.csv', 'w');
fwrite($out, "\xEF\xBB\xBF");               // BOM for Excel
fputcsv($out, ['id', 'name', 'note'], escape: '');
fputcsv($out, [1, 'Ana', 'Says "hi", then leaves'], escape: '');
fclose($out);
// 1,Ana,"Says ""hi"", then leaves"

$fields = str_getcsv('a,"b,c",d', escape: '');   // ['a', 'b,c', 'd']

$file = new SplFileObject('users.csv');
$file->setFlags(SplFileObject::READ_CSV | SplFileObject::SKIP_EMPTY);
foreach ($file as $row) { /* … */ }
```

CODE EXPLAINED

- → First row becomes the keys
- → `escape: ''` avoids the non-standard backslash escape
- → Blank lines come back as `[null]`
- → Quotes inside fields are doubled automatically
- → `SplFileObject` can iterate CSV directly

CSV FUNCTIONS

FUNCTION	DOES
`fgetcsv($fh)`	Read one record
`fputcsv($fh, $fields)`	Write one record
`str_getcsv($line)`	Parse a string
`SplFileObject::READ_CSV`	Iterate rows
`array_combine($head, $row)`	Keyed row

PARAMETERS

PARAM	DEFAULT
`separator`	,
`enclosure`	"
`escape`	\ (pass '' in 8.4+)
`eol (fputcsv)`	\n (8.1+)
`length (fgetcsv)`	0 = unlimited

GOTCHAS

ISSUE	FIX
`Excel mangles UTF-8`	Write BOM
`Semicolon locales`	separator: ';'
`Row length mismatch`	Check before array_combine
`Huge file`	Stream row by row
`Formula injection (=…)`	Prefix with '

⚠ COMMON MISTAKES

- Splitting lines with `explode(',')` — breaks on quoted commas
- Omitting `escape` (deprecated 8.4)
- Exporting user data that starts with `=`, `+`, `-`, `@` unescaped

✓ TIPS & BEST PRACTICES

- Always use the CSV functions, never `explode`
- Stream large imports
- Sanitise cells against spreadsheet formula injection

QUICK SUMMARY

- `fgetcsv` / `fputcsv`
- `str_getcsv` for strings
- Pass `escape: ''`
- Header + `array_combine`

XML & HTML Parsing

READING MARKUP

SimpleXML turns XML into objects you navigate with `->` and `[]` — the easiest option for simple documents. **DOM** (`DOMDocument`, `DOMXPath`) gives full read/write control and XPath queries. **XMLReader** streams huge files node by node. For HTML, PHP 8.4's `Dom\HTMLDocument` parses real-world HTML5 correctly and supports CSS selectors via `querySelector()`.

NOTES

- SimpleXML values are objects — cast with `(string)` or `(int)`
- Attributes: `$node['id']`
- Old `DOMDocument::loadHTML()` is HTML4-based and noisy
- External entity loading is disabled by default (XXE safe since 8.0)
- `XMLWriter` builds large XML efficiently

SIMPLEXML, XPATH, HTML5

```php
<?php
$xml = <<<XML
<catalog>
  <book id="1"><title>PHP</title><price>29.50</price></book>
  <book id="2"><title>SQL</title><price>19.00</price></book>
</catalog>
XML;

$cat = simplexml_load_string($xml);
foreach ($cat->book as $book) {
    echo $book['id'], ': ', $book->title, "\n";
    $total = ($total ?? 0) + (float) $book->price;
}
$cheap = $cat->xpath('//book[price < 20]/title');   // [SQL]

$dom = new DOMDocument();
$dom->loadXML($xml);
$xp = new DOMXPath($dom);
echo $xp->query('//book[@id="2"]/title')->item(0)->textContent;

$html = '<main><a class="nav" href="/a">A</a><a href="/b">B</a></main>';
$doc = Dom\HTMLDocument::createFromString($html, LIBXML_NOERROR);
foreach ($doc->querySelectorAll('a.nav') as $a) {
    echo $a->getAttribute('href');                  // /a
}
```

CODE EXPLAINED

- → SimpleXML exposes child elements as properties
- → Cast values to get scalars
- → SimpleXML supports XPath too
- → `DOMXPath` queries a DOM tree
- → `Dom\HTMLDocument` (8.4) + CSS selectors

WHICH PARSER?

NEED	USE
`Small, simple XML`	SimpleXML
`Edit / build XML`	DOMDocument
`XPath queries`	DOMXPath or SimpleXML
`Huge XML file`	XMLReader
`Write large XML`	XMLWriter
`Parse HTML`	Dom\HTMLDocument (8.4)

SIMPLEXML ACCESS

CODE	GETS
`$x->child`	First child element
`$x->child[1]`	Second child
`$x['attr']`	Attribute
`(string) $x->child`	Text
`$x->children()`	All children
`$x->asXML()`	Back to string

DOM\HTMLDOCUMENT (8.4)

METHOD	DOES
`createFromString($html)`	Parse HTML5
`createFromFile($path)`	Parse file
`querySelector($css)`	First match
`querySelectorAll($css)`	All matches
`->textContent`	Text
`saveHtml()`	Serialise

⚠ COMMON MISTAKES

- Comparing SimpleXML nodes without casting
- Parsing HTML with regex
- Loading huge XML into DOM — memory explodes

✓ TIPS & BEST PRACTICES

- Use XMLReader for large feeds
- Use `Dom\HTMLDocument` for scraping on 8.4+
- Use `libxml_use_internal_errors(true)` to collect parse errors

QUICK SUMMARY

- SimpleXML for easy reading
- DOM + XPath for control
- XMLReader for size
- `Dom\HTMLDocument` for HTML5

Streams & Wrappers

ONE API FOR MANY SOURCES

PHP's file functions work on **streams**, and a **wrapper** prefix decides what a stream points to: `file://` (default), `http://`, `php://`, `data://`, `compress.zlib://` and more. `php://input` reads a raw request body, `php://stdin` / `stdout` / `stderr` are the CLI channels, and `php://memory` / `php://temp` give in-memory buffers. **Stream contexts** pass options such as HTTP headers or timeouts.

NOTES

- `php://temp` spills to disk after 2 MB (configurable)
- `php://input` isn't available for `multipart/form-data`
- `allow_url_fopen` controls remote wrappers
- `stream_get_wrappers()` lists what's registered
- Register your own with `stream_wrapper_register()`

WRAPPERS AND CONTEXTS

```php
<?php
// JSON request body (POST/PUT)
$payload = json_decode(file_get_contents('php://input'), true);

// CLI streams
$name = trim(fgets(STDIN));
fwrite(STDERR, "warn\n");

// In-memory buffer, then read back
$buf = fopen('php://temp', 'w+');
fputcsv($buf, ['a', 'b'], escape: '');
rewind($buf);
$csv = stream_get_contents($buf);     // "a,b\n"

// HTTP with a context
$ctx = stream_context_create(['http' => [
    'method'  => 'POST',
    'header'  => "Content-Type: application/json\r\n",
    'content' => json_encode(['q' => 'php']),
    'timeout' => 5,
]]);
$res = file_get_contents('https://api.example.com/search', false, $ctx);

$gz   = file_get_contents('compress.zlib://archive.log.gz');
$data = file_get_contents('data://text/plain;base64,SGVsbG8=');  // Hello
```

CODE EXPLAINED

- → `php://input` holds the raw body
- → `STDIN` / `STDERR` constants in the CLI
- → `php://temp` buffers CSV in memory
- → Context options configure HTTP method, headers, timeout
- → Wrappers decompress or decode on the fly

PHP:// STREAMS

STREAM	PURPOSE
`php://input`	Raw request body
`php://output`	Same as echo
`php://stdin / stdout / stderr`	CLI I/O
`php://memory`	RAM buffer
`php://temp`	RAM, then temp file
`php://filter/…`	Apply filters

OTHER WRAPPERS

WRAPPER	EXAMPLE
`file://`	Default local files
`http:// https://`	Remote GET/POST
`ftp://`	FTP
`data://`	Inline data URI
`compress.zlib://`	Gzip files
`phar://`	Inside PHAR archives

STREAM FUNCTIONS

FUNCTION	USE
`stream_context_create()`	Options
`stream_get_contents($fh)`	Read rest
`stream_copy_to_stream()`	Pipe streams
`stream_get_meta_data($fh)`	Info/headers
`stream_set_timeout($fh, $s)`	Timeouts
`stream_filter_append()`	Transform

⚠ COMMON MISTAKES

- Reading `php://input` on multipart uploads — use `$_FILES`
- Remote `file_get_contents` without a timeout
- Using `http://` wrappers where a proper HTTP client is needed

✓ TIPS & BEST PRACTICES

- Use `php://temp` for building files in memory
- Use Guzzle or Symfony HttpClient for real HTTP work (Sheet 101)
- Disable `allow_url_include` always

QUICK SUMMARY

- Wrappers pick the backend
- `php://input`, `php://temp`
- Contexts set options
- Same functions everywhere

include & require

LOADING OTHER FILES

`require` and `include` insert and run another PHP file at that point; the included code shares the current variable scope. The difference is failure: `require` stops with a fatal error, `include` emits a Warning and continues. The `_once` variants skip files already loaded. An included file can `return` a value — the standard way to load config arrays. Modern code loads classes through autoloading instead.

NOTES

- They're language constructs: parentheses are optional
- Relative paths are resolved via `include_path`, then the working directory
- Always build paths with `__DIR__`
- `require_once` has a small cost — fine for bootstrapping
- Never include paths built from user input

INCLUDING FILES

```php
<?php
// bootstrap.php
require __DIR__ . '/vendor/autoload.php';    // must exist
require_once __DIR__ . '/src/helpers.php';   // load once

$config = require __DIR__ . '/config/app.php';
echo $config['name'];                        // 'Shop'

if (is_file(__DIR__ . '/config/local.php')) {
    $config = array_replace($config, require __DIR__ . '/config/local.php');
}

include __DIR__ . '/views/header.php';       // Warning if missing

// config/app.php
return [
    'name'  => 'Shop',
    'debug' => (bool) getenv('APP_DEBUG'),
];

// views/header.php sees $config (same scope)
?>
<h1><?= htmlspecialchars($config['name']) ?></h1>
```

CODE EXPLAINED

→ `require` the Composer autoloader first

→ `require_once` prevents double function definitions

→ Included file's `return` becomes the expression value

→ Optional overrides merged only if the file exists

→ Templates included this way can read local variables

FOUR CONSTRUCTS

CONSTRUCT	MISSING FILE / REPEAT
`require`	Fatal error / loads again
`include`	Warning / loads again
`require_once`	Fatal error / skipped
`include_once`	Warning / skipped

RETURN VALUES

INCLUDED FILE	EXPRESSION GIVES
`return [...];`	That array
`return;`	null
`(no return)`	1
`missing (include)`	false
`_once, already loaded`	true

PATH TIPS

CODE	WHY
`__DIR__ . '/x.php'`	Relative to this file
`dirname(__DIR__)`	Parent folder
`get_included_files()`	Debug loaded files
`set_include_path()`	Rarely needed now
`Autoloading`	Classes load themselves

⚠ COMMON MISTAKES

- `include 'config.php'` relative to the working directory
- Including a file built from `$_GET` — remote/local file inclusion
- Using `include` where missing files should stop execution

✓ TIPS & BEST PRACTICES

- Use `require` for anything essential
- Return arrays from config files
- Let Composer autoload classes (Sheet 87)

QUICK SUMMARY

- `require` fatal, `include` warns
- `_once` loads once
- Files can `return` values
- Paths from `__DIR__`

Namespaces

ORGANISING NAMES

Namespaces group classes, functions and constants under a prefix such as `App\Models`, so names from different libraries don't collide. Declare one at the top of a file with `namespace`. Refer to names in other namespaces by their **fully qualified name** (`\App\Models\User`) or import them with `use`. Namespaces map directly to folders under PSR-4 autoloading.

NOTES

- One `namespace` per file is the norm
- Unqualified class names resolve relative to the current namespace
- Unqualified **function** and **constant** names fall back to global
- `use` is per file and resolved at compile time
- `::class` gives the fully qualified string

DECLARING AND IMPORTING

```php
<?php
declare(strict_types=1);

namespace App\Http\Controllers;

use App\Models\User;
use App\Models\{Order, Product};          // group use
use App\Services\Mailer as Mail;          // alias
use function App\Support\slugify;
use const App\Support\VERSION;

final class ShopController
{
    public function show(int $id): string
    {
        $user = new User($id);             // App\Models\User
        $now  = new \DateTimeImmutable();  // global class
        $len  = strlen('x');               // falls back to \strlen
        Mail::send($user);
        return slugify("product $id") . VERSION;
    }
}

echo ShopController::class;    // App\Http\Controllers\ShopController
echo __NAMESPACE__;            // App\Http\Controllers
```

CODE EXPLAINED

- → `namespace` comes right after `declare`
- → Group use imports several classes from one prefix
- → `as` renames an import
- → Leading `\` reaches the global namespace
- → Built-in functions resolve without importing

NAME RESOLUTION

WRITTEN AS	MEANS
`User`	Current ns or imported alias
`Models\User`	Relative to current ns
`\App\Models\User`	Fully qualified
`strlen()`	Current ns, then global
`namespace\f()`	Current ns explicitly

IMPORT FORMS

STATEMENT	IMPORTS
`use A\B;`	Class B
`use A\B as C;`	Class as C
`use A\{B, C};`	Several
`use function A\f;`	Function
`use const A\X;`	Constant

CONVENTIONS (PSR-4)

RULE	EXAMPLE
`Vendor prefix`	App\ or Acme\Shop\
`Folder = namespace`	src/Models → App\Models
`File = class`	User.php → User
`StudlyCaps`	OrderItem
`One class per file`	Always

⚠ COMMON MISTAKES

- Forgetting the `\` before global classes inside a namespace (`DateTime` not found)
- Namespace not matching the folder — autoloading fails
- Importing the same short name twice

✓ TIPS & BEST PRACTICES

- Let your IDE manage `use` statements
- Import global classes explicitly or prefix with `\`
- Match namespaces to directories

QUICK SUMMARY

- `namespace` prefixes names
- `use` imports / aliases
- `\` = global
- Maps to folders (PSR-4)

Autoloading & PSR-4

LOADING CLASSES ON DEMAND

Autoloading loads a class file the first time the class is used, so you never write `require` for classes. You register loaders with `spl_autoload_register()`; in practice **Composer** generates one for you. The **PSR-4** standard maps a namespace prefix to a base directory: `App\Models\User` → `src/Models/User.php`. Run `composer dump-autoload` after changing the mapping.

NOTES

- Autoloaders run only for classes, interfaces, traits and enums — not functions
- Load helper functions via Composer's `files` autoload
- `-o` builds an optimised classmap for production
- `--classmap-authoritative` skips filesystem checks entirely
- `class_exists($c, false)` checks without autoloading

COMPOSER PSR-4 AND A HAND-WRITTEN LOADER

```
// composer.json
{
    "autoload": {
        "psr-4": { "App\\": "src/" },
        "files": ["src/helpers.php"]
    },
    "autoload-dev": {
        "psr-4": { "Tests\\": "tests/" }
    }
}

<?php
// public/index.php
require __DIR__ . '/../vendor/autoload.php';
$user = new App\Models\User();   // loads src/Models/User.php

// What Composer does, simplified
spl_autoload_register(function (string $class): void {
    $prefix = 'App\\';
    if (!str_starts_with($class, $prefix)) return;
    $rel  = str_replace('\\', '/', substr($class, strlen($prefix)));
    $file = __DIR__ . "/src/$rel.php";
    if (is_file($file)) require $file;
});
```

CODE EXPLAINED

→ Namespace prefix (escaped `\\` in JSON) → folder

→ `files` always loads function helpers

→ `autoload-dev` maps test namespaces

→ One `require` of `vendor/autoload.php` is enough

→ A loader turns the class name into a file path

COMPOSER AUTOLOAD TYPES

TYPE	USE
`psr-4`	Namespace → directory
`classmap`	Scan folders for classes
`files`	Always include
`psr-0`	Legacy (avoid)
`exclude-from-classmap`	Skip paths

COMMANDS

COMMAND	EFFECT
`composer dump-autoload`	Regenerate
`composer dump-autoload -o`	Optimised classmap
`--classmap-authoritative (-a)`	Classmap only
`--apcu`	Cache lookups in APCu
`composer install --no-dev -o`	Production

PSR-4 MAPPING

CLASS	FILE
`App\User`	src/User.php
`App\Models\Order`	src/Models/Order.php
`App\Http\Kernel`	src/Http/Kernel.php
`Tests\UserTest`	tests/UserTest.php
`Case`	Must match exactly on Linux

⚠ COMMON MISTAKES

- File name case differs from class name — works on macOS, fails on Linux
- Forgetting `dump-autoload` after editing composer.json
- Expecting functions to autoload

✓ TIPS & BEST PRACTICES

- Use Composer's autoloader everywhere
- Optimise the classmap in production builds
- Keep one class per file, named exactly like the class

QUICK SUMMARY

- Classes load on first use
- PSR-4: namespace → folder
- `vendor/autoload.php`
- `dump-autoload -o` for prod

Composer Basics

composer init · require · require --dev · composer.json · composer.lock · install vs update · ^ and ~ constraints

PHP'S PACKAGE MANAGER

Composer installs libraries from **Packagist** into `vendor/` and generates the autoloader. `composer.json` lists what you need with **version constraints**; `composer.lock` records the exact versions installed. `composer install` installs exactly what the lock file says (use it in CI and production); `composer update` resolves new versions within your constraints and rewrites the lock.

NOTES

- Commit `composer.json` **and** `composer.lock` for applications
- Never commit `vendor/`
- `require-dev` packages are skipped with `--no-dev`
- Packages follow semantic versioning: MAJOR.MINOR.PATCH
- `composer outdated` shows available upgrades

EVERYDAY COMMANDS

```
composer init                       # create composer.json
composer require guzzlehttp/guzzle  # add + install latest stable
composer require monolog/monolog:^3.5
composer require --dev phpunit/phpunit pestphp/pest
composer install                    # from composer.lock
composer install --no-dev -o        # production
composer update                     # newest allowed versions
composer update monolog/monolog     # just one package
composer remove guzzlehttp/guzzle
composer outdated --direct          # what's behind
composer show -t                    # dependency tree
composer validate                   # check composer.json

# composer.json (excerpt)
#  "require": {
#      "php": "^8.3",
#      "ext-pdo": "*",
#      "guzzlehttp/guzzle": "^7.8"
#  }
```

CODE EXPLAINED

- → `require` edits composer.json and installs
- → `:^3.5` pins a constraint at install time
- → `--dev` for test and tooling packages
- → `install` follows the lock file exactly
- → `php` and `ext-*` declare platform requirements

VERSION CONSTRAINTS

CONSTRAINT	ALLOWS
`^1.2.3`	>=1.2.3 <2.0.0
`^0.3`	>=0.3.0 <0.4.0
`~1.2`	>=1.2 <2.0
`~1.2.3`	>=1.2.3 <1.3.0
`1.2.*`	>=1.2.0 <1.3.0
`>=1.0 <2.0`	Range
`1.2.3`	Exact

INSTALL VS UPDATE

ASPECT	INSTALL / UPDATE
Reads	lock / json
Writes lock	no (unless missing) / yes
Versions	exact / newest allowed
Use in CI/prod	yes / no
Use when	cloning / upgrading

PROJECT FILES

FILE	PURPOSE
`composer.json`	Requirements
`composer.lock`	Exact versions
`vendor/`	Installed packages
`vendor/autoload.php`	Autoloader
`vendor/bin/`	Package CLIs (phpunit)

⚠ COMMON MISTAKES

- Running `composer update` on production servers
- Not committing `composer.lock` for an app
- Using `*` constraints

✓ TIPS & BEST PRACTICES

- Use `^` constraints for libraries you depend on
- Run `composer audit` in CI (Sheet 89)
- Call tools via `vendor/bin/…`

QUICK SUMMARY

- `require` adds packages
- `install` = lock, `update` = resolve
- Commit the lock file
- `^` = compatible versions

Composer — Scripts & Publishing

BEYOND INSTALLING

Composer is also a task runner and publishing tool. **Scripts** in `composer.json` define shortcuts such as `composer test`. **Repositories** add sources beyond Packagist — a private VCS repo or a local path for developing packages side by side. `composer audit` checks installed packages against security advisories, and `config.platform` pins the PHP version used for dependency resolution.

NOTES

- Scripts can call other scripts with `@name`
- `composer run-script --list` shows available scripts
- Publish a library by tagging a release and submitting the repo to Packagist
- Libraries commit `composer.json` but usually not the lock file
- `composer bump` raises constraints to installed versions

COMPOSER.JSON WITH SCRIPTS AND REPOSITORIES

```
{
    "name": "acme/shop",
    "type": "project",
    "require": {
        "php": "^8.3",
        "acme/payments": "^2.0"
    },
    "repositories": [
        { "type": "vcs",  "url": "git@github.com:acme/payments.git" },
        { "type": "path", "url": "../packages/*" }
    ],
    "scripts": {
        "test": "pest --parallel",
        "lint": "php-cs-fixer fix --dry-run",
        "stan": "phpstan analyse",
        "check": ["@lint", "@stan", "@test"]
    },
    "config": {
        "platform": { "php": "8.3.0" },
        "sort-packages": true
    },
    "minimum-stability": "stable"
}
```

CODE EXPLAINED

- → `vcs` repository — private Git package
- → `path` repository — symlinks local packages
- → Scripts become `composer test`, `composer check`
- → `@name` chains scripts
- → `platform.php` resolves deps for production's PHP version

USEFUL COMMANDS

COMMAND	DOES
`composer audit`	Security advisories
`composer bump`	Tighten constraints
`composer why vendor/pkg`	Who requires it
`composer why-not vendor/pkg 3.0`	What blocks it
`composer check-platform-reqs`	PHP/ext check
`composer licenses`	License summary

REPOSITORY TYPES

TYPE	SOURCE
`composer`	Packagist-like server
`vcs`	Git / Hg repo
`path`	Local folder (symlink)
`package`	Inline definition
`artifact`	Folder of zips

SCRIPT EVENTS

EVENT	RUNS
`post-install-cmd`	After install
`post-update-cmd`	After update
`post-autoload-dump`	After dump-autoload
`pre-commit (custom)`	Via @script
`custom names`	composer <name>

⚠ COMMON MISTAKES

- Leaving `minimum-stability: dev` without `prefer-stable`
- Ignoring `composer audit` warnings
- Resolving dependencies with a newer local PHP than production

✓ TIPS & BEST PRACTICES

- Add `composer audit` to CI
- Set `config.platform.php` to production's version
- Use `path` repos for monorepo packages

QUICK SUMMARY

- Scripts = task runner
- Repositories beyond Packagist
- `composer audit`
- Pin the platform PHP

Math & Random

NUMBERS AND RANDOMNESS

PHP's math functions cover rounding, extremes, powers, roots, logarithms and trigonometry, plus constants like `M_PI`. For random numbers, use `random_int()` and `random_bytes()` — they are **cryptographically secure**. PHP 8.2's `Random\Randomizer` class adds seedable, object-based random generation with methods for shuffling, picking keys and generating strings. Avoid `rand()` and `mt_rand()` for anything security-related.

NOTES

- `max()` / `min()` accept several values or one array
- `round()` accepts negative precision: `round(1234, -2)` = 1200
- `intdiv()` and `%` for integer math (Sheet 22)
- `Randomizer` with a seeded engine gives repeatable results in tests
- `random_int()` throws if no secure source is available

MATH AND RANDOM FUNCTIONS

```php
<?php
abs(-4.2);              // 4.2
max(3, 9, 1);           // 9
max([3, 9, 1]);         // 9
min([]);                // ValueError
round(1234.567, 1);     // 1234.6
round(1234.567, -2);    // 1200.0
sqrt(16);               // 4.0
pow(2, 8);              // 256
log(M_E);               // 1.0
log10(1000);            // 3.0
hypot(3, 4);            // 5.0
array_sum([1.5, 2.5]);  // 4.0

$dice  = random_int(1, 6);              // secure
$token = bin2hex(random_bytes(16));     // 32 hex chars

$r = new Random\Randomizer();
$r->getInt(1, 100);
$r->shuffleArray(['a', 'b', 'c']);
$r->pickArrayKeys(['x' => 1, 'y' => 2, 'z' => 3], 2);
$r->getBytesFromString('abcdef0123456789', 8);   // 8.3

$seeded = new Random\Randomizer(new Random\Engine\Mt19937(42));
$seeded->getInt(1, 10);                 // same every run
```

CODE EXPLAINED

→ `max` / `min` take values or an array; empty array throws

→ Negative precision rounds to tens, hundreds…

→ `random_int` for games, codes, anything that matters

→ `random_bytes` + `bin2hex` = random token

→ Seeded engine → reproducible sequences

MATH FUNCTIONS

FUNCTION	RESULT
`abs($x)`	Absolute value
`round / floor / ceil`	Rounding
`fmod($a, $b)`	Float remainder
`sqrt / pow / exp`	Roots, powers
`log($x, $base)`	Logarithm
`sin / cos / tan / atan2`	Trig (radians)
`deg2rad / rad2deg`	Convert angles

RANDOM OPTIONS

FUNCTION	SECURE?
`random_int($a, $b)`	Yes
`random_bytes($n)`	Yes
`Randomizer (default engine)`	Yes
`Randomizer(Mt19937)`	No — seeded
`mt_rand() / rand()`	No
`shuffle() / array_rand()`	No

CONSTANTS

CONSTANT	VALUE
`M_PI`	3.14159…
`M_E`	2.71828…
`M_SQRT2`	1.41421…
`PHP_INT_MAX`	Largest int
`PHP_FLOAT_EPSILON`	Float precision
`NAN / INF`	Special floats

⚠ COMMON MISTAKES

- `rand()` or `mt_rand()` for tokens or passwords
- `max([])` on possibly empty data — `ValueError`
- Trig functions with degrees instead of radians

✓ TIPS & BEST PRACTICES

- Use `random_int` / `random_bytes` by default
- Use a seeded `Randomizer` in tests
- Use `Randomizer::getBytesFromString()` for codes from an alphabet

QUICK SUMMARY

- round / floor / ceil / abs
- max / min / sqrt / pow
- `random_int` is secure
- `Random\Randomizer` (8.2)

Dates — date() & Timestamps

time() · date() · format codes · mktime() · strtotime() · checkdate() · date_default_timezone_set()

PROCEDURAL DATE FUNCTIONS

A **Unix timestamp** is the number of seconds since 1970-01-01 00:00:00 UTC; `time()` returns the current one. `date($format, $ts)` formats a timestamp in the default timezone using format letters such as `Y-m-d H:i:s`. `strtotime()` parses English text (`'next monday'`, `'+1 week'`) into a timestamp and `mktime()` builds one from parts. For anything beyond simple formatting, prefer `DateTimeImmutable` (Sheet 92).

NOTES

- Set the default timezone in php.ini (`date.timezone`) or at startup
- Store times in UTC; convert for display
- `strtotime()` returns `false` on unparseable input
- `microtime(true)` gives a float with microseconds
- `hrtime(true)` gives nanoseconds for timing code

TIMESTAMPS AND FORMATTING

```php
<?php
date_default_timezone_set('UTC');

$now = time();                          // e.g. 1791468309
echo date('Y-m-d H:i:s');               // 2026-10-08 14:05:09
echo date('D, d M Y', $now);            // Thu, 08 Oct 2026
echo date('l jS \of F Y');              // Thursday 8th of October 2026
echo date(DATE_ATOM);                   // 2026-10-08T14:05:09+00:00
echo date('N');                         // 4 (ISO: Thu)

$ts = mktime(9, 30, 0, 12, 25, 2026);  // h, i, s, month, day, year
$ts = strtotime('2026-12-25 09:30');
$ts = strtotime('+1 week 2 days');
$ts = strtotime('last day of next month');
if (strtotime('not a date') === false) { /* invalid */ }

checkdate(2, 29, 2027);                 // false (not leap)

$start = hrtime(true);
usleep(1000);
$ms = (hrtime(true) - $start) / 1e6;    // elapsed milliseconds
```

CODE EXPLAINED

→ Always set a timezone

→ Format letters; escape literal letters with `\`

→ `DATE_ATOM` is ISO 8601

→ `mktime` argument order is hour, minute, second, month, day, year

→ `hrtime()` is monotonic — good for measuring durations

FORMAT CODES

CODE	EXAMPLE
`Y` / `y`	2026 / 26
`m` / `n` / `M` / `F`	10 / 10 / Oct / October
`d` / `j` / `D` / `l`	08 / 8 / Thu / Thursday
`H` / `G` / `h` / `A`	14 / 14 / 02 / PM
`i` / `s` / `v`	minutes / seconds / ms
`N` / `W` / `t` / `L`	ISO weekday / week / days in month / leap
`U` / `e` / `T` / `P`	timestamp / tz id / abbr / +00:00

FORMAT CONSTANTS

CONSTANT	OUTPUT
`DATE_ATOM`	2026-10-08T14:05:09+00:00
`DATE_RFC2822`	Thu, 08 Oct 2026 14:05:09 +0000
`DATE_COOKIE`	Thursday, 08-Oct-2026 …
`DATE_RSS`	RFC 822 style
`'c'`	ISO 8601 (same as ATOM)

STRTOTIME PHRASES

INPUT	MEANING
`'now'` / `'today'`	Current / midnight
`'tomorrow'` / `'yesterday'`	Midnight ±1 day
`'+1 day'` / `'-2 weeks'`	Relative
`'next monday'`	Next Monday
`'first day of next month'`	Month start
`'@1700000000'`	From timestamp

⚠ COMMON MISTAKES

- Leaving the timezone unset — results depend on server config
- Adding `86400` to go to tomorrow — wrong across DST
- Confusing `m` (month) and `i` (minutes)

✓ TIPS & BEST PRACTICES

- Store UTC; format in the user's timezone
- Use `DateTimeImmutable` for arithmetic
- Use `hrtime()` for benchmarks

QUICK SUMMARY

- Timestamps = seconds since 1970
- `date()` formats
- `strtotime()` parses text
- Set a timezone

DateTime Objects

OBJECT-ORIENTED DATES

`DateTimeImmutable` represents a moment with a timezone; every change returns a **new** object, so values can't be altered by accident. (`DateTime` is the mutable version — avoid it in new code.) Parse with the constructor or `createFromFormat()`, shift with `modify()` or `add()`, compare with `< > ==`, and measure gaps with `diff()`, which returns a `DateInterval`. `DatePeriod` iterates over ranges.

NOTES

- `DateTimeInterface` covers both classes for type hints
- `setTimezone()` converts the same instant to another zone
- PHP 8.3 throws specific exceptions like `DateMalformedStringException`
- `createFromFormat('!Y-m-d', …)` resets unspecified fields to zero
- Libraries: Carbon, Chronos build on these classes

CREATING, CHANGING, COMPARING

```php
<?php
$tz  = new DateTimeZone('Europe/Berlin');
$now = new DateTimeImmutable('now', $tz);
$due = new DateTimeImmutable('2026-12-24 18:00', $tz);

$d = DateTimeImmutable::createFromFormat('d/m/Y', '25/12/2026');
echo $d->format('Y-m-d');                    // 2026-12-25

$next = $now->modify('+1 month');            // $now unchanged
$later = $now->add(new DateInterval('P1DT2H'));   // +1 day 2 hours
$utc = $now->setTimezone(new DateTimeZone('UTC'));

var_dump($due > $now);                       // compare directly
$gap = $now->diff($due);
echo $gap->days, ' days, ', $gap->h, ' hours';
echo $gap->format('%a days %R');             // total days, sign

$period = new DatePeriod(
    new DateTimeImmutable('2026-10-01'),
    new DateInterval('P1W'),
    new DateTimeImmutable('2026-11-01'),
);
foreach ($period as $week) echo $week->format('M j'), ' ';
// Oct 1 Oct 8 Oct 15 Oct 22 Oct 29
```

CODE EXPLAINED

→ Always pass a timezone (or set a default)

→ `createFromFormat()` parses a known pattern

→ Immutable methods return new objects

→ `DateInterval` spec: P = period, T = time part

→ `DatePeriod` yields each step in the range

KEY METHODS

METHOD	RETURNS
`format($f)`	String
`modify('+1 day')`	New object
`add($i) / sub($i)`	New object
`setTimezone($tz)`	Same instant, new zone
`setDate / setTime`	Changed fields
`diff($other)`	DateInterval
`getTimestamp()`	int

DATEINTERVAL SPECS

SPEC	MEANS
`P1D`	1 day
`P2W`	2 weeks
`P1M`	1 month
`P1Y2M`	1 year 2 months
`PT30M`	30 minutes
`P1DT12H`	1 day 12 hours

IMMUTABLE VS MUTABLE

ASPECT	IMMUTABLE / DATETIME
`modify()`	New object / Changes in place
`Safe to share`	Yes / No
`Convert`	::createFromMutable / ::createFromImmutable
`Type hint both`	DateTimeInterface
`Recommended`	Yes / Legacy

⚠ COMMON MISTAKES

- Using mutable `DateTime` and changing a shared value
- Comparing `format()` strings instead of objects
- Forgetting timezones when parsing user input

✓ TIPS & BEST PRACTICES

- Use `DateTimeImmutable` everywhere
- Inject a clock (PSR-20) to test time-dependent code
- Store UTC, convert at the edges

QUICK SUMMARY

- `DateTimeImmutable` + timezone
- `modify` / `add` return new objects
- `diff()` → `DateInterval`
- `DatePeriod` for ranges

System & Environment

TALKING TO THE SYSTEM

Read environment variables with `getenv()` (or `$_ENV` / `$_SERVER`, depending on `variables_order`). Runtime settings can be read with `ini_get()` and some changed with `ini_set()`. To run external commands, use `exec()`, `shell_exec()`, `system()` or `proc_open()` for full control. **Every** value passed to a shell must be escaped with `escapeshellarg()` to prevent command injection.

NOTES

- `$_ENV` is often empty — `variables_order` lacks `E` by default in production ini
- Libraries like `vlucas/phpdotenv` load `.env` files
- `PHP_OS_FAMILY`: `Linux`, `Darwin`, `Windows`, `BSD`
- Shell functions may be disabled via `disable_functions`
- Symfony Process is a robust `proc_open` wrapper

ENVIRONMENT AND COMMANDS

```php
<?php
$dbHost = getenv('DB_HOST') ?: 'localhost';
$debug  = filter_var(getenv('APP_DEBUG'), FILTER_VALIDATE_BOOL);
putenv('TZ=UTC');                       // for this process

echo PHP_OS_FAMILY, ' ', php_uname('m');  // Linux x86_64
echo ini_get('memory_limit');           // 128M
ini_set('memory_limit', '512M');        // where allowed
echo memory_get_peak_usage(true);       // bytes

$file = $_GET['file'] ?? 'report.txt';
exec('wc -l ' . escapeshellarg($file), $out, $code);
echo $out[0], ' (exit ', $code, ')';

$proc = proc_open(['git', 'log', '-1', '--format=%h'],   // array: no shell
    [1 => ['pipe', 'w'], 2 => ['pipe', 'w']], $pipes);
$hash = trim(stream_get_contents($pipes[1]));
proc_close($proc);

echo gethostname(), ' pid ', getmypid();
```

CODE EXPLAINED

- → `?:` supplies a default when the variable is unset/empty
- → `FILTER_VALIDATE_BOOL` parses "true"/"1"/"on"
- → `ini_set` only works for settings changeable at runtime
- → `escapeshellarg()` quotes user input for the shell
- → `proc_open` with an array bypasses the shell (7.4+)

ENVIRONMENT

SOURCE	NOTES
`getenv('X')`	Reliable; false if unset
`$_ENV['X']`	Needs E in variables_order
`$_SERVER['X']`	Often has env under FPM
`putenv('X=1')`	Set for this process
`.env + phpdotenv`	Local development

RUN COMMANDS

FUNCTION	RETURNS
`exec($cmd, $out, $code)`	Last line; lines in $out
`shell_exec($cmd)`	Full output
`system($cmd)`	Passes output through
`passthru($cmd)`	Raw binary output
`proc_open([...])`	Pipes + exit code
`` `backticks` ``	Same as shell_exec

SYSTEM INFO

FUNCTION	RETURNS
`PHP_OS_FAMILY`	OS family
`php_uname()`	Kernel info
`sys_get_temp_dir()`	Temp folder
`memory_get_usage()`	Bytes used
`getmypid()`	Process ID
`gethostname()`	Host name

⚠ COMMON MISTAKES

- Concatenating user input into shell commands
- Hard-coding secrets instead of reading environment variables
- Relying on `$_ENV` without checking `variables_order`

✓ TIPS & BEST PRACTICES

- Pass commands as arrays to `proc_open`
- Use Symfony Process for timeouts and streaming
- Read config from the environment, with defaults

QUICK SUMMARY

- `getenv()` for config
- `ini_get` / `ini_set`
- `escapeshellarg()` always
- `proc_open([...])` avoids the shell

PHPUnit Basics

TestCase · assertSame() · assertEquals() · setUp() · expectException() · phpunit.xml · vendor/bin/phpunit

UNIT TESTING WITH PHPUNIT

PHPUnit is the standard PHP test framework. A test class extends `PHPUnit\Framework\TestCase`; each public method whose name starts with `test` (or has the `#[Test]` attribute) is a test. Assertions such as `assertSame()` compare expected and actual values. `setUp()` runs before every test for shared fixtures. Configure suites and coverage in `phpunit.xml` and run `vendor/bin/phpunit`.

NOTES

- Install per project: `composer require --dev phpunit/phpunit`
- Test class names end in `Test` ; files mirror `src/`
- Each test runs on a fresh instance of the test class
- PHPUnit 10+ uses attributes instead of docblock annotations
- Arrange · Act · Assert keeps tests readable

A TEST CLASS

```php
<?php
namespace Tests\Unit;

use App\Cart;
use PHPUnit\Framework\TestCase;
use PHPUnit\Framework\Attributes\Test;

final class CartTest extends TestCase
{
    private Cart $cart;

    protected function setUp(): void
    { $this->cart = new Cart(); }

    public function testStartsEmpty(): void
    { $this->assertSame(0, $this->cart->count()); }

    #[Test]
    public function it_totals_items(): void
    {
        $this->cart->add('mug', 2, 8.50);
        $this->assertSame(17.0, $this->cart->total());
    }

    public function testRejectsNegativeQty(): void
    {
        $this->expectException(\InvalidArgumentException::class);
        $this->cart->add('mug', -1, 8.50);
    }
}
```

CODE EXPLAINED

- → Extend `TestCase` ; one class per unit under test
- → `setUp()` creates a fresh cart before each test
- → `assertSame` checks type and value (`===`)
- → `#[Test]` marks methods not named `test…`
- → `expectException` must come before the failing call

COMMON ASSERTIONS

ASSERTION	PASSES WHEN
`assertSame($e, $a)`	$e === $a
`assertEquals($e, $a)`	$e == $a
`assertTrue($x) / assertFalse`	Exactly bool
`assertNull($x)`	null
`assertCount($n, $arr)`	Size
`assertContains($v, $arr)`	Contains value
`assertInstanceOf(C::class, $o)`	Type

LIFECYCLE HOOKS

METHOD	RUNS
`setUp()`	Before each test
`tearDown()`	After each test
`setUpBeforeClass()`	Once, static
`tearDownAfterClass()`	Once, static
`#[Before] / #[After]`	Attribute versions

RUNNING TESTS

COMMAND	DOES
`vendor/bin/phpunit`	All suites
`--filter CartTest`	Match name
`--testsuite unit`	One suite
`--testdox`	Readable output
`--stop-on-failure`	Fail fast
`--coverage-html out`	Coverage report

⚠ COMMON MISTAKES

- `assertEquals` where `assertSame` is needed — `'1' == 1` passes
- Tests depending on each other's state or order
- Calling `expectException` after the code that throws

✓ TIPS & BEST PRACTICES

- Prefer `assertSame` by default
- One behaviour per test, named after the behaviour
- Run tests in CI on every push

QUICK SUMMARY

- Extend `TestCase`
- `test*` methods or `#[Test]`
- `assertSame` & friends
- `setUp()` for fixtures

PHPUnit — Data Providers & Test Doubles

#[DataProvider] · createMock() · createStub() · expects() · willReturn() · code coverage

MANY INPUTS, FAKE COLLABORATORS

A **data provider** feeds many input/expected pairs into one test method, so you write the logic once. Providers are public **static** methods referenced with `#[DataProvider('name')]`. **Test doubles** replace real dependencies: a **stub** returns canned values (`createStub()`), a **mock** also verifies how it was called (`createMock()` + `expects()`). Mock interfaces, not concrete classes.

NOTES

- Name provider rows with string keys for readable failures
- `final` classes can't be doubled — depend on interfaces
- Don't mock what you don't own; wrap third-party APIs
- Coverage needs Xdebug or PCOV
- `#[CoversClass(Cart::class)]` scopes coverage

PROVIDERS AND DOUBLES

```php
<?php
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class SlugTest extends TestCase
{
    public static function slugs(): array
    {
        return [
            'spaces'  => ['Hello World', 'hello-world'],
            'symbols' => ['PHP & You!', 'php-you'],
            'trim'    => ['  x  ', 'x'],
        ];
    }

    #[DataProvider('slugs')]
    public function testSlugify(string $in, string $out): void
    {
        $this->assertSame($out, slugify($in));
    }

    public function testSendsWelcomeMail(): void
    {
        $mailer = $this->createMock(Mailer::class);    // interface
        $mailer->expects($this->once())
               ->method('send')
               ->with('ana@x.io', $this->stringContains('Welcome'))
               ->willReturn(true);

        (new Signup($mailer))->register('ana@x.io');
    }
}
```

CODE EXPLAINED

→ Provider is `public static` and returns rows of arguments

→ Row keys appear in failure messages

→ Each row runs as a separate test

→ `createMock` + `expects(once())` verifies the call

→ `with()` constrains arguments; `willReturn()` sets the result

DOUBLE TYPES

KIND	PURPOSE
`Stub`	Return canned values
`Mock`	Verify interactions
`Fake`	Working lightweight impl
`Spy`	Record calls, assert later
`Dummy`	Fills a parameter

MOCK CONFIGURATION

METHOD	DOES
`expects($this->once())`	Called exactly once
`->method('name')`	Which method
`->with($a, $b)`	Expected arguments
`->willReturn($v)`	Return value
`->willThrowException($e)`	Throw
`->willReturnCallback($fn)`	Compute

INVOCATION MATCHERS

MATCHER	MEANS
`once()`	1 call
`never()`	0 calls
`exactly($n)`	n calls
`atLeastOnce()`	≥1
`any()`	Don't care

⚠ COMMON MISTAKES

- Non-static data providers (PHPUnit 10+ requires static)
- Mocking value objects or the class under test
- Over-specifying calls so refactors break tests

✓ TIPS & BEST PRACTICES

- Prefer simple fakes over complex mock setups
- Use providers for edge-case tables
- Mock interfaces at architectural boundaries

QUICK SUMMARY

- `#[DataProvider]` + static method
- Stubs return, mocks verify
- Mock interfaces
- Coverage via Xdebug/PCOV

Pest

test() · it() · expect() · datasets · beforeEach() · arch tests · pest --parallel

EXPRESSIVE TESTS ON PHPUNIT

Pest is a testing framework built on top of PHPUnit with a function-based syntax. Tests are closures passed to `test()` or `it()`, and assertions use a chainable `expect($value)->toBe(...)` API. Pest supports datasets, hooks, higher-order tests, parallel runs and **architecture tests** that enforce rules like "controllers don't use the database". It runs existing PHPUnit tests unchanged.

NOTES

- Install: `composer require pestphp/pest --dev`, then `./vendor/bin/pest --init`
- `$this` in a Pest closure is the underlying TestCase
- `uses(TestCase::class)->in('Feature')` sets the base class
- `->only()` and `->skip()` focus or skip tests
- Laravel ships with Pest by default

PEST TESTS

```php
<?php
beforeEach(function () {
    $this->cart = new Cart();
});

test('a new cart is empty', function () {
    expect($this->cart->count())->toBe(0);
});

it('totals items', function () {
    $this->cart->add('mug', 2, 8.50);
    expect($this->cart->total())->toEqualWithDelta(17.0, 0.001)
        ->and($this->cart->count())->toBe(1);
});

it('slugifies', function (string $in, string $out) {
    expect(slugify($in))->toBe($out);
})->with([
    ['Hello World', 'hello-world'],
    ['PHP & You!', 'php-you'],
]);

it('rejects negative quantities', fn () =>
    $this->cart->add('mug', -1, 1.0)
)->throws(InvalidArgumentException::class);

arch('controllers do not use the DB directly')
    ->expect('App\Http\Controllers')
    ->not->toUse('Illuminate\Support\Facades\DB');
```

CODE EXPLAINED

- → `beforeEach` sets up `$this->cart`
- → `expect()->toBe()` is a strict `===` check
- → `->and()` chains a second expectation
- → `->with()` supplies a dataset
- → `arch()` enforces dependency rules

EXPECTATIONS

METHOD	CHECKS
`toBe($v)`	`===`
`toEqual($v)`	`==`
`toBeTrue() / toBeNull()`	Exact values
`toHaveCount($n)`	Size
`toContain($v)`	Contains
`toBeInstanceOf(C::class)`	Type
`toThrow(E::class)`	Closure throws

STRUCTURE

FUNCTION	PURPOSE
`test() / it()`	Define a test
`describe()`	Group tests
`beforeEach / afterEach`	Hooks
`dataset('name', [...])`	Shared data
`uses(...)->in('dir')`	Base class/traits
`arch()`	Architecture rule

CLI

COMMAND	DOES
`pest`	Run all
`pest --filter=cart`	Match
`pest --parallel`	Multi-process
`pest --coverage --min=80`	Enforce coverage
`pest --dirty`	Changed files only
`pest --profile`	Slowest tests

⚠ COMMON MISTAKES

- Arrow function tests that need several statements
- Forgetting `->with()` when a test closure has parameters
- Mixing styles inconsistently in one project

✓ TIPS & BEST PRACTICES

- Use `describe()` to group related tests
- Add arch tests for layering rules
- Run `--parallel` in CI

QUICK SUMMARY

- Closures + `expect()`
- Datasets via `->with()`
- Arch tests
- Runs on PHPUnit

Debugging with Xdebug

REAL DEBUGGING

Xdebug is a PHP extension for step debugging, profiling, tracing and code coverage. With `xdebug.mode=debug` it connects to your IDE (PhpStorm, VS Code) so you can set breakpoints, step through code and inspect variables — far faster than scattering `var_dump()`. Turn modes on only when needed: Xdebug slows PHP down significantly. **PCOV** is a lighter alternative for coverage only.

NOTES

- Xdebug 3 uses `xdebug.mode` (debug, develop, coverage, profile, trace)
- Default client port is **9003**
- `start_with_request=trigger` debugs only when asked (cookie/env)
- In Docker, set `client_host=host.docker.internal`
- Never enable Xdebug in production

CONFIGURATION AND TRIGGERS

```
; php.ini / conf.d/xdebug.ini
zend_extension=xdebug
xdebug.mode=debug,develop
xdebug.start_with_request=trigger
xdebug.client_host=127.0.0.1
xdebug.client_port=9003
xdebug.output_dir=/tmp/xdebug

; Profiling instead (writes cachegrind files)
; xdebug.mode=profile

; ── Triggering a session ──────────────────────
; Browser: install "Xdebug helper" → sets XDEBUG_TRIGGER cookie
; URL:     https://app.test/?XDEBUG_TRIGGER=1
; CLI:     XDEBUG_TRIGGER=1 php artisan migrate
; CLI:     php -dxdebug.mode=debug -dxdebug.start_with_request=yes t.php

; Check it's loaded
; php -v            → "with Xdebug v3.x"
; php -r 'xdebug_info();'
```

CODE EXPLAINED

- → `zend_extension` loads Xdebug
- → `develop` mode improves `var_dump` and error output
- → `trigger` avoids slowing every request
- → Profile output opens in KCachegrind / QCacheGrind
- → `xdebug_info()` shows the active configuration

MODES

MODE	USE
`off`	Disabled (default)
`develop`	Better errors, var_dump
`debug`	Step debugging
`coverage`	Code coverage
`profile`	Performance profiling
`trace`	Log every function call

DEBUGGER ACTIONS

ACTION	EFFECT
`Breakpoint`	Pause at a line
`Step over`	Next line, skip calls
`Step into`	Enter the call
`Step out`	Finish current function
`Watch`	Evaluate expressions
`Conditional bp`	Pause when true

TROUBLESHOOTING

SYMPTOM	CHECK
`IDE never stops`	Mode, trigger, port 9003
`Docker can't connect`	client_host
`Wrong file paths`	IDE path mappings
`Very slow`	Disable unused modes
`Not loaded`	php -v, php --ini

⚠ COMMON MISTAKES

- Leaving `xdebug.mode=debug` + `start_with_request=yes` on always
- Using port 9000 from Xdebug 2 docs
- Missing path mappings for Docker or VMs

✓ TIPS & BEST PRACTICES

- Use trigger mode and a browser extension
- Use conditional breakpoints in loops
- Use PCOV in CI when you only need coverage

QUICK SUMMARY

- `xdebug.mode` picks features
- Port 9003
- Trigger-based sessions
- Never in production

Static Analysis — PHPStan & Psalm

PHPStan levels · phpstan.neon · baseline · Psalm · @template generics · @phpstan-ignore

FINDING BUGS WITHOUT RUNNING CODE

Static analysers read your code and report type errors, impossible conditions, undefined methods and dead code before anything runs. **PHPStan** uses levels 0–10 (higher = stricter); **Psalm** offers similar checks plus taint analysis for security. Both understand PHPDoc types such as generics (`@template`, `array<int, User>`), letting you describe what PHP's native type system can't. Start low, fix, raise the level.

NOTES

- Install: `composer require --dev phpstan/phpstan`
- A **baseline** records existing errors so you can adopt it gradually
- Framework extensions add knowledge of magic (Laravel: Larastan)
- `@phpstan-ignore` with an identifier silences one error
- Run it in CI on every pull request

CONFIG, GENERICS & RUNNING

```
# phpstan.neon
parameters:
    level: 8
    paths:
        - src
        - tests
    excludePaths:
        - src/Legacy/*
includes:
    - phpstan-baseline.neon
```

```php
<?php
/**
 * @template T
 */
final class Collection
{
    /** @param list<T> $items */
    public function __construct(private array $items) {}

    /** @return T|null */
    public function first(): mixed { return $this->items[0] ?? null; }
}

/** @var Collection<User> $users */
$users = new Collection([new User()]);
$users->first()?->email;      // PHPStan knows this is ?User
```

```
# vendor/bin/phpstan analyse
# vendor/bin/phpstan analyse --generate-baseline
# vendor/bin/psalm --init && vendor/bin/psalm
```

CODE EXPLAINED

- → Level 8+ checks nullability strictly
- → Baseline file holds known errors
- → `@template T` declares a type parameter
- → Analyser infers `first()` returns `?User`
- → Generate a baseline, then pay it down

PHPSTAN LEVELS

LEVEL	ADDS
0	Unknown classes, functions
3	Return types
5	Argument types
6	Missing typehints
8	Nullable checks
9	Strict mixed
10	Implicit mixed (2.0)

PHPDOC TYPES

TYPE	MEANING
`list<int>`	Sequential int list
`array<string, User>`	Map
`non-empty-string`	Not ''
`positive-int`	> 0
`class-string<T>`	Class name of T
`callable(int): string`	Typed callable

TOOL COMPARISON

FEATURE	PHPSTAN / PSALM
Levels	0–10 / 1–8 (1 strictest)
Generics	Yes / Yes
Taint analysis	No / Yes
Baseline	Yes / Yes
Auto-fix	No / Psalter

⚠ COMMON MISTAKES

- Starting at max level on a legacy codebase
- Silencing errors without an identifier or comment
- Docblock types that lie about runtime behaviour

✓ TIPS & BEST PRACTICES

- Run in CI and fail on new errors
- Raise the level step by step
- Add framework extensions for magic APIs

QUICK SUMMARY

- Analyse without running
- PHPStan levels 0–10
- Generics via `@template`
- Baseline for adoption

Code Style & Refactoring Tools

CONSISTENT, MODERN CODE

PER Coding Style (the successor to PSR-12) defines the community formatting standard: 4-space indents, braces on their own line for classes and methods, one class per file. **PHP-CS-Fixer** and **PHP_CodeSniffer** (`phpcs` / `phpcbf`) check and fix style automatically. **Rector** goes further and rewrites code — upgrading syntax to newer PHP versions or applying refactorings across a project.

NOTES

- Run fixers in a pre-commit hook or CI
- `--dry-run --diff` previews changes
- Laravel Pint is a PHP-CS-Fixer preset
- Rector sets: `UP_TO_PHP_84` , dead code, code quality
- `php -l` checks syntax only

CONFIGS AND COMMANDS

```php
<?php
// .php-cs-fixer.dist.php
$finder = PhpCsFixer\Finder::create()
    ->in([__DIR__ . '/src', __DIR__ . '/tests']);

return (new PhpCsFixer\Config())
    ->setRiskyAllowed(true)
    ->setRules([
        '@PER-CS' => true,
        '@PHP84Migration' => true,
        'declare_strict_types' => true,
        'ordered_imports' => true,
        'no_unused_imports' => true,
    ])
    ->setFinder($finder);

// rector.php
use Rector\Config\RectorConfig;
return RectorConfig::configure()
    ->withPaths([__DIR__ . '/src'])
    ->withPhpSets(php84: true)
    ->withPreparedSets(deadCode: true, codeQuality: true);

// vendor/bin/php-cs-fixer fix --dry-run --diff
// vendor/bin/phpcs --standard=PSR12 src
// vendor/bin/rector process --dry-run
```

CODE EXPLAINED

→ Finder selects which folders to fix

→ `@PER-CS` rule set = community standard

→ Risky rules (like strict types) need opting in

→ Rector upgrades syntax to the target PHP version

→ Always preview with dry-run first

STYLE RULES (PER)

RULE	EXAMPLE
Indent	4 spaces
Class/method brace	Next line
Control brace	Same line
Line length	Soft limit 120
Keywords	lowercase (true, null)
Imports	One per line, after namespace

TOOLS

TOOL	JOB
php -l	Syntax check
PHP-CS-Fixer	Fix formatting
phpcs / phpcbf	Sniff / fix
Laravel Pint	Opinionated fixer
Rector	Automated refactoring
PHPStan	Type analysis (Sheet 98)

NAMING (PSR-1)

ELEMENT	STYLE
Classes	StudlyCaps
Methods	camelCase
Constants	UPPER_SNAKE
Properties	camelCase (consistent)
Files	ClassName.php
Namespaces	Vendor\Package

⚠ COMMON MISTAKES

- Mixing style fixes with logic changes in one commit
- Running Rector without tests to catch regressions
- Arguing style in code review instead of automating it

✓ TIPS & BEST PRACTICES

- Automate formatting in CI
- Run Rector with a clean git tree
- Commit tool configs to the repo

QUICK SUMMARY

- PER-CS / PSR-12 standard
- PHP-CS-Fixer or phpcs fixes style
- Rector upgrades code
- Automate in CI

Logging — Monolog & PSR-3

Monolog Logger · handlers · formatters · log levels · context arrays · PSR-3 LoggerInterface

STRUCTURED LOGGING

PSR-3 defines `Psr\Log\LoggerInterface` with methods for eight severity levels; type-hint against it so any logger can be plugged in. **Monolog** is the standard implementation. A `Logger` has a channel name and a stack of **handlers** (where logs go: files, stderr, Slack…) and **processors** (add data such as request IDs). Pass variable data in the **context array** instead of building strings.

NOTES

- Handlers have a minimum level; lower messages are ignored
- Placeholders `{key}` in messages are filled from context (with a processor)
- Log to `php://stderr` in containers
- Never log passwords, tokens or full card numbers
- Frameworks wire Monolog for you

MONOLOG SETUP AND USE

```php
<?php
use Monolog\Level;
use Monolog\Logger;
use Monolog\Handler\StreamHandler;
use Monolog\Handler\RotatingFileHandler;
use Monolog\Formatter\JsonFormatter;
use Monolog\Processor\PsrLogMessageProcessor;
use Psr\Log\LoggerInterface;

$log = new Logger('shop');
$log->pushHandler(new StreamHandler('php://stderr', Level::Warning));
$file = new RotatingFileHandler(
    __DIR__ . '/var/app.log', maxFiles: 14, level: Level::Debug);
$file->setFormatter(new JsonFormatter());
$log->pushHandler($file);
$log->pushProcessor(new PsrLogMessageProcessor());

$log->info('Order {id} placed', ['id' => 42, 'total' => 19.5]);
$log->error('Payment failed', ['exception' => $e]);

final class Checkout
{
    public function __construct(private LoggerInterface $logger) {}
    public function run(): void { $this->logger->debug('checkout start'); }
}
```

CODE EXPLAINED

- → Channel name tags every record
- → stderr gets warnings and above
- → Rotating JSON file keeps 14 days of everything
- → Processor fills `{id}` from context
- → Depend on `LoggerInterface`, not Monolog

PSR-3 LEVELS

LEVEL	USE FOR
`emergency`	System unusable
`alert`	Immediate action needed
`critical`	Component down
`error`	Runtime errors
`warning`	Unusual but handled
`notice`	Significant normal events
`info / debug`	Events / diagnostics

COMMON HANDLERS

HANDLER	SENDS TO
`StreamHandler`	File or stream
`RotatingFileHandler`	Daily files
`ErrorLogHandler`	PHP error_log
`SyslogHandler`	syslog
`FingersCrossedHandler`	Buffer until error
`SlackWebhookHandler`	Slack

GOOD LOG RECORDS

PRACTICE	EXAMPLE
`Static message`	'Order placed'
`Data in context`	['id' => 42]
`Exception key`	['exception' => $e]
`Correlation id`	Processor adds request id
`No secrets`	Mask tokens

⚠ COMMON MISTAKES

- Interpolating values into messages — breaks grouping
- Logging secrets or personal data
- Using `echo` / `var_dump` for production diagnostics

✓ TIPS & BEST PRACTICES

- Type-hint `LoggerInterface`
- Use `FingersCrossedHandler` to get debug context only on errors
- Log JSON for aggregators

QUICK SUMMARY

- PSR-3 interface, Monolog implementation
- Handlers + processors
- Context arrays
- Eight levels

HTTP Clients — cURL & Guzzle

MAKING HTTP REQUESTS

The **cURL extension** is PHP's low-level HTTP client: create a handle, set options, execute, close. It's powerful but verbose. **Guzzle** wraps it in a clean object API with JSON helpers, timeouts, retries via middleware and async requests. **PSR-18** (`ClientInterface`) and PSR-7 messages let you swap clients — Symfony HttpClient is another popular choice.

NOTES

- Always set connect and total timeouts
- Check both transport errors and HTTP status codes
- Guzzle throws on 4xx/5xx by default (`http_errors`)
- Keep TLS verification on (`CURLOPT_SSL_VERIFYPEER`)
- `curl_close()` is a no-op since 8.0 — handles are objects

CURL VS GUZZLE

```php
<?php
// cURL
$ch = curl_init('https://api.example.com/users?page=1');
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER     => ['Accept: application/json'],
    CURLOPT_CONNECTTIMEOUT => 3,
    CURLOPT_TIMEOUT        => 10,
]);
$body = curl_exec($ch);
if ($body === false) throw new RuntimeException(curl_error($ch));
$status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
$users = json_decode($body, true, flags: JSON_THROW_ON_ERROR);

// Guzzle
use GuzzleHttp\Client;
use GuzzleHttp\Exception\RequestException;

$http = new Client([
    'base_uri' => 'https://api.example.com/',
    'timeout'  => 10,
]);
try {
    $res = $http->post('users', ['json' => ['name' => 'Ana']]);
    $created = json_decode((string) $res->getBody(), true);
    echo $res->getStatusCode();                 // 201
} catch (RequestException $e) {
    $code = $e->getResponse()?->getStatusCode();
}
```

CODE EXPLAINED

→ `RETURNTRANSFER` returns the body instead of printing it

→ Separate connect and total timeouts

→ `false` from `curl_exec` = network error; read `curl_error`

→ Guzzle `json` option encodes the body and sets headers

→ `RequestException` covers 4xx/5xx and transport failures

KEY CURL OPTIONS

OPTION	PURPOSE
`CURLOPT_RETURNTRANSFER`	Return body
`CURLOPT_POST / POSTFIELDS`	POST data
`CURLOPT_CUSTOMREQUEST`	PUT, DELETE…
`CURLOPT_HTTPHEADER`	Headers array
`CURLOPT_TIMEOUT`	Total seconds
`CURLOPT_FOLLOWLOCATION`	Follow redirects

GUZZLE REQUEST OPTIONS

OPTION	DOES
`json`	JSON body
`form_params`	Form-encoded body
`multipart`	File upload
`query`	Query string array
`headers`	Request headers
`timeout / connect_timeout`	Limits

RESPONSE HANDLING

CODE	GETS
`$res->getStatusCode()`	200
`(string) $res->getBody()`	Body text
`$res->getHeaderLine('X')`	Header
`curl_getinfo($ch)`	Status, timing
`curl_errno($ch)`	Transport error code

⚠ COMMON MISTAKES

- No timeouts — requests hang workers
- Disabling TLS verification to "fix" certificate errors
- Ignoring non-200 statuses with raw cURL

✓ TIPS & BEST PRACTICES

- Inject a PSR-18 client for testability
- Add retry middleware for flaky upstreams
- Use Guzzle's `MockHandler` in tests

QUICK SUMMARY

- cURL: init, setopt, exec
- Guzzle: object API + JSON
- Always set timeouts
- PSR-18 for swappable clients

Regular Expressions (PCRE)

PATTERN MATCHING WITH PREG_*

PHP uses the **PCRE2** engine through the `preg_*` functions. A pattern is a string wrapped in **delimiters** (usually `/`, or `#` / `~` when the pattern contains slashes), followed by modifier flags like `i` (case-insensitive) or `u` (UTF-8). `preg_match()` finds the first match and fills a `$matches` array; `preg_match_all()` finds every match; `preg_replace()` and `preg_split()` transform text. See the RegEx sheet set for syntax details.

NOTES

- Use single quotes so backslashes reach PCRE untouched
- Always add `u` when working with UTF-8 text
- `preg_match` returns 1, 0, or `false` on error
- `preg_quote($s, '/')` escapes user input for a pattern
- `preg_last_error_msg()` explains failures (8.0)

PREG_* IN ACTION

```php
<?php
$log = '2026-10-08 ERROR [db] Timeout after 30s';

if (preg_match('/^(?<date>\d{4}-\d{2}-\d{2}) (?<lvl>\w+)/', $log, $m)) {
    echo $m['date'], ' ', $m['lvl'];          // 2026-10-08 ERROR
}

preg_match_all('/\d+/', 'a1 b22 c333', $all);
// $all[0] = ['1', '22', '333']

echo preg_replace('/\s+/', ' ', "too   many \n spaces");
echo preg_replace('#/+#', '/', 'a//b///c');   // a/b/c

echo preg_replace_callback('/\d+/',
    fn($m) => $m[0] * 2, 'x5 y10');           // x10 y20

preg_split('/[\s,]+/', 'a, b  c,d', -1, PREG_SPLIT_NO_EMPTY);
// ['a', 'b', 'c', 'd']

preg_match('/^\p{Lu}/u', 'Élan');             // 1 (Unicode upper)
$safe = '/' . preg_quote($userInput, '/') . '/i';
preg_grep('/^img_/', ['img_1', 'doc', 'img_2']);  // keeps img_*
```

CODE EXPLAINED

→ Named groups `(?<name>…)` appear as keys in `$m`

→ `preg_match_all` fills arrays of all matches

→ `#` delimiter avoids escaping `/`

→ Callback receives the match array

→ `u` flag enables Unicode properties like `\p{Lu}`

FUNCTIONS

FUNCTION	RETURNS
`preg_match($re, $s, $m)`	1 / 0 / false
`preg_match_all($re, $s, $m)`	Count
`preg_replace($re, $r, $s)`	New string
`preg_replace_callback()`	New string
`preg_split($re, $s)`	Array
`preg_grep($re, $arr)`	Matching items
`preg_quote($s, $d)`	Escaped literal

MODIFIERS

FLAG	EFFECT
`i`	Case-insensitive
`m`	^ $ match per line
`s`	. matches newline
`x`	Ignore whitespace, allow comments
`u`	UTF-8 mode
`U`	Ungreedy by default
`n`	No auto-capture (8.2)

REPLACEMENT REFS

SYNTAX	INSERTS
`$0 / \0`	Whole match
`$1 / \1`	Group 1
`${1}0`	Group 1 then '0'
`Named group`	Use callback: $m['name']
`\$`	Literal dollar

⚠ COMMON MISTAKES

- Double-quoted patterns where PHP interprets `\1` or `$x` first
- Forgetting the `u` flag with accented text
- Putting user input into a pattern without `preg_quote()`

✓ TIPS & BEST PRACTICES

- Prefer `str_*` functions for fixed strings
- Use the `x` flag to comment complex patterns
- Check for `false` and call `preg_last_error_msg()`

QUICK SUMMARY

- `preg_match` / `_all` / `replace` / `split`
- Delimiters + flags
- Named groups
- `u` for UTF-8, `preg_quote` for input

Attributes

STRUCTURED METADATA (8.0)

Attributes attach machine-readable metadata to classes, methods, properties, parameters, functions and constants using `#[Name(args)]`. They replace docblock annotations with real, autoloaded classes. Your code (or a framework) reads them with Reflection's `getAttributes()` and `newInstance()`. PHP also ships **built-in** attributes that change engine behaviour, such as `#[\Override]`, `#[\Deprecated]` and `#[\NoDiscard]`.

NOTES

- An attribute class itself is marked `#[Attribute]`
- Targets restrict where it can be used (`Attribute::TARGET_METHOD`)
- Attribute arguments must be constant expressions
- Unknown attributes are ignored until you call `newInstance()`
- Frameworks use them for routes, validation, ORM mapping

DEFINING AND READING ATTRIBUTES

```php
<?php
#[Attribute(Attribute::TARGET_METHOD | Attribute::IS_REPEATABLE)]
final class Route
{
    public function __construct(
        public string $path,
        public string $method = 'GET',
    ) {}
}

final class UserController
{
    #[Route('/users')]
    #[Route('/people')]
    public function index(): array { return []; }

    #[Route('/users', method: 'POST')]
    public function store(#[\SensitiveParameter] string $password): void {}
}

$rc = new ReflectionClass(UserController::class);
foreach ($rc->getMethods() as $m) {
    foreach ($m->getAttributes(Route::class) as $attr) {
        $route = $attr->newInstance();
        echo "$route->method $route->path → {$m->name}\n";
    }
}
// GET /users → index, GET /people → index, POST /users → store
```

CODE EXPLAINED

- → `#[Attribute]` with flags declares a repeatable method attribute
- → Attribute arguments map to constructor parameters
- → Named arguments work in attributes
- → `#[\SensitiveParameter]` hides the value in stack traces
- → Reflection reads and instantiates attributes

BUILT-IN ATTRIBUTES

ATTRIBUTE	EFFECT
`#[\Attribute]`	Declares an attribute class
`#[\Override]`	Must override (8.3)
`#[\SensitiveParameter]`	Redact in traces (8.2)
`#[\AllowDynamicProperties]`	Opt in (8.2)
`#[\ReturnTypeWillChange]`	Silence BC notice
`#[\Deprecated]`	Emit deprecation (8.4)
`#[\NoDiscard]`	Warn if result unused (8.5)

TARGETS

CONSTANT	ALLOWS ON
`TARGET_CLASS`	Classes
`TARGET_FUNCTION`	Functions
`TARGET_METHOD`	Methods
`TARGET_PROPERTY`	Properties
`TARGET_CLASS_CONSTANT`	Constants
`TARGET_PARAMETER`	Parameters
`TARGET_ALL`	Everywhere (default)

REFLECTION API

METHOD	RETURNS
`getAttributes()`	All ReflectionAttribute
`getAttributes(C::class)`	Filtered
`->getName()`	Attribute class
`->getArguments()`	Raw args
`->newInstance()`	Attribute object

⚠ COMMON MISTAKES

- Forgetting `#[Attribute]` on your attribute class — `newInstance()` fails
- Using non-constant expressions as arguments
- Expecting attributes to do anything without code that reads them

✓ TIPS & BEST PRACTICES

- Use attributes instead of docblock annotations
- Restrict targets to catch misuse
- Cache reflection results in production

QUICK SUMMARY

- `#[Name(args)]` metadata
- Classes marked `#[Attribute]`
- Read via Reflection
- Built-ins change engine behaviour

Serialization

TURNING VALUES INTO STRINGS

`serialize()` converts almost any PHP value — including objects — into a string, and `unserialize()` rebuilds it. It's used for caches and sessions. The danger: `unserialize()` on **untrusted input** can instantiate arbitrary classes and trigger their magic methods (**PHP object injection**). For data that crosses trust boundaries, use **JSON**. If you must unserialize, restrict classes with `allowed_classes`.

NOTES

- Closures and most resources can't be serialized
- Objects control their format with `__serialize()` (Sheet 74)
- Serialized data is PHP-specific and version-sensitive
- `igbinary` is a faster, smaller binary serializer extension
- Sign data (HMAC) if it must round-trip through a client

SERIALIZE, UNSERIALIZE & SAFER OPTIONS

```php
<?php
$data = ['id' => 7, 'tags' => ['a', 'b'], 'ok' => true];
$s = serialize($data);
// a:3:{s:2:"id";i:7;s:4:"tags";a:2:{i:0;s:1:"a";i:1;s:1:"b";}s:2:"ok";b:1;}
$back = unserialize($s);                    // same array

// Untrusted input: never allow objects
$value = unserialize($cookie, ['allowed_classes' => false]);

// Or allow-list specific classes
$cart = unserialize($cached, ['allowed_classes' => [Cart::class]]);

// Prefer JSON across trust boundaries
$json = json_encode($data, JSON_THROW_ON_ERROR);
$data = json_decode($json, true, flags: JSON_THROW_ON_ERROR);

// Integrity check when data leaves the server
$payload = base64_encode($s);
$sig = hash_hmac('sha256', $payload, $secretKey);
if (!hash_equals($sig, $_COOKIE['sig'] ?? '')) {
    throw new RuntimeException('Tampered');
}

var_export($data);       // PHP source code form
```

CODE EXPLAINED

- → The format records type and length of each value
- → `allowed_classes => false` turns objects into `__PHP_Incomplete_Class`
- → An allow-list limits which classes can be created
- → JSON can't create objects — safe by design
- → HMAC + `hash_equals` detects tampering

FORMAT CODES

CODE	TYPE
`i:7;`	int
`d:1.5;`	float
`s:3:"abc";`	string (length)
`b:1;`	bool
`N;`	null
`a:n:{…}`	array
`O:4:"User":…`	object

CHOOSING A FORMAT

FORMAT	USE WHEN
`JSON`	APIs, untrusted data, other languages
`serialize()`	Trusted internal caches
`igbinary`	Large caches, speed
`var_export()`	Generating PHP config
`MessagePack`	Compact cross-language

OBJECT HOOKS

METHOD	ROLE
`__serialize()`	Data to store
`__unserialize($d)`	Rebuild
`__sleep()` / `__wakeup()`	Legacy
`__destruct()`	Gadget-chain risk
`JsonSerializable`	JSON instead

⚠ COMMON MISTAKES

- `unserialize($_COOKIE[...])` — object injection vulnerability
- Serializing objects whose class may later be renamed
- Using serialize() for data shared with JavaScript

✓ TIPS & BEST PRACTICES

- Use JSON by default
- Pass `allowed_classes` every time
- Sign anything that round-trips through users

QUICK SUMMARY

- `serialize` / `unserialize`
- Never unserialize untrusted data
- `allowed_classes`
- Prefer JSON

Fibers & Async PHP

COOPERATIVE CONCURRENCY (8.1)

A **Fiber** is a block of code with its own call stack that can **pause** (`Fiber::suspend()`) and **resume** later, exchanging values each way. Fibers don't run in parallel — they let an **event loop** switch tasks while one waits on I/O. Application code rarely uses fibers directly; libraries such as **Revolt** (the shared loop), **AMPHP** and **ReactPHP** build async HTTP clients, servers and database drivers on top of them without `async`/`await` keywords.

NOTES

- Fibers are single-threaded; CPU-bound work still blocks
- Suspending is invisible to callers — no function colouring
- Blocking functions (`sleep` , PDO) block the whole loop
- Use async-aware libraries for sockets, HTTP, DB
- For real parallelism: processes, queues, or `ext-parallel`

FIBER BASICS AND AN ASYNC LIBRARY

```php
<?php
$fiber = new Fiber(function (string $start): string {
    echo "got $start\n";
    $x = Fiber::suspend('paused-1');     // pause, send value out
    echo "resumed with $x\n";
    $y = Fiber::suspend('paused-2');
    return "done $y";
});

$out = $fiber->start('go');              // got go
echo $out, "\n";                         // paused-1
$out = $fiber->resume('A');              // resumed with A
echo $out, "\n";                         // paused-2
$fiber->resume('B');
echo $fiber->getReturn();                // done B

// AMPHP: concurrent HTTP requests
use Amp\Future;
use Amp\Http\Client\HttpClientBuilder;
use function Amp\async;

$client = HttpClientBuilder::buildDefault();
$fetch = fn(string $url) => $client
    ->request(new Amp\Http\Client\Request($url))
    ->getBody()->buffer();
$futures = array_map(fn($u) => async($fetch, $u), $urls);
$bodies = Future\await($futures);       // all run concurrently
```

CODE EXPLAINED

- → `start()` runs until the first `suspend()`
- → `suspend()` returns the value passed to `resume()`
- → Values flow out through `suspend()` and back via `resume()`
- → `getReturn()` reads the final value
- → AMPHP's `async()` + `await()` hide the fiber plumbing

FIBER API

METHOD	DOES
`new Fiber($fn)`	Create
`->start(...$args)`	Run until suspend
`Fiber::suspend($v)`	Pause (inside)
`->resume($v)`	Continue
`->throw($e)`	Throw into fiber
`->getReturn()`	Final value
`->isTerminated()`	Finished?

ASYNC ECOSYSTEM

LIBRARY	PROVIDES
`Revolt`	Shared event loop
`AMPHP`	Fiber-based async (HTTP, SQL, Redis)
`ReactPHP`	Event-driven I/O, promises
`Swoole / OpenSwoole`	Coroutine extension
`FrankenPHP worker mode`	Long-lived workers

CONCURRENCY OPTIONS

APPROACH	PARALLEL?
`Fibers + event loop`	No — interleaved I/O
`pcntl_fork / proc_open`	Yes — processes
`Job queues (Redis, SQS)`	Yes — workers
`ext-parallel`	Yes — threads (ZTS)
`curl_multi`	Concurrent HTTP

⚠ COMMON MISTAKES

- Expecting fibers to speed up CPU-bound code
- Calling blocking functions inside an async loop
- Building your own scheduler instead of using Revolt

✓ TIPS & BEST PRACTICES

- Use AMPHP or ReactPHP rather than raw fibers
- Offload heavy work to queues
- Use `curl_multi` or async clients for many HTTP calls

QUICK SUMMARY

- Fibers pause and resume
- Single-threaded, cooperative
- Libraries provide the loop
- Not for CPU-bound work

CLI Scripts

COMMAND-LINE PHP

PHP is a capable scripting language for the terminal. Arguments arrive in `$argv` (with `$argv[0]` the script name) and their count in `$argc`. `getopt()` parses short (`-v`) and long (`--limit=5`) options. Read input from `STDIN`, write results to `STDOUT` and diagnostics to `STDERR`, and finish with **exit codes** (0 = success) so other tools can react. For larger tools, use Symfony Console.

NOTES

- CLI has no `max_execution_time` by default
- Guard CLI-only scripts with `PHP_SAPI === 'cli'`
- `$_SERVER['argv']` is also available
- `readline()` prompts interactively (readline ext)
- `pcntl_signal()` handles Ctrl-C (SIGINT) gracefully

A SMALL CLI TOOL

```php
#!/usr/bin/env php
<?php
declare(strict_types=1);

if (PHP_SAPI !== 'cli') { exit("CLI only\n"); }

$opts = getopt('vn:', ['limit:', 'dry-run', 'help'], $rest);
if (isset($opts['help'])) {
    echo "Usage: tool.php [-v] [--limit=N] FILE\n";
    exit(0);
}
$verbose = isset($opts['v']);
$limit   = (int) ($opts['limit'] ?? $opts['n'] ?? 10);
$file    = $argv[$rest] ?? null;

if ($file === null || !is_readable($file)) {
    fwrite(STDERR, "error: missing or unreadable FILE\n");
    exit(2);                                // usage error
}

$count = 0;
while (($line = fgets(STDIN)) !== false && $count < $limit) {
    $count++;
    if ($verbose) fwrite(STDERR, "line $count\n");
    echo strtoupper($line);
}
exit(0);
```

CODE EXPLAINED

→ Shebang + `chmod +x` makes it executable

→ `v` = flag, `n:` = required value; long opts in an array

→ `$rest` = index of the first non-option argument

→ Errors go to STDERR with a non-zero exit code

→ Reads piped input: `cat x | ./tool.php -v FILE`

GETOPT SYNTAX

SPEC	MEANS
`'v'`	Flag -v (value false)
`'n:'`	-n VALUE required
`'n::'`	-n[VALUE] optional
`'limit:' (long)`	--limit=VALUE
`'dry-run' (long)`	--dry-run flag
`$rest_index`	First positional arg

STREAMS

CONSTANT	USE
`STDIN`	Read input / pipes
`STDOUT`	Normal output
`STDERR`	Errors, progress
`echo`	Writes to STDOUT
`readline('> ')`	Interactive prompt

EXIT CODES

CODE	CONVENTION
`0`	Success
`1`	General error
`2`	Usage / bad arguments
`126 / 127`	Not executable / not found
`130`	Interrupted (Ctrl-C)
`exit('msg')`	Prints msg, code 0

⚠ COMMON MISTAKES

- Printing errors to STDOUT, breaking pipelines
- Always exiting 0, even on failure
- Loading huge input with `file_get_contents('php://stdin')`

✓ TIPS & BEST PRACTICES

- Use Symfony Console for subcommands and help text
- Stream input line by line
- Return meaningful exit codes

QUICK SUMMARY

- `$argv` / `getopt()`
- STDIN / STDOUT / STDERR
- Exit codes matter
- Symfony Console for big tools

Performance, OPcache & JIT

MAKING PHP FAST

OPcache stores compiled bytecode in shared memory so scripts aren't re-parsed on every request — it's the single biggest win and should always be on in production. Turn off timestamp checks there and clear the cache on deploy. **Preloading** loads framework classes into memory at server start. The **JIT** compiles hot code to machine code; it helps CPU-heavy work far more than typical I/O-bound web apps. Measure with a profiler before optimising.

NOTES

- OPcache is bundled; enable it with `opcache.enable=1`
- `opcache_reset()` or an FPM reload clears it after deploys
- JIT is off by default; enable with `opcache.jit` + buffer size
- Composer's optimised autoloader also matters (Sheet 87)
- Most web latency is database and network, not PHP

PRODUCTION PHP.INI SETTINGS

```
; OPcache
opcache.enable=1
opcache.enable_cli=0
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0     ; reload FPM on deploy
opcache.preload=/app/config/preload.php
opcache.preload_user=www-data

; JIT (helps CPU-bound code)
opcache.jit=tracing
opcache.jit_buffer_size=64M

; Filesystem caching
realpath_cache_size=4096K
realpath_cache_ttl=600

; Check at runtime:
;   php -r 'var_dump(opcache_get_status()["opcache_enabled"]);'
;   memory_get_peak_usage(true), hrtime(true) for timing
;   Profilers: Xdebug profile, Blackfire, Tideways, SPX
```

CODE EXPLAINED

- → Enough memory and file slots for the whole codebase
- → `validate_timestamps=0` skips file checks — reload on deploy
- → Preload script `opcache_compile_file()` s hot classes
- → `tracing` JIT mode suits most workloads
- → Bigger realpath cache cuts filesystem lookups

QUICK WINS

ACTION	IMPACT
`Enable OPcache`	Huge
`composer dump-autoload -o`	Medium
`Upgrade PHP version`	Medium–large
`Fix N+1 queries`	Often largest
`Cache expensive results`	Large
`JIT for CPU work`	Situational

OPCACHE FUNCTIONS

FUNCTION	DOES
`opcache_get_status()`	Stats, hit rate
`opcache_reset()`	Clear all
`opcache_invalidate($f)`	Clear one file
`opcache_compile_file($f)`	Precompile
`opcache_is_script_cached($f)`	Check

MEASURING

TOOL	MEASURES
`hrtime(true)`	Wall time (ns)
`memory_get_usage()`	Current memory
`memory_get_peak_usage()`	Peak memory
`Xdebug profiler`	Call graph
`Blackfire / Tideways`	Production profiling
`ab / wrk / k6`	Load testing

⚠ COMMON MISTAKES

- Running production without OPcache
- `validate_timestamps=0` but no cache reset on deploy — old code runs
- Micro-optimising PHP while queries dominate

✓ TIPS & BEST PRACTICES

- Profile first, then optimise the hot path
- Keep PHP up to date — each release is faster
- Cache at the right layer (HTTP, app, DB)

QUICK SUMMARY

- OPcache always on
- Reset on deploy
- Preload + JIT for extra speed
- Measure before optimising

Memory, Weak References & Lazy Objects

HOW PHP MANAGES MEMORY

PHP frees values automatically using **reference counting**: when the last variable pointing to a value goes away, it's freed immediately. Objects that reference each other in a cycle are cleaned up by the **cycle collector**. `WeakReference` and `WeakMap` hold objects **without** keeping them alive — ideal for caches. PHP 8.4's **lazy objects** create placeholders that initialise only when first used.

NOTES

- Memory resets after each web request (share-nothing)
- Long-running workers must avoid growing arrays and static caches
- `unset()` drops a reference, not necessarily the value
- `memory_limit` caps per-request memory
- Lazy objects are mostly for frameworks (ORMs, DI containers)

WEAK MAPS AND LAZY OBJECTS

```php
<?php
$before = memory_get_usage();
$big = range(1, 1_000_000);
echo memory_get_usage() - $before;     // ~ 16 MB
unset($big);                           // freed immediately

$a = new stdClass(); $b = new stdClass();
$a->other = $b; $b->other = $a;        // cycle
unset($a, $b);
gc_collect_cycles();                   // cycle collector

$cache = new WeakMap();
$user = new User(7);
$cache[$user] = expensiveReport($user);
unset($user);                          // entry vanishes too
count($cache);                         // 0

$ref = WeakReference::create($obj);
$ref->get();                           // object or null

// PHP 8.4: lazy ghost — constructor runs on first access
$rc = new ReflectionClass(Report::class);
$report = $rc->newLazyGhost(function (Report $r) {
    $r->__construct(loadRows());      // expensive work deferred
});
echo $report->title;                   // initialises now
```

CODE EXPLAINED

→ Measure with `memory_get_usage()` before and after

→ Refcount 0 → freed at once

→ Cycles need the collector

→ `WeakMap` entries disappear with their key object

→ Lazy ghost runs its initializer on first property access

MEMORY FUNCTIONS

FUNCTION	RETURNS
`memory_get_usage()`	Bytes now
`memory_get_peak_usage()`	Peak bytes
`gc_collect_cycles()`	Cycles freed
`gc_status()`	Collector stats
`ini_get('memory_limit')`	Cap
`debug_zval_dump($x)`	Refcount info

WEAK TYPES

TYPE	USE
`WeakReference`	One object handle
`WeakMap`	Object → data cache
`SplObjectStorage`	Strong object map
`spl_object_id()`	Identity int
`Array of objects`	Strong references

LAZY OBJECTS (8.4)

METHOD	CREATES
`newLazyGhost($init)`	Same object, init later
`newLazyProxy($factory)`	Proxy to real object
`initializeLazyObject($o)`	Force init
`isUninitializedLazyObject($o)`	Check
`resetAsLazyGhost($o, $init)`	Reuse object

⚠ COMMON MISTAKES

- Unbounded static caches in queue workers
- Assuming `unset()` frees memory held elsewhere
- Loading full result sets instead of streaming

✓ TIPS & BEST PRACTICES

- Use generators for big datasets
- Use `WeakMap` for per-object caches
- Restart long-running workers periodically

QUICK SUMMARY

- Refcounting + cycle collector
- Weak references don't keep objects alive
- Lazy objects (8.4)
- Watch memory in long-running processes

The Request Lifecycle

share-nothing model · SAPI · php-fpm · nginx · Apache · front controller · FrankenPHP

FROM URL TO RESPONSE

A browser request reaches a web server (nginx, Apache, Caddy), which passes PHP requests to **PHP-FPM** over FastCGI. A worker runs your script from the top, builds the response (headers + body) and then throws **all state away** — the *share-nothing* model. Modern apps route every request through one **front controller** (`public/index.php`) that bootstraps the app and dispatches to the right handler. Worker-mode runtimes (FrankenPHP, RoadRunner, Swoole) keep the app booted between requests.

NOTES

- Each request gets fresh globals, superglobals and memory
- Persistent data lives in sessions, databases or caches
- FPM pool size limits concurrent requests
- In worker mode, reset state between requests yourself
- `fastcgi_finish_request()` sends the response, then keeps working

FRONT CONTROLLER + NGINX

```
# nginx: send everything that isn't a file to index.php
server {
    root /var/www/app/public;
    index index.php;

    location / {
        try_files $uri /index.php$is_args$args;
    }

    location ~ \.php$ {
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    }
}
```

```
<?php // public/index.php — the front controller
require __DIR__ . '/../vendor/autoload.php';

$app = App\Kernel::boot(getenv('APP_ENV') ?: 'prod');
$response = $app->handle(App\Http\Request::fromGlobals());
$response->send();                    // headers, then body

fastcgi_finish_request();             // FPM: client done
$app->terminate();                    // slow cleanup after
```

CODE EXPLAINED

- → Document root is `public/`, never the project root
- → `try_files` routes unknown paths to index.php
- → `fastcgi_pass` hands PHP files to FPM
- → One entry point bootstraps and dispatches
- → Post-response work runs after the client has its answer

LIFECYCLE STEPS

STEP	HAPPENS
1	Web server receives HTTP request
2	Passes to FPM worker (FastCGI)
3	PHP fills superglobals
4	Script runs: route, handle, render
5	Headers + body sent
6	State destroyed

RUNTIMES

RUNTIME	MODEL
PHP-FPM	Process pool, share-nothing
mod_php	Inside Apache process
FrankenPHP	Caddy + worker mode
RoadRunner	Go server, workers
Swoole	Coroutine server
php -S	Dev only

FPM POOL SETTINGS

SETTING	CONTROLS
pm = dynamic	Spawn on demand
pm.max_children	Max workers
pm.start_servers	Initial workers
pm.max_requests	Recycle after N
request_terminate_timeout	Kill slow requests

⚠ COMMON MISTAKES

- Pointing the web root at the project folder — exposes `.env` and `vendor/`
- Expecting variables to persist between requests
- Leaking state across requests in worker mode

✓ TIPS & BEST PRACTICES

- Use a single front controller
- Tune `pm.max_children` to available memory
- Offload slow work to queues

QUICK SUMMARY

- Server → FPM → script → response
- Share-nothing per request
- Front controller pattern
- Worker modes keep the app booted

Superglobals

REQUEST DATA ARRAYS

PHP fills several **superglobal** arrays at the start of each request; they're readable in any scope. `$_GET` holds query-string parameters, `$_POST` form fields from urlencoded or multipart bodies, `$_COOKIE` cookies, `$_FILES` uploads, and `$_SERVER` request metadata such as method, URI and headers. Treat every value as **untrusted user input**: validate on the way in, escape on the way out.

NOTES

- Values are strings or arrays — never ints or bools
- `$_POST` is empty for JSON bodies; read `php://input`
- Avoid `$_REQUEST` — it merges sources in config-dependent order
- HTTP headers appear as `$_SERVER['HTTP_*']`
- `filter_input()` reads the original request values

READING REQUEST DATA

```php
<?php
// GET /search.php?q=php&tags[]=web&tags[]=api&page=2
$q     = $_GET['q'] ?? '';                       // 'php'
$tags  = $_GET['tags'] ?? [];                    // ['web', 'api']
$page  = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT,
         ['options' => ['default' => 1, 'min_range' => 1]]);

$method = $_SERVER['REQUEST_METHOD'];            // 'GET'
$path   = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$agent  = $_SERVER['HTTP_USER_AGENT'] ?? '';
$ip     = $_SERVER['REMOTE_ADDR'];
$https  = ($_SERVER['HTTPS'] ?? '') === 'on';

if ($method === 'POST') {
    $email = trim($_POST['email'] ?? '');
}

$json = json_decode(file_get_contents('php://input') ?: 'null', true);
$theme = $_COOKIE['theme'] ?? 'light';
```

CODE EXPLAINED

- → `??` avoids undefined-index warnings
- → `name[]` in the query string becomes an array
- → `filter_input` validates and applies a default
- → `REQUEST_URI` includes the query string — parse the path
- → JSON APIs read the raw body, not `$_POST`

SUPERGLOBALS

ARRAY	CONTAINS
`$_GET`	Query string params
`$_POST`	Form body fields
`$_FILES`	Uploaded files
`$_COOKIE`	Request cookies
`$_SESSION`	Session data
`$_SERVER`	Server + request info
`$_ENV / $GLOBALS`	Env / all globals

USEFUL $_SERVER KEYS

KEY	EXAMPLE
`REQUEST_METHOD`	POST
`REQUEST_URI`	/search?q=php
`QUERY_STRING`	q=php
`HTTP_HOST`	shop.test
`HTTP_ACCEPT`	application/json
`REMOTE_ADDR`	203.0.113.9
`SCRIPT_FILENAME`	/app/public/index.php

TRUST LEVELS

SOURCE	TRUST?
`$_GET / $_POST / $_COOKIE`	Never
`$_FILES name / type`	Never
`HTTP_* headers`	Never
`HTTP_X_FORWARDED_FOR`	Only from your proxy
`REMOTE_ADDR`	Connection IP (behind proxy = proxy)

⚠ COMMON MISTAKES

- Using values without validation
- Trusting `X-Forwarded-For` from the internet
- Reading JSON bodies from `$_POST`

✓ TIPS & BEST PRACTICES

- Wrap globals in a Request object (PSR-7, Symfony HttpFoundation)
- Validate with `filter_input()` / `filter_var()`
- Escape output with `htmlspecialchars()`

QUICK SUMMARY

- `$_GET` `$_POST` `$_COOKIE` `$_FILES`
- `$_SERVER` metadata
- Strings only, untrusted
- JSON via `php://input`

Handling Forms

FORM ROUND TRIPS

An HTML form sends its fields to the `action` URL using GET (query string) or POST (request body). PHP exposes them in `$_GET` or `$_POST` keyed by each input's `name`. A typical handler checks the method, validates fields, re-displays the form with errors and the user's values (**sticky form**), and on success **redirects** (Post/Redirect/Get) so a browser refresh doesn't resubmit.

NOTES

- Unchecked checkboxes send **nothing** — use `isset()`
- `name="tags[]"` sends an array
- Use POST for anything that changes data
- Escape values you echo back into inputs
- Add a CSRF token to every POST form (Sheet 127)

VALIDATE, REDISPLAY, REDIRECT

```php
<?php
$errors = [];
$name = $email = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name  = trim($_POST['name'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $agree = isset($_POST['agree']);

    if ($name === '') $errors['name'] = 'Name is required';
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors['email'] = 'Invalid email';
    }
    if (!$agree) $errors['agree'] = 'Please accept the terms';

    if (!$errors) {
        saveSignup($name, $email);
        header('Location: /thanks', true, 303);   // PRG
        exit;
    }
}
$e = fn($v) => htmlspecialchars($v, ENT_QUOTES);
?>
<form method="post" action="">
  <input name="name" value="<?= $e($name) ?>">
  <?php if (isset($errors['name'])): ?>
    <p class="err"><?= $e($errors['name']) ?></p>
  <?php endif ?>
  <input name="email" type="email" value="<?= $e($email) ?>">
  <label><input type="checkbox" name="agree"> I agree</label>
  <button>Sign up</button>
</form>
```

CODE EXPLAINED

→ Only process on POST

→ `trim` + `??` give clean defaults

→ Collect errors by field name

→ 303 redirect after success (Post/Redirect/Get)

→ Escape every value echoed back (sticky form)

INPUT → PHP

HTML	$_POST VALUE
`<input name="a">`	'text'
`<input type="checkbox" name="c">`	'on' or missing
`<input type="checkbox" name="c" value="1">`	'1' or missing
`<select name="s">`	Selected value
`<select name="s[]" multiple>`	Array
`<input name="user[email]">`	['email' => …]

GET VS POST

ASPECT	GET / POST
Data in	URL / Body
Bookmarkable	Yes / No
Size	Limited / Large
Use for	Search, filters / Changes
Refresh	Safe / Resubmits (use PRG)

FORM ENCODINGS

ENCTYPE	USE
`application/x-www-form-urlencoded`	Default
`multipart/form-data`	File uploads
`text/plain`	Avoid
`JSON (fetch)`	Read php://input

⚠ COMMON MISTAKES

- Missing `exit` after `header('Location: …')`
- Echoing user input back without escaping
- Validating only in JavaScript

✓ TIPS & BEST PRACTICES

- Always validate server-side
- Use Post/Redirect/Get with status 303
- Use a form library or framework validator for larger apps

QUICK SUMMARY

- Fields arrive by `name`
- Checkboxes: `isset()`
- Sticky forms + escaping
- Post/Redirect/Get

Input Validation & Filtering

TRUST NOTHING

Validate every external value before using it. The **filter extension** handles common cases: `filter_var($value, FILTER_VALIDATE_*)` returns the converted value or `false`, with options such as ranges and defaults. `filter_input()` reads directly from the request. `ctype_*` functions check character classes, and regex covers custom formats. Validation decides *accept or reject*; escaping for output is a separate step.

NOTES

- `FILTER_VALIDATE_INT` returns `0` for `"0"` — compare with `=== false`
- `FILTER_NULL_ON_FAILURE` returns `null` instead of `false`
- `FILTER_SANITIZE_STRING` is deprecated (8.1) — escape on output
- Use allow-lists for enums and sort columns
- Libraries: Symfony Validator, Respect\Validation

FILTER_VAR AND FRIENDS

```php
<?php
filter_var('42', FILTER_VALIDATE_INT);              // 42
filter_var('4.2', FILTER_VALIDATE_INT);             // false
filter_var('0', FILTER_VALIDATE_INT);               // 0 (valid!)
filter_var('15', FILTER_VALIDATE_INT,
    ['options' => ['min_range' => 1, 'max_range' => 10]]);  // false

filter_var('ana@example.com', FILTER_VALIDATE_EMAIL);
filter_var('https://x.io/a', FILTER_VALIDATE_URL);
filter_var('192.168.1.1', FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE); // false
filter_var('yes', FILTER_VALIDATE_BOOL, FILTER_NULL_ON_FAILURE);  // true
filter_var('3,5', FILTER_VALIDATE_FLOAT,
    ['options' => ['decimal' => ',']]);             // 3.5

$age = filter_input(INPUT_POST, 'age', FILTER_VALIDATE_INT);
if ($age === false || $age === null) { /* invalid or missing */ }

ctype_digit('123');            // true ('-1' false)
ctype_alnum('abc123');         // true

$sort = in_array($_GET['sort'] ?? '', ['name', 'date'], true)
    ? $_GET['sort'] : 'name';   // allow-list
```

CODE EXPLAINED

→ Returns the typed value on success

→ `0` is valid — test with `=== false`

→ Range options reject out-of-bounds values

→ Flags refine rules (no private IPs)

→ Allow-list user choices like sort columns

VALIDATE FILTERS

FILTER	RETURNS
`FILTER_VALIDATE_INT`	int
`FILTER_VALIDATE_FLOAT`	float
`FILTER_VALIDATE_BOOL`	bool (or null)
`FILTER_VALIDATE_EMAIL`	string
`FILTER_VALIDATE_URL`	string
`FILTER_VALIDATE_IP`	string
`FILTER_VALIDATE_REGEXP`	string

FILTER_INPUT RESULTS

SITUATION	RESULT
`Valid`	Converted value
`Invalid`	false
`Missing`	null
`+ FILTER_NULL_ON_FAILURE`	invalid → null
`Array input`	FILTER_REQUIRE_ARRAY

CTYPE_* CHECKS

FUNCTION	ALLOWS
`ctype_digit`	0-9 only
`ctype_alpha`	Letters
`ctype_alnum`	Letters + digits
`ctype_xdigit`	Hex digits
`ctype_space`	Whitespace
`ctype_upper / lower`	Case

⚠ COMMON MISTAKES

- `if (!filter_var($n, FILTER_VALIDATE_INT))` — rejects `0`
- Sanitising input instead of escaping output
- Trusting client-side validation

✓ TIPS & BEST PRACTICES

- Validate at the boundary, then work with typed values
- Use allow-lists over block-lists
- Return all field errors at once

QUICK SUMMARY

- `filter_var` validates + converts
- `=== false` check
- `filter_input` for requests
- Allow-lists for choices

Headers, Redirects & Status Codes

CONTROLLING THE RESPONSE

`header()` adds or replaces an HTTP response header; `http_response_code()` sets the status. Headers must be sent **before any body output** — even a space before `<?php` sends output and locks them. A redirect is a `Location` header plus a 3xx status, followed by `exit`. For JSON APIs set `Content-Type: application/json` and echo the encoded body.

NOTES

- `header('X: 1', false)` adds instead of replacing
- `headers_sent($file, $line)` tells you who sent output
- Output buffering can delay sending headers
- `header_remove('X-Powered-By')` removes a header
- Default content type is `text/html; charset=UTF-8`

STATUS, HEADERS, REDIRECTS, JSON

```php
<?php
function json(mixed $data, int $status = 200): never
{
    http_response_code($status);
    header('Content-Type: application/json; charset=utf-8');
    header('Cache-Control: no-store');
    echo json_encode($data, JSON_THROW_ON_ERROR | JSON_UNESCAPED_SLASHES);
    exit;
}

function redirect(string $url, int $status = 302): never
{
    header('Location: ' . $url, true, $status);
    exit;
}

if (headers_sent($file, $line)) {
    throw new LogicException("Output started at $file:$line");
}

header_remove('X-Powered-By');
header('X-Frame-Options: DENY');
header('Content-Security-Policy: default-src \'self\'');

$user ?? json(['error' => 'Not found'], 404);
header('Content-Disposition: attachment; filename="report.csv"');
```

CODE EXPLAINED

- → `never` return type: these functions always exit
- → Status first, then headers, then body
- → `Location` + 302/303/301/307/308 = redirect
- → Check `headers_sent()` to debug output leaks
- → Security headers reduce attack surface

COMMON STATUS CODES

CODE	MEANING
`200 / 201 / 204`	OK / Created / No Content
`301 / 308`	Permanent redirect
`302 / 303 / 307`	Temporary redirect
`400 / 401 / 403`	Bad request / Unauth / Forbidden
`404 / 405 / 422`	Not found / Method / Invalid
`429`	Too many requests
`500 / 503`	Server error / Unavailable

HEADER FUNCTIONS

FUNCTION	DOES
`header($h)`	Set header
`header($h, false)`	Add another
`header($h, true, $code)`	With status
`http_response_code($c)`	Status
`headers_sent()`	Already sent?
`headers_list()`	Pending headers
`header_remove($name)`	Remove

USEFUL HEADERS

HEADER	PURPOSE
`Content-Type`	Body format
`Cache-Control`	Caching rules
`Content-Disposition`	Download filename
`Location`	Redirect target
`Content-Security-Policy`	XSS defence
`Strict-Transport-Security`	Force HTTPS

⚠ COMMON MISTAKES

- Output (even whitespace or a BOM) before `header()`
- Missing `exit` after a redirect — code keeps running
- Redirecting to a URL taken from user input (open redirect)

✓ TIPS & BEST PRACTICES

- Centralise responses in helper functions or a Response object
- Use 303 after POST, 301/308 for moved pages
- Remove `X-Powered-By` and add security headers

QUICK SUMMARY

- `header()` before output
- `http_response_code()`
- Redirect + `exit`
- JSON: content type + encode

Cookies

SMALL DATA IN THE BROWSER

A cookie is a name/value pair the browser stores and sends back with every matching request. Set one with `setcookie()` **before output** — it's a `Set-Cookie` header — and read incoming ones from `$_COOKIE` (a newly set cookie appears there only on the **next** request). Protect cookies with the `Secure`, `HttpOnly` and `SameSite` attributes, and never store secrets or trusted data in them unsigned.

NOTES

- Cookie size limit is about 4 KB
- Delete by setting an expiry in the past with the same path/domain
- `SameSite=Lax` is a sensible default; `None` requires `Secure`
- Values are URL-encoded; `setrawcookie()` skips that
- Users can read and edit cookies — treat them as input

SETTING, READING, DELETING

```php
<?php
setcookie('theme', 'dark', [
    'expires'  => time() + 60 * 60 * 24 * 365,   // 1 year
    'path'     => '/',
    'secure'   => true,       // HTTPS only
    'httponly' => true,       // hidden from JavaScript
    'samesite' => 'Lax',      // CSRF protection
]);

$theme = $_COOKIE['theme'] ?? 'light';     // next request

// Delete: same name/path, expiry in the past
setcookie('theme', '', ['expires' => time() - 3600, 'path' => '/']);

// Tamper-evident value
$value = 'user:42';
$sig = hash_hmac('sha256', $value, $key);
setcookie('remember', "$value|$sig", ['httponly' => true,
    'secure' => true, 'samesite' => 'Strict', 'path' => '/']);

[$v, $s] = explode('|', $_COOKIE['remember'] ?? '|', 2);
$valid = hash_equals(hash_hmac('sha256', $v, $key), $s);
```

CODE EXPLAINED

- → Options array form (7.3+) sets every attribute
- → `httponly` blocks `document.cookie` access
- → Read on later requests via `$_COOKIE`
- → Deleting needs matching path/domain
- → HMAC signature detects tampering

SETCOOKIE OPTIONS

OPTION	EFFECT
`expires`	Unix time; 0 = session
`path`	URL path scope
`domain`	Host scope
`secure`	HTTPS only
`httponly`	No JavaScript access
`samesite`	Strict / Lax / None

SAMESITE VALUES

VALUE	SENT ON
`Strict`	Same-site requests only
`Lax`	+ top-level GET navigation
`None`	All (needs Secure)
`(unset)`	Browser default ≈ Lax

COOKIE VS SESSION

ASPECT	COOKIE / SESSION
`Stored`	Browser / Server
`Size`	~4 KB / Large
`User can edit`	Yes / No
`Lifetime`	Your expiry / Session config
`Good for`	Preferences / Auth state

⚠ COMMON MISTAKES

- Calling `setcookie()` after output
- Expecting `$_COOKIE` to update in the same request
- Storing user IDs or roles in plain cookies

✓ TIPS & BEST PRACTICES

- Always set `HttpOnly`, `Secure`, `SameSite`
- Sign or encrypt values you rely on
- Keep auth state in sessions instead

QUICK SUMMARY

- `setcookie()` before output
- Read via `$_COOKIE` next request
- Secure + HttpOnly + SameSite
- Never trust cookie contents

Sessions

session_start() · $_SESSION · session_regenerate_id() · session_destroy() · session_set_cookie_params() · save handlers

SERVER-SIDE STATE

Sessions keep data on the server between requests. `session_start()` sends a session ID cookie (`PHPSESSID`) and loads stored data into `$_SESSION`; changes are saved at the end of the request. Call `session_regenerate_id(true)` whenever privileges change (login, logout) to prevent **session fixation**. By default sessions are files; Redis or a database scale across servers.

NOTES

- Call `session_start()` before any output
- Session files lock: one request per session at a time
- `session_write_close()` releases the lock early
- Set cookie params before `session_start()`
- `session.use_strict_mode=1` rejects unknown IDs

LOGIN, USE, LOGOUT

```php
<?php
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'secure'   => true,
    'httponly' => true,
    'samesite' => 'Lax',
]);
session_start();

// Login
if (password_verify($password, $user->hash)) {
    session_regenerate_id(true);           // new ID, delete old
    $_SESSION['user_id'] = $user->id;
    $_SESSION['login_at'] = time();
}

// Use
$userId = $_SESSION['user_id'] ?? null;
$_SESSION['cart'][] = $productId;
session_write_close();                     // done writing → unlock

// Logout
session_start();
$_SESSION = [];
session_destroy();
setcookie(session_name(), '',
    ['expires' => time() - 3600, 'path' => '/']);
```

CODE EXPLAINED

→ Harden the session cookie before starting

→ Regenerate the ID at login

→ `$_SESSION` behaves like a normal array

→ Release the lock when you're done writing

→ Full logout: clear data, destroy, expire cookie

SESSION FUNCTIONS

FUNCTION	DOES
`session_start()`	Start / resume
`session_regenerate_id(true)`	New ID
`session_write_close()`	Save + unlock
`session_destroy()`	Delete stored data
`session_status()`	NONE / ACTIVE
`session_name()` / `session_id()`	Cookie name / ID

KEY INI SETTINGS

SETTING	RECOMMENDED
`session.use_strict_mode`	1
`session.cookie_httponly`	1
`session.cookie_secure`	1
`session.cookie_samesite`	Lax
`session.gc_maxlifetime`	e.g. 1440 s
`session.save_handler`	files / redis

STORAGE OPTIONS

HANDLER	NOTES
`files`	Default, single server
`redis`	Fast, shared, needs phpredis
`memcached`	Shared, volatile
`database`	Custom SessionHandlerInterface
`Signed cookie`	Stateless (frameworks)

⚠ COMMON MISTAKES

- Not regenerating the ID after login — session fixation
- Long requests holding the session lock
- Storing large objects in the session

✓ TIPS & BEST PRACTICES

- Store IDs, not whole objects
- Use Redis for multi-server setups
- Close the session early in slow requests

QUICK SUMMARY

- `session_start()` + `$_SESSION`
- Regenerate on login
- Harden cookie params
- Destroy fully on logout

File Uploads

RECEIVING FILES

Upload forms need `method="post"` and `enctype="multipart/form-data"`. PHP saves each file to a temporary path and describes it in `$_FILES`. Check the `error` code, verify size and real content type (with `finfo`, not the client's claim), generate your own filename, and move the file with `move_uploaded_file()` to a directory **outside the web root** or one that never executes PHP.

NOTES

- Temp files are deleted at the end of the request
- `upload_max_filesize` limits one file; `post_max_size` the whole body
- If the body exceeds `post_max_size`, `$_POST` and `$_FILES` are empty
- `name` and `type` come from the client — don't trust them
- Multiple files: `name="docs[]"` + `multiple`

A SAFE UPLOAD HANDLER

```php
<?php
$f = $_FILES['avatar'] ?? null;
if (!$f || $f['error'] !== UPLOAD_ERR_OK) {
    $code = $f['error'] ?? 'none';
    throw new RuntimeException("Upload failed: code $code");
}
if ($f['size'] > 2 * 1024 * 1024) {
    throw new RuntimeException('Max 2 MB');
}

$mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
$ext = match ($mime) {
    'image/jpeg' => 'jpg',
    'image/png'  => 'png',
    'image/webp' => 'webp',
    default      => throw new RuntimeException('Images only'),
};

$name = bin2hex(random_bytes(16)) . ".$ext";  // own name
$dest = __DIR__ . '/../storage/avatars/' . $name;
if (!move_uploaded_file($f['tmp_name'], $dest)) {
    throw new RuntimeException('Could not save file');
}
?>
<form method="post" enctype="multipart/form-data">
  <input type="file" name="avatar" accept="image/*">
  <button>Upload</button>
</form>
```

CODE EXPLAINED

- → Check `error` first
- → Enforce your own size limit
- → `finfo` inspects the actual bytes
- → Random filename avoids overwrites and traversal
- → `move_uploaded_file` only moves genuine uploads

$_FILES ENTRY

KEY	HOLDS
`name`	Client filename (untrusted)
`full_path`	Folder path (8.1)
`type`	Client MIME (untrusted)
`tmp_name`	Temp path on server
`error`	UPLOAD_ERR_* code
`size`	Bytes

ERROR CODES

CONSTANT	MEANING
`UPLOAD_ERR_OK (0)`	Success
`UPLOAD_ERR_INI_SIZE (1)`	> upload_max_filesize
`UPLOAD_ERR_FORM_SIZE (2)`	> MAX_FILE_SIZE field
`UPLOAD_ERR_PARTIAL (3)`	Partial upload
`UPLOAD_ERR_NO_FILE (4)`	Nothing sent
`UPLOAD_ERR_NO_TMP_DIR (6)`	Server misconfig

CONFIG LIMITS

SETTING	DEFAULT
`file_uploads`	On
`upload_max_filesize`	2M
`post_max_size`	8M
`max_file_uploads`	20
`upload_tmp_dir`	System temp
`max_input_time`	-1 / 60

⚠ COMMON MISTAKES

- Trusting `$_FILES['x']['type']` or the extension
- Saving uploads in a public, PHP-executing folder
- Using the original filename on disk

✓ TIPS & BEST PRACTICES

- Verify content with `finfo` and re-encode images
- Store outside the web root; serve through a controller
- Use object storage (S3) for scale

QUICK SUMMARY

- multipart/form-data + POST
- Check `error` and size
- `finfo` for real MIME
- Random name + `move_uploaded_file`

Templating & Views

alternative syntax · <?= ?> · partials via include · output buffering layouts · Twig · Blade

SEPARATING LOGIC FROM HTML

PHP itself is a template language: plain PHP files with <?= ?> echo tags and alternative control syntax (`foreach … endforeach`) make readable views. Pass data in, render with output buffering to capture the result, and wrap pages in a layout. **Escape every value** you output. Template engines like **Twig** (Symfony) and **Blade** (Laravel) add auto-escaping, inheritance and a cleaner syntax.

NOTES

- Keep business logic out of templates
- `extract()` turns an array into local variables — use only with trusted keys
- Auto-escaping engines escape by default; mark trusted HTML explicitly
- Compiled templates are cached PHP — fast
- Short helper `e()` keeps escaping concise

A TINY VIEW RENDERER

```php
<?php
function e(?string $v): string {
    return htmlspecialchars($v ?? '', ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}

function render(string $view, array $data = []): string {
    extract($data, EXTR_SKIP);
    ob_start();
    require __DIR__ . "/views/$view.php";
    return ob_get_clean();
}

$content = render('products', ['products' => $list]);
echo render('layout', ['title' => 'Shop', 'content' => $content]);
?>
<!-- views/products.php -->
<ul>
<?php foreach ($products as $p): ?>
    <li class="<?= $p->onSale ? 'sale' : '' ?>">
        <?= e($p->name) ?> — <?= number_format($p->price, 2) ?>
    </li>
<?php endforeach; ?>
</ul>
<!-- views/layout.php -->
<title><?= e($title) ?></title>
<main><?= $content /* already-rendered HTML */ ?></main>
```

CODE EXPLAINED

- → `e()` escapes for HTML attributes and text
- → `extract` + `require` exposes data as variables
- → Output buffering captures the view as a string
- → Alternative syntax reads cleanly inside HTML
- → Layout receives pre-rendered content

TEMPLATE SYNTAX

PLAIN PHP	TWIG / BLADE
`<?= e($x) ?>`	{{ x }} / {{ $x }}
`foreach … endforeach`	{% for %} / @foreach
`if … endif`	{% if %} / @if
`include partial`	{% include %} / @include
`Layout`	{% extends %} / @extends
`Raw HTML`	|raw / {!! $x !!}

ESCAPING CONTEXTS

CONTEXT	USE
`HTML text / attribute`	htmlspecialchars
`URL parameter`	rawurlencode
`JavaScript value`	json_encode + JSON_HEX_*
`CSS value`	Allow-list
`Already safe HTML`	Output as-is (carefully)

ENGINES

ENGINE	NOTES
`Plain PHP`	No dependencies
`Twig`	Sandboxing, auto-escape
`Blade`	Laravel, compiles to PHP
`Latte`	Context-aware escaping
`Plates`	Native PHP with layouts

⚠ COMMON MISTAKES

- Echoing variables without escaping
- Database queries inside templates
- `extract()` on user input — can overwrite variables

✓ TIPS & BEST PRACTICES

- Escape by default, opt out deliberately
- Use an engine with auto-escaping for teams
- Keep templates dumb: data in, HTML out

QUICK SUMMARY

- PHP is a template language
- `<?= e($x) ?>` everywhere
- Buffer + layout
- Twig / Blade auto-escape

Routing & JSON APIs

BUILDING AN API ENDPOINT

A router maps an HTTP method and path to a handler. In plain PHP you read `REQUEST_METHOD` and the path from `REQUEST_URI`, match it against patterns, decode a JSON body from `php://input`, and send a JSON response with the right status code. Real projects use a router library or framework built on **PSR-7** (HTTP message objects) and **PSR-15** (middleware and request handlers).

NOTES

- Return 405 with an `Allow` header for wrong methods
- Return 404 JSON for unknown routes
- Validate JSON input like form input
- Middleware handles auth, CORS, logging
- Libraries: FastRoute, Slim, Laravel, Symfony

A MINIMAL ROUTER

```php
<?php
$method = $_SERVER['REQUEST_METHOD'];
$path   = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$path   = rtrim($path, '/') ?: '/';

$routes = [
    ['GET',  '#^/api/users$#',          fn() => listUsers()],
    ['GET',  '#^/api/users/(\d+)$#',    fn($id) => showUser((int) $id)],
    ['POST', '#^/api/users$#',          fn() => createUser(readJson())],
];

function readJson(): array {
    try {
        return json_decode(file_get_contents('php://input'), true,
            flags: JSON_THROW_ON_ERROR) ?? [];
    } catch (JsonException) {
        sendJson(['error' => 'Invalid JSON'], 400);
    }
}

function sendJson(mixed $body, int $status = 200): never {
    http_response_code($status);
    header('Content-Type: application/json');
    echo json_encode($body);
    exit;
}

foreach ($routes as [$verb, $pattern, $handler]) {
    if (preg_match($pattern, $path, $m) && $verb === $method) {
        sendJson($handler(...array_slice($m, 1)));
    }
}
sendJson(['error' => 'Not found'], 404);
```

CODE EXPLAINED

→ Normalise the path (no trailing slash)

→ Each route: method, regex, handler

→ Capture groups become handler arguments

→ Bad JSON → 400 response

→ Unmatched → 404 JSON

REST CONVENTIONS

METHOD + PATH	ACTION
`GET /users`	List
`GET /users/7`	Show
`POST /users`	Create (201)
`PUT/PATCH /users/7`	Update
`DELETE /users/7`	Delete (204)

PSR STANDARDS

PSR	DEFINES
PSR-7	Request / Response objects
PSR-15	Middleware + handlers
PSR-17	Message factories
PSR-18	HTTP client
PSR-11	Container

API CHECKLIST

ITEM	DETAIL
`Content-Type`	application/json
`Status codes`	Match the outcome
`Errors`	Consistent JSON shape
`Auth`	Bearer token / session
`CORS`	Access-Control-* headers
`Rate limits`	429 + Retry-After

⚠ COMMON MISTAKES

- Returning 200 with an error message in the body
- Matching routes without anchors (`^…$`)
- No validation on JSON input

✓ TIPS & BEST PRACTICES

- Use a router library (FastRoute) or framework
- Keep handlers thin; put logic in services
- Document the API (OpenAPI)

QUICK SUMMARY

- Method + path → handler
- JSON in via `php://input`
- JSON out + status code
- PSR-7/15 for real apps

PDO — Connecting

new PDO() · DSN strings · ERRMODE_EXCEPTION · DEFAULT_FETCH_MODE · utf8mb4 · driver-specific subclasses (8.4)

ONE API, MANY DATABASES

PDO (PHP Data Objects) is the standard database layer: the same methods work with MySQL/MariaDB, PostgreSQL, SQLite, SQL Server and more, chosen by the **DSN** string. Create one connection per request with sensible attributes: throw exceptions on errors, fetch associative arrays by default, and use real prepared statements. Keep credentials in environment variables. PHP 8.4 adds driver subclasses such as `Pdo\Mysql` and the `PDO::connect()` factory.

NOTES

- Since PHP 8.0, `ERRMODE_EXCEPTION` is the default
- Use `charset=utf8mb4` in MySQL DSNs for full Unicode
- `ATTR_EMULATE_PREPARES => false` uses native prepares
- Connection closes when the PDO object is destroyed
- Install drivers: `pdo_mysql`, `pdo_pgsql`, `pdo_sqlite`

CREATING A CONNECTION

```php
<?php
$dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4',
    getenv('DB_HOST'), getenv('DB_PORT') ?: 3306, getenv('DB_NAME'));

try {
    $pdo = new PDO($dsn, getenv('DB_USER'), getenv('DB_PASS'), [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    error_log($e->getMessage());              // never echo it
    throw new RuntimeException('Database unavailable');
}

$pg   = new PDO('pgsql:host=db;dbname=shop', 'app', $pw);
$lite = new PDO('sqlite:' . __DIR__ . '/data.db');
$mem  = new PDO('sqlite::memory:');           // tests

// PHP 8.4: driver-specific class
$db = PDO::connect('sqlite::memory:');        // Pdo\Sqlite
```

CODE EXPLAINED

- → DSN = driver prefix + parameters
- → Exceptions on any SQL error
- → Rows come back as associative arrays
- → Native prepares send parameters separately
- → Messages can contain credentials — log, don't display

DSN FORMATS

DRIVER	DSN
MySQL	mysql:host=h;dbname=d;charset=utf8mb4
PostgreSQL	pgsql:host=h;port=5432;dbname=d
SQLite file	sqlite:/path/app.db
SQLite memory	sqlite::memory:
SQL Server	sqlsrv:Server=h;Database=d
Unix socket	mysql:unix_socket=/run/mysqld.sock

KEY ATTRIBUTES

ATTRIBUTE	RECOMMENDED
ATTR_ERRMODE	ERRMODE_EXCEPTION
ATTR_DEFAULT_FETCH_MODE	FETCH_ASSOC
ATTR_EMULATE_PREPARES	false
ATTR_PERSISTENT	false (usually)
ATTR_TIMEOUT	Driver-specific seconds
ATTR_STRINGIFY_FETCHES	false

CONNECTION TIPS

TIP	WHY
One PDO per request	Reuse connection
Env vars for secrets	Not in code
utf8mb4	Emoji-safe
Least-privilege user	Limit damage
Catch at the boundary	Friendly error

⚠ COMMON MISTAKES

- Echoing `PDOException` messages to users
- Missing `charset` — mojibake and encoding bugs
- Opening a new connection per query

✓ TIPS & BEST PRACTICES

- Create the PDO in one factory or container service
- Use SQLite in-memory for fast tests
- Keep credentials in env vars or a secrets manager

QUICK SUMMARY

- `new PDO($dsn, $user, $pass, $opts)`
- Exceptions + FETCH_ASSOC
- utf8mb4 for MySQL
- Never show connection errors

PDO — Prepared Statements

QUERIES WITHOUT SQL INJECTION

A **prepared statement** sends the SQL with placeholders first and the values separately, so user input can never change the query's structure. Use named (`:email`) or positional (`?`) placeholders, then call `execute()` with an array of values. `bindValue()` binds a value with an explicit type; `bindParam()` binds a variable by reference. Prepare once and execute many times for bulk work.

NOTES

- Placeholders work for **values** only — not table or column names
- Don't mix named and positional placeholders in one query
- `execute()` array values are bound as strings unless you `bindValue` types
- `rowCount()` gives affected rows for INSERT/UPDATE/DELETE
- `lastInsertId()` returns the new auto-increment key

PREPARE, BIND, EXECUTE

```php
<?php
// Named placeholders
$stmt = $pdo->prepare('SELECT id, name FROM users WHERE email = :email');
$stmt->execute(['email' => $_POST['email']]);
$user = $stmt->fetch();                    // array or false

// Positional placeholders
$stmt = $pdo->prepare('UPDATE users SET name = ? WHERE id = ?');
$stmt->execute([$name, $id]);
echo $stmt->rowCount(), ' row(s) updated';

// Typed binding (LIMIT needs an int)
$stmt = $pdo->prepare('SELECT * FROM posts ORDER BY id DESC LIMIT :n');
$stmt->bindValue(':n', $limit, PDO::PARAM_INT);
$stmt->execute();

// Prepare once, run many
$ins = $pdo->prepare('INSERT INTO tags (name) VALUES (?)');
foreach (['php', 'sql', 'web'] as $tag) {
    $ins->execute([$tag]);
}
$newId = $pdo->lastInsertId();

// IN (...) with a dynamic list
$ids = [3, 5, 8];
$in  = implode(',', array_fill(0, count($ids), '?'));
$stmt = $pdo->prepare("SELECT * FROM users WHERE id IN ($in)");
$stmt->execute($ids);
```

CODE EXPLAINED

- → `:email` is filled from the array key
- → `?` placeholders are filled in order
- → `bindValue` with `PARAM_INT` for LIMIT/OFFSET
- → Reuse one statement in a loop
- → Build one `?` per item for `IN` lists

STATEMENT METHODS

METHOD	DOES
`$pdo->prepare($sql)`	Create statement
`->execute($params)`	Run with values
`->bindValue($p, $v, $type)`	Bind value
`->bindParam($p, &$var)`	Bind reference
`->rowCount()`	Affected rows
`->closeCursor()`	Free result set

PARAM TYPES

CONSTANT	FOR
`PDO::PARAM_STR`	Strings (default)
`PDO::PARAM_INT`	Integers
`PDO::PARAM_BOOL`	Booleans
`PDO::PARAM_NULL`	NULL
`PDO::PARAM_LOB`	Binary / large data

DYNAMIC IDENTIFIERS

NEED	SAFE APPROACH
`Sort column`	Allow-list → literal
`Sort direction`	'ASC'/'DESC' allow-list
`Table name`	Allow-list, never input
`IN list`	One ? per value
`LIKE search`	Bind '%' . $term . '%'

⚠ COMMON MISTAKES

- Concatenating input into SQL: `"... WHERE id = $id"`
- Trying to bind a column name as a placeholder
- Passing LIMIT values as strings — breaks with emulated prepares

✓ TIPS & BEST PRACTICES

- Prepare every query that has variables
- Allow-list identifiers you can't bind
- Wrap repeated inserts in a transaction (Sheet 122)

QUICK SUMMARY

- `prepare` + `execute([...])`
- `:named` or `?`
- Values only, never identifiers
- `bindValue` for types

PDO — Fetching Results

GETTING ROWS BACK

After `execute()`, read results with `fetch()` (one row at a time, `false` when done), `fetchAll()` (every row in an array) or `fetchColumn()` (one value). The **fetch mode** decides the shape: associative arrays, objects, instances of your own class, key-value pairs or grouped arrays. A statement is also **iterable** with `foreach`, which streams rows without loading them all.

NOTES

- `fetch()` returns `false` when no row matches
- `fetchAll()` loads everything into memory
- `FETCH_CLASS` sets properties **before** the constructor runs
- Numbers may come back as strings depending on driver/settings
- `query()` is fine for SQL with no variables

FETCH MODES IN PRACTICE

```php
<?php
$stmt = $pdo->prepare(
    'SELECT id, name, email FROM users WHERE active = ?');
$stmt->execute([1]);

$row  = $stmt->fetch();                       // first row (assoc)
$rows = $stmt->fetchAll();                    // remaining rows

$count = $pdo->query('SELECT COUNT(*) FROM users')->fetchColumn();

foreach ($pdo->query('SELECT id, name FROM users') as $r) {
    echo $r['name'];                          // streams rows
}

$names = $pdo->query('SELECT id, name FROM users')
             ->fetchAll(PDO::FETCH_KEY_PAIR);  // [1 => 'Ana', …]
$emails = $pdo->query('SELECT email FROM users')
              ->fetchAll(PDO::FETCH_COLUMN);   // ['a@x', …]
$byRole = $pdo->query('SELECT role, name FROM users')
              ->fetchAll(PDO::FETCH_GROUP | PDO::FETCH_COLUMN);
// ['admin' => ['Ana'], 'user' => ['Bo', 'Cy']]

final class UserRow { public int $id; public string $name; }
$stmt = $pdo->query('SELECT id, name FROM users');
$users = $stmt->fetchAll(PDO::FETCH_CLASS, UserRow::class);

$one = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$one->execute([$id]);
$user = $one->fetch() ?: throw new RuntimeException('Not found');
```

CODE EXPLAINED

→ Default mode from the connection: FETCH_ASSOC

→ `fetchColumn()` → single scalar

→ Iterating the statement streams rows

→ `KEY_PAIR` builds id → name maps

→ `fetch() ?: throw` handles missing rows

FETCH METHODS

METHOD	RETURNS
`fetch()`	Next row or false
`fetchAll()`	Array of rows
`fetchColumn($i)`	One value or false
`fetchObject(C::class)`	One object
`foreach ($stmt as $row)`	Streamed rows
`getIterator()`	Iterator (8.0)

FETCH MODES

MODE	ROW SHAPE
`FETCH_ASSOC`	['id' => 1]
`FETCH_NUM`	[0 => 1]
`FETCH_OBJ`	stdClass
`FETCH_CLASS`	Your class
`FETCH_KEY_PAIR`	[col1 => col2]
`FETCH_COLUMN`	Flat list
`FETCH_GROUP`	Grouped by first col

CHOOSING

NEED	USE
`One record`	fetch()
`Count / scalar`	fetchColumn()
`Small list`	fetchAll()
`Large result`	foreach / fetch loop
`Lookup map`	FETCH_KEY_PAIR
`Typed objects`	FETCH_CLASS / hydrate

⚠ COMMON MISTAKES

- `fetchAll()` on millions of rows
- Treating `false` from `fetch()` as an empty array
- Expecting `FETCH_CLASS` to call the constructor first

✓ TIPS & BEST PRACTICES

- Stream big results with `foreach`
- Use `KEY_PAIR` / `COLUMN` to skip manual loops
- Map rows to DTOs for type safety

QUICK SUMMARY

- `fetch` / `fetchAll` / `fetchColumn`
- Modes shape each row
- `foreach` streams
- `false` = no row

PDO — Transactions & Errors

ALL OR NOTHING

A **transaction** groups several statements so they all succeed or none do. Call `beginTransaction()`, run your queries, then `commit()`; if anything throws, `rollBack()` undoes the changes. With `ERRMODE_EXCEPTION`, failures raise `PDOException`, whose `errorInfo` holds the SQLSTATE and driver codes — useful for handling duplicates or deadlocks specifically. Transactions also speed up bulk inserts.

NOTES

- MySQL DDL statements (CREATE, ALTER) commit implicitly
- Only transactional engines (InnoDB) can roll back
- Nested `beginTransaction()` throws — use savepoints
- Keep transactions short to avoid lock contention
- Retry on deadlock (SQLSTATE 40001)

TRANSFER MONEY SAFELY

```php
<?php
function transfer(PDO $pdo, int $from, int $to, int $cents): void
{
    $pdo->beginTransaction();
    try {
        $debit = $pdo->prepare(
            'UPDATE accounts SET balance = balance - ?
             WHERE id = ? AND balance >= ?');
        $debit->execute([$cents, $from, $cents]);
        if ($debit->rowCount() !== 1) {
            throw new DomainException('Insufficient funds');
        }
        $pdo->prepare('UPDATE accounts
             SET balance = balance + ? WHERE id = ?')
            ->execute([$cents, $to]);
        $pdo->commit();
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) $pdo->rollBack();
        throw $e;
    }
}

try {
    $pdo->prepare('INSERT INTO users (email) VALUES (?)')
        ->execute([$email]);
} catch (PDOException $e) {
    if ($e->errorInfo[1] === 1062) {          // MySQL duplicate key
        echo 'Email already registered';
    } else {
        throw $e;
    }
}
```

CODE EXPLAINED

→ Start the transaction before the first write

→ Conditional UPDATE guards the balance atomically

→ `rowCount()` confirms the debit happened

→ Any exception → roll back, then rethrow

→ `errorInfo[1]` is the driver error code

TRANSACTION METHODS

METHOD	DOES
`beginTransaction()`	Start
`commit()`	Save changes
`rollBack()`	Undo changes
`inTransaction()`	Active?
`exec('SAVEPOINT s1')`	Nested point
`exec('ROLLBACK TO s1')`	Partial undo

PDOEXCEPTION DETAILS

PROPERTY / METHOD	HOLDS
`getMessage()`	Driver message
`getCode()`	SQLSTATE string
`errorInfo[0]`	SQLSTATE
`errorInfo[1]`	Driver error code
`errorInfo[2]`	Driver message

COMMON CODES

CODE	MEANING
`23000 / 1062`	Duplicate key (MySQL)
`23505`	Unique violation (PostgreSQL)
`40001`	Serialization failure / deadlock
`42S02`	Table not found
`HY000`	General error

⚠ COMMON MISTAKES

- Forgetting `rollBack()` on failure — connection stays in a transaction
- Long transactions that wait on HTTP calls
- Assuming MyISAM tables roll back

✓ TIPS & BEST PRACTICES

- Wrap multi-step writes in a helper like `transactional(fn)`
- Use conditional updates to avoid race conditions
- Handle unique-key violations instead of pre-checking

QUICK SUMMARY

- begin → commit / rollBack
- Exceptions trigger rollback
- `errorInfo` codes
- Short transactions

MySQLi

THE MYSQL-SPECIFIC EXTENSION

MySQLi (MySQL improved) is an alternative to PDO that works only with MySQL and MariaDB. It offers both an object-oriented and a procedural API, supports prepared statements with `bind_param()`, and exposes MySQL-specific features such as multi-queries and async queries. Since PHP 8.1, errors throw `mysqli_sql_exception` by default. New projects usually prefer PDO for its portability and named placeholders.

NOTES

- `bind_param` type string: `i` int, `d` double, `s` string, `b` blob
- `get_result()` needs the mysqlnd driver (standard today)
- 8.2 added `execute_query()` — prepare + bind + execute in one
- The old `mysql_*` functions were removed in PHP 7
- Set the charset with `set_charset('utf8mb4')`

MYSQLI OBJECT-ORIENTED STYLE

```php
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);   // throw

$db = new mysqli('localhost', 'app', $password, 'shop');
$db->set_charset('utf8mb4');

$stmt = $db->prepare(
    'SELECT id, name FROM products WHERE price < ? AND cat = ?');
$stmt->bind_param('ds', $maxPrice, $category);   // double, string
$stmt->execute();
$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    echo $row['name'], "\n";
}

// PHP 8.2+: one call
$rows = $db->execute_query(
    'SELECT * FROM orders WHERE user_id = ?', [$userId]
)->fetch_all(MYSQLI_ASSOC);

$db->query("INSERT INTO log (msg) VALUES ('started')");
echo $db->insert_id, ' ', $db->affected_rows;

// Procedural equivalent
$link = mysqli_connect('localhost', 'app', $password, 'shop');
$res  = mysqli_query($link, 'SELECT COUNT(*) AS n FROM products');
echo mysqli_fetch_assoc($res)['n'];
```

CODE EXPLAINED

→ Make MySQLi throw exceptions (default since 8.1)

→ Always set utf8mb4

→ `'ds'` — one type letter per placeholder

→ `execute_query()` shortens the common case

→ Procedural functions take the link as first argument

PDO VS MYSQLI

FEATURE	PDO / MYSQLI
Databases	12+ drivers / MySQL only
Named placeholders	Yes / No
API style	OO / OO + procedural
Typed binding	bindValue / bind_param types
MySQL extras	Limited / Async, multi-query
Recommendation	Default choice / Legacy, MySQL-specific

BIND_PARAM TYPES

LETTER	TYPE
i	integer
d	double (float)
s	string
b	blob (sent in packets)

RESULT METHODS

METHOD	RETURNS
`fetch_assoc()`	Assoc row or null
`fetch_row()`	Numeric row
`fetch_object()`	Object
`fetch_all(MYSQLI_ASSOC)`	All rows
`num_rows`	Row count
`free()`	Release memory

⚠ COMMON MISTAKES

- Interpolating variables into `query()` strings
- Wrong type letters in `bind_param()`
- Code still using removed `mysql_*` functions

✓ TIPS & BEST PRACTICES

- Prefer PDO for new code
- Use `execute_query()` on 8.2+
- Enable strict error reporting

QUICK SUMMARY

- MySQL-only extension
- `prepare` + `bind_param('si', …)`
- `execute_query()` (8.2)
- PDO is the portable default

ORMs & Query Builders

HIGHER-LEVEL DATABASE ACCESS

A **query builder** (Doctrine DBAL, Laravel's query builder) composes SQL with method calls and binds values for you. An **ORM** maps rows to objects. **Eloquent** (Laravel) uses *Active Record* — each model saves itself. **Doctrine ORM** (Symfony) uses *Data Mapper* — plain entities plus an `EntityManager` that tracks changes. Both include **migrations** for versioning the schema. Watch out for the **N+1 query** problem and use eager loading.

NOTES

- ORMs still use prepared statements under the hood
- Raw SQL is fine for reports and complex queries
- Migrations belong in version control
- Log queries in development to spot N+1
- Eager loading: Eloquent `with()`, Doctrine fetch joins

ELOQUENT, DOCTRINE, AND THE N+1 FIX

```php
<?php
// Eloquent (Active Record)
class Post extends Illuminate\Database\Eloquent\Model
{
    protected $fillable = ['title', 'body'];
    public function author()
    { return $this->belongsTo(User::class); }
}
$post = Post::create(['title' => 'Hi', 'body' => '…']);
$recent = Post::where('published', true)->latest()->take(10)->get();

foreach (Post::all() as $p) echo $p->author->name;    // N+1!
foreach (Post::with('author')->get() as $p) {         // 2 queries
    echo $p->author->name;
}

// Doctrine ORM (Data Mapper, attributes)
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Product
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    public ?int $id = null;
    #[ORM\Column(length: 120)]
    public string $name;
}
$product = new Product();
$product->name = 'Mug';
$em->persist($product);
$em->flush();                                  // INSERT here
$found = $em->getRepository(Product::class)
    ->findOneBy(['name' => 'Mug']);
```

CODE EXPLAINED

- → Eloquent models map to tables by convention
- → Query methods chain and bind values
- → Lazy relation in a loop = one query per row
- → `with()` eager-loads in a single extra query
- → Doctrine tracks entities and writes on `flush()`

PATTERNS

PATTERN	EXAMPLE
Active Record	Eloquent: $post->save()
Data Mapper	Doctrine: $em->persist()
Query builder	DBAL, Laravel DB
Raw SQL	PDO prepared statements
Repository	Query methods per entity

MIGRATIONS

TOOL	COMMAND
Laravel	php artisan migrate
Doctrine	bin/console doctrine:migrations:migrate
Phinx	vendor/bin/phinx migrate
Rollback	migrate:rollback / :prev
Generate	make:migration / diff

N+1 FIXES

ORM	EAGER LOAD
Eloquent	Post::with('author')
Eloquent nested	with('author.team')
Doctrine DQL	JOIN FETCH
Doctrine	fetch: 'EAGER' (careful)
Detect	Debugbar, query log

⚠ COMMON MISTAKES

- Accessing relations inside loops without eager loading
- Mass assignment without `$fillable` guards
- Loading whole tables into objects for a count

✓ TIPS & BEST PRACTICES

- Enable query logging during development
- Use DTO/array hydration for read-heavy reports
- Keep migrations small and reversible

QUICK SUMMARY

- Builders compose SQL
- Eloquent = Active Record
- Doctrine = Data Mapper
- Eager-load to avoid N+1

XSS & Output Escaping

CROSS-SITE SCRIPTING

XSS happens when user-controlled text is output as HTML or JavaScript, letting an attacker run scripts in other users' browsers. The fix is **escaping on output, for the right context**: `htmlspecialchars()` for HTML text and attributes, `rawurlencode()` for URL parts, `json_encode()` with hex flags for data inside `<script>`. A **Content-Security-Policy** header is a strong second layer.

NOTES

- Escape at output time, not when saving input
- Default flags (8.1+): `ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401`
- Always quote HTML attributes
- `javascript:` URLs slip past HTML escaping — validate schemes
- Template engines with auto-escaping prevent most XSS

ESCAPE FOR EACH CONTEXT

```php
<?php
$name = $_GET['name'] ?? '';        // e.g. <script>alert(1)</script>
$e = fn(string $s) => htmlspecialchars($s,
    ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
?>
<!-- HTML text -->
<p>Hello, <?= $e($name) ?></p>

<!-- Quoted attribute -->
<input value="<?= $e($name) ?>">

<!-- URL parameter -->
<a href="/search?q=<?= rawurlencode($name) ?>">Search</a>

<!-- User-supplied link: allow-list the scheme -->
<?php $url = $_GET['site'] ?? '';
      $safe = preg_match('#^https?://#i', $url) ? $url : '#'; ?>
<a href="<?= $e($safe) ?>">Website</a>

<!-- Data for JavaScript -->
<script>
  const user = <?= json_encode(['name' => $name],
      JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT) ?>;
</script>
<?php
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
```

CODE EXPLAINED

→ `ENT_QUOTES` escapes both quote types

→ Same escaper works for text and quoted attributes

→ URL parts need URL encoding

→ Block `javascript:` by allowing only http(s)

→ JSON hex flags make data safe inside `<script>`

CONTEXT → ESCAPER

OUTPUT CONTEXT	USE
`HTML body text`	htmlspecialchars
`Quoted attribute`	htmlspecialchars + ENT_QUOTES
`URL query value`	rawurlencode / http_build_query
`href / src URL`	Validate scheme, then escape
`<script> data`	json_encode + JSON_HEX_*
`CSS`	Allow-list values only

HTMLSPECIALCHARS CONVERTS

CHAR	BECOMES
`&`	&
`<`	<
`>`	>
`"`	"
`'`	'

DEFENCE LAYERS

LAYER	HELPS BY
`Output escaping`	Primary fix
`Auto-escaping templates`	Safe by default
`CSP header`	Blocks inline scripts
`HttpOnly cookies`	Protects session
`Input validation`	Reduces attack surface

⚠ COMMON MISTAKES

- Escaping on input and then outputting raw elsewhere
- Unquoted attributes: `<input value=<?= $v ?>>`
- Using `strip_tags()` as an XSS filter

✓ TIPS & BEST PRACTICES

- Escape every dynamic value in templates
- Add a strict CSP
- Use HTML Purifier if users may submit rich HTML

QUICK SUMMARY

- Escape on output
- Match the context
- `htmlspecialchars` + `ENT_QUOTES`
- CSP as backup

SQL Injection

KEEPING INPUT OUT OF YOUR SQL

SQL injection happens when user input is pasted into a query string, letting an attacker change the query — reading other users' data, bypassing logins or deleting tables. The fix is simple and absolute: pass **every value** through prepared-statement placeholders. Placeholders can't hold identifiers, so column names, sort directions and table names must come from an **allow-list** in your code.

NOTES

- Escaping functions are not a substitute for placeholders
- Second-order injection: data from your DB is still untrusted
- ORMs and builders are safe until you use their raw-SQL methods
- Give the app's DB user only the privileges it needs
- Log and alert on SQL syntax errors — they can signal probing

VULNERABLE VS SAFE

```php
<?php
// ❌ Vulnerable: input becomes SQL
$id = $_GET['id'];                       // "1 OR 1=1"
$pdo->query("SELECT * FROM users WHERE id = $id");

// ✅ Safe: value travels separately
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([$_GET['id']]);

// ✅ Identifiers: allow-list, never bind or interpolate input
$columns = ['name' => 'name', 'date' => 'created_at'];
$col = $columns[$_GET['sort'] ?? ''] ?? 'name';
$dir = strtoupper($_GET['dir'] ?? '') === 'DESC' ? 'DESC' : 'ASC';
$stmt = $pdo->prepare("SELECT * FROM posts ORDER BY $col $dir LIMIT ?");
$stmt->bindValue(1, 20, PDO::PARAM_INT);
$stmt->execute();

// ✅ LIKE: bind the pattern, escape wildcards in the term
$term = str_replace(['!', '%', '_'], ['!!', '!%', '!_'], $_GET['q'] ?? '');
$stmt = $pdo->prepare("SELECT * FROM posts WHERE title LIKE ? ESCAPE '!'");
$stmt->execute(["%$term%"]);

// ✅ IN list: one placeholder per value
$ids = array_map('intval', (array) ($_GET['ids'] ?? []));
$in  = implode(',', array_fill(0, max(1, count($ids)), '?'));
$stmt = $pdo->prepare("SELECT * FROM tags WHERE id IN ($in)");
$stmt->execute($ids ?: [0]);
```

CODE EXPLAINED

- → Interpolated input can rewrite the WHERE clause
- → Placeholders keep data and code separate
- → Map user choices to known column names
- → Escape `%` and `_` so users can't widen LIKE searches
- → Generate placeholders for variable-length lists

ATTACK EXAMPLES

INPUT	EFFECT IF INTERPOLATED
`1 OR 1=1`	Returns every row
`' OR '1'='1`	Bypasses login
`1; DROP TABLE users`	Stacked query
`1 UNION SELECT password …`	Data theft
`SLEEP(5)`	Blind timing probe

SAFE TECHNIQUES

SITUATION	TECHNIQUE
`Values`	Placeholders
`Column / table names`	Allow-list map
`ASC / DESC`	Ternary to literal
`LIKE`	Bind + escape wildcards
`IN (...)`	Generated placeholders
`Raw ORM SQL`	Still bind values

DEFENCE IN DEPTH

LAYER	BENEFIT
`Least-privilege DB user`	Limits damage
`No stacked queries`	Driver setting
`Generic error pages`	No SQL leaks
`WAF / monitoring`	Detects probing
`Code review + SAST`	Finds concatenation

⚠ COMMON MISTAKES

- "Sanitising" input instead of using placeholders
- Binding the sort column as a value — it sorts by a constant
- Trusting values because they came from your own database

✓ TIPS & BEST PRACTICES

- Grep for `query("` and `->query($` in reviews
- Use static analysis rules that flag string-built SQL
- Run the app with a restricted DB account

QUICK SUMMARY

- Placeholders for every value
- Allow-list identifiers
- Escape LIKE wildcards
- Least privilege as backup

CSRF & Session Security

CSRF tokens · random_bytes() · hash_equals() · SameSite · session fixation · session.cookie_secure

REQUESTS THE USER DIDN'T MEAN TO SEND

Cross-Site Request Forgery tricks a logged-in user's browser into submitting a request to your site from another site — the browser attaches the session cookie automatically. Defend every state-changing request with a **CSRF token**: a random secret stored in the session, embedded in forms, and compared with `hash_equals()` on submit. `SameSite` cookies add a strong second layer. Protect the session itself against fixation and hijacking.

NOTES

- Only `POST` / `PUT` / `PATCH` / `DELETE` should change state
- One token per session is fine; per-form tokens are stricter
- Send the token in a header for AJAX (`X-CSRF-Token`)
- Regenerate the session ID at login and privilege changes
- Frameworks include CSRF middleware — use it

TOKEN GENERATION AND CHECKING

```php
<?php
session_start();

function csrfToken(): string
{
    return $_SESSION['csrf'] ??= bin2hex(random_bytes(32));
}

function checkCsrf(): void
{
    $sent = $_POST['_token'] ?? $_SERVER['HTTP_X_CSRF_TOKEN'] ?? '';
    $ok = is_string($sent)
        && hash_equals($_SESSION['csrf'] ?? '', $sent);
    if (!$ok) {
        http_response_code(403);
        exit('Invalid CSRF token');
    }
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    checkCsrf();
    // … perform the change …
}
?>
<form method="post" action="/account/delete">
  <input type="hidden" name="_token"
         value="<?= htmlspecialchars(csrfToken()) ?>">
  <button>Delete account</button>
</form>
<meta name="csrf-token" content="<?= htmlspecialchars(csrfToken()) ?>">
```

CODE EXPLAINED

→ `??=` creates the token once per session

→ `random_bytes(32)` = unpredictable secret

→ Accept the token from a form field or header

→ `hash_equals` compares in constant time

→ Meta tag lets JavaScript send the header

SESSION HARDENING

THREAT	DEFENCE
Fixation	session_regenerate_id(true) at login
Hijacking via XSS	HttpOnly cookie + CSP
Sniffing	Secure cookie + HTTPS + HSTS
Cross-site sends	SameSite=Lax/Strict
Unknown IDs	session.use_strict_mode=1
Idle sessions	Timeout + re-auth

CSRF DEFENCES

DEFENCE	STRENGTH
Synchronizer token	Strong (standard)
SameSite cookies	Strong second layer
Origin / Referer check	Good supplement
Custom header (AJAX)	Good for APIs
GET for changes	Never

SECURE RANDOM

FUNCTION	USE
random_bytes($n)	Tokens, keys
bin2hex($bytes)	Hex string
random_int($a, $b)	Codes, PINs
uniqid()	Not secure — avoid
md5(time())	Not secure — avoid

⚠ COMMON MISTAKES

- Comparing tokens with `==` / `===` — use `hash_equals()`
- Changing data on GET requests
- Generating tokens with `uniqid()` or `rand()`

✓ TIPS & BEST PRACTICES

- Use your framework's CSRF protection
- Set SameSite=Lax on the session cookie
- Regenerate the session ID on login and logout

QUICK SUMMARY

- Token in session + form
- `hash_equals()` to compare
- SameSite cookies
- Regenerate session IDs

Password Hashing

password_hash() · password_verify() · PASSWORD_ARGON2ID · PASSWORD_BCRYPT · password_needs_rehash() · why not md5/sha1

STORING PASSWORDS SAFELY

Never store passwords in plain text or with fast hashes like MD5 or SHA-1 — they're cracked at billions of guesses per second. `password_hash()` uses a **slow, salted** algorithm (bcrypt by default, Argon2id if available) and returns a self-describing string that includes the algorithm, cost and salt. `password_verify()` checks a login; `password_needs_rehash()` upgrades hashes when you change settings.

NOTES

- Store the hash in a `VARCHAR(255)` column
- Bcrypt only uses the first 72 bytes of the password
- PHP 8.4 raised the default bcrypt cost from 10 to 12
- Salt is generated automatically — don't supply one
- Rate-limit login attempts as well

HASH, VERIFY, REHASH

```php
<?php
// Registration
$hash = password_hash($plain, PASSWORD_DEFAULT);
// $2y$12$9bL1y0kq… (algorithm, cost, salt, hash)
$hash = password_hash($plain, PASSWORD_ARGON2ID, [
    'memory_cost' => 65536,     // KiB
    'time_cost'   => 4,
    'threads'     => 1,
]);

// Login
$row = findUserByEmail($email);
if (!$row || !password_verify($plain, $row['password_hash'])) {
    throw new RuntimeException('Invalid email or password');
}

// Upgrade old hashes transparently
if (password_needs_rehash($row['password_hash'], PASSWORD_DEFAULT)) {
    saveHash($row['id'], password_hash($plain, PASSWORD_DEFAULT));
}

password_get_info($hash);
// ['algo' => 'argon2id', 'algoName' => 'argon2id', 'options' => [...]]
```

CODE EXPLAINED

- → `PASSWORD_DEFAULT` follows PHP's recommendation
- → Argon2id is memory-hard (better against GPUs)
- → Same error message for unknown user and wrong password
- → Rehash at login when settings change
- → `password_get_info` decodes a hash's settings

ALGORITHMS

CONSTANT	NOTES
`PASSWORD_DEFAULT`	bcrypt today; may change
`PASSWORD_BCRYPT`	cost 4–31 (default 12 in 8.4)
`PASSWORD_ARGON2I`	Argon2i
`PASSWORD_ARGON2ID`	Recommended if available
`md5 / sha1 / sha256`	Never for passwords

FUNCTIONS

FUNCTION	RETURNS
`password_hash($p, $algo)`	Hash string
`password_verify($p, $h)`	bool
`password_needs_rehash($h, $algo)`	bool
`password_get_info($h)`	Settings array
`password_algos()`	Available algorithms

LOGIN HARDENING

MEASURE	WHY
`Generic error message`	No user enumeration
`Rate limiting`	Slows brute force
`Account lockout / CAPTCHA`	Stops automation
`MFA`	Stolen passwords not enough
`Breached-password check`	Blocks known leaks

⚠ COMMON MISTAKES

- `md5($password)` or unsalted SHA hashes
- Comparing hashes with `==` instead of `password_verify()`
- Truncating the hash column to 60 chars (Argon2 is longer)

✓ TIPS & BEST PRACTICES

- Use `PASSWORD_DEFAULT` or `PASSWORD_ARGON2ID`
- Rehash on login to migrate old hashes
- Tune cost so hashing takes ~100–300 ms

QUICK SUMMARY

- `password_hash` / `password_verify`
- Bcrypt or Argon2id
- Rehash when settings change
- Never fast hashes

Hashing, HMAC & Encryption

CHOOSING THE RIGHT PRIMITIVE

A **hash** (`hash('sha256', $data)`) fingerprints data — fine for checksums, never for passwords. An **HMAC** (`hash_hmac()`) adds a secret key so only key holders can create a valid signature — use it to verify tokens and webhooks. **Encryption** hides data and is reversible with the key: use **libsodium** (`sodium_crypto_secretbox()` for one shared key, `sodium_crypto_box()` for public-key) — it's built in and hard to misuse.

NOTES

- Never invent your own crypto schemes
- Keys come from `random_bytes()` or `sodium_crypto_*_keygen()`
- A nonce must never repeat with the same key
- Prefer authenticated encryption (secretbox, AES-GCM)
- Wipe secrets with `sodium_memzero()` when done

HASH, HMAC, ENCRYPT

```php
<?php
// Integrity / fingerprint
hash('sha256', 'hello');               // 2cf24dba5fb0a30e…
hash_file('sha256', 'release.zip');    // checksum

// Authenticity: HMAC-signed webhook
$expected = hash_hmac('sha256', $rawBody, $webhookSecret);
if (!hash_equals($expected, $_SERVER['HTTP_X_SIGNATURE'] ?? '')) {
    http_response_code(401); exit;
}

// Secret-key authenticated encryption (libsodium)
$key    = sodium_crypto_secretbox_keygen();          // store safely
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$box    = sodium_crypto_secretbox('card ****4242', $nonce, $key);
$stored = base64_encode($nonce . $box);

$raw   = base64_decode($stored);
$n     = substr($raw, 0, SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$c     = substr($raw, SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$plain = sodium_crypto_secretbox_open($c, $n, $key);  // false if tampered

// OpenSSL AES-256-GCM (when interoperability requires it)
$iv  = random_bytes(12);
$enc = openssl_encrypt('data', 'aes-256-gcm', $aesKey, 0, $iv, $tag);
```

CODE EXPLAINED

- → Plain hash: anyone can recompute it
- → HMAC: only holders of the secret can sign
- → `hash_equals` prevents timing attacks
- → Store nonce + ciphertext together; never reuse a nonce
- → Decryption returns `false` if data was modified

WHICH TOOL?

GOAL	USE
Checksum	hash('sha256')
Password storage	password_hash()
Sign / verify messages	hash_hmac() + hash_equals()
Encrypt with shared key	sodium_crypto_secretbox
Encrypt to a public key	sodium_crypto_box
Digital signatures	sodium_crypto_sign

HASH ALGORITHMS

ALGORITHM	STATUS
sha256 / sha512	Good
sha3-256	Good
xxh3 / xxh128	Fast, non-crypto (8.1)
crc32c	Checksums only
md5 / sha1	Broken for security

KEY HANDLING

PRACTICE	DETAIL
Generate	random_bytes / *_keygen()
Store	Env var / secrets manager
Rotate	Version your keys
Encode	base64 / hex for storage
Never	Commit keys to git

⚠ COMMON MISTAKES

- Using `md5` / `sha1` for security
- Reusing a nonce/IV with the same key
- Comparing signatures with `==`

✓ TIPS & BEST PRACTICES

- Prefer libsodium over OpenSSL for new code
- Use HMAC for webhooks and signed URLs
- Keep keys out of the codebase

QUICK SUMMARY

- Hash ≠ HMAC ≠ encryption
- `hash_hmac` + `hash_equals`
- libsodium secretbox
- Unique nonces, safe keys

Secure Configuration & File Safety

HARDENING THE ENVIRONMENT

Many PHP breaches come from configuration, not code. In production, hide errors from visitors, stop advertising the PHP version, block remote file inclusion and keep secrets out of the web root. **File inclusion** bugs (LFI/RFI) let attackers run or read files when paths come from user input — never build `include` paths from requests. Keep PHP and dependencies patched and run `composer audit`.

NOTES

- Only `public/` should be web-accessible
- Uploaded files must never be executable as PHP
- `open_basedir` limits which paths PHP can open
- `disable_functions` can block shell functions you don't use
- Use HTTPS everywhere with HSTS

PRODUCTION PHP.INI HARDENING

```
; Errors: log, don't display
display_errors = Off
display_startup_errors = Off
log_errors = On
error_log = /var/log/php/error.log
error_reporting = E_ALL

; Don't advertise
expose_php = Off

; Remote code / file access
allow_url_include = Off
allow_url_fopen = Off          ; if you use an HTTP client instead

; Limit the blast radius
open_basedir = /var/www/app:/tmp
disable_functions = exec,passthru,shell_exec,system,proc_open,popen

; Sessions
session.use_strict_mode = 1
session.cookie_secure = 1
session.cookie_httponly = 1
session.cookie_samesite = Lax

; Uploads
file_uploads = On
upload_max_filesize = 10M
post_max_size = 12M
```

CODE EXPLAINED

→ Visitors see a generic page; details go to the log

→ `expose_php=Off` removes the X-Powered-By version header

→ No remote `include`, ever

→ `open_basedir` jails file access to app paths

→ Disable shell functions if the app doesn't need them

FILE INCLUSION RISKS

PATTERN	PROBLEM
`include $_GET['page']`	LFI / RFI
`include "views/$p.php"`	Traversal (../)
`readfile($_GET['f'])`	Arbitrary file read
Fix	Allow-list map: 'home' => 'home.php'
Fix	realpath() + base dir check

DEPLOYMENT CHECKLIST

ITEM	CHECK
`Web root`	public/ only
`.env / vendor`	Not web-accessible
`Upload dir`	No PHP execution
`Debug tools`	Disabled (Xdebug, debug bars)
`composer install`	--no-dev -o
`Dependencies`	composer audit clean

SECURITY HEADERS

HEADER	PURPOSE
`Strict-Transport-Security`	Force HTTPS
`Content-Security-Policy`	Limit script sources
`X-Content-Type-Options: nosniff`	No MIME sniffing
`X-Frame-Options / frame-ancestors`	Clickjacking
`Referrer-Policy`	Limit referrer leaks

⚠ COMMON MISTAKES

- `display_errors=On` in production
- Project root as document root — `.env` downloadable
- Including files chosen by request parameters

✓ TIPS & BEST PRACTICES

- Start from `php.ini-production`
- Serve uploads through a controller, from outside the web root
- Patch PHP and run `composer audit` regularly

QUICK SUMMARY

- Log errors, don't display
- No remote include
- public/ is the only web root
- Patch and audit

PHP 8.x Feature Timeline

8.0 match & named args · 8.1 enums & fibers · 8.2 readonly classes · 8.3 typed constants · 8.4 property hooks · 8.5 pipe operator

WHAT ARRIVED WHEN

PHP 8 transformed the language with a steady stream of features. Use this sheet to check whether a feature is available on the version you deploy to, and to spot modern replacements for older idioms. Each release also deprecates older behaviour — read the migration guide on php.net before upgrading, and let tools like Rector and PHPStan do the mechanical work.

NOTES

- Releases land each year around late November
- Check your floor with `"php": "^8.3"` in composer.json
- Rector `withPhpSets()` upgrades syntax automatically
- `PHP_VERSION_ID` (e.g. 80400) for numeric checks
- Sheet numbers in parentheses point to the details

MODERN PHP IN ONE SNIPPET

```php
<?php
// 8.1 enum
enum Status: string { case Active = 'a'; case Banned = 'b'; }

// 8.2 readonly class + 8.0 constructor promotion
final readonly class User
{
    public function __construct(
        public int $id,
        public string $email,
        public Status $status = Status::Active,
    ) {}
}

// 8.0 match (exhaustive with enums)
function label(User $u): string
{
    return match ($u->status) {
        Status::Active => 'ok',
        Status::Banned => 'blocked',
    };
}

$u = new User(id: 1, email: 'ana@x.io');     // 8.0 named args
echo $u?->email ?? 'none';                   // 8.0 nullsafe
$len = strlen(...);                          // 8.1 callable
json_validate('{"a":1}');                    // 8.3
$big = array_find([3, 8], fn($n) => $n > 5); // 8.4
$s = ' Hi ' |> trim(...) |> strtolower(...); // 8.5 pipe
```

CODE EXPLAINED

- → Enums and readonly replace constant classes and getters
- → Constructor promotion + named args cut boilerplate
- → `match` + enums give exhaustive branching
- → First-class callables and the pipe compose functions
- → Array helpers in 8.4 replace common loops

8.0 – 8.2

VERSION	HIGHLIGHTS
8.0	JIT, union types, match, named args, nullsafe ?->, attributes, promotion, str_contains
8.1	Enums, readonly props, fibers, never, first-class callables, new in initializers, array_is_list
8.2	Readonly classes, DNF types, true/null/false types, Random extension, dynamic props deprecated

8.3 – 8.5

VERSION	HIGHLIGHTS
8.3	Typed class constants, #[\Override], json_validate, dynamic const fetch, mb_str_pad
8.4	Property hooks, asymmetric visibility, new without (), array_find/any/all, lazy objects, Dom\HTMLDocument, #[\Deprecated]
8.5	Pipe |>, clone with, array_first/last, #[\NoDiscard], URI extension, closures in const exprs

OLD → NEW

OLD IDIOM	MODERN REPLACEMENT
`strpos() !== false`	str_contains() (8.0)
`switch for values`	match (8.0)
`class constants as enum`	enum (8.1)
`private + getter`	readonly / private(set)
`isset($a) ? $a : $b`	$a ?? $b
`Closure::fromCallable`	f(…) (8.1)

⚠ COMMON MISTAKES

- Using 8.4/8.5 syntax when production runs an older version
- Upgrading without reading deprecation notices
- Mixing old and new idioms inconsistently

✓ TIPS & BEST PRACTICES

- Pin the minimum PHP version in composer.json
- Let Rector apply new idioms project-wide
- Run tests on the next PHP version in CI early

QUICK SUMMARY

- 8.0 match/named args
- 8.1 enums/readonly
- 8.4 hooks/visibility
- 8.5 pipe/clone with

Quick Reference — Syntax

SYNTAX AT A GLANCE

A one-page reminder of PHP's core syntax, from tags and variables to classes and namespaces. Each line shows the shape of a construct; the detailed sheets cover rules and edge cases. Start every file with `<?php` and `declare(strict_types=1);`, type everything, and prefer strict comparisons.

NOTES

- Statements end with `;`
- Variables start with `$`; constants don't
- Blocks use braces; templates can use `: … end*;`
- Strings: `'literal'`, `"interpolated $x"`
- Comments: `//`, `#`, `/* */`, `/** */`

CORE SYNTAX

```php
<?php
declare(strict_types=1);
namespace App;
use App\Models\User;

const MAX = 10;
$name = 'Ana';              $list = [1, 2, 3];
$map  = ['k' => 'v'];       [$a, $b] = [1, 2];

if ($x > 0) { /* … */ } elseif ($x < 0) { /* … */ } else { /* … */ }
$r = match ($code) { 200, 201 => 'ok', default => 'err' };
foreach ($map as $k => $v) { /* … */ }
for ($i = 0; $i < MAX; $i++) { /* … */ }
try { /* … */ } catch (A|B $e) { /* … */ } finally { /* … */ }

function add(int $a, int $b = 0): int { return $a + $b; }
$double = fn(int $n): int => $n * 2;
$greet  = function (string $n) use ($name) { return "$n & $name"; };

interface Shape { public function area(): float; }
final class Circle implements Shape
{
    public function __construct(private float $r) {}
    public function area(): float
    { return M_PI * $this->r ** 2; }
    public static function unit(): static
    { return new static(1); }
}
echo (new Circle(2))->area();
```

CODE EXPLAINED

- → File header: tag, strict types, namespace, imports
- → Arrays and destructuring
- → Control flow, match and exceptions
- → Functions, arrow functions, closures
- → Interfaces, classes, promotion, static factory

OPERATORS

GROUP	OPERATORS
Arithmetic	+ - * / % **
Comparison	== === != !== < > <= >= <=>
Logical	&& || ! xor
String	. .=
Null	?? ??= ?->
Other	?: … instanceof clone |>

DECLARATIONS

CONSTRUCT	SYNTAX
Constant	const X = 1;
Function	function f(T $a): R { }
Class	final class C extends P implements I
Property	public readonly T $p;
Enum	enum E: string { case A = 'a'; }
Attribute	#[Attr(arg)]

TYPE SYNTAX

FORM	EXAMPLE
Nullable	?int
Union	int|string
Intersection	A&B
DNF	(A&B)|null
Special	void never mixed static
Variadic	int ...$n

⚠ COMMON MISTAKES

- Missing `$` on variables or `;` at line ends
- Using `+` to join strings (use `.`)
- Loose `==` where `===` was meant

✓ TIPS & BEST PRACTICES

- Copy the file header into every new file
- Let your IDE and PHP-CS-Fixer format code
- Look up the detailed sheet when in doubt

QUICK SUMMARY

- `<?php` + strict types
- `$vars`, `[]` arrays
- match / foreach / try
- Typed functions and classes

Quick Reference — Core Functions

string · array · math · date · file · JSON · type checks

MOST-USED BUILT-IN FUNCTIONS

PHP's standard library is large; these are the functions you'll reach for daily. Remember the two classic gotchas: **argument order** isn't consistent (`array_map($fn, $arr)` vs `array_filter($arr, $fn)`), and many functions return **`false` on failure** — compare strictly. Use `mb_*` versions for user-visible text. Detailed sheets are referenced in each table title.

NOTES

- Array functions return new arrays except sort/push/pop/shift
- String functions never modify the original string
- Pass `true` for strict `in_array` / `array_search`
- Use `JSON_THROW_ON_ERROR` with JSON functions
- Check `php.net/function-name` for full signatures

DAILY TOOLKIT

```php
<?php
// Strings (16-21)
strlen($s); mb_strlen($s); trim($s); strtolower($s);
str_contains($s, 'x'); str_starts_with($s, 'a');
str_replace('a', 'b', $s); substr($s, 0, 5);
explode(',', $s); implode(', ', $arr); sprintf('%05.2f', $n);

// Arrays (51-59)
count($a); in_array($v, $a, true); array_key_exists($k, $a);
array_map($fn, $a); array_filter($a, $fn); array_reduce($a, $fn, 0);
array_keys($a); array_values($a); array_merge($a, $b);
array_slice($a, 1, 2); array_column($rows, 'name', 'id');
sort($a); usort($a, fn($x, $y) => $x <=> $y); array_unique($a);

// Math & dates (90-92)
round($n, 2); max($a); random_int(1, 6);
date('Y-m-d'); new DateTimeImmutable('+1 day');

// Files & JSON (77-81)
file_get_contents($p); file_put_contents($p, $data, FILE_APPEND);
json_encode($v, JSON_THROW_ON_ERROR); json_decode($j, true);

// Types (14-15)
is_int($x); is_string($x); is_array($x); get_debug_type($x);
```

CODE EXPLAINED

- → Strings: length, case, search, replace, split
- → Arrays: search, transform, combine, sort
- → Math & dates: rounding, extremes, random, formatting
- → Files & JSON: read, write, encode, decode
- → Types: runtime checks

STRINGS & ARRAYS

NEED	FUNCTION
Length	strlen / mb_strlen / count
Contains	str_contains / in_array(..., true)
Find position / key	strpos / array_search
Replace	str_replace / array_replace
Split / join	explode / implode
Transform each	array_map
Keep some	array_filter

RETURNS FALSE ON FAILURE

FUNCTION	CHECK WITH
`strpos()`	!== false
`array_search()`	!== false
`file_get_contents()`	=== false
`fopen()` / `fgets()`	=== false
`strtotime()`	=== false
`PDOStatement::fetch()`	=== false

ARGUMENT ORDER

FUNCTION	ORDER
`array_map`	($fn, $array)
`array_filter`	($array, $fn)
`array_reduce`	($array, $fn, $init)
`in_array`	($needle, $haystack)
`strpos`	($haystack, $needle)
`str_replace`	($search, $replace, $subject)

⚠ COMMON MISTAKES

- Ignoring `false` returns
- Swapping needle/haystack order
- Byte functions on UTF-8 text

✓ TIPS & BEST PRACTICES

- Let IDE parameter hints guide argument order
- Wrap `false` -returning calls in small helpers that throw
- Prefer the newer, clearer functions (str_contains, array_find)

QUICK SUMMARY

- Strings: str_*, mb_*
- Arrays: array_* + sort
- Check for false
- Mind argument order